

The Ledger: A Closed-System Framework for Financial Settlement

Unified Design Specification: Architecture, Lifecycle, and CDM Integration

R. Delloye

July 2026

Abstract

Within its scope, the post-trade activity addressed by this framework reduces to one operation: the atomic move of a quantity between wallets. From this single primitive the Ledger builds a unified system in which financial instruments—equities, bonds, derivatives, swaps, managed accounts, Quantitative Investment Strategies, listed futures—are deterministic contracts that emit sequences of moves. One law governs the system: conservation, which holds by construction. From it follow path-independent profit and loss, complete time travel over an immutable event log, and idempotent lifecycle processing through pure functions. Several classes of internal reconciliation failure become structurally unreachable. Unit state lives in three homes—immutable `ProductTerms`, the shared `UnitStatus`, and per-position `PositionState`; `UnitStatus` is a materialised projection of the event log, not an authoritative mutable store. The scalar balance suffices for buy/sell/exercise workflows; for securities lending it extends to a six-coordinate position vector. Throughout, the design is illustrated in Haskell, with types chosen so that illegal states are unrepresentable. This design specification presents the three-home state model, the `UnitStatus` characterisation, the futures lifecycle, and the managed-account workflow in one self-contained development.

Contents

Roadmap and Reading Guidance	7
I Foundations: The Closed System and Conservation	8
1 Purpose, Scope, and Properties	8
1.1 The Problem	8
1.2 The Solution	8
1.3 Three Unreachable Internal Breaks	9
1.4 Architecture	9
1.5 Scope	9
1.6 The Ledger Boundary	9
2 The Closed Ledger System	11
2.1 Wallets and Units	11
2.2 Ownership as Balance	11
2.3 Atomic Moves	12
2.4 Transactions and the Conservation Law	13
2.5 Virtual Wallets	15
2.6 Books and Reference Wallets	16
2.7 The Self-Consistency Principle	17

3	The Unit Store	18
3.1	Units	18
3.2	Unit Identity: OTC versus Listed	18
3.3	Three-Tier Architecture	18
3.4	Registration	19
3.5	Two-Stage Validation	21
3.6	Guarantees	21
3.7	CDM Alignment	21
II	The Three-Home State Model and UnitStatus	22
4	Where Unit State Lives: The Three-Home State Model	22
4.1	The placement rule and the 2×2	23
4.2	Why three homes, and why the fourth cell is empty	23
4.3	The UnitStatus discipline, stated once	25
4.4	The construction	25
4.5	The conditions C1–C14	36
4.6	Why exactly three maps	37
4.7	The unique design	37
4.8	Invariants made unrepresentable	37
4.9	Reference implementation	38
III	Valuation, Contracts, and Instrument Lifecycles	40
5	Portfolio Valuation and PnL	41
5.1	Quantity versus value	41
5.2	Mark-to-market valuation	41
5.3	State-sufficiency	42
5.4	Path-independent PnL	43
5.5	Price and flow decomposition	43
5.6	Dual valuation	44
6	Smart Contracts as Move Generators	44
6.1	Contract Definition	44
6.2	Composition: Modular over Composite	46
6.3	Equity, Dividend, and Corporate-Action Contracts	46
6.4	European Put—Full Move Schedule	47
6.5	Interest Rate Swap—Full Move Schedule	48
7	Lifecycle Management	49
7.1	The Transition Function	49
7.2	Idempotence	50
7.3	The Single Door	50
7.4	Purity	51
7.5	Time Travel	52
7.6	State Storage and Embedded Defence	53
7.7	Lifecycle Risk Profile	53
7.8	Strategy as Unit: QIS and Leveraged ETFs	54

8	The State-Basis Discipline	54
8.1	The basis coordinate	55
8.2	The adjustment operators and their composition law	58
8.3	The adjustment schedule	63
8.4	The stamp and the ingest door	65
8.5	The market-data contract	65
8.6	The datum-kind registry	76
8.7	The market-data integrator	79
8.8	The invariant	80
8.9	Type-level enforcement	80
8.10	Time travel and late corporate actions	83
8.11	Failure semantics as workflow	84
8.12	The 2-for-1 split, end to end	85
8.13	CDM corporate-action mapping	89
9	The Futures Lifecycle	92
9.1	Notation, and where the future’s state lives	92
9.2	The instrument and the worked example	94
9.3	Listing	95
9.4	First trade	96
9.5	Daily variation-margin settlement: the mechanism and the clean case (day 1) . .	98
9.6	A subsequent, intraday trade (day 2)	99
9.7	Settlement under intraday trading (day 2): the anchor	99
9.8	Closing a position to flat	100
9.9	Expiry, final settlement, and the Close	100
9.10	After expiry: state versus derivation	101
9.11	Invariants threaded through the life	101
9.12	Escalations	105
IV	Accounts, Reporting, Operations, Settlement, and CDM Integration	105
10	Managed Accounts, Virtual Portfolios, and TRS	105
10.1	The mandate is a unit	106
10.2	Subscription	107
10.3	Trading under mandate	107
10.4	Valuation and NAV	108
10.5	Fee logic	108
10.6	Segregation	109
10.7	CSA margin and collateral	110
10.8	TRS as synthetic managed account	110
10.9	Redemption and wind-down	111
10.10	Account-level balance-sheet substantiation	111
10.11	Worked example	111
10.12	Conformance and flags	112
11	Balance Sheet Substantiation and Dual Valuation	113
11.1	Balance Reconstruction	113
11.2	Substantiation Scope	114
11.3	Dual Valuation	115
11.4	Snapshots and Audit Trail	116

12 Implementation and Operations	116
12.1 Three-Layer Architecture	116
12.2 Balance-Update Algorithm	117
12.3 Constant-Cost Profit and Loss	117
12.4 Fault Tolerance	117
12.5 Corrections as Events	118
12.6 From Moves to Settlement Instructions	118
12.7 ISO 20022 Interface	118
13 The Settlement Layer Interface	119
13.1 The Settlement Projection	119
13.2 Boundary Separation	121
13.3 Transaction Types and Settability	122
13.4 Lifecycle-Originated Settlement	122
13.5 DvP: The Two-Level Guarantee	123
13.6 Gross-to-Net Reconciliation	123
13.7 The Confirmation Return Path	124
14 ISDA CDM Integration	124
14.1 The CDM	124
14.2 Derivation: CDM Maps to the Framework Primitives	125
14.3 The Synonym Layer and Ingestion	126
14.4 The Forgetful Map F	126
14.5 CDM Enumerations as Generator Universe	128
V Obligations, Generalised Positions, and Securities Lending	128
15 Orchestration and Obligation Liveness	129
15.1 Execution-engine requirements	129
15.2 Processor requirements	129
15.3 The executor as an activity	131
15.4 The due-event scheduler	131
15.5 Concurrency: single-writer by construction	132
15.6 Obligation liveness	132
16 Generalised Positions and Securities Borrowing and Lending	138
16.1 Where the Scalar Model Breaks	138
16.2 The Generalised Coordinate Vector	138
16.3 Graceful Degeneration to the Scalar Model	141
16.4 The Lending Resolution: Named Wallets	141
16.5 Analytical Projections and Encumbrance	141
16.6 The SBL Smart Contract	142
16.7 SBL State Machine	143
16.8 Atomic Moves for SBL Lifecycle Events	143
16.9 Title Transfer vs Security Interest	144
16.10 Collateral and Margin	144
16.11 Short Selling and Inventory	145
16.12 Cash Collateral Reinvestment	145
16.13 On-Lending Chains	146
16.14 Locates	147
16.15 Cross-Border Regulatory Treatment	150

16.16	External Reconciliation	150
16.17	CDM Alignment and Regulatory Reporting	151
16.18	SBL-Specific Invariants (P11–P20)	151
VI	Formal Properties, Limitations, and Context	153
17	Invariants, Conservation Laws, and Property-Based Testing	153
17.1	Conservation as Central Invariant and Universal Oracle	153
17.2	The Core Invariants P1–P10	154
17.3	SBL Invariants P11–P20	155
17.4	Obligation Liveness Invariants P21–P23	156
17.5	Basis Invariants P24 and P25	156
17.6	Locate Invariant P26	156
17.7	Producer and Vocabulary Invariants P27–P31	157
17.8	The Conditions C1–C14	158
17.9	Which Condition Discharges Which Invariant	159
17.10	Invariants as Specification and Oracle	162
17.11	Property-Based Testing over the CDM Generator Universe	162
17.12	Specification-First Development	163
18	Regulatory Obligations and the Direction of Travel	163
19	Scope and Limitations	165
19.1	What the Framework Does Not Do	165
19.2	Theorems and Modelling Choices	166
19.3	Architecture and Operational Reality	166
20	Frequently Asked Questions	167
21	Conclusion and Open Problems	169
21.1	Summary	169
21.2	Resolved Design Problems	169
21.3	Open Problems	169
21.4	Flagged Open-Items Register	170
VII	Appendices	172
A	CDM Type Mapping Tables	172
B	Property Test Catalogue	173
C	Reconciliation Failure Mode Taxonomy	178
D	Coordination Between Pricing, Contract State, and Market Data	179
E	Glossary	180
F	The CDM Product Model—A Developer’s Guide	186
F.1	The Six-Layer Hierarchy	186
F.2	Unit Identity: Trade Is the Unit	187
F.3	Payout Composition and the TransferableProduct Boundary	188
F.4	Tokenised Securities: Double-Counting and the Flat Custodian	189

F.5	Date Handling: Types, Resolution, and Gotchas	190
G	Available Inventory Identity: Eight-Scenario Verification	192
H	European SBL Worked Example: GMSLA 2010 Title Transfer with Rehy- pothecation	193
H.1	Participants and Setup	193
H.2	Step 1: Loan Initiation (Day 0)	194
H.3	Step 2: Delivery to Hedge Fund and Short Sale (Day 0)	194
H.4	Step 3: Rehypothecation of Bund Collateral (Day 1)	195
H.5	Step 4: Mark-to-Market (Day 3)	196
H.6	Step 5: Manufactured Dividend (Day 7)	196
H.7	Step 6: Partial Recall and Return (Days 10–12)	196
H.8	Step 7: Full Return and Settlement (Day 20)	197
I	US SBL Worked Example: SEC Rule 15c3-3 with Cash Collateral	198
I.1	Participants and Setup	199
I.2	Step 1: Loan Initiation (Day 0)	199
I.3	Step 2: Rebate Mechanics and Cash Reinvestment	200
I.4	Step 3: Rehypothecation Constraints under Rule 15c3-3	201
I.5	Step 4: Daily Mark-to-Market (Day 5)	201
I.6	Step 5: FINRA SLATE Reporting	202
I.7	Step 6: Partial Recall and Return (Day 15)	202
I.8	Step 7: Full Return and Close-Out (Day 25)	203
I.9	Cross-Jurisdictional Comparison	203

Roadmap and Reading Guidance

Each audience has a shortest route to a working understanding. The table below gives that route, the sections to add for more depth, and the sections a first pass may skip without loss.

Audience	Minimal path	Then if needed	May skip on first pass
Regulator	§1, the conservation core of §2, §18, §19	§13, §20, §21, App. C, App. H, App. I	the Haskell, §4 construction detail, §6, §9 detail, §§15–17 internals
Implementer	§§2–4, §6, §7, §12, §17	§9, §13, §14 + App. F, §15 incl. §15.2, §16, App. A/B/G	§5 profit-and-loss derivations, §10, §11, §18, worked examples
Quant	§2, §5, §11, App. D	§9, §16, §8	§3, §12, §13, §§14–15 mechanics, §18
Market-data integrator	§8.5 (the whole boundary contract: obligations, direction of flow, datum classes, operating modes), App. D	the rest of §8, §7	everything outside §8 on a first pass
Risk manager	§2, §5, §7, §10, §11, §16	§18, App. C	§3, §6 internals, §12, §§14–15, the Haskell

The scalar wallet balance carries the whole argument through Parts I–IV. The six-coordinate position vector and securities lending enter only in Part V, once obligations are established.

One reading aid recurs throughout. Where a concept is introduced, a short note headed MENTAL MODEL offers a concrete, picturable image—water sealed in pipes, a ledger written in ink, a cloakroom ticket—whose behaviour mirrors the behaviour the concept fixes. These notes are aids to intuition and nothing more: no definition, proof, or condition rests on one, and none is cross-referenced. Deleting every such note would leave the specification complete and unchanged.

Part I

Foundations: The Closed System and Conservation

1 Purpose, Scope, and Properties

1.1 The Problem

Conventional post-trade infrastructure records the same economic reality in many places: trading systems, risk engines, general ledgers, custodian records, regulatory reports. Each carries its own copy of positions, valuations, and histories. Reconciliation is the work of verifying that these independent records agree. It is the dominant operational cost and the primary source of breaks. The cause is architectural: a design with many sources of truth admits inconsistency by construction.

1.2 The Solution

The post-trade activities within scope use a single primitive: the atomic move of a quantity between wallets. A financial instrument is a deterministic contract that emits a sequence of moves; this covers equities, bonds, derivatives, swaps, managed accounts, and quantitative investment strategies (QIS). Only the moves are stored. Everything else is recomputed from them on demand and never kept as a second copy. An immutable move stream records every move with full provenance; it is the sole canonical record. Every other view is a projection of it: balances, profit and loss, balance sheets, reports.

The framework satisfies six properties by construction. Four constrain the record itself: atomicity, conservation, determinism, state-sufficiency. Two describe its behaviour over time: value invariance across deterministic lifecycle events, and reconstruction of any historical state.

1. **Atomicity.** Every transaction is an indivisible operation with complete provenance.
2. **Conservation.** The system is closed: the *quantity* of each unit is neither created nor destroyed. This is double-entry accounting reduced to one equation, $\sum_w w(u) = 0$. Conservation governs quantities per unit, not market values; value changes from price movements are captured by the valuation layer (Section 5).
3. **Determinism.** Given inputs and conditions, execution is reproducible.
4. **State-Sufficiency.** Portfolio value depends only on current state—wallet balances, unit state (including embedded lifecycle state: accrued coupons, exercise status, barrier flags), and the current price vector—not on transaction history. Where unit state lives is fixed by the three-home model (Section 4).
5. **Lifecycle Value Invariance** (restricted form). For contractual lifecycle events involving only deterministic cash flows—coupon and dividend payments, resets, principal redemptions—the total value of all wallets under a given price vector is invariant: the jump in the instrument’s price is exactly offset by the explicit cash move. Lifecycle processing reshuffles value between units and cash; it creates and destroys no PnL. This invariance is conservation’s counterpart for value.

Qualification. For lifecycle events involving optionality—exercise, expiry, barrier knock-in/out, callable redemption—the invariance holds only for intrinsic value. Time value and embedded option value are extinguished when optionality resolves: exercising an option destroys time value; calling a bond extinguishes the holder’s above-market coupon

stream. This is not a conservation violation, because those components reflect probabilistic expectations that cease to exist when uncertainty resolves.

6. **Time Travel.** The exact ledger state at any historical timestamp is reconstructed by replaying the immutable move stream and unit-state events up to that time; the operation `clone_at( $t$ )` performs this reconstruction as of time t . Combined with pure lifecycle functions, this yields historical truth: the ledger is restored to what was known at time t and re-simulated. “Time travel to what was known at t ” (market data as captured at t) and “time travel to t with today’s corrected data” (restated data) are distinct and are not conflated.

These properties make the ledger a deterministic function of its event history: given the same transactions and market data, the state at any time is uniquely determined.

1.3 Three Unreachable Internal Breaks

When the ledger is the sole system of record for the activities it covers, three of the most damaging reconciliation failures are structurally impossible within that scope: trading-system / general-ledger divergence, PnL / balance-sheet mismatch, and inter-entity breaks. Each is a disagreement between two views of the same activity. By construction both views are projections of one record, so the disagreement is not representable. The mechanism per break, and the scope of the guarantee, are given in Section 2.

1.4 Architecture

A *wallet* holds quantities of *units* (currencies, securities, derivatives, or any contractual obligation). A *move* transfers a quantity of a unit between two wallets. A *transaction* is a finite set of simultaneous moves satisfying conservation. A *smart contract* is a deterministic program mapping inputs and state to sequences of moves. Unit state records each unit’s lifecycle position. The immutable *move stream* records every move with full provenance. All other views are derived projections of it. External authorities—central securities depositories (CSDs), share registrars, regulators—maintain independent records at the boundary.

1.5 Scope

The scope is trading books measured at fair value through profit or loss under International Financial Reporting Standards (IFRS) and US Generally Accepted Accounting Principles (US GAAP).¹ Positions are marked to market using price vectors consistent with IFRS 13 and ASC 820 fair-value guidance. Banking-book activities are out of scope: amortised-cost loans, held-to-collect debt, hedge accounting for structural positions. When added, they extend the same primitives.

The International Swaps and Derivatives Association (ISDA) Common Domain Model (CDM) is the canonical vocabulary. It defines the unit universe $\mathcal{U}$, drives lifecycle state transitions, bounds the input space for property-based testing, and maps ledger transactions to ISO 20022 settlement messages. CDM integration, the product hierarchy, and its layer-by-layer justification are developed in Section 14 and Appendix F.

1.6 The Ledger Boundary

The framework’s guarantees hold for activity recorded inside the ledger boundary. The boundary is stated here, once. The dividing line is uniform: inside is what the move stream and

¹IFRS 9 §§4.1.2, 4.1.4, 5.7.1–5.7.7; ASC 815, ASC 320/321, ASC 820.

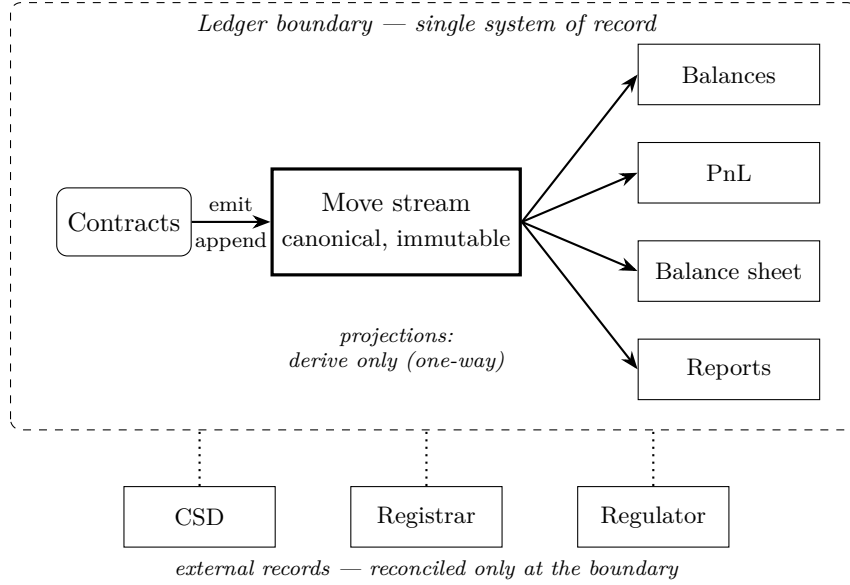


Figure 1: The single source of truth. Contracts emit moves that are appended to one immutable move stream; every internal view—balances, PnL, the balance sheet, and reports—is a one-way *projection* of that stream. The arrows are derivations, not synchronisations: there is no second internal record, hence nothing to reconcile within the boundary. External authorities lie outside the boundary and are reconciled against only at its edge.

its projections determine; outside is what the ledger consumes from, or reports to, external authorities but does not govern.

Inside (governed by the framework’s invariants):

- Positions: wallet balances, the generalised position vector, the conservation law $\sum_w w_t(u) = 0$.
- Moves: atomic transfers of quantity between wallets, with full provenance.
- Lifecycle events: state transitions on units, recorded as transactions with CDM event metadata.
- PnL: portfolio valuation $V_t = \sum_w w_t(u) P_t(u)$ for an externally supplied price vector P_t .
- Time travel: reconstruction of historical state by move-stream and unit-state replay.
- Internal reconciliation: the guarantee that trading-system, general-ledger, and balance-sheet views are projections of one record.

Outside (external systems, not governed by the framework):

- Price formation: P_t is an external input; the framework does not compute, validate, or guarantee executable prices.
- Legal agreements: ISDA Master Agreements, Global Master Securities Lending Agreements (GMSLAs), GMRA, CSAs, and custody agreements determine legal rights. The ledger records economic positions, not legal ownership or enforceability.
- Settlement infrastructure: CSD connectivity, payment-system access, settlement finality, and real-world delivery-versus-payment (DvP) depend on external infrastructure (DTC, Euroclear, Clearstream, Continuous Linked Settlement (CLS), TARGET2).
- Regulatory gateways: trade, transaction, and prudential reporting (EMIR, MiFIR, Securities Financing Transactions Regulation (SFTR), Dodd-Frank, MiFID II RTS 25, CRR, FRTB) require submission infrastructure beyond the ledger.
- Reference-data authority: instrument master data (International Securities Identification Numbers (ISINs), contract specifications, corporate-action terms) is consumed from external sources, not created by the ledger.

- Model governance: pricing and risk models and their validation.
- Accounting policy: IFRS 9 classification (FVTPL, FVOCI, amortised cost), hedge-accounting designation, and impairment assessment.

Every “within scope” qualifier elsewhere in this document refers to the inside list above.

2 The Closed Ledger System

The ledger is built from four primitives: *wallet*, *unit*, *move*, and *transaction*. Everything else is a projection of the move stream they generate: balances, valuation, profit and loss, lifecycle state, reports. This section establishes the primitives, the conservation law that closes the system, and the structural consequence that internally inconsistent states are unreachable. It is the spine on which the remainder of the specification stands.

2.1 Wallets and Units

A wallet is a named place inside the ledger where quantities sit. Its entire state is one signed number per unit; the sign carries direction, so a short position needs no separate machinery.

Definition 2.1 (Wallet). *A wallet w is a named account within the ledger holding quantities of units. It is a logical partition of position space—not a bank account, a custody account, or a legal entity. Let $\mathcal{W}$ denote the set of all wallets and $\mathcal{U}$ the set of all units. A wallet state at time t is a function*

$$w_t : \mathcal{U} \rightarrow \mathbb{R},$$

mapping each unit u to a quantity $w_t(u)$. A negative balance $w_t(u) < 0$ is a short position or obligation.

Units include currencies, securities (identified by ISIN), commodities, derivatives, over-the-counter (OTC) instruments, swaps, loans, and any other contractual obligation. $\mathcal{U}$ is extensible: a new instrument type is a new element of $\mathcal{U}$, not a change to existing infrastructure. The Unit Store (Section 3) manages $\mathcal{U}$ concretely: identification, validation, registration. The mapping from wallets to external accounts (custody, settlement, regulatory) belongs to the settlement layer (Section 13), not to the wallet.

2.2 Ownership as Balance

What a wallet owns is the number standing in its balance, nothing more. There is no separate register of title to keep in step with the holding.

Principle 2.2 (Ownership). *Economic ownership is the wallet balance: to own 100 shares of a stock is for that wallet’s balance in that stock to be 100. Where ownership and possession diverge—nominee structures, securities lending, repo, pledged collateral—the generalised position model (Section 16) extends the scalar balance to a vector distinguishing economic ownership from operational state.*

This eliminates the internal reconciliation problem at its root. Conventionally, ownership is recorded in several systems: trading, back-office, custodian, general ledger. Reconciliation then verifies their agreement. Here there is one record. The ledger is the canonical internal record of economic positions. External authorities (CSDs, registrars, legal documentation) maintain independent records at the boundary, where reconciliation remains necessary (Section 11).

2.3 Atomic Moves

A move is the one way any balance ever changes. It takes a positive quantity out of one wallet and puts the same quantity into another, in a single indivisible step.

Definition 2.3 (Move). *An atomic move m is an indivisible transfer of a positive quantity q of a unit u from a source wallet w_s to a destination wallet w_d at timestamp t , tagged with a source contract identifier s and metadata (event description, external references). Its semantics are*

$$w_s(u) -= q, \quad w_d(u) += q.$$

External references include counterparty identifiers and message identifiers under ISO 20022, the financial-messaging standard. The source s names the smart contract or component that generated the move.

Every change to the ledger is an explicit, time-stamped event; nothing happens off the record.

Principle 2.4 (Atomicity). *Every ledger change occurs as a discrete, time-stamped move. There are no implicit adjustments or retrospective calculations.*

The move is the additive operation of the position algebra. A quantity is an element of an additive abelian group. A move subtracts an element from one wallet and adds the same element to another. In Haskell the group is `Qty`, where `<>` is addition, `empty` the zero position, and `negQty` the additive inverse. Quantities are exact `Integer` minor units, never floating point. Conservation and deterministic replay are arithmetic facts; floating point would forfeit them. The listing realises this algebra: the group, the move, and the one function that applies a move to the ledger.

```
-- Qty: the additive abelian group of exact minor units.
-- A negative Qty is a short position; the group must host it, so Qty is unsigned-free.
newtype Qty = Qty Integer deriving (Eq, Ord)

instance Semigroup Qty where Qty a <> Qty b = Qty (a + b)    -- addition
instance Monoid Qty where empty = Qty 0                      -- the zero position

negQty :: Qty -> Qty                                          -- the additive
    inverse
negQty (Qty n) = Qty (negate n)
-- negQty q <> q == empty    -- the group's inverse axiom, for every q

newtype WalletId = WalletId String deriving (Eq, Ord, Show)
newtype UnitId = UnitId String deriving (Eq, Ord, Show)
newtype Timestamp = Timestamp Integer deriving (Eq, Ord, Show)
newtype SourceId = SourceId String deriving (Eq, Ord, Show)

-- A move transfers a *positive* magnitude; direction is carried by (from,to).
-- Illustrative form, local to this section: the reference (reference/Ledger.hs)
-- carries the magnitude as PosQty -- positive by type, sole door mkPosQty
-- (sec:futures) -- so positivity is a type guarantee there; here it is the
-- guard in 'move', and the mini-ledger below is this section's own.
data Move = Move
    { mFrom :: !WalletId, mTo :: !WalletId, mUnit :: !UnitId
    , mQty :: !Qty, mTime :: !Timestamp, mSource :: !SourceId } deriving (Eq,
    Show)

-- 'move' is the sole ingest path; it is the parse boundary that rejects a
-- non-positive magnitude (Qty itself stays signed, to hold balances).
```

```

move :: WalletId -> WalletId -> UnitId -> Qty -> Timestamp -> SourceId -> Maybe
      Move
move ws wd u q t s | q > mempty = Just (Move ws wd u q t s)
                   | otherwise  = Nothing

-- The per-wallet deltas a move induces; their sum is the conservation witness.
moveDelta :: Move -> [(WalletId, Qty)]
moveDelta (Move s d _ q _ _) = [(s, negQty q), (d, q)] -- negQty q <> q = mempty

-- The ledger: each wallet maps units to quantities. An unread balance is the
-- group identity, so 'balance' is total; 'adjust' adds a (possibly negative) dq.
type Ledger = Map WalletId (Map UnitId Qty)

balance :: WalletId -> UnitId -> Ledger -> Qty
balance w u = maybe mempty (Map.findWithDefault mempty u) . Map.lookup w

adjust :: WalletId -> UnitId -> Qty -> Ledger -> Ledger
adjust w u dq = Map.insertWith (Map.unionWith (<>)) w (Map.singleton u dq)

-- applyMove realises w_s -= q ; w_d += q as one act.
applyMove :: Move -> Ledger -> Ledger
applyMove (Move ws wd u q _ _) = adjust wd u q . adjust ws u (negQty q)

```

`Qty` and its group instances are the same definitions the reference of Section 4 uses; that reference abbreviates the inverse `negQty` as `qneg`. The `Move` and mini-ledger of this listing are illustrative. In the reference, `Move` carries its magnitude as `PosQty`, so positivity is a type guarantee rather than a guard; the reference form in Section 4 is the verbatim one.

2.4 Transactions and the Conservation Law

One economic event is recorded as one transaction. Its moves carry every balance change; a state delta carries everything else the event writes.

Definition 2.5 (Transaction). *A transaction τ is the one atomic event recorded in the log: an ordered list of moves $[m_1, \dots, m_n]$ together with the non-balance state delta they ride with—per-wallet bookkeeping (accumulated cost, high-water mark, entry NAV), shared unit-status writes, and a product-terms introduction or append. The moves carry every balance change; the delta carries everything else. When several transactions share a timestamp, the move stream assigns a total order (by sequence number within the timestamp) for deterministic replay. Conservation holds regardless of order; intermediate balances, margin, and lifecycle state depend on it.*

Because every move removes from one wallet exactly what it adds to another, no transaction changes the total quantity of any unit held across the whole system.

Principle 2.6 (Conservation Law). *For any transaction τ and any unit $u \in \mathcal{U}$, the net change in total quantity across all wallets is zero.*

Conservation follows from the move semantics. Each move contributes $-q + q = 0$ to the total for its unit, so the sum over the transaction is zero. This is not a runtime check. It is an algebraic consequence of the additive-group structure: adding an element's inverse to one wallet and the element to another nets to the group identity.

The Haskell makes this exact. `foldMap` over a transaction's moves is the unique homomorphism from the free monoid of moves into `Qty`. It maps each move to its contribution and combines the results with `<>`. Every move's contribution is already `negQty q <> q = mempty`, so the fold lands at `mempty` by the monoid laws alone. The empty move list is `mempty`, so a move-less event is conserved for free; a registration or a preserving amendment is such an event,

carrying only a state delta. Conservation is therefore a property of the type, not a checked side-condition. There is no separate validation step and no unconserved transaction to reject.

```

-- Illustrative Qty form; the reference weave of sec:states is the verbatim one
-- (magnitude PosQty, read through unPosQty).
-- The ONE atomic event: the conserved economic flow (txMoves, in edge form)
-- PLUS the non-balance state delta it rides with. A balance changes ONLY via a
-- Move -- there is no balance writer in the per-wallet rows -- so "a balance
-- change without a conserving move" is unrepresentable, and conservation needs
-- no validation gate. txIntroduce = Just marks the introduction of txUnit
-- (registration); Nothing operates on an existing unit (trade/settlement).
data Transaction = Transaction
  { txUnit      :: UnitId          -- the unit this event
    concerns
  , txMoves     :: [Move]          -- conserved flow as edges (
    by construction)
  , txRows      :: Map WalletId [SomeWrite] -- non-balance bookkeeping (
    ac/hwm/entryNav)
  , txStatus    :: [StatusWrite]   -- shared UnitStatus writes
  , txIntroduce :: Maybe (TermsVersion, UnitStatus) -- Just => introduce txUnit (
    registration)
  , txAppend    :: Maybe TermsVersion -- Just => append a version (
    amendment)
  , txBoundary  :: Maybe BoundaryEvent -- Just => declare a
    corporate-action
  } deriving (Show)                -- boundary on txUnit (Sec.
    state-basis)

-- Conservation, per unit. Each move's contribution to the system-wide total of
-- its unit is negQty q <> q = mempty; foldMap over the moves is therefore
-- mempty -- conservation by the monoid laws, not by a runtime sum. The empty
-- move list (registration, amendment) folds to mempty too: conserved for free.
unitDelta :: UnitId -> Transaction -> Qty
unitDelta u tx =
  foldMap (\m -> if mUnit m == u
    then negQty (mQty m) <> mQty m
    else mempty) (txMoves tx)
-- unitDelta u tau == mempty -- for every unit u and
-- transaction tau (this is P1's premise)

```

This is the type-level statement of the Conservation Law (Principle 2.6). System Closure (Theorem 2.7) is its consequence over the whole stream: balances start at zero, every transaction nets to zero, so the system-wide total of every unit stays at zero.

Theorem 2.7 (System Closure). *If every transaction satisfies the conservation law, then*

$$\sum_{w \in \mathcal{W}} w_t(u) = 0 \quad \forall u \in \mathcal{U}, \forall t.$$

Proof. By induction on transactions. At $t = 0$ all balances are zero, so $Q(u) \equiv \sum_w w_t(u) = 0$. Each transaction preserves $Q(u) = 0$ by the conservation law. Hence $Q(u) = 0$ at all t . $\square$

System Closure (Property P1) is the closed double-entry invariant: units in real wallets (assets) are exactly offset by units in virtual wallets (liabilities). No move creates or destroys units. Every move transfers between wallets. The property is stated with the other invariants in Section 17.

MENTAL MODEL. Picture the quantity of one unit as an incompressible fluid filling a system of sealed pipes that joins every wallet. A move is a transfer along one pipe: the litres leaving the

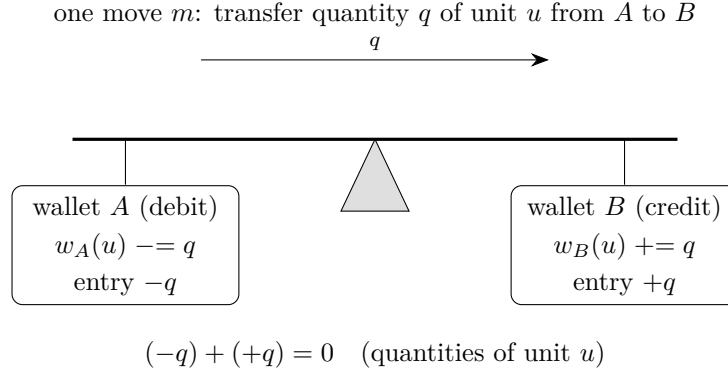


Figure 2: Conservation of quantity, per unit u . A move records two equal and opposite entries—a debit of q in wallet A and a credit of q in wallet B —which sum to zero, so $\sum_w w(u) = 0$ holds for every unit. What is conserved is *quantity*, not market value: prices move and so does value, and across optionality value is not conserved (§1). The beam weighs units, not money.

source arrive, to the litre, at the destination, and no pipe opens to anything outside, so the total volume in the system never changes. What the pipes seal in is volume, not worth—the price of the fluid is posted on a sheet outside the seal, and unlike the volume, that value is neither held constant nor conserved.

2.5 Virtual Wallets

Closure requires that every move have both a source and a destination. The framework therefore represents every external counterparty (market maker, exchange, clearinghouse, custodian) as a virtual wallet alongside the real positions.

Definition 2.8 (Real vs Virtual Wallets).

- **Real wallets** ($\mathcal{W}_{\text{real}} \subset \mathcal{W}$): the firm’s own portfolio positions; changes affect financial position and PnL.
- **Virtual wallets** ($\mathcal{W}_{\text{virtual}} \subset \mathcal{W}$): external counterparties and market entities; their balances track external positions for reconciliation but do not affect portfolio value.

MENTAL MODEL. Extend the pipework: the counterparty is the tank at the far end of every pipe, standing inside the plumbing rather than beyond it, so no pipe ever opens to the outside and the loop stays sealed. Its level moves exactly opposite to the firm’s own vessels—a litre out of the portfolio is a litre into the tank—so reading the tank against the counterparty’s own gauge is a reconciliation. Unlike the firm’s own vessels, the tank’s level is no part of what the firm holds; it closes the loop and invites comparison, and counts toward no portfolio value.

A virtual wallet serves three purposes. First, it gives every move a counterparty, keeping the system closed. Second, its balance records the firm’s position with that counterparty, for direct comparison against external statements. Third, the metadata trail through it connects each internal position to its external origin. The external world is thus inside the ledger: within its scope the ledger has no outside. This is the architectural step that makes System Closure (Theorem 2.7) hold.

A share purchase realises the construction. Buying 500 AAPL at \$100 through a broker is one transaction of two moves against the broker’s virtual wallet:

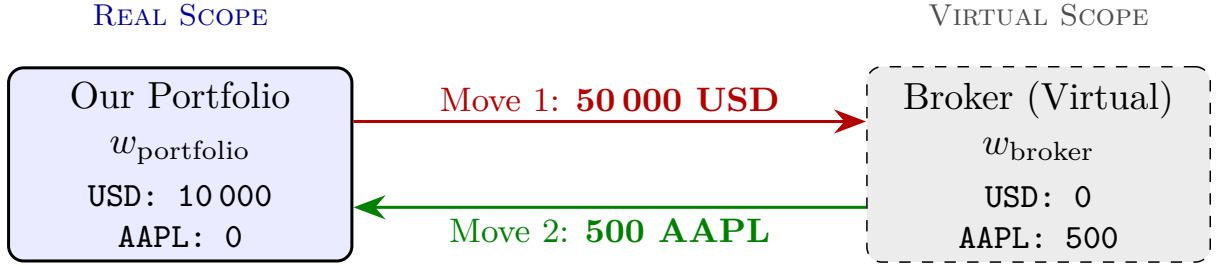
```
-- A trade carries moves and no introduction (txIntroduce = Nothing); the broker's
-- virtual wallet is the counterparty that keeps the system closed.
buyAAPL :: Transaction
buyAAPL = Transaction
```

```

{ txUnit = UnitId "AAPL"
, txMoves =
  [ Move portfolio      brokerVirtual (UnitId "USD") (Qty 50000) t (SourceId
    "broker")
    , Move brokerVirtual portfolio      (UnitId "AAPL") (Qty 500) t (SourceId
    "broker") ]
, txRows      = Map.empty
, txStatus    = []
, txIntroduce = Nothing      -- operates on already-registered units
, txAppend    = Nothing
, txBoundary  = Nothing }

-- unitDelta (UnitId "USD") buyAAPL == Qty 0      -- cash conserved
-- unitDelta (UnitId "AAPL") buyAAPL == Qty 0      -- shares conserved
-- after replaying both legs onto the empty ledger:
-- balance portfolio      (UnitId "AAPL") l == Qty 500
-- balance brokerVirtual (UnitId "AAPL") l == Qty (-500)  -- totals to 0

```



$$\begin{aligned}
&\text{Conservation: } \Delta USD_{total} = -50,000 + 50,000 = 0; \quad \Delta AAPL_{total} = +500 - 500 = 0
\end{aligned}$$

Figure 3: A share purchase: 500 AAPL at \$100. Cash moves from the real portfolio wallet to the virtual broker wallet; shares move the opposite way. Both moves execute atomically as one transaction; each unit's total across all wallets is conserved.

Initialisation. At inception all wallets start at zero, so $Q(u) = 0$ holds trivially. Opening positions are ordinary conservation-preserving moves from a virtual wallet to a real one: seeding 1,000 AAPL is a move of 1,000 AAPL from the custodian's virtual wallet to the portfolio, paired with its cash leg. No `set_balance` primitive exists. Conservation is never violated, not even at inception. Where the system starts mid-life, the initialisation moves capture the state at migration time t_{init} . The move stream begins at t_{init} ; pre-migration history resides in legacy systems.

2.6 Books and Reference Wallets

Firms group activity into *books* for reporting and governance. A book is associated with one or more real wallets, and often with a distinguished *reference wallet* that holds the book's positions and cash. This grouping is a reporting convention: every definition and smart contract operates at the wallet level. The managed-account contracts (Section 10) act on reference wallets whether classified real or virtual.

2.7 The Self-Consistency Principle

The move stream is the one record the system keeps. Every balance, valuation, and report is that same stream read through a different lens, never a second copy stored alongside it.

Principle 2.9 (Self-Consistency). *The ledger’s conservation law, atomic transactions, and canonical event log make internally inconsistent states structurally unreachable, not merely detectable after the fact. Internal reconciliation—comparing two independent records of the same reality within the firm—is a symptom of a multi-source-of-truth architecture. In a self-consistent system the balance sheet is the ledger viewed through an accounting lens; within the ledger there is no second record to reconcile against. External systems—custodians, CSDs, counterparties, regulators—maintain independent records, and reconciliation at the boundary remains necessary.*

Three of the most damaging reconciliation failures are thereby unreachable within scope:

- **Trading system / general ledger divergence.** The trade and its accounting entry are one object: a transaction in the move stream. There is no second system to diverge from.
- **Profit and loss (PnL) / balance sheet mismatch.** $PnL = V_{t_1} - V_{t_0}$ is a theorem (Section 5); both values derive from the same position set and price vector. There is no separate PnL system.
- **Inter-entity breaks.** An inter-entity move is a single atomic event recorded once; within a shared ledger, disagreement is impossible.

The guarantee is scoped. During any migration in which legacy systems run in parallel, it covers only activity recorded in the ledger. Activity remaining in legacy systems retains the legacy failure modes.

The remaining failure modes are boundary conditions where the external world is an independent actor: custodian breaks, nostro/vostro discrepancies, counterparty disputes. The architecture cannot prevent an external custodian from disagreeing. It makes disagreement detectable (virtual-wallet comparison) and narrowly scoped (ISO 20022 feedback at the boundary).

Settlement timing. The atomic transaction records the *economic intent* of simultaneous settlement. Legs settle asynchronously in practice. Foreign-exchange (FX) currency legs settle in different time zones. CLS provides payment-versus-payment within a window, not instantaneous atomicity. The framework records the trade as an atomic transaction at trade time and tracks per-leg settlement progress as unit state (Section 4) or move metadata. Between trade and settlement the residual is an unsettled receivable in the counterparty’s virtual wallet. The receivable represents Herstatt risk; recording it does not eliminate it. The settlement interface that consumes this state is Section 13.

3 The Unit Store

Section 2 fixed the unit universe $\mathcal{U}$ and the balance function $w_t : \mathcal{U} \rightarrow \mathbb{R}$ but left $\mathcal{U}$ abstract. The **Unit Store** makes $\mathcal{U}$ concrete. It decides which units exist, records their identity and static terms, binds each unit to a smart contract, and validates every unit at registration. The executor, the settlement projection, and the valuation function all read $\mathcal{U}$ from here.

3.1 Units

A unit is any financial instrument that can appear as a wallet balance: cash currencies, listed equities and bonds (by ISIN), listed derivatives (by exchange contract specification), OTC derivatives (each bilateral trade), and structured products (each issuance). A unit has *identity* (how it is distinguished from every other unit), *type*, static *terms*, and a bound *smart contract*. A unit's evolving state is not stored on the unit entry. Its lifecycle stage and product-specific state live in the three-home state model (Section 4).

3.2 Unit Identity: OTC versus Listed

Two positions are in the same unit exactly when they are fungible.

Principle 3.1 (Unit Identity). *Two positions are in the same unit if and only if they are fungible: a holder is economically indifferent between them. Fungible positions share one unit entry and net against each other; non-fungible positions are distinct units with distinct entries.*

Fungibility fixes a different identity key for each instrument class.

Instrument class	Identity key	Fungibility
Cash	Currency code	All balances of a currency net
Listed equity	ISIN	All shares of an ISIN net
Listed derivative	Contract spec (exchange, underlier, type, strike, expiry)	All positions in a series net (via CCP novation)
OTC derivative	Full CDM Trade (includes counterparty, Collateral)	Non-fungible: counterparty and CSA make each trade unique
Bond	ISIN	All holdings of an ISIN net
Structured note	ISIN + product terms	Same issuance is fungible
Tokenised equity	Contract address + chain ID	All tokens on a contract net

For OTC instruments the identity is the CDM Trade object *including* its Collateral field. Two trades with identical payoffs under different CSAs (USD versus EUR) are therefore different units. They carry different discount curves, margin flows, and settlement currencies. For listed instruments the CCP novates itself in as sole counterparty, so all positions in one contract specification are fungible regardless of original counterparty. A tokenised security is a unit distinct from its underlying: NVDA and tokenised-NVDA carry the same exposure under different settlement mechanisms. The double-counting this raises is resolved in Appendix F: the custodian nets to zero economic exposure.

3.3 Three-Tier Architecture

The store has three tiers, each answering one question.

Tier 1 — Reference Data. *What instruments exist in the market?* Instrument master data: ISIN, exchange, contract specification, issuer, coupon schedule. It is read from exchanges, vendors, and CSD records. The Ledger consumes reference data; it does not author it. CDM: TransferableProduct for securities, NonTransferableProduct for derivatives and structured products.

Tier 2 — Product Registry. *What lifecycle logic governs this kind of instrument?*

One smart-contract template per product *type*, not per contract. “European equity index option” maps to one `OptionPayout` template, and that template covers every such option regardless of underlier, strike, or expiry. CDM `ProductQualification` classifies `EconomicTerms` into a product type; each type maps to a template (Section 6). An entry is created automatically the first time a Tier-3 registration encounters a new product type. Thereafter the entry is immutable.

Tier 3 — Unit Registry. *Which specific units can appear as balances?* The elements of $\mathcal{U}$. For listed instruments the unit is the contract specification; for OTC instruments it is the full CDM `Trade`, `Collateral` included. Each entry carries identity, type, the static terms drawn from `EconomicTerms`, the bindings to its Tier-2 template and smart contract, and provenance:

```
UnitEntry:
  unit_id      : UniqueId      -- deterministic, from the CDM object
  unit_type    : UnitType      -- CASH | EQUITY | LISTED_DERIV
                                -- | OTC_DERIV | BOND | STRUCTURED
  -- Identity (varies by type)
  isin         : Option<ISIN>   -- securities, bonds, notes
  contract_spec : Option<ContractSpec> -- listed derivatives
  cdm_trade_ref : Option<TradeRef> -- OTC (includes Collateral)
  -- Bindings
  product_ref   : ProductRegistryRef -- Tier 2 template
  smart_contract : SmartContractRef  -- lifecycle logic instance
  -- Static terms (from CDM EconomicTerms)
  currency      : CurrencyCode
  multiplier    : Option<Decimal>
  expiry        : Option<Date>
  -- Provenance
  created_by    : TransactionId    -- registering transaction
  created_at    : Timestamp
```

The `unit_id` is derived deterministically and injectively from the CDM object: a hash of the contract-specification fields for listed instruments, the UTI or metadata key for OTC. Distinct CDM objects always yield distinct unit IDs.

Where evolving state lives. The unit entry holds identity and static terms only. The lifecycle stage and the product-specific state (which coupons a bond has paid, the accumulated cost of a futures position) are not entry fields. A unit carries immutable, versioned terms in `ProductTerms[u]` and a shared lifecycle status in `UnitStatus[u]`. Per-position state lives in `PositionState[w, u]`. These are the three-home state model (Section 4). The lifecycle stage is read from `UnitStatus`.

`UnitStatus` is a projection, not a mutable store. It is one shared value per unit: a read cache of the immutable event log, rebuilt exactly by replay (established in §4.3).

3.4 Registration

Units enter the registry through three channels.

1. **Cash (pre-registered).** Currency units are permanent entries created at system initialisation: no expiry, no lifecycle events, stage `ACTIVE` permanently, smart contract the identity function.
2. **Listed instruments (reference-data feed).** On a new listing or initialisation against an existing market, the feed supplies the contract specification; the store creates the Tier-

1, Tier-2 (if the product type is new), and Tier-3 entries. This is administrative, not a trade: the unit enters $\mathcal{U}$ upon listing, before any position is taken.

3. **OTC instruments (trade execution).** The executed CDM Trade *is* the unit definition; the store creates the Tier-3 entry from it. The unit ID derives from the Trade’s metadata key (counterparty and Collateral included), so identical payoffs under different CSAs receive different IDs. The unit enters $\mathcal{U}$ at execution.

Registration is atomic: a unit is fully validated and added, or rejected with no partial state. Units are never deleted. A unit at end of life reaches a terminal stage in *UnitStatus* (matured, terminated, settled), and its entry remains for audit and time travel. The executor rejects new moves against a non-ACTIVE unit, except to settle existing obligations.

Two disciplines bind registration: it writes the first value of every per-unit map, and it happens once.

Condition (C7: ProductTerms registration-total). *ProductTerms is registration-total: its first version is written at registration, so $\text{product_terms}(u)$ is defined for every registered u . (UnitStatus is registration-total by the same write, C5, established in Section 4.)*

Condition (C10: no re-registration). *Re-registering an existing unit id is a hard error, never a silent reset of its state.*

Registration writes *ProductTerms* and *UnitStatus* together, so “registered in ProductTerms $\iff$ registered in UnitStatus” holds by construction. C10 makes the write once-only, so a re-registration cannot overwrite live state behind the log.

The registering function writes the *singleton* first version of *ProductTerms* and the default *UnitStatus* together, in one operation. It returns the updated ledger, or rejects an already-registered id with a **Left** (C10).

ProductTerms is a **NonEmpty** list of versions, so a registered-but-versionless unit is unrepresentable, and the in-force-version accessor is total: it needs no **Maybe** for emptiness (C7).

register is exported; *ProductTerms* and **Ledger** are exported abstract, with no field setter. That single write is therefore the only origin of a unit’s terms, and no caller can falsify the *ProductTerms–UnitStatus* coupling. In code, registration is a move-less transaction applied through the same function as trades, settlements, and amendments:

```
-- ProductTerms abstract: NonEmpty makes "registered but versionless"
-- unrepresentable; currentTerms is then total (no Maybe).      -- C7
newtype ProductTerms = ProductTerms (NonEmpty TermsVersion)

data LedgerError = ReRegistration UnitId | UnknownUnit UnitId    -- C10; the two
    registration errors: the full LedgerError is in Sec. 4

-- Registration is ONE move-less Transaction whose txIntroduce carries
-- the unit's first terms version and its default status.
registerTx :: UnitId -> TermsVersion -> UnitStatus -> Transaction
registerTx u tv us = Transaction
    { txUnit = u, txMoves = [], txRows = Map.empty, txStatus = []
    , txIntroduce = Just (tv, us), txAppend = Nothing, txBoundary = Nothing }

-- register is not a separate door: it is applyTx of that transaction, so
-- registration, trade, settlement, and amendment share one apply. The
-- introduce branch of applyTx writes the singleton first ProductTerms
-- version (C7) and the default UnitStatus (C5) together, and rejects an
-- already-registered id with Left (ReRegistration u) (C10). [Section 4]
register :: UnitId -> TermsVersion -> UnitStatus
    -> Ledger -> Either LedgerError Ledger
register u tv us = applyTx (registerTx u tv us)
```

A first registration succeeds. The same id a second time is rejected, and the prior ledger and its live state are left untouched. The rejection is a hard error, never a silent reset:

```
register uES tvES (defaultStatus uES) emptyLedger == Right l0
register uES tvDup (defaultStatus uES) l0          == Left (ReRegistration uES)
```

Conservation does not arise at registration. A registration is a move-less transaction, so its edge-sum is `mempty`, which is vacuously conserved, and the transaction touches no *PositionState* row. A non-vacuous balance, and the conservation that constrains it, appears only when the first position is taken (Section 4).

3.5 Two-Stage Validation

Validity is enforced at registration and at transaction time.

At registration, a unit must satisfy:

1. **Uniqueness**: no existing unit shares its `unit_id` (C10).
2. **Smart-contract binding**: a contract reference exists for its product type.
3. **CDM qualification**: for CDM-native units, `ProductQualification` returns a valid classification for the stated `EconomicTerms`; an unclassifiable product is rejected.
4. **Term consistency**: static terms cohere. Expiry is in the future at creation, the multiplier is positive, the currency code is valid.

At transaction time, the executor checks that every unit a move references (i) exists in the store, (ii) is in a stage compatible with the operation (read from *UnitStatus*), and (iii) has a smart contract invocable for the requested event.

Invalid units cannot enter $\mathcal{U}$, and moves cannot reference non-existent or retired units. With the conservation law (Section 2) and the executor’s purity (Section 7), two-stage validation completes the correctness chain. The ledger operates only on validated, contract-governed units.

3.6 Guarantees

The Unit Store provides four guarantees to every downstream consumer.

1. **Existence**: every unit a move references exists and is in a valid lifecycle stage.
2. **Immutability of identity and terms**: a `unit_id` always denotes the same instrument with the same static terms. Lifecycle stage and product-specific state evolve (Section 4); identity and terms do not.
3. **Smart-contract availability**: every registered unit has a bound contract the executor can invoke for lifecycle events.
4. **CDM alignment**: every CDM-native unit maps to a CDM object (`Trade`, `NonTransferableProduct`, or `TransferableProduct`) with a successful qualification.

These guarantees make referential integrity (P3, Section 17) hold by construction: $m.unit \in \mathcal{U}$ because the executor consults the store before accepting any move.

3.7 CDM Alignment

Each tier maps to a CDM concept.

Tier	Content	CDM equivalent
1 Reference Data	Instrument master data	<code>TransferableProduct</code> (securities), <code>NonTransferableProduct</code> (derivatives)
2 Product Registry	Smart-contract templates	<code>ProductQualification</code> + <code>EconomicTerms</code>
3 Unit Registry	Position-bearing units ($\mathcal{U}$)	<code>Trade</code> (OTC), contract spec (listed)

CDM has no native catalogue concept. It models individual trades, not registries. It serves both listed and OTC derivatives through one `NonTransferableProduct` type, with no dedicated listed-contract-specification type and no model for exchange reference-data feeds (listings, expiry calendars, lot sizes). These are documentation gaps, not design incompatibilities. The store uses CDM objects as its internal representation; catalogue management sits alongside CDM as a Ledger-specific concern.

Key Invariants and Consequences

The foundations rest on one primitive: the atomic move of a quantity between two wallets. From that primitive the following hold by construction. Each is catalogued, with its executable oracle, in §17.

- **Conservation (P1).** $Q(u) = \sum_w w_t(u) = 0$ for every unit u and time t . Each move debits and credits equal magnitude, so the signed per-unit sum of any transaction is the group identity. A conservation failure is an implementation bug, never a financial event. (§2, Theorem 2.7.)
- **Atomic commitment (P2).** A transaction applies completely or not at all. No partial state is observable, so a lifecycle event commits as one indivisible unit. (§2.)
- **Log monotonicity (P4).** The event stream is append-only and hash-chained; no entry is retroactively rewritten. (§3.)
- **Idempotency (P5, P6).** Re-applying a transaction by `tx_id`, or a lifecycle event to the same unit state, has no incremental effect. (§3.)
- **Determinism and time travel (P8, P9).** Replay of the immutable stream is a pure, total fold, and lifecycle functions are pure. `clone_at(t)` reconstructs balances and unit state identical to the ledger at time t . The ledger is a deterministic function of its event history. (§3.)
- **Consequence—unreachable internal breaks.** Trading-system/general-ledger divergence, PnL/balance-sheet mismatch, and inter-entity breaks are disagreements between projections of one record. For activity inside the boundary they are not representable. (§1.)

Part II

The Three-Home State Model and UnitStatus

4 Where Unit State Lives: The Three-Home State Model

Three kinds of fact describe a unit, and each is kept in its own home. The first is what the contract is: its fixed terms, written once at registration and thereafter only added to, never rewritten. The second is where the contract stands in its life: its lifecycle stage and last settlement price, a single shared value read the same way by everyone who holds it. The third is what a given holder holds: that holder's own quantity and cost, which differ from one holder to the next. These three facts are the three homes named below.

Unit state lives in three homes, one map each: immutable `ProductTerms[u]` keyed by unit; shared `UnitStatus[u]` keyed by unit; and per-position `PositionState[w, u]` keyed by (wallet, unit). There is no map keyed by wallet alone: every per-wallet economic fact is keyed by a mandate or strategy unit, so it lives in `PositionState[w, uMA]`. Each economic fact has exactly one home and one writer, so internal reconciliation failure cannot arise. Conservation and deterministic replay then hold by construction: the single-writer discipline is sealed, and replay is pure and total.

This placement is the minimum basis of the problem (§4.2) and unique under any correctness gate (§4.7).

A *unit* is anything held and conserved (§2): an instrument, and also a mandate or a strategy contract (§4.2). The framework primitives are those of §2: wallet, unit, holding $h(w, u)$, move, transaction, and conservation $\sum_w h(w, u) = 0$.

4.1 The placement rule and the 2×2

One rule governs every placement: each economic fact has exactly one home and one writer. A fact violates it in exactly two ways. *Keyed wrong*: one home collapses two facts, or one fact duplicates across homes that drift. *Authored wrong*: two authorities keep one fact, and their records drift apart. Two questions forestall the two failures. Because there are only two failures, there is no third question.

Does the fact depend on the holder, or only on the unit? The scope is one unit’s state, so a unit and a holder of it are the only terms in view. Every economic fact — one entering conservation, valuation, or profit and loss — is then a (holder, unit) fact or a unit fact. What stays keyed by the wallet alone (KYC, permissions, an audit cursor) enters none of these three reckonings. It is identity, not economic state. It lives in a *WalletRegistry* that carries no quantity and joins no conservation law.

Is the fact’s record owned by an outside authority or by the ledger? The criterion is who owns the record’s history, not who sources the number. An externally authored fact is one an outside authority — the exchange, the contract, the reference-data provider — owns and restates; the ledger preserves it version by version. A ledger-authored fact is one the ledger’s own events produce and overwrite. One provider can supply both: the benchmark level, which the ledger records and overwrites, is ledger-authored; the benchmark identity, which the provider restates, is the provider’s own versioned record and externally authored. The two sort apart though both come from the same provider.

The two questions give a 2×2 ; three of its four cells are occupied, and each occupied cell is one home.

	ledger-authored	externally authored
per unit	Status: lifecycle stage, last settlement price, current weights, benchmark level	Terms: multiplier, currency, expiry, strike, ISIN, fee schedule, mandate text, benchmark identity
per (holder, unit)	Position: held quantity, accumulated cost, high-water mark, entry NAV	— <i>empty</i> —

Terms and status share the unit key but cannot be one value, since their authorities of record differ (§4.2). They ride together on the unit key as a co-present pair: two homes whose mutation disciplines differ, each kept in its own map (§4.4).

4.2 Why three homes, and why the fourth cell is empty

Each occupied cell exists for one reason, and the empty cell is empty for one. Exactly three maps are forced, no fewer and no more.

Position is keyed by (holder, unit), because two holders of one contract hold different state. A buyer and a seller hold the same future. The buyer’s quantity is +1000, the seller’s is −1000, and their accumulated costs differ. Substituting one holder’s position for the other’s changes the value, so any unit-only keying identifies the two and loses the difference. The holder is in the key.

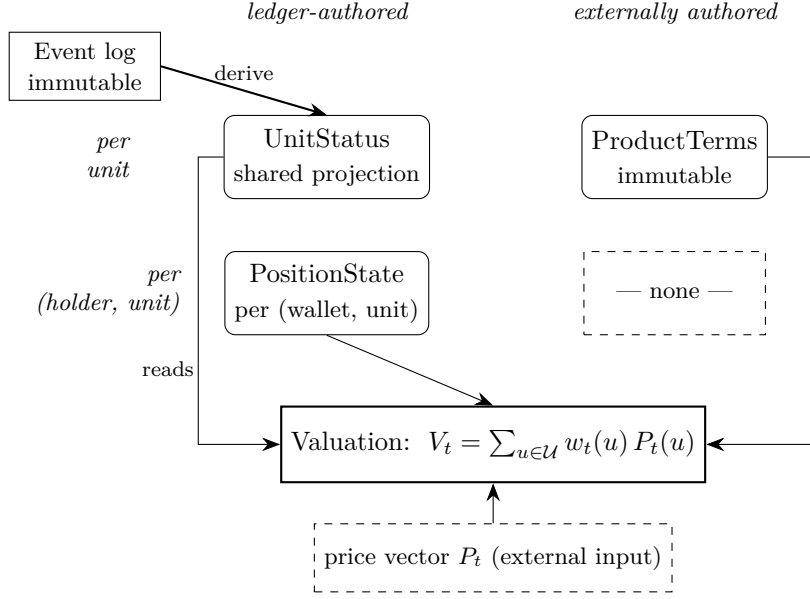


Figure 4: The three state homes and their dataflow. Each economic fact has one home, fixed by the placement above: **UnitStatus** (per unit, ledger-authored), **ProductTerms** (per unit, externally authored), and **PositionState** (per (wallet, unit)); the fourth cell—an externally authored per-position fact—is empty by construction (§4). **UnitStatus** is not an authoritative store but a projection derived from the immutable event log, shown by the incoming arrow. Valuation reads all three homes together with an externally supplied price vector P_t .

Status is keyed by the unit alone, because one value is read identically by every holder. A future settles at one price on the day; an index stands at one level. Stored per holder, the number is copied across thousands of rows free to drift — a reconciliation break by construction. Keyed by the unit, it is one value with one writer.

Terms are a home distinct from status, because the two have different authorities of record. Terms are externally authored; status is ledger-authored. Co-mingling the authority’s record with the ledger’s own is the single-source-of-truth violation the system exists to prevent. The mutation disciplines differ too. Terms are append-only and versioned for reconstruction. The status cache is overwritten on every settlement, while the settling event is appended immutably to the log (§4.3). Merging the two would put two mutation disciplines under one name.

The fourth cell is empty, because no authority issues a fact about one holder’s position. A position is the ledger’s own from origin: the held quantity is folded from the move stream, and the high-water mark and entry NAV are written by ledger events. A custodian statement or prime-broker position report is a reconciliation input the ledger checks against, not a record it adopts as an authority’s own. So the (holder, unit) key carries only ledger-authored facts, and the externally authored cell is empty by construction.

The W -sector collapses (C12). The managed account, whose state is the client’s rather than the instrument’s, appears to demand a wallet-keyed economic-state map — call it the W -sector, $\text{Map}[\text{WalletId}, \text{WalletState}]$. No such map exists, because the mandate is a unit. The manager issues a mandate unit u_{MA} to the client (§10), and it conserves like any issued unit:

$$h(w_{\text{manager}}, u_{\text{MA}}) = -1, \quad h(w_{\text{client}}, u_{\text{MA}}) = +1, \quad \sum_w h(w, u_{\text{MA}}) = 0.$$

Every field that looked per-wallet is then a fact about the position $(w_{\text{client}}, u_{\text{MA}})$ and lives in *PositionState*: high-water-mark value, accrued management and performance fee, mandate

breach flags, benchmark NAV at this wallet’s inception. Two mandates on one wallet are two rows, each with its own high-water mark and fees; a wallet-keyed scalar would have merged them. This is why C12 keys on the mandate unit, not the wallet.

A relationship spanning several instruments that does *not* reduce to a single (holder, unit) row would be a (holder, several-units) fact. Such a fact would require a fourth home under a new key shape; the three-map count excludes it, and it lies outside this scope.

4.3 The UnitStatus discipline, stated once

MENTAL MODEL. Picture a bound ledger-book written only in pen: each event is a line entered in ink, and no line is ever erased. On the cover sits a running total in pencil, rubbed out and rewritten after every new line. Rubbed out entirely, the pencil costs nothing: re-adding the ink lines reproduces the exact total that stood at any chosen line. Unlike a chequebook, where only the balance need be kept, here the pencilled total is the disposable part and the ink lines never are — the total answers at a glance but certifies nothing on its own; the ink is the record.

UnitStatus is a read cache. It holds one value per unit, shared: every holder reads it identically, and that value changes over time. The source of truth is the immutable event log, not UnitStatus (justified by the idempotent replay of §7): every change to the stored value is caused by a logged event and overwrites it in place, and replaying the events up to any point rebuilds the exact value that held then. The cache can therefore be discarded and rebuilt from the log without loss. There is no other way to change the value, which exists from registration onward.

Equivalently, the stored value always equals what replaying the unit’s events produces, and no part of the system writes it except through a logged event. This is the catamorphism property: the fold of the event stream always rebuilds the exact cached value. In the reference (§4.9) it holds by construction. Every *UnitStatus* field is written only by `applyStatus`, folded over the unit’s status writes inside the sole transition `applyTx`; `register` is `applyTx` of a move-less introduction. The `Ledger` constructor is unexported, so no out-of-band `setStatus` exists. Externally sourced inputs — a settlement price, the current benchmark level — enter only as a logged observation event, so the fold stays pure at the boundary. Every later reference to cached unit state in this specification is read in this sense.

4.4 The construction

The homes are built from the smallest pieces, each forced by the one before. The conditions of the register (§4.5) are introduced where the construction forces them: C1–C12 here, C13 and C14 at the data plane’s basis edge (§8). Quantities are exact integer minor units, never floating point, so determinism and conservation are arithmetic facts.

Scalars and keys. The smallest pieces are the number types and the keys that name them. A balance is a quantity with a zero, associative addition, and a negation — an abelian group. The negation forces the two legs of a transfer to cancel. The zero makes the sum over no wallets conserved with no special case, which discharges the vacuous base of conservation (C9). A price is a number but not a quantity: prices are never summed, so a price is a separate type with neither identity nor inverse. A balance is named by two keys, wallet and unit, the holder in the key.

```
{-# LANGUAGE GADTs, DataKinds, KindSignatures, StandaloneDeriving #-}
import           Control.Monad          (foldM)
import           Data.List              (foldl')
import           Data.List.NonEmpty     (NonEmpty (..))
import qualified Data.List.NonEmpty as NE
import           Data.Map.Strict        (Map)
```

```

import qualified Data.Map.Strict as Map
import           Data.Set       (Set)
import qualified Data.Set       as Set

newtype Qty = Qty Integer deriving (Eq, Ord)           -- exact minor units, never
               Float
instance Show Qty      where show (Qty n) = show n
instance Semigroup Qty where Qty a <> Qty b = Qty (a + b)
instance Monoid Qty    where mempty = Qty 0           -- the zero discharges the
               vacuous base (C9)

qneg :: Qty -> Qty                                     -- group inverse: a transfer'
      s legs cancel
qneg (Qty n) = Qty (negate n)
qmax :: Qty -> Qty -> Qty
qmax (Qty a) (Qty b) = Qty (max a b)

newtype WalletId = WalletId String deriving (Eq, Ord, Show)
newtype UnitId   = UnitId   String deriving (Eq, Ord, Show)

newtype Price = Price Integer deriving (Eq, Ord, Show) -- minor units per unit;
               NO Monoid

-- The corporate-action basis coordinate (sec:state-basis). BoundaryId is exported
-- ABSTRACT: the address of a Boundary -- a corporate-action boundary -- is
               assigned
-- by content at the attested notice gateway, never minted by a caller. A BasisId
               is
-- a content address, never an ordinal, so a retro-effective boundary rennumbers
-- no stored value.
newtype BoundaryId = BoundaryId String deriving (Eq, Ord, Show)
data   BasisId     = Origin UnitId | Boundary BoundaryId deriving (Eq, Ord, Show)

```

ProductTerms — the authority’s record, grown only by appending (C6, C7). The picture is a stack of versions that only grows: the first is laid down at registration, later ones are laid on top, and nothing beneath is ever touched. “One or more versions, append-only” is a non-empty list, so a registered unit always has a current version and “registered but versionless” is not representable. The type is abstract: its constructor is unexported, so no importer can lay down a fresh one-version value and discard history. The only growth operations are **register**, which writes version one, and **appendVersion**, which appends. No setter exists, so the append-only discipline holds by absence, not by convention (C6). The first version is written at registration, so terms are total on the registered units (C7).

```

data TermsVersion = TermsVersion
  { tvLabel :: String, tvFields :: Map String String } -- opaque payload:
    multiplier, ISIN, ...
  deriving (Eq, Show)

newtype ProductTerms = ProductTerms (NonEmpty TermsVersion) deriving (Show)
-- exported ABSTRACT: no data constructor, no setter -> append-only holds by
  absence (C6)

currentTerms :: ProductTerms -> TermsVersion           -- total: NonEmpty always
               has a last
currentTerms (ProductTerms vs) = NE.last vs
allVersions  :: ProductTerms -> NonEmpty TermsVersion
allVersions  (ProductTerms vs) = vs

```

```

appendVersion :: TermsVersion -> ProductTerms -> ProductTerms    -- C6: append,
    never rewrite
appendVersion tv (ProductTerms vs) = ProductTerms (vs <> (tv :| []))

productTerms :: Ledger -> UnitId -> Maybe ProductTerms    -- Nothing = unregistered
productTerms l u = Map.lookup u (ledgerPT l)

```

UnitStatus — the shared read cache (C5). The picture is one shared record per unit, overwritten in place, with the log keeping the event behind every overwrite. Status records the unit’s lifecycle stage, its corporate-action basis (state basis, §8), its last settle mark, and any successor, under the discipline of §4.3. The settle mark is dimensionally a *Price*, paired with the basis it was struck at. The lifecycle is a closed sum, *Registered*→*Active*→*Expired*→*Closed*. *Registered* is the at-registration stage: a unit recorded in and known to the ledger before any settlement. An OTC option, a bespoke mandate, or an internal strategy reaches it exactly as an exchange instrument does. Every *UnitStatus* field changes only through a *StatusWrite*, the closed writer set whose four constructors each name one field. Each write is applied by the single total writer *applyStatus*, the projection step of §4.3. No other code overwrites a status field, so each write is last-write-wins and idempotent (P6). Product-declared defaults are written at Unit Store registration (§3), so status is total on the registered units (C5).

```

data Lifecycle = Registered | Active | Expired | Closed deriving (Eq, Show)

data UnitStatus = UnitStatus
  { usLifecycle    :: Lifecycle
  , usBasis        :: BasisId          -- corporate-action basis; tip-
    welded (sec:state-basis)
  , usLastSettle   :: Maybe (Price, BasisId) -- a settle mark names its basis;
    None at default (C5)
  , usSupersededBy :: Maybe UnitId      -- set only by a Breaking amendment
    (C8)
  } deriving (Eq, Show)

defaultStatus :: UnitId -> UnitStatus -- C5: the per-unit default written at
    registration;
defaultStatus u = UnitStatus Registered (Origin u) Nothing Nothing
    -- the basis origin is forced by the unit
    itself

data StatusWrite -- the closed status-writer set; each names
    ONE field
  = SetLifecycle    Lifecycle -- a lifecycle event
  | SetBasis        BasisId   -- a corporate-action boundary crossed (sec:
    state-basis)
  | SetLastSettle   (Price, BasisId) -- a settle event (a logged observation)
  | SetSupersededBy UnitId     -- a Breaking amendment (C8)
  deriving (Eq, Show)

applyStatus :: StatusWrite -> UnitStatus -> UnitStatus -- the ONLY writer of a
    status field;
applyStatus (SetLifecycle l) s = s { usLifecycle = l } -- total; every
    equation an
applyStatus (SetBasis b) s = s { usBasis = b } -- ABSOLUTE
    replacement write,
applyStatus (SetLastSettle m) s = s { usLastSettle = Just m } -- never an
    increment:

```

```

applyStatus (SetSupersededBy u) s = s { usSupersededBy = Just u } -- last-write-
wins, idempotent (P6)

unitStatus :: Ledger -> UnitId -> Maybe UnitStatus      -- Nothing = unregistered
unitStatus l u = Map.lookup u (ledgerUS l)

```

PositionState — the per-position home (C1). The picture is one row per (holder, unit): the holder’s own quantity, and beside it the holder-specific facts that ride with it. A position carries more than a balance. Its primary field is the held quantity, which conserves. Beside it ride non-conserved per-(holder, unit) facts — a high-water mark, an entry NAV — which no zero-sum law binds. A first touch starts a position at 0_P , the all-zero state. Two orthogonal disciplines govern the home (C1). (a) The accessor returns Option, so None (never held) and Some(0_P) (held, now flat) stay distinct; variation-margin settlement, wash-sale lookback, and record-date entitlement each act on the difference. (b) The carrier is monotone: a row, once created, is never removed, and close-out leaves it flat. The two are independent: (a) constrains the return type, (b) the domain. The monotone carrier keeps the key set stable across any checkpoint cut, which is what makes replay a literal fold (P8). The high-water-mark and entry-NAV writers are valuation events specified in the valuation satellite (§5); they are out of scope here and leave those fields at their defaults throughout.

```

data PositionState = PositionState
{ psAc      :: !Qty          -- conserved, additive      <- Settle / Trade
, psBalance :: !Qty          -- conserved, additive      <- Transfer
, psHwm     :: !Qty          -- NOT conserved, monotone  <- FeeCrystallise (
  qmax)
, psEntryNav :: !(Maybe Qty) -- NOT conserved, write-once <- Subscribe
} deriving (Eq, Show)

zeroP :: PositionState      -- flat row on first touch; also the held-and-
flat baseline
zeroP = PositionState mempty mempty mempty Nothing

position :: Ledger -> WalletId -> UnitId -> Maybe PositionState -- C1 Option
accessor:
position l w u = Map.lookup (w, u) (ledgerPS l)
-- Nothing = never held; Just zeroP = held once, now flat. The two stay
distinct.
-- Monotone carrier (C1, storage half): applyTx only inserts/updates a row, and
no
-- row deleter is exported anywhere -- close-out leaves a flat row, never
removes it.

```

The three homes, three maps — the sealed Ledger. The picture is the three maps travelling as one sealed value, built and read only through the operations that follow. Terms and status share the unit key but keep separate maps — `ledgerPT` append-only, `ledgerUS` overwrite-in-place — because their mutation disciplines differ. Their co-presence, “ u registered in terms $\iff u$ registered in status,” is preserved by construction, not policed: `register` is the sole introducer and writes both, and every later writer touches them only for an already-registered unit. The position map is the balance map enriched. The `Ledger` constructor and its field selectors are unexported: a `Ledger` is built only by the writers below and read only through the accessors. Were a selector exported, record update through it would install a non-conserving map without naming the constructor, bypassing the single-writer discipline. Coherence is carried by the single introducer; the seal keeps conservation true by construction. A fourth field, `ledgerBounds`, is not a home. It caches the committed boundary projection read

by the basis welds — the basis admission conditions `applyTx` checks (§8). Like `ledgerUS`, it is rebuilt by replay and is never a source of truth.

```
data Ledger = Ledger
  { ledgerPT :: Map UnitId          ProductTerms    -- append-only terms, by
    unit
  , ledgerUS :: Map UnitId          UnitStatus       -- overwrite status, by
    unit
  , ledgerPS :: Map (WalletId, UnitId) PositionState -- positions, by (holder,
    unit)
  , ledgerBounds :: Map UnitId (Set (Timestamp, Integer, BoundaryId))
    -- committed-boundary projection: each unit's (t_eff, prec, bid) triples,
    whose
    -- derived Ord IS the effective order. A DERIVABLE CACHE of the log, exactly
    -- like ledgerUS -- rebuilt by replay, never a source of truth.
  } deriving (Show)
-- exported ABSTRACT: constructor and selectors unexported. A Ledger is built
-- only by the
-- writers below and read only through the accessors -> single-writer discipline
-- ; with no
-- row deleter, the carrier is monotone (C1).

emptyLedger :: Ledger
emptyLedger = Ledger Map.empty Map.empty Map.empty Map.empty

data LedgerError = ReRegistration UnitId | UnitAlreadyExists UnitId | UnknownUnit
  UnitId
  | DanglingSupersede UnitId    -- SetSupersededBy target is not a
    registered unit
  | BasisNotTip UnitId         -- the three basis welds at the
    same single
  | InvarianceViolation UnitId -- door, specified in sec:state-
    basis and
  | ForeignOrigin UnitId       -- implemented in reference/
    Ledger.hs
  | ScheduleIncomplete UnitId  -- C14: the written version's
    schedule fails
  )
  | CauseNotCommitted UnitId    -- txCause names no strictly-
    earlier
  )
  -- committed boundary id

deriving (Eq, Show)
```

The Transaction — the one atomic event (C2, C3). The picture is one value carrying everything one event does to one unit; it lands whole or not at all. One event records, for a single unit, the whole change across the three maps, in two complementary parts. First, the conserved economic flow in *edge* form: a list of moves, each debiting one wallet and crediting another by the same magnitude. The signed per-unit sum of any move list is therefore the group identity *by construction*: $\text{qneg } q \diamond q = \text{mempty}$ for each leg, summed by the monoid laws. The empty list (a move-less registration or amendment) is included; it is the vacuous base (C9). Second, the non-balance state delta that rides alongside the moves: per-wallet bookkeeping (accumulated cost, high-water mark, entry NAV, via the C11 `FieldWrite` table), the shared `UnitStatus` writes, and a `ProductTerms` introduction or append.

Conservation is then a property of the type, not a checked side-condition. There is no unconserved `Transaction` to reject, hence no `validate` gate (C2). A transaction applies in full

or not at all; a partially applied transaction is not a representable state (C3). One structural field, `txIntroduce`, is the sole asymmetry between registration and every other event: `Just` introduces the unit, writing terms and status together; `Nothing` operates on an existing one. Registration is thus the move-less instance of this one type, not a separate operation.

An event touching several units — a rebalance trading three futures, or a future bought against cash — is a transaction per unit applied as a single fold. Per-unit conservation composes to event-level conservation. Induction over the stream lifts the per-event identity to the global invariant P1.

```
-- One value, two notations: txMoves is the per-edge view of the conserved flow;
-- the
-- remaining fields are the non-balance bookkeeping that rides with it.
-- (Move and moveDelta are the sec:ledger primitives in their reference form: the
-- magnitude mQty is a PosQty -- positive by type, sole door mkPosQty, the parse
-- boundary of sec:futures -- read back through unPosQty. sec:ledger presents the
-- same algebra in illustrative Qty form.)
data Transaction = Transaction
  { txUnit      :: UnitId          -- the unit this event
    concerns
  , txMoves     :: [Move]          -- conserved flow as edges (by
    construction)
  , txRows      :: Map WalletId [SomeWrite] -- non-balance per-wallet
    bookkeeping
  , txStatus    :: [StatusWrite]   -- shared UnitStatus writes
  , txIntroduce :: Maybe (TermsVersion, UnitStatus) -- Just => introduce txUnit (
    registration)
  , txAppend    :: Maybe TermsVersion -- Just => append a version (
    C6, Preserving)
  , txBoundary  :: Maybe BoundaryEvent -- Just => declare a corporate
    -action boundary
    prec, declaration
    sec:state-basis)
  , txCause     :: Maybe BoundaryId -- PROVENANCE ONLY: the parent
    notice's
    on or cascade;
    boundary tx (its
    txBoundary -- one fact,
    requires the referent
    earlier (refusal
    self-reference is
    only by the
    P27/P28) and by
    branches on it.
  } deriving (Show)

-- Conservation witnesses. Each move contributes qneg q <> q = mempty, so the
-- foldMap lands
```

```

-- at mempty BY THE MONOID LAWS -- not by a sum computed and compared at run time
   (C2, C9).
unitDelta :: UnitId -> Transaction -> Qty
unitDelta u tx =
  foldMap (\m -> if mUnit m == u then let n = unPosQty (mQty m) in qneg n <> n
    else mempty) (txMoves tx)
-- law: unitDelta u tau == mempty for every unit u and transaction tau

netDelta :: Transaction -> Qty
netDelta tx = foldMap (\m -> let n = unPosQty (mQty m) in qneg n <> n) (txMoves tx)
-- law: netDelta tau == mempty for every tau (the empty/registration case too)

-- The canonical move-less instances. Trade/settlement/transfer use the record
   constructor
-- directly (txIntroduce = Nothing, moves + rows present).
registerTx :: UnitId -> TermsVersion -> UnitStatus -> Transaction -- C5+C7+C10:
   introduction
registerTx u tv us = Transaction
  { txUnit = u, txMoves = [], txRows = Map.empty
    , txStatus = [], txIntroduce = Just (tv, us), txAppend = Nothing
    , txBoundary = Nothing, txCause = Nothing }

appendTx :: UnitId -> TermsVersion -> Transaction -- C6
   Preserving amendment
appendTx u tv = Transaction
  { txUnit = u, txMoves = [], txRows = Map.empty
    , txStatus = [], txIntroduce = Nothing, txAppend = Just tv
    , txBoundary = Nothing, txCause = Nothing }

supersedeTx :: UnitId -> UnitId -> Transaction -- C8 Breaking
   : stamp old unit
supersedeTx u uFresh = Transaction
  { txUnit = u, txMoves = [], txRows = Map.empty
    , txStatus = [SetSupersededBy uFresh], txIntroduce = Nothing, txAppend = Nothing
    , txBoundary = Nothing, txCause = Nothing }

-- withCause stamps a follow-on or cascade with the parent notice's boundary id:
-- withCause :: BoundaryId -> Transaction -> Transaction

```

Per-field canonical writer (C11). The picture is a table naming, for each position field, the only writers allowed to touch it; the type system holds every writer to that table. Each *PositionState* field is tagged with the closed set of field-writers permitted to mutate it: accumulated cost by settle and trade; high-water mark by fee-crystallisation; entry NAV by subscription. The balance field is not in this table. Its sole canonical writer is the Move edge (handler **Transfer**), applied in **applyTx**, so a balance change without a conserving move is unrepresentable. A write by a field-writer outside the permitted set is a type error at the writer's authorship site. The tag is erased once writes share one delta row, so the guarantee binds at authorship, not at the stored row (C11).

These field-writers are a different axis from the event classes of C2: one event class drives one or more field-writers, and the two name-sets are not meant to coincide. The discipline is carried by a GADT (a generalised algebraic data type, in which each constructor fixes its own type index) whose constructors *are* the field-to-writer table.

```

data Handler = Settle | Trade | Transfer | FeeCrystallise | Subscribe deriving (Eq
, Show)

```

```

-- The GADT constructors ARE the C11 field->writer table; no other pairing is
  representable.
-- The balance field is absent: its sole writer is the Move edge (handler '
  Transfer), applied
-- in applyTx -- so "balance written off-ledger" is unrepresentable.
data FieldWrite (h :: Handler) where
  WAc      :: Qty -> FieldWrite 'Settle      -- ac, by settle
  WAcTrade :: Qty -> FieldWrite 'Trade      -- ac, by trade
  WHwm     :: Qty -> FieldWrite 'FeeCrystallise -- hwm, by fee crystallisation
  WEntryNav :: Qty -> FieldWrite 'Subscribe  -- entry_nav, by subscribe
deriving instance Show (FieldWrite h)

data SomeWrite where SomeWrite :: FieldWrite h -> SomeWrite -- index erased once
  writes share a row
instance Show SomeWrite where show (SomeWrite w) = show w

applyWrite :: FieldWrite h -> PositionState -> PositionState -- total over all
  constructors
applyWrite (WAc q)      p = p { psAc      = psAc p <> q }
applyWrite (WAcTrade q) p = p { psAc      = psAc p <> q }
applyWrite (WHwm q)     p = p { psHwm     = qmax (psHwm p) q }
  -- monotone
applyWrite (WEntryNav q) p = p { psEntryNav = maybe (Just q) Just (psEntryNav p) }
  -- write-once

-- A settle handler authors ONLY 'Settle writes -- the result type IS the C11
  checkpoint.
settleHandler :: [(WalletId, Qty)] -> Map WalletId [FieldWrite 'Settle]
settleHandler legs = Map.fromList [ (w, [WAc q]) | (w, q) <- legs ]

erase :: Map WalletId [FieldWrite h] -> Map WalletId [SomeWrite] -- authorship ->
  erasure (S3)
erase = fmap (map SomeWrite)

_c11_ok_settle :: Qty -> FieldWrite 'Settle
_c11_ok_settle = WAc
_c11_ok_fee    :: Qty -> FieldWrite 'FeeCrystallise
_c11_ok_fee    = WHwm
-- _c11_bad :: Qty -> FieldWrite 'Settle
-- _c11_bad = WHwm -- TYPE ERROR: WHwm :: FieldWrite 'FeeCrystallise, not '
  Settle

```

Registration, settlement, moves, and amendment — one door, applyTx. The picture is a single door: every write to the three maps passes through `applyTx`, and nothing else writes. The ledger is sealed, and there is no second write path. Registration is the move-less transaction whose `txIntroduce` is `Just`. It writes its first terms version and default status as one entry, and it refuses a unit already present: re-registration is a hard error, never a silent reset (C10). Because terms and status are written together, “registered in terms $\iff$ registered in status” holds by construction. `register` is therefore `applyTx` of `registerTx`, not a separate operation.

A move’s effect is its per-wallet net delta, written into the balance field and gated on registration. Its two legs are exact inverses, so it sums to zero over wallets. A zero-quantity or self-move nets to zero on every wallet and writes no net change. *Held* means named in a move that nets nonzero.

Amendment has two tracks, chosen by a product-declared total predicate (C8). A fungibility-preserving amendment is one move-less transaction appending a terms version, leaving positions

untouched. A fungibility-breaking amendment spans two units, so it is the composition of two transactions, run atomically through `applyTx` by `amend`: introduce the successor u_{new} , then stamp `superseded_by( $u_{\text{old}} \rightarrow u_{\text{new}}$ )` on the predecessor's status. Moving the holders is a separate paired-issuance event, never part of the amendment. The two transactions are ordered so the successor is introduced before the predecessor is stamped; the supersede pointer therefore always names a registered unit. A `SetSupersededBy` whose target is absent is rejected as a `DanglingSupersede`, so the pointer can never dangle. Reads are capability-scoped: a cross-overlay read of another holder's position is forbidden, and strategy exports flow only through `UnitStatus` (C4).

```
-- C5+C7+C10: register is applyTx of the move-less introduction. It is not a
    primitive
-- standing outside the fold; registration, trade, settlement and amendment all
    flow through
-- the ONE apply.
register :: UnitId -> TermsVersion -> UnitStatus -> Ledger -> Either LedgerError
    Ledger
register u tv us = applyTx (registerTx u tv us)

-- The single door. C3 (all-or-nothing) is structural: the WHOLE new Ledger or a
    Left, never
-- a partial state. txIntroduce=Just must introduce an ABSENT unit (else
    ReRegistration, C10),
-- writing PT and US together (C5/C7); txUnit and every unit a move names must be
    registered
-- (else UnknownUnit), so PT<->US co-presence is preserved; rows insert/update,
    never delete
-- (C1 monotone). Conservation is already a property of txMoves (edges) -- nothing
    to validate.
applyTx :: Transaction -> Ledger -> Either LedgerError Ledger
applyTx tx l0 = do
    l1 <- introduce l0
    () <- requireKnown l1
    () <- requireSupersedeTargets l1
    () <- requireBasisTip l1      -- tip weld      \ bodies elided; the origin
    weld guards
    () <- requireInvariance l1    -- invariance weld / introduce. Code in the
    reference.
    Right (commit l1)
where
    u = txUnit tx

    introduce l = case txIntroduce tx of
        Nothing -> Right l
        Just (tv, us)
            | u 'Map.member' ledgerPT l -> Left (ReRegistration u)          -- C10
            | usBasis us /= Origin u    -> Left (ForeignOrigin u)  -- origin weld: C5
    stays a construction
            | otherwise -> Right l
                { ledgerPT = Map.insert u (ProductTerms (tv :| [])) (ledgerPT l) -- C7
                , ledgerUS = Map.insert u us                                     (ledgerUS l) -- C5
                }

    requireKnown l =
        case filter (\x -> not (x 'Map.member' ledgerPT l)) (u : map mUnit (txMoves
            tx)) of
            (bad : _) -> Left (UnknownUnit bad)
```

```

    [] -> Right ()

-- Referential integrity for the supersede pointer: a SetSupersededBy target
must name a
-- registered unit, else the stamp would dangle. A Breaking amendment
introduces the
-- successor first, so its id is registered by the time supersedeTx is applied
(C8).
requireSupersedeTargets l =
  case filter (\x -> not (x `Map.member` ledgerPT l))
    [ t | SetSupersededBy t <- txStatus tx ] of
    (bad : _) -> Left (DanglingSupersede bad)
    [] -> Right ()

commit l = l
{ ledgerBounds = maybe (ledgerBounds l)
  (\be -> Map.insertWith Set.union u
    (Set.singleton (bevTEff be, bevPrec be,
      bevId be))
    (ledgerBounds l))
  (txBoundary tx)
  , ledgerPT = maybe (ledgerPT l)
    (\tv -> Map.adjust (appendVersion tv) u (ledgerPT l))
  -- C6 append
    (txAppend tx)
  , ledgerUS = if null (txStatus tx)
    then ledgerUS l
    else Map.adjust (\s -> foldl' (flip applyStatus) s (txStatus
tx)) u (ledgerUS l)
  , ledgerPS = foldl' (flip applyMoveRow)
    (Map.foldrWithKey applyRow (ledgerPS l) (txRows tx))
    (txMoves tx)
  }

applyRow w writes ps = -- non-balance
bookkeeping
  let key = (w, u)
  cur = Map.findWithDefault zeroP key ps -- first touch ->
flat row
  new = foldl' (\acc (SomeWrite fw) -> applyWrite fw acc) cur writes
  in Map.insert key new ps -- insert/update
only

-- A Move writes ONLY psBalance, via the conserving pair from moveDelta (from -= q
, to += q),
-- keyed by the move's own unit -- the sole writer of the balance field.
applyMoveRow :: Move -> Map (WalletId, UnitId) PositionState
  -> Map (WalletId, UnitId) PositionState
applyMoveRow m ps0 = foldl' step ps0 (moveDelta m)
where
  step ps (w, dq) =
    let key = (w, mUnit m)
    cur = Map.findWithDefault zeroP key ps
    in Map.insert key cur { psBalance = psBalance cur <> dq } ps

-- C8: a product-declared total predicate chooses the track. Either track reaches
the maps
-- ONLY through applyTx -- there is no second write path.

```

```

data Fungibility = Preserving | Breaking deriving (Eq, Show)
type FungibilityPredicate = ProductTerms -> TermsVersion -> Fungibility
data AmendResult = Appended UnitId | Superseded UnitId UnitId deriving (Eq, Show)

amend :: FungibilityPredicate -> UnitId -> TermsVersion -> UnitId -> Ledger
      -> Either LedgerError (AmendResult, Ledger)
amend isFungible u tvNew uFresh l =
  case Map.lookup u (ledgerPT l) of
    Nothing -> Left (UnknownUnit u)
    Just pt -> case isFungible pt tvNew of
      Preserving ->                                     -- ONE move-less Transaction;
        positions untouched
          (\l' -> (Appended u, l')) <$> applyTx (appendTx u tvNew) l
      Breaking                                           -- spans two units: a
        composite of TWO Transactions
          | uFresh 'Map.member' ledgerPT l -> Left (UnitAlreadyExists uFresh)
          | otherwise -> do
            l1 <- applyTx (registerTx uFresh tvNew (defaultStatus uFresh)) l --
            introduce the successor
            l2 <- applyTx (supersedeTx u uFresh)                               l1 --
            stamp the predecessor
            Right (Superseded u uFresh, l2)                                     -- old terms never rewritten (
            C7/P7)

```

Deterministic replay. The picture is the ledger rebuilt from nothing: the events are folded, in order, over the empty ledger. Replay is the monadic left-fold of the event stream over the empty ledger. It is deterministic because application is a pure, total function of the event and the prior ledger: the same events give the same ledger. On a well-formed stream it returns the rebuilt ledger. On an ill-formed one — a repeated registration, or a move or settlement on an unregistered unit — it returns an error, the fold halting at the first refusal. Checkpointing is sound by the monadic left-fold law: the fold over a concatenation splits at any cut. Registration, settlement, and moves are all events, so replay from the empty ledger rebuilds every unit's terms, status, and positions. Every view — balances, profit and loss, the cached *UnitStatus* of §4.3 — is a projection of the stream. Row retention serves audit, not determinism (P8).

```

-- A Transaction IS the event; applyTx is the pure, total transition. Replay is
  the
-- error-stopping left fold of the stream. Determinism (P8) is the fold-
  homomorphism law,
--   replay (xs <> ys) = replay xs >=> replay ys,
-- so the result is independent of where a checkpoint is cut. On a well-formed
  stream (every
-- event on a registered unit) no Left arises, so replay is total there. (Balances
  and PnL are
-- further projections of the same stream -- netBal, in the balance-sheet section
  .)
replay :: [Transaction] -> Ledger -> Either LedgerError Ledger
replay ds l0 = foldM (\l d -> applyTx d l) l0 ds

```

Remark 4.1 (Projection generalisation (optional)). *balances* = *foldMap contribution* is a monoid homomorphism — *balances* (xs <> ys) = *balances* xs <> *balances* ys — into the Balances monoid; *UnitStatus* is a left fold of its status-write stream and value is a fold of the materialised position state, so every named view is a catamorphism of the log (directly for *balances* and *UnitStatus*, by composition with *replay* for value).

4.5 The conditions C1–C14

Each condition renders one illegal state unrepresentable. The labels are stable tags, not a sequence. Each is stated once and forced at the site named. C1–C12 are forced at the construction steps and instruments worked in §4.4 and §4.2: a future whose state depends on past trades, a managed account, a strategy that trades futures, and an instrument registered but never traded. C13 and C14 are stated and forced at the data plane’s basis edge, in §8.

	Condition	Forced at
C1	Position accessor returns <code>Option</code> (never-held vs. held-and-flat distinct) <i>and</i> the carrier is monotone (no row deleted); both.	<code>PositionState</code>
C2	Every <code>Transaction</code> carries its conserved flow as edge moves, whose signed per-unit sum is <code>mempty</code> by construction; vacuous base case (empty move list) included.	<code>Transaction</code>
C3	A <code>Transaction</code> spans all three maps for one unit and applies in full or not at all; partial application unrepresentable.	<code>Transaction</code>
C4	Reads are capability-scoped; cross-overlay reads forbidden; strategy exports flow only through <code>UnitStatus</code> .	strategy
C5	<code>UnitStatus</code> is registration-total: product-declared defaults written at registration, including the basis origin.	untraded unit
C6	<code>ProductTerms</code> is a non-empty version list, append-only; no version mutated in place.	<code>ProductTerms</code>
C7	<code>ProductTerms</code> is registration-total: first version written at registration.	<code>ProductTerms</code>
C8	Two-track amendment by a total fungibility predicate: <code>Preserving</code> appends a version; <code>Breaking</code> allocates a fresh <i>u</i> and records <code>superseded_by</code> .	amendment
C9	A move-less event has an empty move list, which folds to <code>mempty</code> , so it is conserved vacuously; conservation sums edge legs and never divides by a holder count.	untraded unit
C10	Re-registering an existing unit is a hard error, never a silent reset.	registration
C11	Each <code>PositionState</code> field has a canonical writer set; a writer outside it is a type error at authorship, erased at the delta row.	<code>PositionState</code>
C12	All per-(wallet, mandate/strategy) economic state lives at <code>PositionState[w, u_{MA}]</code> ; the <i>W</i> -sector is empty by schema.	managed account
C13	Every consumed datum names its joint basis; consumption requires pointwise agreement with the prevailing total assignment or a ledger-authored arrow (§8).	basis edge
C14	Adjustment-schedule totality: registration and every terms append admitted only if the schedule yields exactly one well-formed entry per (corporate-action class, field); an append introducing a field extends the schedule in the same transaction (§8.3).	basis edge

4.6 Why exactly three maps

Three independent constraints each force one map, and removing any one breaks its constraint. (i) Distinct per-holder state under the same contract forces a (w, u) -keyed map: *PositionState*. (ii) Shared observables read identically by every holder force a u -keyed map distinct from per-holder state: *UnitStatus*. (iii) The two mutation disciplines — append-only versioned terms versus an overwrite-in-place status cache — force a third, append-only map distinct from status: *ProductTerms*. A fourth map, the W -sector, adds a sector whose economic content is empty (C12). Three maps are the minimum basis of the problem, not a compromise.

4.7 The unique design

The adopted design — three maps under the conditions of §4.5, design B — is the unique Pareto-optimum on testability, correctness, and simplicity under any correctness gate. Each rejected alternative fails on one forcing point.

- **A** (per-unit state, plus per- (w, u) for futures only) has no sector for shared observables distinct from immutable terms, so settlement prices and terms share one overwrite cell; the two mutation disciplines of §4.6(iii) collapse.
- **C** (a sentinel unit carrying wallet state) has a sentinel with no issuer and no conservation partner, so $\sum_w h(\cdot) = 0$ fails by fiat.
- **D** (an explicit W -sector) adds a sector whose economic content is empty: every candidate datum is (w, u_{MA}) -keyed (C12).
- **E** (state indexed only over the held set) equals B on correctness but has no implementation in available tooling and is far less simple; shippability decides, not elegance.
- **F** (a universe wallet $w_\star$ folding *UnitStatus* into *PositionState* $[w_\star, u]$) re-introduces the per-unit/per- (w, u) split as a runtime predicate on the wallet axis, and conservation must then carve out $w_\star$ by hand. *UnitStatus* is the type-level expression of what F hides as a wallet-id convention.

4.8 Invariants made unrepresentable

Under design B with C1–C12, seven illegal states are placed beyond reach rather than merely tested; the further states C13’s welds place beyond reach at the same door are enumerated in §8. Each line names the hub invariant (§17) it discharges; where the guarantee is a contributing mechanism rather than the invariant itself, the line names the hub invariant it underpins. The P-numbers are the canonical hub numbering of §17. The mechanism differs by invariant and is named in each line. “Unrepresentable” is precise. An abstract type whose only constructor is a checked smart constructor keeps the illegal value from reaching the operation that would act on it. A non-empty list makes the versionless state untypable. A per-field tag is a type error at authorship. Conservation is carried by the edge form of a move, whose two legs cancel by construction. The claim is not that every guarantee is a type-level fact: read-scoping, in particular, is enforced at the capability layer wrapping the accessors, not in the data reference.

Invariant	Mechanism	Discharged by
P1 (conservation of quantity)	Conserved flow is carried as Move edges, each naming both legs ($\text{qneg } q \diamond q = \text{mempty}$), so the signed per-unit sum of any transaction is mempty by construction; there is no unconserved Transaction to reject, hence no validate gate.	C2 (edge-conserved flow), C3 (atomic transaction)
P8 (snapshot/replay determinism)	Replay is the foldM of the stream, a fold homomorphism: replaying a concatenated log equals replaying each part and composing; the monotone carrier keeps the key set stable across any cut.	C1(b) (monotone carrier)
P6 (lifecycle idempotency)	Lifecycle stage is u -keyed in <i>UnitStatus</i> , written by replacement, so re-applying a stage write is the identity; non-conserved fields carry the same algebra (high-water mark by max, entry NAV write-once).	C5 (registration-total status)
Terms immutability (underpins P4)	Append-only non-empty version list; no in-place mutation, no exported setter.	C6 (append-only), C7 (registration-total)
Identity immutability (underpins P4)	Re-registration rejected; a breaking amendment allocates u_{new} and never rewrites u_{old} .	C8 (two-track), C10 (no re-registration)
P7 (virtual/real isolation; read-scoping)	Cross-overlay reads forbidden at the capability layer wrapping the accessors, not in the data reference.	C4 (capability-scoped reads)
Handler-field canon (underpins P9)	Canonical writer set per field; a writer outside it is a type error at authorship, erased once writes share a delta row.	C11 (per-field writer)

No other candidate design places more than three of the core invariants beyond reach.

4.9 Reference implementation

The reference is `reference/Ledger.hs`, woven through §4.4 and collected here. It is total and deterministic, and makes the illegal states unrepresentable rather than merely checked:

- **Ledger** is sealed, with no row deleter (C1 monotone half) and an **Option** accessor (C1 Option half);
- a **Transaction** carries its conserved flow as **Move** edges, so the signed per-unit sum is **mempty** by construction and no **validate** gate is needed (C2, the empty move list giving C9);

- `ProductTerms` is abstract over a non-empty list grown only by registration and `appendVersion` (C6, C7);
- the `FieldWrite` GADT is the C11 field-to-writer table;
- `applyTx` is the single door, returning the whole ledger or an error with no partial state (C3);
- registration is the move-less `txIntroduce=Just` instance, rejecting re-registration (C10) and writing terms and status together;
- `amend` is the two-track C8.

Every `UnitStatus` field is written only by `applyStatus`, folded inside `applyTx` — the projection-of-log discipline of §4.3 held by construction.

The pieces woven through §4.4 are excerpted from `reference/Ledger.hs` and constitute its canonical core: the scalar layer, the three homes, the `Transaction`, and the single door `applyTx`. Two elisions are made, both marked in the listings: the basis-weld bodies, and the `BoundaryEvent` and `Declaration` declarations, specified in §8 and carried in the reference. The remaining parts of the reference serve their own sections and appendices. The `Move` the core builds on is the reference form, magnitude `PosQty`; §2 presents the same algebra in illustrative `Qty` form. Run on the reference source, the following evaluation makes the behaviour concrete; the comments are the printed results.

```
expect :: Show e => Either e a -> IO a
expect = either (error . show) pure

main :: IO ()
main = do
  let uES = UnitId "CME-ES"; wB = WalletId "BUYER"; wS = WalletId "SELLER"
  l0 <- expect $ register uES (TermsVersion "ES-v1" (Map.fromList [("mult","50")]))
    )
    (defaultStatus uES) emptyLedger
  print (position    l0 wB uES)      -- Nothing : never held (C1 Option half)
  print (unitStatus l0    uES)
  -- Just (UnitStatus {usLifecycle = Registered, usBasis = Origin (UnitId "CME-ES"
  -- ),
  --   usLastSettle = Nothing, usSupersededBy = Nothing}) : the C5 default,
  --   basis at origin
  -- a conserving trade: +1000 to BUYER as a Move EDGE (balance), with accumulated
  -- cost rows
  -- authored at 'Settle then erased (C11). (move builds the PosQty magnitude via
  -- mkPosQty.)
  Just mv <- pure (move wS wB uES (Qty 1000) (Timestamp 0) (SourceId "demo"))
  let tradeTx = Transaction uES [mv]
    (erase (settleHandler [(wB, Qty 1000), (wS, Qty (-1000))]))
    [] Nothing Nothing Nothing
  print (netDelta tradeTx)           -- 0 : the move legs cancel BY CONSTRUCTION
    (C2)
  l1 <- expect $ applyTx tradeTx l0
  print (psBalance <$> position l1 wB uES) -- Just 1000 : balance, written by the
    Move edge
  print (psAc      <$> position l1 wB uES) -- Just 1000 : accumulated cost, via
    the row
  -- There is no unconserved Transaction to reject: a one-legged flow is not a
  -- Move, so it is
  -- not representable in txMoves -- conservation is a property of the type, not a
  -- validate gate.
  -- close-out: a Move back keeps a flat row (monotone carrier): held-and-flat /=
  -- never-held
  Just mvC <- pure (move wB wS uES (Qty 1000) (Timestamp 1) (SourceId "demo"))
```

```

12 <- expect $ applyTx (Transaction uES [mvC] Map.empty [] Nothing Nothing
  Nothing) 11
print (psBalance <$> position 12 wB uES) -- Just 0 : retained, not Nothing
print (register uES (TermsVersion "dup" Map.empty) (defaultStatus uES) 12)
-- Left (ReRegistration (UnitId "CME-ES")) (
C10)

```

Remark 4.2 (Discharged by construction). *The listings above, save the evaluation driver, are excerpted verbatim from `reference/Ledger.hs`, the reference core, under the two marked elisions. Here and wherever an excerpt is claimed verbatim, the claim is code-level: the code tokens match the reference exactly; comments and line layout are re-set for the page. Quantities are exact **Integer** minor units, a correctness choice that makes determinism and conservation arithmetic facts. Three structural points carry the discharge: the per-unit homes are three coupled maps (`ledgerPT`, `ledgerUS`, `ledgerPS`) with co-presence preserved by construction, as §4.6 requires; the lifecycle is the closed sum *Registered/Active/Expired/Closed* with the last settle mark — a *Price* paired with its basis — a separate *Maybe* field written only through a logged *StatusWrite*, rather than carried on an *Active* constructor; and one atomic *Transaction* carries both the conserved move edges and the non-balance state delta, so conservation holds by construction and there is no separate validate step.*

The testing commitments for this construction — handler mutation score 85–90%, lifecycle-guard 70–80%, core overall $\geq 80\%$, and a state-machine model bounded by a finite generator universe — are carried by the testing agenda (§21, Appendix B), not restated here.

Key Invariants and Consequences

Unit state lives in three homes under one placement rule. The conditions of the register (§4.5) each render an illegal state unrepresentable rather than merely tested; C13’s identical claim is stated in §8. Each guarantee carries its canonical number in §17.

- **One home, one writer.** Each economic fact has exactly one home and one writer, so internal reconciliation failure cannot arise. The homes are immutable *ProductTerms*[*u*], shared *UnitStatus*[*u*], and per-position *PositionState*[*w*, *u*]; the externally-authored per-(holder, unit) cell is empty by construction (C12). (§4.)
- **UnitStatus is a projection, not a source of truth.** It holds one value per unit, overwritten in place only as a read cache. The immutable event log is authoritative, and replaying the events to any point rebuilds the exact value that held then. This discipline discharges lifecycle idempotency P6—write-by-replacement at one key—via C5. (§4.3.)
- **Conservation by construction (P1).** A *Transaction* carries its flow as *Move* edges whose signed per-unit sum is the group identity; C2 (edge-conserved flow) and C3 (atomic transaction across all three maps) leave no unconserved transaction to reject. (§4.)
- **Snapshot/replay determinism (P8).** Replay is the fold of the stream; the monotone carrier C1 keeps the key set stable across any checkpoint cut, so the result is independent of where the cut falls. (§4.)
- **Terms and identity immutability (underpin P4).** *ProductTerms* is an append-only non-empty version list (C6, C7); re-registration is a hard error and a breaking amendment allocates a fresh unit, never rewriting the old one (C8, C10). (§4.)
- **Read scoping (P7).** Cross-overlay reads are forbidden at the capability layer wrapping the accessors; strategy exports flow only through *UnitStatus* (C4). (§4.)

Part III

Valuation, Contracts, and Instrument Lifecycles

5 Portfolio Valuation and PnL

Valuation follows from the wallet structure. Ownership is wallet balance. Conservation (§2) keeps quantities consistent. Valuation multiplies each position by its price and sums.

5.1 Quantity versus value

MENTAL MODEL. The units in the wallets are water sealed in a system of pipes: no operation changes the total volume of each kind, however the water is shifted. Value is not in the pipes. It is what the water would fetch if drained and sold at the rates on a price sheet posted outside the seal, carried back as a single reference currency. Because the sheet hangs outside the seal and its rates move on their own, this money total rises and falls with no water flowing through any pipe; unlike the sealed volumes, it obeys no conservation law.

Conservation guarantees *quantity* balance: the total quantity of each unit across all wallets is zero. This is an algebraic invariant of the move semantics. Conservation does not guarantee *value* balance. Total portfolio value $V_t = \sum_u w_t(u) P_t(u)$ moves with prices. Prices are external inputs. They change independently of ledger activity. An FX trade conserves EUR and USD quantities separately. Yet at any rate other than the trade rate, the market value of the EUR received differs from the USD paid. Value conservation holds exactly only for deterministic lifecycle events (Property 5). Conservation secures quantity integrity. The valuation layer accounts for value change due to price movement.

5.2 Mark-to-market valuation

The value of a set of wallets is what unwinding every position at current prices would leave in the reference currency: each balance times its price, summed.

Definition 5.1 (Portfolio value). *Let $P_t : \mathcal{U} \rightarrow \mathbb{R}$ be a price function at time t , with reference currency $P_t(\text{USD}) = 1$. A portfolio is a non-empty set of wallets W —the real wallets of a book. Its value is the quantity of reference currency that would remain were every position in W unwound at P_t :*

$$V_t(W) = \sum_{w \in W} \sum_{u \in \mathcal{U}} w_t(u) P_t(u).$$

Summing over all wallets gives zero by closure (§2): real and virtual legs cancel, which is what makes the system closed. A meaningful valuation is therefore always of a chosen wallet scope, never the whole ledger. P_t is an external input, not computed by the framework; it requires only a designated reference currency and a price per unit in that currency. Pricing methodology — exchange quote, model output, or dealer quote — and the choice of reference currency and FX rates are fixed by the valuation specification (satellite, out of scope here).

Qualification. $P_t(u)$ presumes orderly, liquid markets. Under halts, closures, or counterparty default it may be stale, absent, or above achievable liquidation value.

Valuation is one total function from a portfolio scope, a price vector, and a ledger to reference-currency cash. The full price methodology and reference-currency machinery live in the valuation satellite. Here, only the signature and the law it carries.

```

-- Qty, Cash are exact additive groups (Integer minor units, never Float).
-- Price carries NO Monoid: adding two prices is meaningless, so the type
-- forbids the one wrong combination. Qty and Price meet only via markValue.
newtype Qty    = Qty    Integer
newtype Cash   = Cash   Integer    -- a Monoid: Cash a <> Cash b = Cash (a+b)
newtype Price  = Price  Integer

markValue :: Qty -> Price -> Cash      -- markValue (Qty q) (Price p) = Cash (q
    *p)
cashSub    :: Cash -> Cash -> Cash      -- pointwise difference: a plus the
    -- group inverse of b

-- A price vector prices EVERY unit (a total function, not a partial lookup):
-- "a held unit with no price" is unrepresentable; the reference unit prices to 1.
newtype PriceVec = PriceVec { priceOf :: UnitId -> Price }

-- A PORTFOLIO is a NON-EMPTY set of a book's real wallets; mkPortfolio is the
    sole
-- door, so the empty scope -- which values to Cash 0 by the empty sum, the same
-- degeneracy that makes a whole-ledger fold identically zero -- is
    unrepresentable.
-- value is scope-first, total, and history-free: it depends ONLY on current
-- balances in scope and prices. State-sufficiency is the signature, not a claim.
newtype Portfolio = Portfolio (Set WalletId)    -- constructor NOT exported
mkPortfolio :: [WalletId] -> Maybe Portfolio    -- Nothing on the empty scope

value :: Portfolio -> PriceVec -> Ledger -> Cash
value (Portfolio ws) (PriceVec p) l =
    foldMap (\((w, u), ps) -> if w `Set.member` ws
        then markValue (psBalance ps) (p u) else mempty)
        (Map.toList (ledgerPS l))

-- A Snapshot pairs one price vector with one ledger, so an endpoint cannot be
-- valued at the OTHER endpoint's prices. PnL is the difference of two snapshots'
-- valuations (P10): each is fixed by its own endpoint, so any intermediate
-- snapshot adds (v `cashSub` v) = mempty -- telescoping, hence path-independence.
data Snapshot = Snapshot PriceVec Ledger
pnl :: Portfolio -> Snapshot -> Snapshot -> Cash
pnl port (Snapshot p0 l0) (Snapshot p1 l1) = value port p1 l1 `cashSub` value port
    p0 l0

```

Five illegal states fall out of the types. *A held unit with no price* cannot occur: `PriceVec` is a total function, not a partial map. *Adding two prices* cannot be written: `Price` has no `Monoid`. The only way out of `Price` is `markValue`, which lands in `Cash`. *Path-dependence* cannot enter: `value` takes a portfolio scope, a `PriceVec`, and a `Ledger` — and no transaction history. The type itself certifies state-sufficiency. *An empty valuation scope* cannot be built: `Portfolio` is abstract, and `mkPortfolio` rejects the empty wallet set. The identically-zero whole-book valuation cannot be reached by omission. *Valuing one endpoint at another's prices* cannot be written: an endpoint is one `Snapshot`, a price vector bound to its ledger. The two arguments of `pnl` cannot be cross-paired. The fold is total over a finite balance projection, and `markValue` is exact integer multiplication; `value` is therefore total and deterministic.

5.3 State-sufficiency

To value the book, the only inputs are where positions stand now and what they are worth now. How they got there does not enter.

Principle 5.2 (State-sufficiency). *Given that every economically relevant state change is recorded, portfolio value at time t depends only on current wallet balances, current unit state, and current prices. Transaction history is irrelevant.*

Unit state carries all embedded lifecycle state: accrued interest, exercise flags, barrier status. It is a materialised projection of the immutable event log (§3), not a separate source of truth. Computing V_t therefore needs one snapshot of balances and one price vector, not a replay; that snapshot is coherent only when its balances and prices stand in one corporate-action basis, which single-basis consumption (Invariant 8.17) requires. PnL is $O(n)$ in instruments, not $O(m)$ in moves.

5.4 Path-independent PnL

Profit and loss over a period depends only on its two endpoints. The trading in between, however much of it, leaves the total unchanged.

Theorem 5.3 (Path-independent PnL). *Profit and loss over $[t_0, t_1]$ is*

$$\text{PnL} = V_{t_1} - V_{t_0},$$

a function of the portfolio states at t_0 and t_1 alone, not of intermediate transactions. (Property P10.)

Proof. Each of V_{t_0} , V_{t_1} is fixed by wallet balances and prices at its time, and balances are the cumulative effect of all prior moves. For any intermediate times $t_0 < s_1 < \dots < s_k < t_1$,

$$V_{t_1} - V_{t_0} = (V_{t_1} - V_{s_k}) + (V_{s_k} - V_{s_{k-1}}) + \dots + (V_{s_1} - V_{t_0});$$

the interior values cancel, and no path information survives. $\square$

The mechanism is telescoping. The content is independence of trading activity. A portfolio that executed ten thousand intermediate trades has the same total PnL as one that executed none, given matching final positions and prices. PnL never requires replaying intermediate transactions: `pnl` above takes the two endpoints and nothing between them.

Qualification. The result captures a state change only if it is recorded as a move. An unrecorded fee or an unbooked bilateral agreement falls outside it. PnL is defined as the change in the ledger's valuation, and the ledger records all economic activity within its scope.

Scope. This is *economic* PnL: the change in fair value. Accounting PnL under IFRS 9 or US GAAP depends on instrument classification (FVTPL, FVOCI, amortised cost), which is outside the ledger's scope. The framework supplies the position quantities and economic valuations from which it is derived.

5.5 Price and flow decomposition

Total PnL splits in two. Price PnL is the revaluation of the opening holding at the new prices. Flow PnL is the net effect of the period's cash and position moves.

Proposition 5.4 (PnL attribution).

$$\text{PnL} = \text{PnL}_{\text{price}} + \text{PnL}_{\text{flow}},$$

$$\text{PnL}_{\text{price}} = \sum_i w_{t_0}(i) [P_{t_1}(i) - P_{t_0}(i)], \quad \text{PnL}_{\text{flow}} = \Delta w(\text{USD}) + \sum_i \Delta w(i) P_{t_1}(i).$$

Worked example: USD reference, one risk unit AAPL.

Event	USD	AAPL	Price
State at t_0 ($V_{t_0} = 1000 + 10 \cdot 100 = 2000$)	1000	10	100
Buy 5 @ 100	-500	+5	
Dividend	+20		
Fees	-5		
Financing	-8		
State at t_1 ($V_{t_1} = 507 + 15 \cdot 110 = 2157$)	507	15	110

$$\begin{aligned}
\text{PnL} &= V_{t_1} - V_{t_0} = 157, \\
\text{PnL}_{\text{price}} &= 10(110 - 100) = 100, \\
\text{PnL}_{\text{flow}} &= (507 - 1000) + 5 \cdot 110 = -493 + 550 = 57, \\
100 + 57 &= 157.
\end{aligned}$$

Every cash flow is an explicit USD move: purchase, dividend, fee, financing. Price change revalues existing positions only.

5.6 Dual valuation

The same positions can hold more than one value at once: value is read from a price vector, and more than one price convention can apply.

V_t is a function of an externally supplied price vector, so distinct price conventions value the same positions differently. Applying Definition 5.1 under two conventions, mark-to-market and mark-to-mid, yields the dual valuation. Its construction, the multi-exchange futures case, and its balance-sheet use are developed in §11.

6 Smart Contracts as Move Generators

A financial instrument is a deterministic program that emits moves, not an opaque record in a database. A bond emits coupons and a redemption; an option, a premium and a conditional payoff; a swap, periodic net payments. The logic varies; the output is always one list of moves. **MENTAL MODEL.** Picture a set of automatic valves bolted into a sealed system of water pipes, where each sealed tank is a wallet and the water it holds is units. The contract is the clockwork that drives the valves: set the same dials — the same inputs, state, and observed conditions — and it always steps the valves through the same sequence, routing water from one tank to another. Because it sits inside the seal, its only available action is to route water already in the pipes; it has no tap that adds water and no drain that removes it, so even a mis-cut cam can only send water to the wrong tank, never conjure or lose a drop. Unlike a real valve that could jam or burst and spill, this one cannot leak: routing is the whole of its vocabulary.

6.1 Contract Definition

Definition 6.1 (Smart Contract). *A smart contract is a deterministic function from inputs, state, and observed conditions to a sequence of atomic moves:*

$$\text{Contract} : (\text{Input}, \text{State}, \text{Conditions}) \rightarrow [\text{Move}].$$

A contract's sole output is moves. A move transfers a quantity from one wallet to another without creating or destroying it (Section 2). Conservation therefore holds by construction: a contract can move units, never create or destroy them.

Equal inputs, state, and conditions yield equal moves. There is no randomness, no hidden state, no dependency beyond the passed parameters. The contract is not stored apart from the ledger; its effects *are* ledger entries. Product complexity is isolated in the contract, and the ledger sees only moves. A new product type is a new contract; the move stream, balance computation, and reporting are untouched, and at the ledger level its moves are indistinguishable from any other product's.

A smart contract is the machine-readable, executable form of the legal confirmation. An ISDA derivatives confirmation already states the product, economics, lifecycle events, and settlement obligations in structured terms. The CDM is the formal vocabulary of that structure. Writing the contract from the confirmation is transcription, not engineering: CDM enums map directly onto product types, lifecycle events, and settlement obligations. The contract adds executability; it adds nothing to the information content.

The output type is a `Move` carrying a quantity `Qty`. A quantity is an exact integer in the unit's smallest denomination (minor units: cents, satoshis, basis-point-notional); never a decimal, never a float. Integer minor units make addition associative and every balance computation exact. Identical inputs therefore produce bit-identical outputs across hardware and compilers. This is the precondition for reproducible time travel and cross-system verification. Rounding (round-half-to-even, IEEE 754) applies once, at instruction projection, where an exact internal amount becomes a settlement or payment message. It never applies within balance or payment arithmetic.

The types make this exactness structural. A `Qty` is an integer count of minor units, and the contract is a total function emitting a list of moves. The contract is fully determined by the moves it lists, with no hidden step.

```
-- A quantity is an EXACT integer in the unit's smallest denomination
-- (minor units: cents, satoshis, basis-point-notional). Never a decimal float.
newtype Qty = Qty Integer deriving (Eq, Ord, Show)

-- Qty is the additive abelian group of Section 2 (the position algebra).
instance Semigroup Qty where Qty a <> Qty b = Qty (a + b) -- associative, exact
instance Monoid Qty where mempty = Qty 0 -- identity (zero)
qneg :: Qty -> Qty -- inverse
qneg (Qty n) = Qty (negate n)

newtype WalletId = WalletId String deriving (Eq, Ord, Show)
newtype UnitId = UnitId String deriving (Eq, Ord, Show)

-- A move: a positive Qty of one unit, one wallet to another (Section 2),
-- tagged with the emitting contract and metadata. Direction carries the sign,
-- so mQty is always > 0. Illustrative form, local to this section: the reference
-- Move carries the magnitude as PosQty and elides the free-text mMeta field.
data Move = Move
  { mFrom :: WalletId, mTo :: WalletId
  , mUnit :: UnitId, mQty :: Qty -- exact minor units, > 0
  , mTime :: Integer, mSource :: String -- timestamp; emitting contract id
  , mMeta :: String -- event description / external refs
  } deriving (Eq, Show)

-- A smart contract is a DETERMINISTIC, TOTAL function emitting a move schedule.
-- The ledger sees only [Move]; product complexity stays inside the contract.
type Contract i s c = (i, s, c) -> [Move]
```

Instruction projection converts an exact internal amount to a settlement message, and a rate or an FX cross can make that conversion produce a fraction. Round-half-to-even applies there and nowhere else.

```

-- The SOLE rounding site. Prelude 'round' IS round-half-to-even (IEEE 754).
projectAmount :: Rational -> Qty
projectAmount r = Qty (round r)

-- ghci> Qty 250 <> Qty 12500          -- exact integer addition
-- Qty 12750
-- ghci> qneg (Qty 12500) <> Qty 12500  -- conservation: no rounding error
-- Qty 0
-- ghci> projectAmount (5 % 2)          -- 2.5 -> nearest even -> 2
-- Qty 2
-- ghci> projectAmount (7 % 2)          -- 3.5 -> nearest even -> 4
-- Qty 4
-- ghci> Move (WalletId "wB") (WalletId "wS") (UnitId "USD")
-- (Qty 250000) 0 "put_ref" "PUT_PREMIUM" -- Move 1 below, typed

```

The listed code carries its own guarantees: every function is total and deterministic, and rounding cannot reach balance arithmetic. The remark makes this precise.

Remark 6.2 (Totality and determinism). *The Qty group ($\langle \rangle$), mempty, qneg) is reused verbatim from the reference `reference/Ledger.hs`. The Move record, Contract synonym, and projectAmount are total (round is total on Rational; Move is a plain product; Contract is a type synonym) and deterministic, with rounding confined to the single projectAmount boundary so no balance arithmetic can round.*

6.2 Composition: Modular over Composite

An instrument is a bundle of obligations, and each obligation has its own inputs and its own logic. The principle keeps them separate.

Principle 6.3 (Modular contracts). *One smart contract per payout or obligation type, not one composite contract per instrument.*

An option payoff depends on strike, spot, and exercise terms. A CSA margin depends on aggregate mark-to-market, threshold, minimum transfer amount, and eligible-collateral currency. These are independent calculations on independent inputs. Separation keeps each a pure function testable in isolation. Combining them makes the input space the product of the two, most of which is irrelevant. A margin contract written once attaches to any collateral wallet, whatever the instrument behind it (option, swap, or structured note), eliminating duplication.

Modularity requires clear interfaces. The executor, the single component permitted to mutate the ledger (Section 7), orchestrates the contract invocations for a given lifecycle event. This orchestration is simpler than maintaining a composite script for every product–CSA combination.

6.3 Equity, Dividend, and Corporate-Action Contracts

A stock contract emits moves for trade execution, dividend payment, and corporate actions through one primitive: a set of moves in a transaction. No separate machinery is involved.

Trade execution. A purchase of n shares at price p is a transaction of two atomic moves: $n \cdot p$ cash from buyer to seller, and n of the stock from seller to buyer.

Dividend. On the ex-dividend date the contract emits cash from an issuer virtual wallet to each holder wallet, proportional to shares held. The unit state — materialised *UnitStatus*, a projection of the event log (§4.3), governed in Section 7 — records the event (date, amount per share) for idempotency.

Bond accrued interest. Accrued interest is not a separate unit. The bond price function $P_t(u)$ returns the *dirty* price (including accrual). A between-coupon trade settles as cash-for-bond at the dirty price. At the next coupon date the contract pays the full coupon and

transitions the bond ex-coupon. Clean versus dirty is a reporting convention derived from unit state (last coupon date, accrual convention), not a separate position: the clean price is reported by subtracting the accrued interest computed from that state.

Corporate actions. A corporate action is a sequence of state transitions and moves anchored to its dates — announcement, record, ex/effective, payment. At the record date the unit state records the entitlement snapshot; at the ex/effective date the contract emits position-adjustment moves from a corporate-action virtual wallet, and the same multi-date mechanism handles splits, mergers, spin-offs, and rights issues. Each step is a transaction with full metadata (dates, ratios, entitlement rules), so time travel reconstructs both pre- and post-action views; the frame change the action causes, and every pricing rule across it, are governed by the State-Basis Discipline (§8).

6.4 European Put—Full Move Schedule

A put is two transfers: an unconditional premium at inception and a conditional payoff at expiry. The full schedule follows.

Parameters: strike K , expiry T , underlying S , notional N , premium P , buyer w_B , seller w_S .

Move 1 (inception, $t = 0$) — unconditional, on contract creation:

```
Move(from: w_B, to: w_S, unit: USD, quantity: P * N,
      timestamp: t_0, source: "put_ref", metadata: "PUT_PREMIUM")
```

Cash settlement. *Move 2 (expiry $t = T$)* executes only if $S_T < K$ (in-the-money). The seller pays the buyer the intrinsic value $(K - S_T) \cdot N$. If $S_T \geq K$, no move.

```
Move(from: w_S, to: w_B, unit: USD, quantity: (K - S_T) * N,
      timestamp: T, source: "put_ref", metadata: "PUT_SETTLEMENT_CASH")
```

Physical delivery. If $S_T < K$, the buyer delivers N of the underlying and receives $K \cdot N$ cash. The two moves are one atomic transaction; delivery requires the buyer to hold the underlying at expiry. If $S_T \geq K$, no moves.

```
Move(from: w_B, to: w_S, unit: S, quantity: N, ... metadata: "
      PUT_DELIVERY_ASSET")
Move(from: w_S, to: w_B, unit: USD, quantity: K * N, ... metadata: "
      PUT_DELIVERY_CASH")
```

Lot-size constraint. Equity delivery requires whole multiples of the board lot L (reference data in the Unit Store). For lot L ,

$$N_{\text{deliver}} = \left\lfloor \frac{N}{L} \right\rfloor \cdot L, \quad N_{\text{residual}} = N - N_{\text{deliver}}.$$

Whole lots settle as a DvP pair at strike; the residual settles in cash at intrinsic value. All moves are one atomic transaction:

```
Transaction (settlement: txIntroduce = Nothing, carrying the three moves below):
  Move(from: w_B, to: w_S, unit: S, quantity: N_deliver) -- whole
  lots
  Move(from: w_S, to: w_B, unit: USD, quantity: K * N_deliver) -- strike
  cash
  Move(from: w_S, to: w_B, unit: USD, quantity: (K - S_T) * N_residual) -- cash
  residual
```

Splitting the notional changes the form of settlement, not its value.

Proposition 6.4 (Lot-split conservation). *The underlying nets to zero ($-N_{\text{deliver}} + N_{\text{deliver}} = 0$); cash nets to zero, the seller's outflow $K \cdot N_{\text{deliver}} + (K - S_T) \cdot N_{\text{residual}}$ equalling the buyer's inflow. The economic value transferred is the put payoff $(K - S_T) \cdot N$, independent of the lot split.*

The lot-adjustment is a pure function of (K, N, S_T, L) , tested by the property suite (Appendix B). The settlement projection (Section 13) yields two instructions: a securities delivery (`sese.023`) for N_{deliver} shares and a cash payment for $K \cdot N_{\text{deliver}} + (K - S_T) \cdot N_{\text{residual}}$. The pattern generalises to any physically settled instrument with lot constraints: convertible conversion, warrant exercise, equity-forward delivery. Only the payoff formula differs.

The contract is fully defined by its move schedule: a finite list of conditional transfers parameterised by market observables. Pricing complexity (Black–Scholes, Monte Carlo, finite differences) is irrelevant to the ledger, which sees only the emitted moves.

6.5 Interest Rate Swap—Full Move Schedule

Each reset pays the difference between the floating and fixed rates on the notional for the period. The swap is that payment repeated over the schedule.

Parameters: notional N , fixed rate r_{fixed} , floating reference $r_{\text{float},k}$ (e.g. SOFR) at reset k , reset dates $\{t_1, \dots, t_n\}$, day-count fraction δ_k , fixed-payer w_F , floating-payer w_L . CDM date types and conventions fix the dates and δ_k (Section F.5). For each reset t_k :

$$\text{Payment}_k = N \cdot (r_{\text{float},k} - r_{\text{fixed}}) \cdot \delta_k.$$

```
Move(from: w_F, to: w_L, unit: USD, quantity: Payment_k,
     timestamp: t_k, source: "irs_ref", metadata: "IRS_NET_PAYMENT")
```

Move quantities are positive (Section 2). The contract emits $|\text{Payment}_k|$: $w_F \rightarrow w_L$ when $\text{Payment}_k > 0$, $w_L \rightarrow w_F$ when $\text{Payment}_k < 0$. Direction encodes the sign; quantity is the magnitude. Each reset needs only the current floating rate and the fixed parameters; no historical path is required. The complete contract is the sequence $\{m_1, \dots, m_n\}$, one move per reset, plus the unit state recording settled resets for idempotency.

Worked example (CDM lifecycle). A 5-year USD IRS, notional \$10M, fixed 3%, quarterly resets.

- *Inception* (CDM `ExecutionEvent`, intent `OPEN`). Unit state initialised: fixed rate, floating index, schedule, reset counter = 0. No cash move (par swap, no upfront premium).
- *Reset at t_1* (CDM `ResetEvent`). SOFR observed at 3.5%; unit state records `float_rate_t1 = 3.5%`, counter incremented.
- *Coupon at t_1* (CDM `TransferEvent`). $10\text{M} \times (3.5\% - 3\%) \times 0.25 = 12,500$. `Move(from: w_F, to: w_L, unit: USD, quantity: 12,500)`; period t_1 marked settled.
- *Maturity at t_n* (CDM `TerminationEvent`, intent `TERMINATE`). Final coupon as above; unit state transitions to `TERMINATED`. No further resets or payments.

7 Lifecycle Management

A lifecycle event is a transaction. A coupon payment, a dividend, a margin call, a barrier hit, an exercise, a reset, and a rebalancing step are each a contractual obligation becoming due. Each is realised as value moving between wallets together with a change of contract state. To the ledger, executing a trade and processing a lifecycle event are the same operation: a finite list of atomic moves, possibly accompanied by unit registrations and state changes, obeying conservation. There is no separate lifecycle engine, data format, or pipeline per product type. Every product type uses the move and the state transition.

This identity has four consequences.

1. **Uniformity.** Trade execution, coupon payments, margin flows, conversions, and rebalancing are one primitive—a transaction carrying moves and optional state updates.
2. **Conservation.** Every transaction satisfies the conservation law of §2; lifecycle events preserve closed-system integrity by construction.
3. **Auditability.** The evidence of a lifecycle event is the transaction: dated moves with origin and metadata, plus explicit state changes.
4. **Composability.** Lifecycle events sequence and simulate exactly as trades do. “Execute a trade, process a coupon, settle a margin call” is three transactions applied in order, each conserving.

7.1 The Transition Function

A lifecycle event is computed from the present, not from history. It reads the unit, that unit’s current state, and the observed market data. It returns the moves to make together with the next state. It commits nothing.

A lifecycle event is a pure function

$$f : (\text{unit}, \text{state}_t(u), \text{market_data}) \rightarrow (\text{moves}, \text{state}').$$

It mutates nothing and reads no hidden global. It returns a description: a list of wallet-to-wallet moves and the next state. The function is local to the unit and the event; it requires no knowledge of other units, wallets, or pending events. `applyTx` commits its output atomically.

Lifecycle is therefore state-transition logic plus a collection of moves. The definition presupposes no pricing model. This generality is required. The ledger carries convertible bonds, margin loans with bespoke collateral rules, and rule-based strategies whose economics live in state transitions rather than closed-form prices. The framework demands only the transition function above, so each is handled at the lifecycle level on the same footing as a plain-vanilla instrument.

In code `f` is `handle`. It reads the ledger, which supplies `statet(u)`. It takes the event, which carries the unit and any market datum. It returns the proposal as a `Transaction`: the moves and the state delta they carry. It never returns a changed ledger.

```
-- A lifecycle event is a pure function of the current state: it reads and
-- proposes, it commits nothing. The proposal is a Transaction (sec.4) -- the one
-- atomic event: the moves plus the non-balance state delta they ride with -- not
-- a changed Ledger.
handle :: Event -> Ledger -> Either LedgerError Transaction

-- The concrete Event type is given per product family: the three-home model
-- (sec.4) and the futures lifecycle (sec.9). The Transaction it builds carries
-- * txMoves : the conserved flow in edge form, each leg debiting one wallet and
--               crediting another by the same magnitude, so the signed per-unit
--               sum is empty BY CONSTRUCTION (conservation, C2/C9);
-- * the state delta : the status / stage writes the event causes (txStatus,
--                       txRows, txIntroduce, txAppend).
```

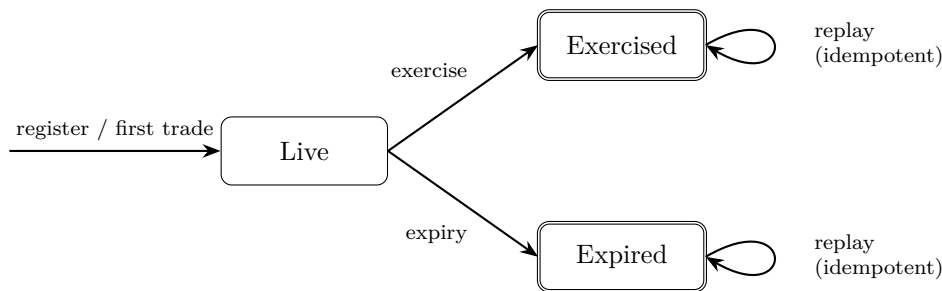


Figure 5: Lifecycle as a state machine, for a generic instrument: from *Live*, exercise leads to *Exercised* and expiry to *Expired*. The terminal states are absorbing; their self-loops depict the idempotence of replay—re-applying a settled event produces no further effect (§7). There is no single global machine: the transitions are determined by **ProductTerms** per unit-type. The concrete futures machine REGISTERED → ACTIVE → EXPIRED → CLOSED is one instance (§9).

-- Market data enters only inside the Event, as a logged observation (e.g. a
 -- settlement price), so handle stays pure -- a fresh replay reproduces the same
 -- moves bit for bit.

7.2 Idempotence

A lifecycle event that has already taken effect does nothing when applied again. The second application reads a state that already records the first, so it proposes no moves and no change of state.

Principle 7.1 (Idempotence). *Applying the same lifecycle event twice to the same state has no additional economic effect.*

Idempotence holds because moves are derived from $(\text{state}_t(u), \text{event})$, not from history. The state records which events have occurred. If it already shows the coupon at date T paid, f produces no moves and no state change. Paying a coupon twice at the same date pays it once. **MENTAL MODEL.** Picture the coupon recorded in a notebook that is only ever added to: the line “coupon at date T : paid” is already on the page. Running the event again reads that line, finds the payment already entered, and writes nothing. The second pass is empty because the page shows the work done, not because a separate list remembers the event was seen—strike the line and the same event pays the coupon once more, since the emptiness lives in the state read, never in a memory of past attempts.

7.3 The Single Door

One function changes the ledger; nothing else does. A lifecycle function only proposes a transaction. The proposal becomes part of the ledger only when this one function commits it.

Principle 7.2 (Single door). ***applyTx** is the only function permitted to change the ledger.*

The Transaction returned by a lifecycle function is a proposal. It takes effect only when **applyTx** commits it. Conservation is not checked at this boundary; it is a property of the Transaction type. The signed edge-sum of the moves is **empty** by construction (§2, C2/C9), so there is no unconserved Transaction to reject. **applyTx** requires that the unit be registered. It inserts or updates rows and never deletes one. It commits the whole Transaction atomically to the immutable event log of §2. Product guards (sufficient source balance, stage legality) live in **handle**, which builds the proposal.

Confining mutation to one door has three consequences. First, conservation holds in exactly one place: the Transaction type. A defective contract therefore cannot break an invariant: there is no unconserved proposal for `applyTx` to admit, and a balance moves only via a conserving `Move`. What it can still do is book wrong economics — a conserving transaction whose amounts disagree with the declared terms — and that error is detected by recomputation from declared data (§15.2). Second, lifecycle functions stay pure: they compute a proposal and cannot alter state, so they are testable, composable, and replaceable. Third, the audit trail is complete: every state change passes through `applyTx` and is logged. The attack surface for state corruption is bounded to this single door, whose properties P1–P10 are stated and tested in §17. Economic correctness of what the door admits is governed by §15.2. The role is that of an operating-system kernel: all privileged operations pass through one narrow, auditable interface. In code the step is the two halves in sequence: `handle` builds the proposal and `applyTx` commits it. Replay folds this step over the event stream.

```
-- handle PROPOSES a Transaction and commits nothing; applyTx (sec.4) is the SOLE
-- door that returns a changed Ledger. The step stops at the first error (the bind
-- ):
step :: Event -> Ledger -> Either LedgerError Ledger
step ev l = handle ev l >>= \tx -> applyTx tx l

-- There is no separate validate gate. Conservation is a property of
-- the Transaction type -- the signed edge-sum of txMoves is mempty BY
-- CONSTRUCTION
-- (C2/C9) -- so an unconserved move is unrepresentable, not rejected at run time.
-- applyTx (sec.4) is the single mutation boundary: it requires the unit be
-- registered, inserts/updates rows and NEVER deletes one (monotone carrier, C1),
-- and returns the whole new Ledger or a Left, never a partial state (atomicity,
-- C3). Product guards -- sufficient source balance, stage legality -- live in
-- handle (sec.9, G1-G5).
-- applyTx :: Transaction -> Ledger -> Either LedgerError Ledger -- sec.4

-- Time travel is this same step FOLDED over the event stream (foldM threads the
-- ledger, stopping at the first error); equivalently, once the proposals are
-- built, it is foldM applyTx over the transactions (sec.4). Replaying to any t
-- rebuilds the exact ledger as of t -- this is clone_at t.
replay :: [Event] -> Ledger -> Either LedgerError Ledger
replay evs l0 = foldM (flip step) l0 evs
```

Replay is a *fold homomorphism*: replaying two event batches joined, `replay (xs <> ys)`, equals replaying each in turn, `replay xs >=> replay ys`. Here `f >=> g` runs `f`, feeds its result to `g`, and stops at the first error. The reconstructed ledger is therefore independent of where a checkpoint is cut. Checkpoint-independence (§17, P8) is a consequence of the law, not a separate test. Determinism follows from `applyTx` being pure and total.

The event log is therefore canonical. Every state change passes through the single door and is logged. Replay rebuilds any state from those events alone. Any ledger, current or historical, is reproducible by replaying its log. The projections of §4, `UnitStatus` among them, are rebuilt this way.

7.4 Purity

A lifecycle function computes; it does not observe its surroundings. Run it now or in a later replay, on any machine: the same inputs give the same outputs.

Principle 7.3 (Purity). *For fixed inputs—view, parameters, market data—a lifecycle function produces identical outputs, independent of when or where it runs.*

Purity is a design requirement, not a coding convention. A lifecycle function has no hidden dependency: no runtime database lookup, no configuration read, no ambient state. Every input affecting the output is passed explicitly.

Purity alone does not give reproducibility. Reproducibility also requires a deterministic market-data oracle: a fixed specification of price source, snapshot time, and fallback chain. The market data used by each invocation is captured and stored at execution time as a versioned snapshot. Replays read the stored snapshot, not a live feed. Vendor corrections are separate data versions. A replay “as known at t ” uses the snapshot from t ; a replay “with corrected data” uses the restated snapshot. Each is deterministic given its inputs. Market data is governed by the valuation and market-data satellite specifications. This section depends only on the determinism the oracle provides.

Purity reduces testing to three mechanical forms. Unit tests assert “given view and inputs, expect these moves and state changes.” Property tests, such as idempotence (Principle 7.1), run the same function repeatedly on cloned views. Regression tests construct the appropriate historical view and feed the stored snapshot.

7.5 Time Travel

MENTAL MODEL. The ledger’s history is a notebook written only by adding lines, never by erasing them. To see any past date, cover every line entered after it and re-total the page from the top: the holdings that stood at that date reappear exactly, because the earlier lines were never overwritten. No daily snapshot is stored and read back—each past total is recomputed from the surviving lines, which is why the reconstruction is exact only for a record that appends and never erases.

Time travel reconstructs the exact ledger state at any historical timestamp t . This is not merely today’s totals replayed: it is what each position was, what it referenced, and how its value evolved. Four situations make this non-trivial. Any implementation must reconstruct each.

1. **Futures and margin PnL.** The margining story is reconstructed step by step: the sequence of variation-margin calls and the prices at which each was computed, not only the final cash total.
2. **Corporate actions and share denominators.** A 2-for-1 split turns 100 pre-split shares at a pre-split price into 200 post-split shares at roughly half the price; economic value is preserved but the unit changes. Replaying trade tickets against the current share definition mis-describes the t_0 position. Correct time travel reconstructs both views and explains the jump as a corporate action, not a reference-table edit.
3. **Exercise and novation.** On exercise the derivative is replaced by a cash or physical position; on novation the original contract is extinguished and a new one created with a different counterparty. The reconstruction must show why the derivative is present at t_0 and absent at t_1 , not only the cash that moved.
4. **Basket composition changes.** A basket of 40% A, 30% B, 30% C is redefined when B is acquired, substituting the acquirer D at a merger-dictated ratio. Time travel before the change shows the original basket; after, the updated basket with the conversion explained. A basket read from today’s static file cannot distinguish the two.

These are routine features of real portfolios. Unit state makes their reconstruction possible. Each event (split, exercise, novation, redefinition) is recorded as an explicit state change on the relevant unit. A view cloned at any historical date therefore carries the correct state, and re-applying f to that state with the stored market data reproduces exactly the moves generated

then. Without explicit unit state, time travel is limited to balance replay: enough for cash totals, insufficient to explain what the portfolio meant.

Cloning. Two operations realise reconstruction.

- **clone()** deep-copies the current view—units, states, and moves to the current time—for simulation or testing without affecting the original.
- **clone_at(t)** replays the move stream and all state changes up to t and returns the ledger as of t ; later moves are ignored.

Any lifecycle function re-run against a clone reproduces the same pending transaction given the same market data. Backtesting and scenario analysis reduce to: take a historical state, apply f , inspect the moves and state deltas. Time travel is a design property, not a debugging aid. Any state is reachable again by replaying the same transactions, and any lifecycle logic re-applied yields identical outcomes (§17, P8).

7.6 State Storage and Embedded Defence

Unit state is not stored as a private dictionary on the unit. It lives in the three-home state model of §4. There the per-unit lifecycle projection is **UnitStatus**: a read cache of the event log, rebuilt exactly by replay (§4.3).

A lifecycle function never writes **UnitStatus** off-log. It returns a **Transaction**; **applyTx** logs the event; the projection is rebuilt from the log. This is why time travel is exact: **clone_at(t)** rebuilds the projection from the events up to t .

Attaching every lifecycle state change to the unit gives an embedded explanation layer. For any instrument u , its current state $\text{state}_t(u)$ and recorded changes reconstruct its lifecycle history: coupons paid, dividends accrued, barriers triggered, margin calls made. Three consequences follow.

1. **Valuation justification.** Current valuation is explained directly from the state, without chasing flags across systems: remaining cash flows, exercised versus unexercised rights, triggered versus untriggered features.
2. **Client notification.** Coupon advices and margin-call letters are generated mechanically from the state-change stream, since every economically relevant event is an explicit transition.
3. **Traceability.** Any amount or flag in the current state traces back to the exact transaction that created it.

7.7 Lifecycle Risk Profile

Every lifecycle function category is either a pure function or a deterministic projection, and each is covered by named properties (§17). Contract evaluation maps (input, state, conditions) to moves, pure (§6), covered by P1 and the totality oracle. State transition maps (unit, state, event) to (moves, new state), pure (Principle 7.3), covered by P5 and P6 and the valid-transitions oracle. Transaction commit, **applyTx**, is the sole door — registration check, monotone atomic commit — deterministic and the sole mutator, covered by P1 (conservation by construction) and P2 (atomic commitment); no function bypasses it. Views and snapshots are read-only accessors, covered by P8 (snapshot and time-travel consistency). The PnL projection $V_t = \sum w_t(u) \cdot P_t(u)$ is pure, derived from state and prices, covered by P10 (path-independence). Balance computation, $w_t(u)$ from move-stream replay, is a pure projection of the log, covered by P1 and P3 (referential integrity).

7.8 Strategy as Unit: QIS and Leveraged ETFs

A quantitative investment strategy or leveraged ETF is itself a unit. Its state carries the last rebalance timestamp, the current weights or holdings, configuration (target leverage, rebalancing frequency, reset rules), and path-dependent flags (triggered barriers, suspension status). This is strategy-as-unit. The collapse of its holdings sector is governed by condition C12 (§4).

At each rebalance or leverage reset, the lifecycle function computes the trades required to hold the target allocation: moves between the strategy wallet and market wallets. It also computes an updated state recording new weights, reference level, and performance lock-in. Subscriptions and redemptions are lifecycle events. A subscription creates units, triggers the underlying rebalance, and updates composition and leverage. A redemption destroys units and unwinds correspondingly.

Time travel follows the general rule. A view cloned at t reconstructs the strategy unit's state and all past rebalances. Re-applying the rebalance logic with the stored market data reproduces the same transactions. Client-level exposure, leverage, and performance at t are explained directly from the reconstructed state and its moves.

8 The State-Basis Discipline

The State-Basis Discipline stops a valuation from combining a quantity and a price that were measured in different corporate-action states of the same instrument. In this framework the ledger, not the market-data feed, decides which state a unit is in: it reads that state from the unit's own logged history, and it treats a price from a feed as an outside reading to be stamped and then either used or refused.

Every corporate action — a split, a dividend, an index change — rewrites the frame in which a unit's numbers mean anything, and the same number can be right in one state and wrong in another. The rule the discipline enforces is narrow: a quantity and a price may be multiplied only when they are in the same frame, or after one has been converted into the other's frame by the exact rule the action itself published. The discipline does not keep the ledger “in sync” with the feed, and it never lets the feed decide a unit's state; where two numbers are in different frames and no declared conversion connects them, the combination is refused, not guessed.

Here is the error it makes impossible. A wallet holds 1,000 shares at €100, so its marked value is €100,000. The issuer runs a 2-for-1 split: the holding becomes 2,000 shares, and the price reference halves to €50. The value is unchanged, because $2,000 \times 50 = €100,000$. Now suppose a price feed still carries the last pre-split print, €100. A valuation that multiplies the new quantity by the old price computes $2,000 \times 100 = €200,000$ — a phantom €100,000 that never existed. Each factor is correct on its own: 2,000 is the real share count, and €100 was a real trade. Nothing inside the ledger disagrees, so no reconciliation catches it. The one mistake is combining two numbers that belong to different frames.

The frame a unit's numbers are quoted in is called its *basis*. A basis makes two numbers comparable; it does not certify what a holding is worth. A split changes the numbers but not the value; a spin-off changes past numbers precisely because real value has left the unit for the child that was spun off.

Unit state already lives in three homes (§4), and the discipline asks one thing of each; it adds no fourth. *ProductTerms*, the immutable first home, holds the one rounding rule used when an exact price is rounded to a whole number of minor currency units (cents, for euros); because that rule lives in a single place, converting a number between bases stays exact. *UnitStatus*, the shared second home, gains one field: the unit's current basis, which the ledger computes from the unit's own logged corporate-action events. *PositionState*, the third home, holds the balances, and a balance is read in the current basis; a balance and a price are brought together only inside a single valuation, whose basis the ledger fixes.

Observation time cannot stand in for the basis. A feed can lag the moment the ledger marks a unit *ex* — the point at which its price detaches from a distribution such as a dividend — the *ex* date can differ between venues, and a late-announced action can change the frame of a past reading after the fact. The basis is therefore stored explicitly, as a field of the second home, and is never inferred from a clock.

The whole discipline reduces to one rule, stated formally later as Invariant 8.17, *single-basis consumption* (§8.8), and referred to by that name throughout: a valuation runs in one basis, and every balance and every price it uses must be in that basis — either because it already was, or because the ledger converted it there by a declared rule. Each price carries a *stamp* that records which basis it was observed in (§8.4); the check covers every unit named on that stamp, not only the units being valued. The rule is enforced by the types, not checked after the fact: a quantity and a price in different bases cannot be multiplied in a well-typed program, and at the boundary where types no longer reach, the combination is refused rather than guessed (§8.9).

The rest of this section builds the machinery that makes the rule hold: the basis coordinate and the two checks that keep it honest; the declared conversion operators; the door through which outside prices enter and are stamped; the typed read seam; and what happens when a basis is missing. The same rule governs every place a mark is consumed downstream — portfolio valuation (§5), the futures lifecycle (§9), and managed accounts (§10).

8.1 The basis coordinate

A unit’s basis changes only at declared events, never by inference. A boundary event is the logged fact that a unit’s frame changed. It records three things: which unit changed, when the change takes economic effect, and how each kind of stored datum converts across the change. The rule for each datum kind is a declared conversion — the formal conditions and the code also call it an *arrow*. Two sources declare boundary events, and both enter through the log: corporate-action transactions on held units, and external basis notices with no lifecycle leg, such as index divisor maintenance.

Definition 8.1 (Boundary event). *A boundary event on unit u is a logged event carrying the unit, an effective time t_{eff} , a declaration — a finite partial map from datum kinds to operator specifications (§8.2) — and provenance. Boundary events are content-addressed; *bid* denotes the address. The two sources are corporate-action lifecycle transactions on held units (§6) and external basis notices with no lifecycle leg — index divisor maintenance, composition true-ups — entering as logged observation events with a canonical handler.*

The 2-for-1 split enters as one such event: it names the unit u , carries the split’s effective time, and declares **Scale 2** — the quantity factor — for each affected *price-like* datum kind, so a stored price of 100 reads as 50 on the far side. The doubling of the share count travels not as a conversion but as the entitlement moves of the same transaction ($1,000 \rightarrow 2,000$), welded to the declaration by the admission check stated later in this subsection (Principle 8.6); a datum kind the declaration does not mention has no conversion at all and does not cross the boundary. The event’s *bid* is a content address, computed from the event’s content alone: the same event always has the same id, and the id never depends on when the event was booked.

Basis ids name a unit’s frames. The *origin* is the frame a unit starts in at registration. Each boundary event opens a new frame, named by the event’s own id. The *chain* is the unit’s boundary events sorted by when they take effect, not by when they were booked. Two boundaries can take effect on the same day; for that case the attested notice declares an intra-day precedence — a declared rank that says which same-day boundary applies first, required by the same- t_{eff} rule W4 (§8.11) whenever the order matters.

Definition 8.2 (Basis id, chain, effective order, epoch). *A basis id of u is either the origin of u or a boundary id of u . The basis chain of u , relative to a replay view, is the sequence of u ’s*

boundary events known to that view in effective order: the lexicographic total order on

$$(t_{\text{eff}}, \text{prec}, \text{bid}),$$

where *prec* is the intra-day precedence declared in the attested notice (§8.11, W_4) and *bid* is a deterministic final tie-break. The epoch ordinal of a basis id is its position in the chain — a derived, per-view quantity, never stored.

After the split, u has two basis ids: the origin b and the split’s id b' , and the chain holds one event. A back-dated boundary discovered later — one whose effective time precedes the split — inserts *before* the split in the chain. The insertion shifts b' one position later, so its epoch ordinal changes; the id b' itself does not. This is why a stamp is a basis stamp, not a timestamp, and why stamps and state store basis ids, never ordinals: ordinals renumber under retro-effective insertion, while content addresses are stable.

The effective order is total on every log that can arise, so replay is deterministic. Ordering keys on effective time first, then on the precedence the two notices declare; only when both of these coincide does a content id *bid* break the remaining tie. That final tie-break would change the numbers in just one situation: two boundaries on the same unit, at the same effective time and with no declared precedence between them, whose conversions do not commute, so that applying them in the two possible orders would give different results. The ledger does not let the *bid* silently decide that case. The W_4 rule requires the two notices to declare which comes first; if neither does, the colliding notice is refused and the unit is held in *pending-transition* — no conversion crosses it — until the conflict is resolved. So the order in which a boundary was booked never affects the result; only its declared effective position does.

The coordinate lives where every other unit-level status fact lives: as one field of **UnitStatus**, written through the existing status machinery.

Definition 8.3 (The basis field). *A unit’s basis is one field of its status, **usBasis** :: **BasisId**, declared with **UnitStatus** in §4.4. The field is total from registration onward: **defaultStatus** u carries **Origin** (C_5), so every registered unit always has a basis. Only one operation writes it — the **SetBasis** case of the closed **StatusWrite** set — and **applyStatus** applies that write as an outright replacement, never an increment. So the field is last-write-wins and idempotent (P_6), exactly like its sibling status fields. A separate settle mark, **usLastSettle**, records the basis a unit was last settled at; it is dimensionally a **Price**.*

At registration u ’s status carries **usBasis** = b , the origin; committing the split writes **SetBasis** b' , and every later read returns b' . The origin is fixed at registration; no later boundary rewrites it. Re-applying **SetBasis** b' to a status already at b' is the identity, so P_6 is untouched and no counter is stored.

The intended invariant is simple: **usBasis** equals the effective-order tip of the chain as known to the view. The field does not maintain this invariant by itself. Replay folds the log in booking order (§7), while the chain is projected in effective order, and the two orders disagree exactly on retro-effective insertions. Under naked last-write-wins, a back-dated boundary booked after the split would overwrite b' with a mid-chain basis — the coordinate would move backward. The invariant is therefore welded at the door, not assumed.

Principle 8.4 (Tip weld). ***applyTx** admits a transaction carrying **SetBasis** b on unit u only if b is the last element, in the effective order of Definition 8.2, of u ’s chain after insertion of the boundary the transaction itself carries. The condition is checkable at admission from the committed log and the transaction alone.*

For a chain-tip boundary, the common case, the mandated write is the boundary’s own id: the split writes b' . For a retro-effective boundary, the post-insertion tip is the already-committed tip. The mandated write therefore re-asserts the standing tip, still b' ; this is idempotent under

P6, and the coordinate never moves backward: *basis regressed by a late notice* is an unrepresentable ledger state. The inserted boundary changes the mid-chain conversions, but conversions are per-view projections of the log, never stored (Definition 8.8), so every subsequent snapshot recomputes them. Observations stamped in the window repair through re-stamp events (§8.10); moves repair through the existing compensating path. The booking-order status fold and the effective-order chain projection therefore agree on `usBasis` at every log prefix.

A corporate action acts on two planes at once, and the machinery keeps them apart. The split’s quantity leg travels as moves while its price rule travels as the declaration; that separation is general.

Principle 8.5 (Two planes). *Entitlements and elections are per-holder consequences, and they settle as moves: this is the entitlement plane. The quotation plane records what the market decides about the quoted series: each boundary carries exactly one declaration, read identically by every holder, however elaborate the election. Operators live on the quotation plane. A declaration never depends on any holder’s election, and an election never produces a conversion.*

The second weld ties the coordinate to the economics. A basis change and the entitlement moves that make it value-neutral travel in one transaction, and the transaction is admitted only if the two halves agree. In the identity below, f is the quantity factor the declaration carries, and c is the per-share cash the boundary books.

Principle 8.6 (Invariance weld). *A basis-changing corporate action is one atomic transaction (C3) carrying its entitlement moves, its `SetBasis`, and its declaration. `applyTx` admits the transaction only if the booked moves and the declaration jointly satisfy the invariance identity of Theorem 8.11, in the case the notice declares. The uniform case covers every holder-independent declaration — every class of the taxonomy of §8.13 except the elective class — and its check is per holder. Each holder’s quantity legs must realise the map $q \mapsto q \cdot f$: under a split the holder’s units scale by f and no cash moves. Each holder’s booked cash legs must equal $q \cdot c$: under an ordinary dividend the holder’s units stand and a cash leg of $q \cdot c$ is booked. Two refinements to these equalities, cash-in-lieu and withholding, are stated after this principle. The elective case (`CAMergerElective`, §8.13) checks the same identity in aggregate at the confirmed blend; its statement accompanies the worked elective merger (§8.12). Case selection reads declared notice data only — the notice’s class and election structure — never a name or a heuristic. Absent a confirmed elective declaration, the uniform case applies: the door fails closed into the strict per-holder weld.*

Two refinements qualify the uniform equalities. Each is an explicit cash leg of the same transaction, never an adjustment outside it.

- *Cash-in-lieu.* A fractional entitlement is paid as real cash in place of the fraction, as an explicit leg of the same transaction. The quantity and cash equalities then hold up to exactly the legs so declared.
- *Withholding.* Where tax is withheld at source, two cash legs travel in the same transaction: the holder’s net cash leg, and a second leg, booked on that holder’s entitlement, paying the withheld amount to the designated tax wallet. The two sum to $q \cdot c$. Absent an explicit withholding leg, the gross identity applies.

The split declares $f = 2$ and $c = 0$: the weld admits it only with quantity legs taking 1,000 shares to 2,000 and no cash leg. A €2 dividend on 1,000 shares declares $f = 1$ and $c = 2$: the weld admits it only with a cash leg of $1,000 \times 2 = €2,000$. The check needs no market data. It is the same admission idiom as `move`’s positivity gate. Consequence: “positions doubled but basis not advanced,” “basis advanced but declaration missing,” “declaration inconsistent with moves,” and, by the tip weld, “basis regressed by a late notice” are all unrepresentable ledger states.

The uniform case is the weld of Principle 8.6 at full strength, holder by holder: a split that misallocates shares between holders while totals match is refused at the door. An elective

boundary cannot be checked holder by holder, because no single holder’s legs match the blend — that is what an election means. The door therefore checks the quotation-plane fact (Principle 8.5): the blend identity at unit level, stated as the elective aggregate weld with the worked elective merger of §8.12. Per-holder allocation is checked separately, as economic correctness: a wrong allocation is not refused at the door but caught by recomputing the transaction from the log and comparing. Each holder’s legs are compared against that holder’s logged election and the confirmed proration; elections and proration are logged, attested data, so the comparison is recomputable from the log, and differential recomputation (P27) performs it. Each case keeps its own witness: the per-holder weld witness for uniform boundaries, and elective aggregate invariance (P33) for elective ones. Both run, and neither substitutes for the other.

8.2 The adjustment operators and their composition law

Each corporate-action event carries its own adjustment operator as data — a 2-for-1 split carries **Scale 2**, a €2 dividend carries **Shift -2** — and one generic evaluator applies whatever the event declares. The system never derives an adjustment from an event’s name, and never computes one from a model: an event that declares no operator adjusts nothing — the safety clause of the confirmation gate’s pending window (§8.13). An operator acts on every declared field at once, each by the map its dimension fixes (Definition 8.12): under **Scale** f , quantities multiply by f and prices divide by f ; under **Shift** x , prices move by x ; cash and dimensionless values never move. A €2 dividend lowers the price by 2, so it is **Shift -2**: 102 becomes 100, and the matching cash leg pays the 2 to holders.

Definition 8.7 (Operator specification). *A boundary’s declaration is a map that sends each affected datum kind to one operator from the list below. Every parameter comes from the declared terms of the notice, never from the event’s classification: a 5% stock dividend that declares $f = 21/20$ is a scale, whatever its name. The list is closed: extending it is a change to this specification, never a change to data, and a declaration naming an operator outside it is refused at the door.*

```
data OpSpec
  = Scale Rational      -- f /= 0; invertible
  | Shift Rational      -- invertible
  | Subst UnitId Rational -- r /= 0; cross-unit succession, invertible
  | Recompose Sigma     -- invertible iff sigma is declared bijective
  | AId                 -- identity: DECLARED, never assumed
  | Pending BoundaryRef -- boundary known, parameter unpublished: no arrow yet
  | Terminal            -- series ends: no arrow in either direction
  deriving (Eq, Show)
-- Side conditions (non-zero, bijectivity) are enforced by unexported smart
-- constructors at the single admission door (applyTx), the ‘move’ idiom: the
-- invalid spec is never stored, so the evaluator is total over stored values.
```

The map is partial on purpose: a datum kind outside the declaration’s domain has no conversion and cannot cross that boundary, so a declaration covering only the spot kind converts spot and refuses every other kind — the door fails closed rather than guess. **AId** must be declared; it is never a default. All data-plane arithmetic is exact rational. The single crossing to integer minor units is **toPrice**, whose rounding rule is fixed once in *ProductTerms* (round-half-even, one site). Composition of exact maps is therefore path-independent, and caching intermediate values is sound.

A **Recompose** σ whose target names successor units declares one forward conversion onto a jointly-stamped basket of the successors; the basket mechanics are stated with the spin-off worked example (§8.12).

A unit’s bases are its basis ids: the origin, then one new basis after each boundary. The declared operators connect them. Each boundary gives, for each datum kind it declares, one

conversion from the basis before it to the basis after it, and carrying a datum across several boundaries is the composition of those conversions in order.

Definition 8.8 (Basis conversions and their composition). *The bases are the basis ids of all units. For each datum kind k , the declared conversions are these: for each boundary event g with k in the domain of its declaration, one conversion from the basis before g to $\text{bid}(g)$, acting by the interpretation of the declared operator; and for each declared succession (**Subst**, welded to **usSupersededBy**, C8), one cross-unit conversion from the predecessor’s final basis to the successor’s origin. A reverse conversion exists exactly where the declared operator is invertible; **Pending** and **Terminal** give no conversion. Within one unit the chain is totally ordered and successions are unique, so between any two bases on one (possibly succession-extended) chain there is at most one conversion per datum kind.*

Uniqueness of the conversion makes multi-boundary transport a computation, not a choice: the composite across several boundaries is the composition of the boundary conversions, in effective order, and it exists exactly where every one of them does. A **Recompose** targeting successor units adds one cross-unit basket step to this law (§8.12); each successor’s own chain then composes as above.

Proposition 8.9 (Composition law). *For basis ids $b \leq b'$ on the (possibly succession-extended) chain, with intervening boundaries $g_1, \dots, g_n$ in the effective order of Definition 8.2,*

$$A_k(b \rightarrow b') = \text{interp}(g_n) \circ \dots \circ \text{interp}(g_1), \quad \text{defined iff } k \text{ is in every } g_i \text{'s declaration;}$$

$$A_k(b' \rightarrow b) = A_k(b \rightarrow b')^{-1} \quad \text{defined iff every intervening conversion is invertible;}$$

$$A_k(b \rightarrow b) = \text{id}, \quad A_k(b \rightarrow b'') = A_k(b' \rightarrow b'') \circ A_k(b \rightarrow b').$$

The declared operator is written into the log once and consumed twice. The two consumers share no code path and run at different times: the corporate-action processor consumes it once, at the effective time; the read seam consumes it on every later read.

Principle 8.10 (One declaration, two consumers). *A boundary’s declared operator is consumed twice: at the effective time, by the corporate-action processor that produces the boundary’s one-time transactions; and from then on, at the read seam, as the standing conversion applied to every pre-boundary arrival. Neither consumer interprets the event; both apply the same logged declaration.*

The first use is the effective-time work. At t_{eff} the corporate-action processor (§15.2) reads the declaration — for the 2-for-1 split, quantity factor $f = 2$ and no cash — and produces the transactions the door admits: entitlement moves taking 1,000 shares to 2,000; a terms append taking the option series’ strike from €100 to €50 (C6); a terms append taking the performance option’s stored initial spot from €100 to €50. Which fields move, and by what map, is fixed by the unit’s adjustment schedule (C14, Principle 8.15). Each follow-on transaction carries the boundary’s id in **txCause**. That field is provenance, never control: no admission, valuation, or transport decision reads its value. It is necessary — it is what lets an auditor walk from the €50 strike back to the notice that caused it — and it is insufficient, because it names the cause and not the conversion. Audit needs the tag; arithmetic needs the operator.

The second use never ends. Prices do not stop arriving because a split happened. The 09:59 print for a 10:00 split carries a pre-boundary stamp; so do last month’s history pulled next week, a vendor’s late file tonight, and every observation inside a historical view that **clone_at(t)** rebuilds (§8.10). The read seam converts each one by the composite of Proposition 8.9: the declared conversions, in effective order, across however many boundaries the stamp lies behind. Nothing is reprocessed and nothing is configured. A backtest also reads the other way, expressing post-boundary data in an earlier frame, and the reverse conversion exists exactly where the

declared operator is invertible (Definition 8.8): **Scale** and **Shift** always invert; **Recompose** inverts only where its σ is declared bijective; **Pending** and **Terminal** give no conversion in either direction.

This is why the declaration is an operator and not a name. “Stock split” is an open name: consuming it means interpreting it, identically, in every consumer, forever. **Scale 2** is an entry in a closed menu: consuming it means applying it. With a bare tag the second use is impossible, because nothing in the log says divide by 2. The first use is unverifiable, because the log would show a strike of €50 and no longer carry the fact that lets anyone check that €50 is €100 halved.

One June-1 timeline carries both uses. The unit’s chain already holds May’s €2 dividend, **Shift** -2 ($b \rightarrow b_1$). At 09:59 a vendor prints 100, stamped b_1 under the source’s convention on file. At 10:00 the split takes effect ($b_1 \rightarrow b_2$): the processor consumes the declaration once, and the door admits its transaction — 2,000 shares, the **SetBasis**, the two terms appends, each tagged with the boundary’s id. At 10:03 a valuation asks for spot; the seam reads the 09:59 print, one boundary behind, and returns $100/2 = 50$. Next week a history pull fetches May prints stamped at the origin b : a spot of 102 crosses both boundaries, and the composite is fixed by effective order, $(102 - 2)/2 = 50$. The reversed composite, $102/2 - 2 = 49$, is not a valid conversion: no pair of basis ids has it as its unique conversion, and no exported constructor can forge it. Had the dividend and the split shared one t_{eff} , the pair would be inadmissible without a declared mutual precedence; the 49-versus-50 ambiguity is refused at the door, never resolved by arrival order. Tonight’s late vendor file, rows stamped b_1 , converts the same way on read, and so does every later **clone_at**(t). On June 1 exactly one component did one-time work: the processor, once.

The theorem states the economics the invariance weld enforces: crossing a boundary changes the numbers but not the value, except for exactly the cash the boundary itself books. In its statement, f is the declared quantity factor and c is the cash per share the boundary books. The split declares $f = 2$, $c = 0$: a wallet of 1,000 shares at 100 becomes 2,000 at 50, and $2,000 \times 50 = \text{€}100,000$ — value unchanged, no cash booked. The dividend declares $f = 1$, $c = 2$ on 1,000 shares priced at 102: the price becomes 100 and the same transaction books $1,000 \times 2 = \text{€}2,000$ of cash — €102,000 before, €100,000 in shares and €2,000 in cash after.

Theorem 8.11 (Lifecycle value invariance across boundaries). *Let a boundary declare price action $p \mapsto (p - c)/f$ with quantity legs $q \mapsto q \cdot f$ and cash legs $q \cdot c$ (up to explicit cash-in-lieu and withholding legs, each booked in the same transaction), as Principle 8.6 requires. Then*

$$(q \cdot f) \cdot \frac{p - c}{f} = q \cdot p - q \cdot c :$$

marked value is continuous through every boundary up to exactly the cash the same atomic transaction books.

Proof. The identity is algebraic; the weld makes its hypotheses admission conditions, so every committed boundary satisfies them. For an elective boundary the weld’s hypotheses hold in aggregate, and the identity applies to the aggregate position by linearity: $(\sum q \cdot f) \cdot \frac{p - c}{f} = \sum q \cdot p - \sum q \cdot c$, with (f, c) holder-independent because operators live on the quotation plane (Principle 8.5). Fractional entitlements land as explicit cash-in-lieu moves in the same transaction, so the identity holds exactly, down to the last minor unit: any leftover fraction is paid as real cash in the same transaction, and is never lost to rounding. Withholding likewise: the withheld amount is real cash to the tax wallet in the same transaction, so the holder’s net and the withheld part partition $q \cdot c$ exactly — recorded, never leaked. $\square$

For basis boundaries this proves the lifecycle value-invariance property stated in restricted form in §1: the price jump at the boundary exactly offsets the booked cash. The theorem’s scope follows the weld’s two cases: per-position continuity across uniform boundaries, and aggregate

continuity across elective boundaries. Across an elective boundary the per-holder differences are exactly the elected consideration, and that consideration is recorded in the log. On the worked merger of §8.12 the differences are $\pm\text{€}1,000$ per holder and sum to zero.

The theorem speaks of one quantity, one price, and the booked cash. A product’s terms carry more numbers than that: the vanilla call carries a strike and a deliverable per contract; the performance option carries a stored initial spot; a single datum may carry several components, as an implied-vol point carries a strike coordinate and a vol value. Every one of these must move, or stay, at every boundary, and which it does is not a property of the event. It is a property of what the number measures.

Definition 8.12 (Dimension declaration). *Every terms field and every component of a registered datum kind declares exactly one dimension at its definition: quantity-of- u , price-in-basis, cash, or dimensionless. A boundary declaring quantity factor $f \neq 0$ and per-share cash c acts on each field by the map its dimension fixes — the per-dimension action of the table below.*

Dimension	Measures	Action under (f, c)	Examples
quantity-of- u	units of u held or delivered	$q \mapsto q \cdot f$	position; deliverable per contract
price-in-basis	€ per unit of u , in a frame	$p \mapsto (p - c)/f$	spot; strike; stored initial spot
cash	€, in no frame	fixed	booked dividend cash; premium paid
dimensionless	a pure number	fixed	moneyiness; a vol value; a proportion

The dimension is the constructor: the only way to build a terms value is to build it as a quantity, a price, cash, or a dimensionless number, so declaring the dimension is the same act that creates the field. An unclassified field is therefore unconstructible, not merely invalid:

```
data Dim          = DimQuantity | DimPrice | DimCash | DimNone
data TermsValue = TVQty Rational | TVPrice Rational
               | TVCash Rational | TVFree String   -- dimension = constructor

transportField :: Rational -> Rational -> TermsValue -> Maybe TermsValue
-- the per-dimension action: quantities  $\times f$ , prices  $(p-c)/f$ , cash and
-- dimensionless fixed;  $f == 0$  is refused before any field is touched.
```

This extends the scalar discipline of §9, under which quantities and cash add and prices do not. Here the same separation, plus the dimensionless case, becomes a declaration every field must make before any boundary exists to test it. A field with no dimension cannot be written into any terms version, so “the strike was never classified” is not a latent bug. It is a value that cannot be built. The action column is the re-expression map of the declared boundary. Whether a given corporate-action class applies that map to a given terms field is a separate, per-product question; the adjustment schedule of §8.3 answers it, through a per-class default reaction called the *class default*. A quote is not a terms field and faces no such question: it always crosses by the action column, at the ingest seam (§8.4).

The vanilla call shows the action. The split declares $f = 2$, $c = 0$. The strike is price-in-basis, so €100 reads €50; the deliverable is quantity-of- u , so 100 shares read 200. At any post-split spot s the position pays $200 \cdot \max(s - 50, 0)$; at the corresponding pre-split spot $2s$ it paid $100 \cdot \max(2s - 100, 0)$ — the same number, for every s . Value invariance holds field by field, from two declarations and no option-specific rule.

The performance option shows what the declaration buys. It pays on the ratio S_T/S_0 , with the initial spot $S_0 = \text{€}100$ stored as a terms field. S_0 measures euros per share in a frame, so it is declared price-in-basis, and the split takes it to €50; spot observations cross by the same map. A post-split valuation re-derives the ratio from transported inputs and gets $(S_T/2)/(S_0/2) = S_T/S_0$: invariant automatically. No rule was ever written for performance options. The field declared what it measures, and the dimension did the rest. The dimension

settles the split; the ordinary dividend falls to the class default. Under an ordinary dividend the class default (§8.3) holds S_0 fixed. A performance confirmation fixes S_0 at inception, and holding it fixed on an ordinary dividend keeps exactly that contract term. The ex-date price drop then shows up in the ratio through fresh prints of S_T , never through any change to S_0 .

Theorem 8.13 (Dimension invariance across boundaries). *Let a boundary declare quantity factor $f \neq 0$ and per-share cash c , as Principle 8.6 requires. Under the per-dimension action, in the same boundary, each price-dimension value maps by $p \mapsto (p - c)/f$, each quantity-dimension value maps by $q \mapsto q \cdot f$, and cash and dimensionless values are fixed. Hence for every quantity–price pair a product’s terms and data put together,*

$$(q \cdot f) \cdot \frac{p - c}{f} = q \cdot p - q \cdot c :$$

the identity of Theorem 8.11 holds field by field, for every product, at every boundary on the closed operator menu.

Proof. The identity is algebraic in (f, c) and never mentions which field carries q or which carries p . The action is fixed by the dimension alone, the dimensions are a closed four-element set, and every field carries one by construction. The proof therefore quantifies over four dimensions, once — never over products or event classes. A product registered next year adds fields, not cases. $\square$

The theorem moves each value by the dimension it declares; it does not judge whether the declaration is true. For an observation — a datum-kind entry — the correctness of the declaration is a registration fact. It is fixed at the point where the kind’s dimension is declared, and the per-entry invariance witness the entry carries checks it there (§8.6). If a declared dimension is wrong, the theorem still applies that dimension’s map, and the resulting settlement pays the wrong amount. That mispayment is what the invariance witness, not this theorem, is built to catch.

The declarations relocate the expert work; they do not eliminate it. Someone who knows the product still decides, field by field, what each number measures, and that judgement is as hard as it ever was. But it is made once per field, at definition, as a declaration — not once per product per event class, as adjustment code. The relocation changes the count: N products crossing M event classes need N sets of field declarations and one class-default table of constant size, declared once in this specification — not $N \times M$ hand-written cases. Totality is completeness, not correctness. A schedule total in the sense of §8.3 — one entry for every pair of a class and a field — answers every such pair from its own declarations, and it answers wrongly wherever a declaration is wrong. Differential recomputation (P27) checks that the schedule was applied faithfully; it does not check that the convention the schedule encodes is the right one. That is why the default table is fixed in this specification and witnessed, rather than left to each product to remember.

Pending crystallisation. Some corporate actions become effective before their parameter is published: the boundary is known, the number is not. A **Pending** boundary advances the basis id at effectiveness. Its moves and its **SetBasis** commit in one atomic transaction (C3). The parameter arrives later as a parameter-publication event referencing the boundary id. That event is a versioned resolution under the *ProductTerms* append discipline (C6). The chain projection then resolves each boundary to its latest parameter version. No committed element mutates and the epoch does not move, because a publication is not a boundary. Until publication no conversion crosses that boundary for the affected kinds, so quarantine is forced (§8.11). The quarantine is correct, because any interim mark is a guess.

Transport applies only to what the ledger observed; what the ledger computed is recomputed at the target basis, never transported.

Principle 8.14 (Re-derivation canon). *Primitive data are transported by declared conversions; derived data are re-derived from transported inputs.*

The framework never assumes that a derivation commutes with adjustment: whether it does is a fact about the model that produced the datum, and the model-free directive — the rule that the ledger assumes nothing about a pricing model’s internals — forbids depending on it. The index divisor shows why. An index I has constituents A at 60 and B at 84, and its published level $(60 + 84)/D = 100$ fixes the divisor $D = 1.44$. B splits 2-for-1, and fresh prices arrive: A at 60, B at 42. Consuming the stale divisor gives $(60 + 42)/1.44 \approx 70.83$, a phantom -29.2% move built from individually correct inputs. The ledger refuses it. The divisor is derived, not observed: it was computed from A ’s and B ’s prices, so it means something only while those prices sit in the bases it was built from. Once B splits, one of those bases has moved and the stored divisor no longer matches, so it is refused rather than reused. The formal check, which ranges over every unit the divisor’s stamp names, is Invariant 8.17(ii) (§8.8). The repair is a maintenance boundary declaring **Recompose** with $D' = 1.02$, after which $(60 + 42)/1.02 = 100$: the divisor is re-derived, never transported. A derivation performed inside a snapshot scope consumes transported primitive inputs, so its result is in that basis as well: the types carry the basis through ordinary computation, with no separate check (§8.9).

8.3 The adjustment schedule

The operators of §8.2 say how a declared boundary converts stored data. One question they leave open: for *this* product, what does each class of event do to each of its terms? A vanilla call and a performance option both face the same 2-for-1 split, and their strikes react by different rules. That answer is product data, and it is demanded where all product data is laid down: every unit’s *ProductTerms* carries, from its first version at registration, an *adjustment schedule* — for every corporate-action class (**CAClass**, §8.13) and every terms field, exactly one entry stating how that field reacts. An entry comes from one of three layers.

Class default.

Every terms field already declares its dimension, and this specification declares once, for every pair of a corporate-action class and a dimension, the default reaction: quantities scale by f ; cash and dimensionless fields stand; price-in-basis fields move to $(p - c)/f$ under every class except **CADividendOrdinary**, where they stand.² That one exception follows from the invariance arithmetic itself, not from market custom; its derivation follows this list. Quotes are not terms fields; they cross by the declared operator at the ingest seam (§8.4), so an old cum print still reads $p - c$ past the ex-date. A term sheet that deviates declares an override. The table is written out over the closed **CAClass**: a new class re-opens it entry by entry, never through a wildcard.

Declared override.

Where the term sheet deviates from the default, the deviation is written as a rule in a closed, first-order language: a rule reads the boundary’s declared parameters and the field’s current value, combines them with arithmetic and comparison, and chooses by condition. The language has no recursion and no partial operation, so every well-formed rule evaluates to exactly one value. Widening the language is a revision of this specification, never a property of data, and the totality gate C14 below is re-run under the revised language. Example: a term sheet says a special dividend adjusts the strike only above €0.50, and only by the excess. The rule reads the declared per-share cash c and sets the strike to $K - (c - 0.50)$ when $c > 0.50$, else leaves it. A €2 special dividend takes a €100 strike to €98.50, where the class default would give €98 — the override exists precisely because the confirmation says €98.50.

²In the reference implementation the table is **classDefault**; its price-dimension column is **priceTermsDefault**.

Designated discretion.

Where the confirmation empowers a party — a calculation agent, an exchange — to determine the adjusted value, no formula exists to write down. The schedule then declares *who owes the determination*, and nothing else. The determination later enters the log as *resolved terms*: a W4-attested terms append referencing the boundary, transcribed from the designated party’s decision. Resolved terms are the second admissible source of adjusted terms; the first is the schedule’s evaluator — the generic component of §8.2 that applies declared entries — and the principle below closes the pair. Until the determination lands, the affected fields have no adjusted value. The boundary is **Pending** for them and consumption quarantines: the pending-crystallisation discipline of §8.2, extended from a parameter no one has published to a determination no one has yet made.

The **CADividendOrdinary** exception derives in three steps. First, the $-c$ shift is the value-invariance re-expression of Theorem 8.11, and it is valid only when the same transaction that shifts the field also pays this field’s holder $q \cdot c$. Second, an ordinary dividend pays the shareholder, not the option holder, and an anticipated distribution is already priced into every contractual reference struck against the underlying — a strike, a stored initial spot. Third, shifting such a field on an ordinary dividend would therefore hand its holder value the boundary has already booked to the shareholder — an unearned transfer. So price-in-basis terms fields stand under **CADividendOrdinary**. In numbers: a €100-strike call crosses an ordinary €2 dividend. The wrong reaction re-expresses the strike to €98; the correct one leaves it at €100. The €2 of option value the wrong shift would create is exactly the €2 per share the boundary pays the shareholder — the same cash booked twice if the strike moved. Listed and OTC convention — ordinary distributions adjust no strike; splits, specials, and successions do — is thus the arithmetic’s own answer, declared here once for every product.

Principle 8.15 (Schedule authority). *Adjusted terms enter the log from exactly two sources: the schedule’s evaluator applying the unit’s declared entry, and the attested resolved terms of the party the schedule designates. No other source of adjusted terms exists.*

The schedule is total, and totality is checked where the schedule is written. The condition below joins the register of §4.5:

C14 (adjustment-schedule totality).

Registration, and every terms append, is admitted only if the schedule yields exactly one well-formed entry for every pair of a corporate-action class and a field of the version being written; an append that introduces a field extends the schedule to cover it in the same transaction. The refusal is **ScheduleIncomplete**.

The gate quantifies over the closed **CAClass**, so it is a finite check. It is also strict: an ill-formed declared override is refused; it never falls through to the default. The consequence is that “the framework does not know how to handle this event for this product” is unrepresentable past inception. For every event class and every field, a registered unit’s terms state the answer, the formula that produces it, or the party who owes it.

Adjustment never edits terms. The evaluator’s output is a new terms version appended under C6, carrying the boundary as its cause, so replay re-runs the evaluator against the logged notices and re-derives every historical strike.

The schedule governs one plane only: how the unit’s single set of terms reacts. The per-holder entitlement moves of the same transaction are the other plane, checked by the invariance weld (Principle 8.6).

One duty falls on schedule authors, and it is per class. A listed series declares the exchange as designated party for exactly the classes on which the exchange renders adjustment decisions — special distributions, spin-offs, elective and prorated events — and keeps defaults and overrides

for the rest. An ordinary dividend adjusts nothing and the exchange publishes nothing, so a wholesale designation would hold the series **Pending** on every ex-date for a determination no one owes. The schedule is keyed by (class, field); a designation is an entry, never a blanket.

8.4 The stamp and the ingest door

A raw vendor number becomes a ledger observation in exactly one way: it passes through one total ingest function. This is the parse-boundary idiom of **move** — validate at the door, so that everything past the door is already well formed.

Two terms the door depends on are fixed now. A datum kind is *registered* when a registration event has declared one dimension for each of its components (§8.6). An unregistered kind is unconsumable: the door refuses it outright.

```
ingest :: Ledger -> SourceId -> Timestamp -> RawDatum
        -> Either IngestError StampedObs
data StampedObs    -- ABSTRACT: constructor unexported; ingest is the only door.
    -- Carries: value (exact), t_obs, source, and stamp :: Map UnitId BasisId,
    -- inside the attestation envelope (provider key, signature, content address).
```

The stamp is assigned per (unit, source) pair. A source can lag the unit: after the 2-for-1 split the unit’s chain has advanced to the post-split basis, yet a source may go on publishing pre-split prices for a while — the ex-transition lag of §D, whose one-bit quote-ex flag and hard-coded one-step adjustment are the special case of a chain with a single boundary. The lag is recorded as a per-source basis offset, from one of two places: transcribed from the source’s declared convention under signature, or asserted by the gateway against the committed corporate-action log. The stamp is never defaulted from the ledger’s prevailing state — the door never assumes a value is in the current frame merely because it arrived now. **ingest** also refuses a stamp naming an unregistered unit, so the condition that every stamped unit be registered (Invariant 8.17) is discharged at the door, never at consumption. Sourcing remains the market-data satellite’s concern; the stamp and the coherence check are ledger-owned.

The stamp assignment rests on a named trust assumption:

TA-BASIS.

Owner: market-data operations. *Content:* the per-(unit, source) basis stamp assigned at ingest is true of the source’s dissemination. *Violation consequence:* mis-based valuation. *Detection:* W3 partition quarantine (§8.11) — a large innovation coinciding with no committed boundary quarantines the series — and daily reconciliation of vendor-published adjustment factors against applied declarations.

8.5 The market-data contract

The boundary chain has one author: the ledger. Every component outside the ledger reads the chain and never writes it. The contract below keeps that so.

The ledger publishes one read-only value, the basis projection: a versioned map from each registered unit to its prevailing basis id, with the unit’s committed boundary chain behind it. The market-data layer holds that value and presents every raw number at the single ingest door (§8.4). There the ledger attaches the stamp — the record of which frame the value inhabits — from the pinned projection and the source’s declared convention (Figure 6). Two flows and nothing else: the projection flows out, observations flow in, and no basis coordinate is authored anywhere but inside the ledger.

The market-data layer is deliberately passive — no per-datum query, no callback, no re-implemented corporate-action arithmetic, which the arithmetic-free projection makes impossible even by mistake; the pinned view is an immutable value, so a boundary committing mid-run

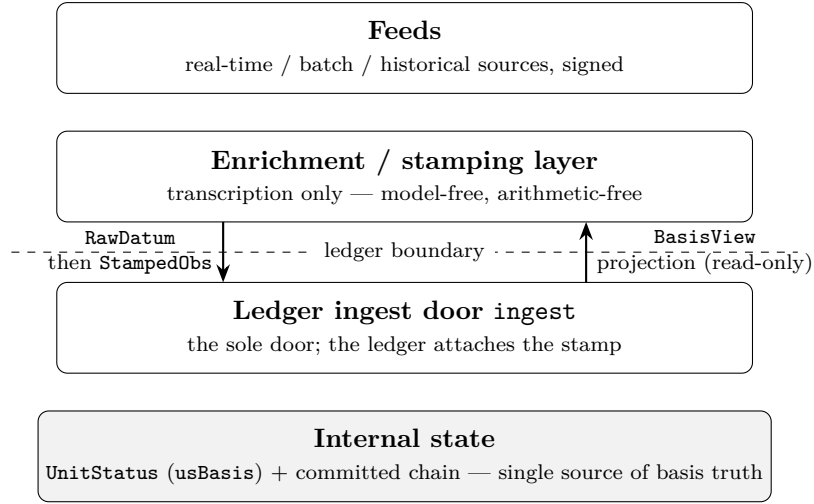


Figure 6: The four layers as an architecture. External feeds and the enrichment/stamping layer sit outside the ledger; the ingest door and the internal state sit inside. The single source of basis truth is the committed boundary chain, materialised in `UnitStatus` at the base (§8.4, Principle 8.16); every layer above reads a derived, read-only projection of it and none authors, defaults, or adjusts a basis coordinate. Only two arrows cross the ledger boundary: raw observations descend through `ingest` and return as stamped, while the `BasisView` projection ascends.

changes nothing the layer holds, and a source lagging the chain — the pre-split publisher above — is not an error to be chased down but data in the convention on file, stamped as such.

Principle 8.16 (Stamping authority). *The ledger’s committed boundary chain is the single source of basis truth: the ingestion layer stamps by consulting a derived, read-only projection of that chain, never by re-implementing corporate-action logic, and no component outside the ledger authors, defaults, or adjusts a basis coordinate.*

The basis projection. One interface serves every ingestion layer — in-process, gateway, batch loader — and diagnostics. `basisView` projects a `BasisView` value from the ledger. `viewAsOf` constructs the view as of any log position (the P8 machinery). `betaAt` returns a unit’s basis id; it returns `Nothing` exactly when the unit is unregistered, so the door’s refusal is a pattern match that cannot be forgotten. `chainAt` returns the unit’s boundary chain, ascending in the effective order of Definition 8.2. `onChain` answers whether a claimed basis id is a committed point of the unit’s chain. `kindAt` is the registry’s `betaAt`: it returns a registered datum kind’s entry from the projection of §8.6, pinned by the same view version. Each view carries its version: the as-of log position. Nothing else is exposed.

```

data BasisView      -- ABSTRACT: a READ-ONLY projection of the log, same rank as
                    -- UnitStatus -- derivable, discardable, never authoritative.
                    -- Immutable once constructed; monotone under append (P24
                    -- corollary: the tip never regresses); arithmetic-free -- no
                    -- operator kinds, no parameters, no arrows are exposed.

basisView    :: Ledger -> BasisView                -- the projection
viewAsOf     :: [Transaction] -> Timestamp -> BasisView -- the P8 clone machinery
viewVersion  :: BasisView -> ViewVersion           -- the pin a stamp
              records

-- / Nothing = the unit is not registered (the door refuses, 05); Just b = the
--   prevailing basis (replayed usBasis; by P24, the committed chain’s tip).

```

```

betaAt :: BasisView -> UnitId -> Maybe BasisId

-- / The unit's committed chain, ascending in the effective order (t_eff, prec,
--   bid) -- the read a re-stamp repair or a backward transport pins against.
chainAt :: BasisView -> UnitId -> [(Timestamp, Integer, BoundaryId)]

-- / The admission predicate: is b a committed point of u's chain? The origin
--   of a REGISTERED unit is the chain's base point; a boundary id is on-chain
--   iff the chain carries it. A claim naming anything else is refused
--   (OffChainClaim) -- in particular a pro-forma claim on a not-yet-committed
--   boundary, which is re-presented after the boundary commits at effectiveness.
onChain :: BasisView -> UnitId -> BasisId -> Bool

-- / The registry's betaAt (sec:basis-kinds): Nothing = the kind is not in the
--   pinned vocabulary, and the door refuses it as its FIRST clause.
kindAt :: BasisView -> KindId -> Maybe DatumKind

data Convention
  = TracksChain          -- caught up: the value is in the chain position at t_obs
  | LagsBy Integer       -- still publishing k boundaries behind that position
  | PreAdjusts           -- adjusts at commit: the value is at the pinned view's
    tip
  | DeclaredAt BasisId   -- file declared adjusted-through a named basis (06);
    -- the door checks the claim against the chain
  | NoClaim              -- nothing on file: refused, quarantined (05)
  deriving (Eq, Show)

data RawDatum = RawDatum
  { rdUnitRef :: String          -- provider's unit reference (01)
  , rdValue   :: Integer         -- exact disseminated value (01)
  , rdConv    :: (ConvVersion, Convention) -- convention consulted (02/04)
  , rdKind    :: KindId          -- the claimed datum kind: a NAME
    resolved
    -- against the registry at the door,
    never
    -- a declaration -- the declaration
    lives
    -- on the registered entry (sec:
    basis-kinds)
  } deriving (Eq, Show)

data StampedObs -- ABSTRACT: constructor unexported; ingest is the only door.
  -- Carries value, t_obs, source, stamp :: Map UnitId BasisId
  -- (singleton for a plain observation), and the envelope: the
  -- projection version and convention version consulted (02),
  -- so every stamp replays as a pure function of recorded data.

-- The Left branch IS the quarantine: refused with payload retained, never
-- given a guessed basis, never defaulted from the ledger's prevailing state.
data IngestError
  = UnregisteredUnit UnitId RawDatum -- unresolvable reference: refused
    outright
  | UndeterminedBasis UnitId RawDatum -- no convention on file:
    quarantined
  | OffChainClaim UnitId BasisId RawDatum -- claim names no committed chain
    point
  | UnregisteredKind KindId RawDatum -- claimed kind not in the pinned
    registry

```

```

-- (sec:basis-kinds): the error
names the
guard is
deriving (Eq, Show)

retained :: IngestError -> RawDatum

-- / Stamp one raw observation against one pinned view. Total; deterministic in
-- its visible arguments (pDet). The door refuses (Left, payload retained)
-- rather than guess.
ingestAt :: BasisView -> SourceId -> Timestamp -> RawDatum
        -> Either IngestError StampedObs

-- / The published door: the sole path from a raw vendor number to a
-- consumable observation -- the parse-boundary idiom of 'move'. The
-- market-data layer links against 'ingestAt' only: the Ledger value never
-- crosses the boundary; the projection does.
ingest :: Ledger -> SourceId -> Timestamp -> RawDatum
        -> Either IngestError StampedObs
ingest l = ingestAt (basisView l)

```

The exhibit is drawn from the reference (`reference/Ledger.hs`, Part M). The signatures, `Convention`, `RawDatum`, `IngestError`, and the `ingest` factoring are verbatim, under the code-level claim of §4.9: tokens exact, comments and layout re-set for the page. `BasisView` and `StampedObs` appear in interface form — declaration plus stated contract, constructors withheld, the idiom of §8.4 — because the reference exports neither constructor. For an abstract type the interface is the specification, and the record body is implementation.

`BasisView` is a projection of the log like every other: it has the rank of `unitStatus`, under the cache discipline of §4.3 — derivable, discardable, never authoritative. Three of its facts carry the contract. First, it is an immutable value, so a stamp computed from it is a pure function of its arguments. Second, it is monotone under append, a corollary of P24: the tip never regresses, so an on-chain stamp stays on-chain under every later view. Third, it is arithmetic-free: it exposes no operator kinds, no parameters, no conversions.

The published door keeps the signature of §8.4, factoring as `ingest l = ingestAt (basisView l)`. The market-data layer links against `ingestAt` alone, so the `Ledger` value never crosses the boundary. Three facts fix the door's shape: `ingestAt` is a pure, total function of a value argument; `BasisView` has no mutators; and `StampedObs` exports no constructor. The `Left` branch is the quarantine: `IngestError` is `UnregisteredKind`, `UnregisteredUnit`, `UndeterminedBasis`, or `OffChainClaim` — the door's refusal order — each retaining the refused payload, and the quarantine store is the accumulation of the `Lefts`, inspected under the W3 workflow (§8.11). *Consumable* is therefore a type fact: an observation is consumable exactly when it is in the `Right` image of `ingest`. No unstamped observation is consumable, by that fact together with door soundness (P25, below) and C13 at the seam. The door is idempotent by content address: a duplicate raw datum returns the existing `StampedObs`. This is the same idempotence P6 gives moves, here at the observation plane.

Direction of flow. Exactly two data flows cross the market-data boundary (Figure 7). The basis projection flows out: read-only, versioned, a view of the log. Raw observations flow in through `ingest`, the sole door, where the ledger itself attaches the stamp. The market-data layer never writes ledger state, and the ledger never accepts a basis it did not stamp or verify against its own committed chain. The traffic is pull-based: the layer pulls the projection when it opens a run and pushes observations through the door, and there is no per-datum traffic in

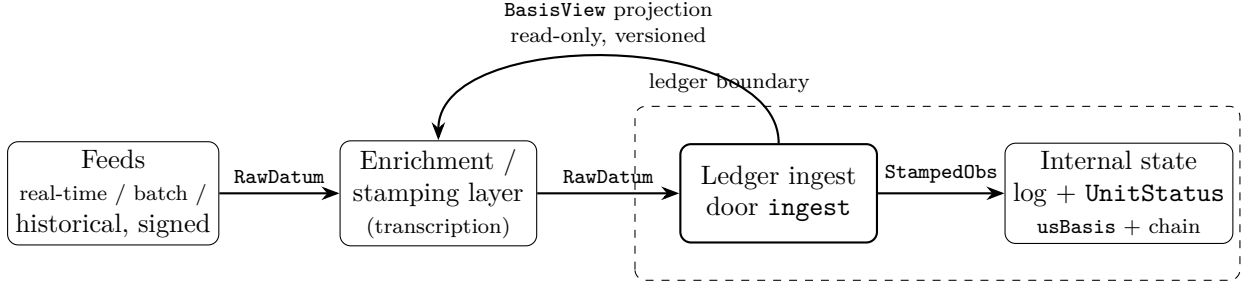


Figure 7: The market-data contract as a data flow. Real-time and batch feeds hand raw vendor numbers to the external enrichment/stamping layer, whose whole competence is transcription—of values, of times, of conventions—never adjustment. Exactly two arrows cross the ledger boundary (§8.5, Principle 8.16): the read-only, versioned **BasisView** projection flows out, and raw observations flow in through **ingest**, the sole door, where the ledger itself attaches the basis stamp and commits the accepted observation as a logged event; the stamp is checked against **UnitStatus** (**usBasis**, Definition 8.3) over the committed boundary chain. The market-data layer never writes ledger state; the ledger never accepts a basis it did not stamp or verify against its own committed chain.

either direction.

The obligation set.

O1 (raw observation).

A raw observation carries exactly: the value, the observation time t_{obs} , a signed source id, and a unit reference resolvable to a registered **UnitId**. Nothing more is demanded of a provider. t_{obs} is attested data; it is never compared against a wall clock to decide semantics.

O2 (the stamp).

The ledger attaches the stamp at ingestion, from the pinned basis projection and the source basis convention (O4); the attestation envelope records the projection version and the convention version consulted. Every stamp therefore replays as a pure function of recorded data: same projection version, convention version, source, t_{obs} , and raw datum give a bit-identical **StampedObs**.

O3 (no corporate-action awareness).

A provider is not required to track corporate actions: a pre-adjusting source and a lagging source are both representable, because the stamp states which basis the value is in — the quote-ex flag of §D generalised from one bit to the coordinate.

O4 (source basis convention).

Each (unit, source) pair has a *source basis convention* on file: how the source’s dissemination relates to the chain — tracks it, lags by a declared offset, pre-adjusts, declares a fixed basis, or makes no claim. Conventions enter as logged observation events under the canonical-handler idiom of Definition 8.1: versioned, as-of-queryable, no new state home. Owner: market-data operations, under TA-BASIS (§8.4).

O5 (fail-closed).

A datum whose basis the convention and the pinned view cannot determine is refused with payload retained — never given a guessed basis, never defaulted from the ledger’s prevailing state; an unresolvable or unregistered unit reference is refused outright (§8.4).

O6 (back-adjusted files).

A historical series pulled as a back-adjusted file is stamped from the file’s declared adjusted-through convention, never from the basis prevailing at each row’s observation time.

O7 (signature).

Feeds are signed; an unsigned feed enters only through a gateway that counter-signs under a named key, carrying its own scoped trust assumption.

O8 (corrections).

A vendor value correction is a new observation, superseding by (source, unit, t_{obs}); a duplicate content address is idempotent at the door; basis re-coordination is exclusively the ledger’s re-stamp event with lineage (§8.10).

Primitive and derived. A datum is *derived* exactly when it is computed inside a snapshot scope from ledger-stamped inputs; it carries the joint stamp of its inputs and is never transported (Principle 8.14). A value computed outside the ledger from data the ledger never stamped — a provider-published index level, a vendor-supplied curve — is a *primitive* observation of the publisher’s unit, with a singleton stamp; the publisher’s rebalance and divisor events enter as W4 basis notices on that unit. The dividing line is that authority follows computation: the log has no authority over arithmetic it did not perform, and a joint stamp on a provider-computed composite would attest to nothing, because the ledger never saw the inputs.

The per-kind run of this contract — for each registered datum kind, what the stamp covers and what fail-closed means — is the table of §8.6, whose registry fixes each kind’s component dimensions.

One contract, three modes. A real-time feed, an end-of-day batch, and a historical pull are three modes of arrival, and all three face the same door under the same contract: mode is not among the stamp’s arguments. One ingestion run pins one projection version. Within the run, each stamp is computed by the source basis convention from the datum and its t_{obs} , against the pinned chain. t_{obs} is an input the convention may use; it is never the coordinate itself. Where the convention names the basis directly — a pre-adjusting source, or a file declared adjusted-through (O6) — that declaration overrides any positional reading off the chain at t_{obs} , and the stamp is never defaulted from the basis prevailing at arrival (§8.4). A boundary committing mid-run cannot tear the run, because the run’s view is one immutable value. Batch and real-time therefore agree by construction: the stamp is a pure function of projection version, convention version, source, t_{obs} , and raw datum. Multiple vendors are the same door, one per `SourceId`. Two vendors disagreeing on value within one basis partition is a data-quality question, outside this contract; two vendors disagreeing on basis is two different stamps, resolved by W3’s partitions (§8.11), across which no aggregate can be computed.

Two consumers across one boundary. The 2-for-1 split is effective at the market open. The stored pre-split print, 100, is stamped b_3 (the unit’s basis before the split; the subscript is just its position in this unit’s boundary chain, which began before the split). Consumer A calls `withSnapshot` — the operation that opens a read scope fixed to one basis assignment, defined fully at the typed seam (§8.9) — at 09:29. In A’s scope $\beta(u) = b_3$: the print is consumed as stored, and the mark is $1,000 \times 100 = \text{€}100,000$. Consumer B opens its scope at 09:31. In B’s scope $\beta(u) = b_4$: the same stored observation is transported along the unique declared conversion to 50, and the mark is $2,000 \times 50 = \text{€}100,000$. One stored series, read from opposite sides of the boundary, gives two correct marks, and the totals agree by Theorem 8.11 with per-share cash $c = 0$. The phantom $2,000 \times 100$ can be formed in neither scope: A’s snapshot holds 1,000 shares, and in B’s scope the raw 100 has no way in — `snapPrice` and `transport`

yield 50 or refuse. Neither consumer knows the split’s terms; each knows only its own β and the door.

No corporate-action logic exists outside the ledger, and none can be built there: the projection is arithmetic-free, so an external component does not hold the information adjustment would need. Adjustment conversions are declared once, in the log, and applied by the one evaluator. The ingestion layer’s whole competence is transcription — of values, of times, of conventions — never adjustment.

Boundary faults. The consequence classes are the four regimes of §8.11. The table below lists the fault scenarios that arise at this boundary and where each one discharges.

Fault	Discharge
F1 Late boundary notice	Tip weld (Principle 8.4); re-stamp events with lineage (§8.10); W3 detects and W1 blocks in the window.
F2 Duplicated notice	Content address; SetBasis idempotent (P6).
F3 Reordered notices	Every admissible booking permutation yields the identical chain projection — the tip weld again; property form P-PERM-N.
F4 Omitted notice	W3 innovation-without-boundary quarantine — detection, not guarantee; the window is fail-closed.
F5 Duplicated observation	The door is idempotent by content address and returns the existing StampedObs .
F6 Reordered or late observations	Valuation is invariant under ingestion-order permutation at fixed pins; property form P-PERM-O.
F7 Omitted observation	snapPrice returns Left ; Unpriced enumerated (W2).
F8 Clock skew	t_{obs} is data (O1); the only clock read is the W1 staleness check, whose threshold lives in <i>ProductTerms</i> and whose clock is an explicit invocation argument, never ambient.
F9 Crash mid-ingestion	A StampedObs exists only as a logged event: a crash before append is nothing-happened, resubmission dedups (F5), and recovery reaches a committed prefix; property form P-CRASH.
F10 Multi-vendor basis disagreement	W3 partitions by stamped basis; no cross-partition aggregate is representable — the property form of the W3 rule.

Property forms. The boundary contributes nine property forms, each stated once.

P25, door soundness.

On **Right**, every stamped unit is registered and every stamped basis id is on the unit’s chain; on **Left**, nothing is admitted and the payload is retained.

P-DET, stamp determinism.

The stamp is a pure function of recorded data — the property form of O2.

P-MODE, mode equivalence.

Mode is not an argument of the stamp: real-time, batch, and historical arrival produce the identical stamp from identical inputs.

P-PERM-N, P-PERM-O.

Permutation invariance: booking the same notices in any admissible order yields the identical chain, and ingesting the same observations in any order at fixed pins yields the identical valuations.

P-CRASH.

Recovery reaches a committed prefix.

P-REPRO.

Same log, same stamped observations, same pinned chain version — bit-identical valuations.

P-CLONE-STAMP.

The as-known replay reproduces the honest mistake; the corrected replay applies the re-stamps; both replays are deterministic (§8.10).

P-LAG, lag arithmetic.

A `LagsBy k` stamp sits exactly k effective-order steps behind the caught-up position at the same t_{obs} , floored at the origin. This catches an off-by-one that door soundness cannot see, because any on-chain id is sound.

The executable forms follow, verbatim from the reference (`reference/Ledger.hs`, Part M), in the catalogue shape of Appendix B. A claim about a single invocation (P25) is written as a property, following P1 and P2; a claim that quantifies over a whole log or feed is written as a boolean oracle — a function that returns true exactly when the property holds — following P8 and P24.

```
-- P25 Ingest-door soundness. Run o = ingest l s t rd. On Right: every stamped
-- unit is registered and every stamped id is a committed point of that unit's
-- chain (origin included) -- nothing enters the consumable plane claiming a
-- basis the ledger has not committed. On Left: nothing is admitted and the
-- payload is retained verbatim -- the Left IS the quarantine. Wrongful-
-- rejection is the vacuous half, as in P1/P2. The no-side-door half is the
-- type fact stated at StampedObs; the consumption half is C13 at withSnapshot.
p25 :: Property (SourceId, Timestamp, RawDatum) (Either IngestError StampedObs)
p25 = Property (\_ _ -> True) $ \l (_, _, rd) o -> case o of
  Left e   -> retained e == rd
  Right so -> all (\(u, b) -> registered l u && onChain (basisView l) u b)
    (Map.toList (obsStamp so))

-- P-DET Stamp determinism, restated as the purity witness (the pureL
-- precedent): the stamp is a total pure function of (projection version,
-- convention version, source, t_obs, raw) and of NOTHING else -- every input
-- is a value argument; no clock, no ambient log read.
pDet :: BasisView -> SourceId -> Timestamp -> RawDatum -> Bool
pDet v s t rd = ingestAt v s t rd == ingestAt v s t rd

-- P-MODE Mode equivalence -- the executable shadow of a one-sentence theorem:
-- the stamp is a pure function of the pinned view and the datum, so arrival
-- MODE (tick-by-tick, end-of-day file, resumed run) cannot enter the result.
-- One datum through the door equals the same datum inside any batch at the
-- same pin; a boundary committing mid-run cannot tear the run, because the
-- run never re-reads the ledger.
pMode :: BasisView -> [(SourceId, Timestamp, RawDatum)] -> Bool
pMode v feed =
  fst (ingestRun v feed)
    == Set.fromList [ so | (s, t, rd) <- feed, Right so <- [ingestAt v s t rd] ]
```



```

-- P-PERM-N Notice-order irrelevance, strong form: the effective order is data
-- in the notices ((t_eff, prec, bid)), so booking order cannot enter the chain
-- projection or the tip. A naive permutation of a committed log is mostly
-- REFUSED by the tip weld -- each SetBasis re-assertion is derived for its own
-- booking position -- and a refused replay asserts nothing (P2's half), so the
-- naive trial is vacuous almost always. The oracle therefore re-derives what
-- is derived and permutes only what is data: the boundary NOTICES are permuted
-- among their own positions (registrations stay put, permuteBoundariesBy) and
-- each SetBasis is re-asserted to the post-insertion tip of the NEW order
-- (rebook), exactly as genChain mints it. Every permutation of a generated
-- chain then replays Right, and the trial is non-vacuous by construction; the
-- vacuous branch remains only for adversarial inputs, where refusal is P2's
-- half, as before.
pPermN :: [Transaction] -> [Int] -> Bool
pPermN txs perm =
  case ( replay txs emptyLedger
        , replay (rebook (permuteBoundariesBy perm txs)) emptyLedger ) of
    (Right a, Right b) ->
      all (\u -> chainAt (basisView a) u == chainAt (basisView b) u
          && betaAt (basisView a) u == betaAt (basisView b) u)
        (map txUnit txs)
    _ -> True

-- P-PERM-O Observation-order irrelevance: at a fixed pin, the committed set of
-- a run is invariant under any permutation of arrivals -- each datum's stamp
-- is a function of the pin and the datum, never of its neighbours. (The
-- quarantine LIST keeps arrival order and is not claimed; the committed SET is.)
pPermO :: BasisView -> [(SourceId, Timestamp, RawDatum)] -> [Int] -> Bool
pPermO v feed perm =
  fst (ingestRun v feed) == fst (ingestRun v (permuteBy perm feed))

-- P-CRASH Crash recovery: a run interrupted after k data leaves exactly its
-- committed prefix -- a StampedObs exists only as an appended event, so
-- crash-before-append is nothing-happened -- and resubmitting the WHOLE feed
-- against the SAME pin dedups by content address and lands on the
-- uninterrupted run's committed set. Every crash point recovers to a committed
-- prefix, and then to the run itself.
pCrash :: BasisView -> [(SourceId, Timestamp, RawDatum)] -> Int -> Bool
pCrash v feed k =
  fst (ingestResume v (fst (ingestRun v (take k feed))) feed)
    == fst (ingestRun v feed)

-- P-REPRO Reproducibility (as-of stability): the view as-of booking position c
-- is unchanged by anything booked after c -- viewAsOf is prefix-stable -- so a
-- stamp replayed today against the recorded pin is bit-identical to the stamp
-- minted then. Same log, same observations, same pinned version: the same
-- StampedObs, and (with P8 and C13) the same valuations.
pRepro :: [Transaction] -> Timestamp -> Int -> (SourceId, Timestamp, RawDatum) ->
  Bool
pRepro txs (Timestamp c) ext (s, t, rd) =
  let m = fromInteger (max 0 c) + max 0 ext -- any later extension
  in ingestAt (viewAsOf (take m txs) (Timestamp c)) s t rd
    == ingestAt (viewAsOf txs (Timestamp c)) s t rd

-- P-CLONE-STAMP clone_at stamp reconstruction: for a stamp minted at ANY
-- booking cut, the log alone reconstructs the pin -- some cut's rebuilt view
-- carries the recorded projection version (a content address, so a

```

```

-- retro-effective boundary booked later rennumbers nothing) -- and EVERY cut
-- carrying that version re-mints the stamp bit-identically. The as-known-at-t
-- replay thus reproduces the honest mistake rather than repairing it; the
-- corrected reading is the same door at the tip pin (deterministic by pDet);
-- re-coordination beyond that is exclusively the ledger's re-stamp event with
-- lineage (O8), outside this oracle.
pCloneStamp :: [Transaction] -> (SourceId, Timestamp, RawDatum) -> Bool
pCloneStamp txs (s, t, rd) = all cutOK cuts
  where
    cuts = [0 .. toInteger (length txs)]
    viewAt k = viewAsOf txs (Timestamp k)
    cutOK k = case ingestAt (viewAt k) s t rd of
      Left _   -> True -- a refusal asserts nothing (P2's half)
    )
    Right so ->
      let hits = [ j | j <- cuts, bvVersion (viewAt j) == obsView so ]
      in not (null hits)
      && all (\j -> ingestAt (viewAt j) s t rd == Right so) hits

-- The convention's positional readings, internal to the door: the chain
-- position at t in EFFECTIVE order (the maximal committed boundary with
-- t_eff <= t, or the origin), and the position k boundaries behind it,
-- floored at the origin. Total; consumed by TracksChain / LagsBy; never a
-- stamp default -- a datum with no convention is refused, never positioned.
chainPosAt :: BasisView -> UnitId -> Timestamp -> BasisId
lagPosAt   :: BasisView -> UnitId -> Timestamp -> Integer -> BasisId

-- P-LAG Lag-convention arithmetic: a LagsBy k stamp sits exactly k
-- effective-order steps behind the caught-up position at the same t_obs,
-- floored at the origin -- and LagsBy 0 IS TracksChain. P25 cannot see an
-- off-by-one here, because any on-chain id is sound; the arithmetic therefore
-- carries its own oracle, computed independently by positional indexing where
-- lagPosAt drops from the reversed chain. Drivers: views from genChain via
-- viewAsOf, t and k drawn by nextInt, k shrunk by shrinkInteger.
pLag :: BasisView -> UnitId -> Timestamp -> Integer -> Bool
pLag v u t k =
  let path = [ bid | (te, _, bid) <- chainAt v u, te <= t ] -- ascending
      k'    = fromInteger (max 0 k)
      want | length path <= k' = Origin u
            | otherwise        = Boundary (path !! (length path - 1 - k'))
  in lagPosAt v u t k == want && lagPosAt v u t 0 == chainPosAt v u t

-- F10 Basis-partition separation: observations aggregate only within one stamp
-- cell. The oracle witnesses the partition's soundness; that no AGGREGATE can
-- consume two cells is the consumption seam's type fact (C13), not re-proved
-- here.
pPartition :: [StampedObs] -> Bool
pPartition sos =
  all (\(st, cell) -> all ((== st) . obsStamp) cell)
    (Map.toList (partitionByStamp sos))

```

The generators and shrinkers that drive the catalogue are deterministic and pure, with no external dependency: a `Seed` drives a splittable linear-congruential generator (splittable so independent draws share no state), every generator is a total function of its `Seed`, and every shrinker enumerates purely toward the structurally smaller candidate. A full body is shown only where the way a value is built is itself part of the argument: a `StampedObs` can be generated only by calling the door, and a chain only by building a committed log. Where the generator is just plain seed-driven drawing, only its signature and contract are shown.

```

newtype Seed = Seed Integer deriving (Eq, Show)

nextInt    :: Integer -> Seed -> (Integer, Seed)    -- uniform-ish in [0, n); total
splitSeed :: Seed -> (Seed, Seed)

-- / genVendorBehaviour: the source basis convention alphabet -- caught-up,
-- lagging, pre-adjusting, declared-through (on-chain ids AND an off-chain id,
-- so the OffChainClaim refusal is exercised), and nothing-on-file. Shrinks
-- toward TracksChain, the best-behaved vendor. The forged off-chain id is
-- harmless by construction: the door refuses it, and the tip weld refuses it
-- as a SetBasis -- it can never name a committed state.
genVendorBehaviour :: BasisView -> UnitId -> Seed -> Convention

shrinkVendorBehaviour :: Convention -> [Convention]
shrinkVendorBehaviour TracksChain = []
shrinkVendorBehaviour (LagsBy k)  = TracksChain : [ LagsBy k' | k' <-
    shrinkInteger k ]
shrinkVendorBehaviour _           = [TracksChain]

-- / genRawDatum: draws the unit reference from the view's register, PLUS one
-- unregistered reference so the UnregisteredUnit refusal is exercised; an
-- exact value; and a generated vendor behaviour with its convention version.
genRawDatum    :: BasisView -> Seed -> RawDatum
shrinkRawDatum :: RawDatum -> [RawDatum]
shrinkRawDatum (RawDatum ref val (cv, conv)) =
    [ RawDatum ref v' (cv, conv) | v' <- shrinkInteger val ]
    ++ [ RawDatum ref val (cv, c') | c' <- shrinkVendorBehaviour conv ]

-- / genStampedObs: ONLY by calling the door. A generator that built a
-- StampedObs by record syntax would mint values outside ingest's image --
-- precisely the states the abstract type excludes -- so generation is
-- ingestAt over generated inputs, Nothing when the door refuses, and
-- shrinking shrinks the RAW DATUM and re-ingests (shrinkStampedObs): a
-- stamp is never shrunk in place.
genStampedObs :: BasisView -> Seed -> Maybe StampedObs
genStampedObs v sd0 =
    let (s1, s2) = splitSeed sd0
        (tv, _ ) = nextInt 16 s1
    in either (const Nothing) Just
        (ingestAt v (SourceId "gen") (Timestamp tv) (genRawDatum v s2))

shrinkStampedObs :: BasisView -> SourceId -> Timestamp -> RawDatum -> [StampedObs]
shrinkStampedObs v s t rd =
    [ so | rd' <- shrinkRawDatum rd, Right so <- [ingestAt v s t rd'] ]

-- / genFeed: a multi-vendor feed against one view -- several sources, mixed
-- vendor behaviours, observation times straddling the chain's boundaries.
-- genPerm and genCrashPoint drive the pPerm0 / pCrash schedules over the
-- same feed; partitionByStamp splits its committed set for pPartition.
genFeed      :: BasisView -> Int -> Seed -> [(SourceId, Timestamp, RawDatum)]
genPerm      :: Int -> Seed -> [Int]
genCrashPoint :: Int -> Seed -> Int

-- / Shrinkers, paired: a feed shrinks by prefix, then pointwise through
-- shrinkRawDatum (source and t_obs name the trial; the datum carries the
-- structure); a permutation shrinks one step, to the identity; a crash
-- point shrinks toward 0, then halfway.
shrinkFeed      :: [(SourceId, Timestamp, RawDatum)]

```

```

--> [[(SourceId, Timestamp, RawDatum)]]
shrinkPerm      :: [Int] -> [[Int]]
shrinkCrashPoint :: Int -> [Int]

-- / The P-PERM-N drivers: permuteBoundariesBy permutes the boundary NOTICES
-- among their own positions (registrations stay put); rebook re-asserts
-- each derived SetBasis to the post-insertion tip of the new order, per
-- unit, exactly as genChain mints it -- so every permutation of a
-- generated chain replays Right, and no trial is vacuous.
permuteBoundariesBy :: [Int] -> [Transaction] -> [Transaction]
rebook              :: [Transaction] -> [Transaction]

-- In-module chain minting (the P24 precedent). BoundaryId is a content address
-- assigned at the attested notice gateway; the oracle harness mints
-- deterministic addresses HERE, in-module, and these minters are deliberately
-- NOT exported -- no production caller gains a minting door.

-- / genBoundaryEvent: covers retro-effective t_eff (the small window makes a
-- later booking with an earlier t_eff frequent), same-t_eff precedence
-- collisions (tie broken by the content address, data in the notice), and
-- every Declaration constructor -- parameterised (with and without a cash
-- leg), Pending, Terminal.
genBoundaryEvent  :: UnitId -> Seed -> BoundaryEvent
shrinkBoundaryEvent :: UnitId -> BoundaryEvent -> [BoundaryEvent]

-- / genChain: a committed log for one unit -- registration, then n boundary
-- transactions, each SetBasis re-asserting the post-insertion effective tip
-- so the tip weld admits it (retro-effective notices included). Holder-free
-- by construction, so the invariance weld is the vacuous zero-holder case
-- and the generator exercises the CHAIN, not entitlement arithmetic; every
-- generated log replays Right. Shrinking is prefix-closed (shrinkChain):
-- every prefix of a committed log is a committed log.
genChain          :: UnitId -> Int -> Seed -> [Transaction]
shrinkChain       :: [Transaction] -> [[Transaction]]
shrinkChain txs = [ take k txs | k <- [1 .. length txs - 1] ]

-- / genBoundaryStraddle: the boundary-straddle scenario -- one unit, one
-- committed boundary, and the two booking cuts on either side of it. A
-- consumer pinned at the earlier cut reads the stored series at the old
-- point; a consumer pinned at the later cut reads it transported along the
-- declared arrow; both are correct, related by value invariance, and the
-- cross-reading is refused at the seam (C13).
genBoundaryStraddle :: Seed -> ([Transaction], Timestamp, Timestamp)

```

Properties, generators, and shrinkers are verbatim from the reference ([reference/Ledger.hs](#), Part M), authored and verified by inspection; execution is pending the toolchain and is not reported as passed.

8.6 The datum-kind registry

A market datum is not one number under one name. A dividend forecast lists entries, some a cash amount per share, some a proportion of price; a vol quote carries a strike coordinate and a vol value. Under one boundary operator the components of one datum can have different fates, so the declaration that governs transport is made per component, never per name.

Each consumable datum kind is therefore *registered*, once, before any datum of that kind can enter: a kind registration event fixes the kind's identity and its *component dimension declaration* — one declared dimension per component: price, cash, or dimensionless. A datum

is an observation, never a holding, so the quantity dimension is excluded at the registration door. The per-dimension action then fixes how each component crosses a boundary, and the one declaration serves both consumers (Principle 8.10): the ingest door stamps against it, and the read seam converts by it. Spot registers a single price component. The dividend forecast registers its cash entries as prices and its proportional entries as dimensionless. A vol quote registers its strike coordinate as a price and its vol value as dimensionless.

The vol declaration decides a question of identity. Absolute-strike vol and moneyness vol are two registered kinds, not one kind with a convention flag. An absolute strike transforms as a price under **Scale** while a moneyness ratio stands, and two quantities that transform differently under the same operator are different kinds: the kind id says so, and no conditional enters the transport path. Within a kind, each component carries its own boundary behaviour. The vol value is dimensionless and is never transformed. An absolute-strike coordinate transforms by transcription of the exchange’s published contract adjustment, never by a model. A moneyness coordinate crosses the boundary unchanged only where the boundary notice declares **AId**, the declared identity conversion. Identity is declared, never assumed: the default declaration is empty, so by default transport is refused. The quotation convention stays on the entry as attested provenance, checkable but never branched on.

The dividend forecast shows two fates under one operator. A wallet holds 1,000 shares at €100; the forecast lists €3 cash per share and a proportional entry of 1% of price. The 2-for-1 split declares **Scale** 2. The cash component is a price, so €3 reads as €1.50 post-split, and the cash delivered is invariant: $1,000 \times 3 = 2,000 \times 1.50 = \text{€}3,000$. The proportional component is dimensionless, so 1% stands, and its leg is likewise invariant: $1\% \times 100 \times 1,000 = 1\% \times 50 \times 2,000 = \text{€}1,000$ — one datum, two components, two fates, and total cash unchanged on both, which is Theorem 8.13 applied per component. Now misdeclare the proportional entry as a price. **Scale** halves it to 0.5%, and the leg pays $0.5\% \times 50 \times 2,000 = \text{€}500$ — half the true entitlement, from one wrong word in a declaration. The mandatory witness below exists to catch this class of error.

The registry is vocabulary, and it lives where vocabulary must: in the log. Registration and retirement are attested events under the notice-admission rule W4 (§8.11). The registry the door consults is a projection of those events, under the cache discipline of §4.3: derivable, discardable, never itself authoritative. Each ingestion run consults the registry through the same pinned view it stamps against — the snapshot fixed when the run opens (§8.5) — so every stamp and every refusal replays as a pure function of recorded data (O2). A registration is immutable: a wrong declaration is corrected by registering a new kind and logging a retirement that closes the old kind to new ingestion, never by editing an entry.

An unregistered kind is unconsumable. The door refuses it as **UnregisteredKind**, retaining the payload, as its first clause — before unit resolution, convention lookup, or claim checking. This refusal is distinct from an undeclared conversion: an unregistered kind never enters the ledger at all, while a registered kind that a boundary’s declaration omits enters and is stopped only at that crossing (the declared partiality of Definition 8.7). The first is a vocabulary failure at the door; the second is a conversion gap at a boundary.

Every registry entry carries an *invariance witness*, and an entry without one is inadmissible: registration demands the witness with the declaration. For a scalar kind the witness is an instance of the invariance theorem itself (Theorem 8.11); a composite kind names an executable oracle — the dividend-list kind cites **dkDivSplitOK**, which replays a scale boundary against the declaration, checks the delivered-cash identity above, and fails the misdeclared variant. The declaration the witnesses pin is a named trust assumption:

TA-KIND.

Owner: data governance, the W4 owner. *Content:* each registry entry’s component dimension declaration is true of the kind it names. *Violation consequence:* dimensionally wrong transport — the €500 leg above. *Detection:* the entry’s invariance witness at regis-

tration, W3 partition divergence (§8.11), and differential recomputation (P27). *Accepted residual*: a misdeclared kind whose data never cross a boundary.

TA-BASIS trusts the stamp; TA-KIND trusts the declaration of what the stamp covers. The scope of that trust is explicit: each registered kind is trusted only to the extent that its per-entry invariance witness holds, and a kind without its witness cannot be registered.

The registry classifies what the ledger transports; it grants no authority over arithmetic the ledger did not perform. A provider-computed composite — an index level, a vendor curve — registers as a primitive kind of the publisher’s unit, never as a composite of its inputs, and the re-derivation canon (Principle 8.14) stands: derived data are re-derived from transported inputs, never transported. Boundary action on registered kinds stays within the closed operator menu of Definition 8.7: an index publisher’s rebalance or divisor change enters as a W4 basis notice declaring **Recompose**, and a benchmark cessation or succession is **Subst** composed with **Shift** at the published fallback spread — instances of existing operators, not new cases.

The table below runs the contract across the registered kinds: for each kind, what the stamp covers and what fail-closed means for that kind.

Raw datum	What the stamp covers	Fail-closed meaning
Spot, quote	Disseminated trade or quote; singleton stamp $\{u \mapsto b\}$ per (unit, source) convention; a lagging source is the per-source convention case (§8.4).	No convention on file: the series is refused into quarantine; around ex dates W3 partitions by stamped basis.
Implied-vol observation	The vol value with its labelled coordinates — absolute strike, expiry, moneyness or delta — transcribed from the source, never inferred; singleton stamp on the quoted unit (the option series where registered, else the underlying). No parametrisation is mandated.	An undeclared (coordinate, operator) pair has no conversion: the point cannot cross the boundary; an unpriced vol point is an enumerated Unpriced , never a transported value.
Index level	Provider-published level: a primitive observation of the index unit I ; singleton stamp $\{I \mapsto b_I\}$. The divisor snapped in-scope from stamped constituents is the derived case and keeps its joint stamp (§8.12).	A composition change with no committed maintenance boundary is a large innovation without a boundary: W3 quarantines the level series.
Composition, weights, divisor files	The publisher’s declared parameters for a maintenance boundary (Recompose σ), with its t_{eff} ; logged as a W4 notice.	Pro-forma data claiming a basis id not yet on the committed chain are refused (OffChainClaim) and represented after the boundary commits at effectiveness; a stale divisor consumed across the boundary is the refused 70.83 phantom (§8.12).

Raw datum	What the stamp covers	Fail-closed meaning
Fixing, benchmark	Value, fixing time, publication time; singleton stamp on the benchmark unit at the basis the administrator’s convention names.	A republication within the correction window is a new observation on the value axis, basis untouched; a fixing meeting Terminal or a Pending gap blocks under W1 — real money never moves on an unwitnessed basis.
Curve, derived data	Derived in-scope from transported primitives; the joint stamp of its inputs, inherited compositionally (Principle 8.14, Proposition 8.9); never transported — the inputs transport and the derivation re-runs at the target assignment. A vendor-supplied curve is the primitive case of the index-level row.	One refusing input refuses the derived point, enumerated per unit.

8.7 The market-data integrator

An integrator is the component that brings external market data — prices, implied-volatility points, index levels — to the ingest door. It does four things, and nothing else.

First, it *pins*. At the start of a run it takes one **BasisView** and holds that view for the whole run. The view is immutable, so a boundary that commits in the middle of the run cannot change what the run sees: every datum in the run is judged against the same picture.

Second, it *transcribes*. From the feed it copies the disseminated value, the observation time, the signed source id, and the provider’s unit reference into a **RawDatum**, under the (unit, source) convention on file (O4). Copying values, times, and conventions is the whole of its competence; it never adjusts a number.

Third, it *presents*. It passes each datum through **ingestAt** against the pinned view. The integrator links against **ingestAt** alone, and the **Ledger** value never crosses the boundary. The ledger attaches the stamp; the integrator never computes a basis id.

Fourth, it *keeps the refusals*. Each **Left** it receives retains its payload, and the quarantine store — the accumulated record of those refusals — is inspected under the W3 workflow (§8.11).

The integrator therefore carries no live picture of the ledger. The stamp is a pure function of projection version, convention version, source, t_{obs} , and raw datum (O2, P-DET), so a source that lags a boundary needs no intervention. After the 2-for-1 split creates the post-split basis, a source that keeps publishing pre-split prints is on file as **LagsBy**, and the door stamps its print of 100 at the pre-split chain point. The 100 is correct there; consumption at the post-split basis either transports it to 50 along the declared conversion or refuses it, so the stamped 100 can never meet the 2,000-share balance unexamined. The **mQuoteEx** bit of **§D** is exactly this stamp collapsed to one step, generalised by the coordinate to boundary chains of any length.

Real time and batch share one code path. A tick-by-tick feed presents each datum at the door as it arrives. An end-of-day file presents the same door, datum by datum, within one run. A historical back-adjusted pull is stamped from the file’s declared adjusted-through convention (O6), never from the basis prevailing at each row’s observation time. Mode is not among the

stamp’s arguments (P-MODE): a datum presented in real time and the same datum presented inside a batch at the same pin receive the same stamp. The only operational difference between the modes is when the integrator refreshes its pin — between runs, never within one.

8.8 The invariant

The discipline reduces to one statement. A valuation run works against a single basis assignment: one basis per unit, fixed for the whole run. Every balance the run reads and every datum it consumes must agree with that assignment, and the agreement is checked over every unit the datum’s stamp names, not only the units being valued.

Invariant 8.17 (Single-basis consumption). *Let V be a valuation invocation over position scope S at time t , and let $\beta_t : \text{UnitId} \rightarrow \text{BasisId}$ be the total basis assignment given by replayed `usBasis` over all registered units (total by Definition 8.3: `defaultStatus` carries `Origin`). Then: (i) every balance consumed by V is read from S at $\beta_t|_S$; (ii) every datum d consumed by V satisfies $\text{stamp}(d)(u) = \beta_t(u)$ for every $u \in \text{dom}(\text{stamp}(d))$ — the check ranges over the stamp’s whole domain, whether or not $u \in S$; (iii) every value formed from several data is formed from data satisfying (ii) under one and the same β_t .*

In the small world: a valuation whose assignment β_t places the share at the post-split basis may read the 2,000-share balance and a price stamped post-split (50), but not the pre-split print of 100 — clause (ii) refuses the stale stamp even though each number is individually correct, and the €200,000 phantom of the opening never forms. Clause (ii) also ranges beyond the valuation scope: the level of index I is stamped over the constituents A and B it was computed from, so a valuation whose scope contains only I still checks A and B , and a level computed with the stale divisor is refused.

Well-definedness. `withSnapshot` (§8.9) fixes β as β_t restricted to the *stamp-closure* of S — the set $S \cup \bigcup \{\text{dom}(\text{stamp}(d)) : d \text{ consumed}\}$. This restriction is well-defined, because β_t is total over registered units and `ingest` admits no stamp naming an unregistered unit. Thus S restricts which balances are read, never the domain on which admissibility is checked.

Clause (iii) is what makes the invariant meaningful for a black box: the framework legislates which data it is legal to feed a pricing function, never what the function does with them. State-sufficiency (§5) presupposes that prices and balances agree in basis, and the snapshot’s type variable carries exactly that agreement. A held unit priced in the wrong basis remains representable as an error but is unrepresentable as a `Cash` value: one more illegal state that the valuation types exclude, beyond those of §5.

The same rule is also recorded as a condition, C13, in the register of §4.5. Its statement keeps the word *arrow* of the formal register for what this section’s prose calls a declared conversion.

C13 (basis edge).

Every consumed datum names its joint basis; consumption requires pointwise agreement, over the stamp’s whole domain, with the prevailing total assignment β_t , or a ledger-authored arrow. C13 does for a datum’s meaning what P3 does for a move’s existence: P3 refuses a move to a wallet that does not exist, and C13 refuses a datum consumed against state that sits in a different basis.

8.9 Type-level enforcement

The guarantee is the two-tier discipline of C11: inside authored code the basis is a type index; in the store it is plain stamped data; at the read seam the index is re-established by checking the stamps. Which epoch a unit occupies is known only by replaying the committed log at run time, so the compiler cannot know an epoch’s value. The honest static guarantee is therefore *sameness* — two values carry the same index — never the index’s value. A type-level ordinal in the normative layer would claim static knowledge the system cannot have.

Tier 1 — the didactic law. The listing below states the invariant on one page; it is not the system mechanism. It gives a price the type `PriceAt e` and a balance the type `BalAt e`, where `e` is an epoch index that appears only in the type, never inside the value — such a parameter is called a *phantom* parameter. A quantity and a price meet only at equal index, and the only function that changes an index applies a ledger-authored adjustment. All constructors are unexported, so the sole author of an `Adj` value is the ledger’s fold; an `Adj` value is therefore itself a proof that the log carries exactly these boundaries, in effective order. The commented `_bad` at the end shows the phantom doubling of the opening as a rejected program: the post-split balance of 2,000 shares priced with the pre-split quote of 100 is not a mistaken number; it is not a well-typed program at all.

One language fact must be handled before the listing is sound. GHC assigns every type parameter a *role*, which governs when two instantiations of the type may be coerced into one another. For a phantom parameter the inferred role permits coercion between any two indices — which would forge exactly the crossing the types exist to forbid. The declaration `type role ... nominal` withdraws that permission: coercion becomes derivable only at equal index. The annotations in the listing are therefore normative, not stylistic.

```
{-# LANGUAGE DataKinds, GADTs, KindSignatures, RoleAnnotations #-}
data N = Z | S N

newtype PriceAt (e :: N) = PriceAt Rational -- constructor NOT exported
newtype BalAt (e :: N) = BalAt Integer -- constructor NOT exported
type role PriceAt nominal -- GHC would infer PHANTOM, and Data.Coerce.coerce
type role BalAt nominal -- (Safe Haskell, no unsafe feature) would then forge
-- the crossing at ANY pair of indices. Nominal makes
-- the coercion derivable only at equal index.

data Adj (m :: N) (n :: N) where
  AIdA :: Adj n n -- constructors NOT exported:
  author -- the ledger fold is the only
  AStep :: Adj m n -> OpSpec -> Adj m ('S n) -- one boundary, in effective order

adjust :: Adj m n -> PriceAt m -> PriceAt n -- the ONLY epoch-crossing function

markValue :: BalAt e -> PriceAt e -> Cash -- Qty and Price meet ONLY at equal
index
markValue (BalAt q) (PriceAt p) = ...

-- _bad :: BalAt ('S e) -> PriceAt e -> Cash
-- _bad = markValue
-- TYPE ERROR. The epoch-(n+1) position priced with the epoch-n quote is not a
-- wrong number; it is not a program. And AStep extends only to 'S n, so the
-- mis-ordered composite of the June-1 timeline's 49-vs-50 has no inhabitant.
```

Tier 2 — the normative seam. `withSnapshot` is the one constructor of coherent read scopes, mirroring `applyTx` as the one write door. The caller hands it a function of type `forall b. Snapshot b -> r`; the type variable `b` is chosen fresh at each call, and the caller can neither name it nor equate it with any other — a variable with this status is called a *skolem*. It denotes a whole basis assignment, not one unit’s epoch, and every accessor is unit-keyed. The ledger is sealed inside the scope: no `Ledger` parameter escapes downstream. The per-unit price accessor is fallible, so quarantine is representable.

```
data Snapshot b -- ABSTRACT. Constructed only by withSnapshot, which fixes the
-- target assignment beta = beta_t on the STAMP-CLOSURE of the
-- scope (Invariant B: beta_t is total over registered units;
```

```

-- Scope restricts which balances are read, never beta's domain)
,
-- reads balances from the SAME sealed ledger, and transports
-- every admissible stamped observation to beta along declared
-- arrows -- refusing, per unit and kind, where none exists.
type role Snapshot nominal

newtype BalAt' b = ...      -- constructors NOT exported
newtype PriceAt' b = ...
newtype At b a = ...
type role BalAt' nominal
type role PriceAt' nominal
type role At nominal nominal

withSnapshot :: Ledger -> Timestamp -> Scope
              -> (forall b. Snapshot b -> r) -> r      -- quantifier where the proof
              lives

snapBal :: Snapshot b -> WalletId -> UnitId -> BalAt' b
snapPrice :: Snapshot b -> UnitId -> Either BasisError (PriceAt' b) -- per-unit
          refusal

markV :: BalAt' b -> PriceAt' b -> Cash -- the seam; Cash is
      basis-free
zipAt :: (x -> y -> z) -> At b x -> At b y -> At b z -- in-scope derivation

data Valuation = Priced Cash
               | Unpriced UnitId BasisError -- quarantined,
               enumerated
               | PricedStale UnitId BasisId BasisId Cash -- logged election

value :: Snapshot b -> [WalletId] -> (Cash, [Valuation]) -- book total +
          exclusions

-- The single cross-scope door, ledger-authored, factoring ONLY through declared
-- arrows:
transport :: Snapshot b0 -> Snapshot b1 -> Kind a -> At b0 a
          -> Either BasisError (At b1 a)

```

Unreachability, for an arbitrary black-box pricing function. Let $F :: \forall b. \text{Snapshot } b \rightarrow r$ be any pricing function; nothing is assumed about its body. Because F must work for every b , and the fragment stated below closes every side channel, the type alone constrains what F can do with b -indexed values. This consequence of the quantifier is called *parametricity*, and it yields the argument in two steps.

First, every b -indexed value that F manipulates comes from its one `Snapshot` argument. The carriers' constructors are unexported, so F cannot build a `BalAt' b` or a `PriceAt' b` of its own; the only ways to obtain one are `snapBal`, `snapPrice`, `transport`, and the index-preserving combinators, and each of these yields values at the snapshot's target assignment β by construction of the door. A quantity and a price that F combines therefore both descend from the same snapshot, and both sit at the same assignment.

Second, values from two different snapshots cannot be mixed. Each call to `withSnapshot` introduces its own skolem, and two skolems never unify: a price indexed by one snapshot's variable does not typecheck where the other's is required. The single function that crosses snapshots, `transport`, factors through the unique declared conversion or refuses with a typed error.

Hence every (quantity, price-like) pair any well-typed F can form lies in a single basis assignment. In the small world: F may pair 2,000 shares with 50 inside the post-split snapshot, or 1,000 with 100 inside the pre-split one, but no well-typed F pairs 2,000 with 100 — the €200,000 phantom is not a wrong output of F ; it is not a program. What the types cannot check is the store boundary, where stamps are erased data. There the runtime complement is a runtime check inside `withSnapshot` that the two bases really are equal, which returns a typed `BasisError`, never a silent pick. This is the C11 guarantee restated at the read seam: compile-time exclusion at authorship, typed refusal at the boundary — claimed as exactly that and no more.

Language-fragment proviso. The parametricity argument holds only in a stated fragment of the language, because GHC offers features that would defeat it. The central one is coercion. GHC infers the role *phantom* for a type parameter that occurs in no constructor field. At role *phantom*, the `Coercible` relation holds between all index pairs. So `Data.Coerce.coerce` — which Safe Haskell accepts, with no unsafe feature involved — would convert a value at one index into a value at any other, forging the epoch and skolem crossings past every unexported constructor. Every basis-indexed carrier therefore declares `type role ... nominal` on its basis index; the annotations are normative, not stylistic. The unreachability claim above is made for exactly this fragment:

- Safe Haskell;
- nominal roles on every basis-indexed parameter;
- constructors, field selectors, and eliminators unexported;
- no `Generic`, `Data`, or `Typeable`-derived path exposing representation;
- no `GeneralizedNewtypeDeriving` on the carriers;
- no Template Haskell in consuming modules.

Within the fragment the free-theorem reading of $\forall b$ — the parametricity argument above — is sound, and `coerce` contributes no way to build or convert a basis-indexed value.

8.10 Time travel and late corporate actions

Historical valuation needs no new machinery. `clone_at(t)` already rebuilds every `UnitStatus` field by replaying the log up to t (§7), and `usBasis` is such a field, so the clone carries the basis that prevailed at t (P8, extended). Valuing history is `withSnapshot` over the clone, at the target assignment β_t . Raw observations are immutable and stamped, so they never migrate. A datum stamped after β_t can be carried backward only if every conversion between the two bases is invertible (Definition 8.7): the split’s `Scale 2` is invertible, so a post-split print reaches a pre-split valuation, while a `Pending` spin-off declares no conversion, so nothing crosses and the datum is refused into the workflow of §8.11.

Two version axes run through every historical question, and they are independent. The *basis axis* records the order in which boundaries take economic effect; it is ledger-authored, and conversions compose along it (Proposition 8.9). The *knowledge axis* records what the ledger knew at each moment; it is bitemporal: an “as known at t ” replay uses the chain as the log held it at t , and a “restated” replay uses the chain with corrections applied. A historical question fixes a point on each axis, and the two choices compose.

A late or retro-effective boundary books at t_{notify} but takes economic effect at the earlier t_{eff} . It changes no stored stamp, because stamps are stable ids. Instead the ledger appends *re-stamp derivation events* for the observations captured in $[t_{\text{eff}}, t_{\text{notify}})$: each keeps its value, receives the corrected coordinate, and carries lineage to the late transaction. That transaction’s `SetBasis` is tip-welded (Principle 8.4): it re-asserts the standing effective tip, so the live coordinate never moves backward. The corrected replay, meanwhile, projects the chain with the boundary at its true effective position mid-chain and recomputes the conversions that span it. The two replays

then diverge exactly as intended: the as-known replay reproduces the honest mistake, and the corrected replay applies the re-stamps. Moves committed inside the window repair through the existing compensating-transaction path (§12.5), now with a citable coordinate lineage. Pinning both the snapshot and the chain version gives a valuation that is both repeatable and correct; pinning the snapshot alone, as the old machinery did, repeated the earlier mistake faithfully instead of correcting it.

8.11 Failure semantics as workflow

Every refusal carries a typed error, a **BasisError** value. What the refused datum was for decides what happens next: the consequence class selects the regime. No regime is silent.

Regime	Trigger	Consequence	Mandatory for
W1 Block	Move-emitting consumption (settlement projection, NAV strike, fee crystallisation) meets any BasisError .	The lifecycle event defers as a recorded workflow event with a named unblock condition — a fresh observation at β , or the crystallising publication event — queued under obligation liveness (P21, §15.6). Real money never moves on an unwitnessed basis.	The stale-data gate (§12.4) and the pre-invocation check (§15): basis equality first, timestamp threshold second.
W2 Per-unit quarantine	snapPrice returns Left for a unit: no admissible datum, a Pending gap, an undeclared kind.	The book values on time as (Cash , [Unpriced ...]) — a total minus a named, enumerated exclusion list. Blocking the whole book is rejected: one late notice must not halt thousands of units; shippability decides.	Daily marks, risk, management PnL.
W2' Flagged-stale carry	An explicit, logged election event with its canonical writer row in the C11 table.	The unit is valued <i>coherent-at-β^-</i> — the latest assignment $\beta^- \leq \beta_t$ at which its data are pointwise complete — never a mixed vector. The result is PricedStale , a distinct constructor carrying its gap; downstream must pattern-match. <i>Coherent-at-β</i> means all inputs share assignment β ; <i>current</i> means $\beta = \beta_t$; W2' is coherent, not current, and says so.	Management PnL where quarantining a large unit is worse than a flagged carry.

Regime	Trigger	Consequence	Mandatory for
W3 Partition quarantine	Aggregation partitions sources by asserted basis id; quorum and divergence are computed within a partition only.	$\{100 @ b_3\}$ and $\{50 @ b_4\}$ are two singleton partitions — no false divergence, no median 75 that exists in no basis. A large innovation with no committed boundary is the signature of an undeclared corporate action or a bad feed: the series quarantines. This is TA-BASIS's detection signal; the window in which the world has split but the log has not is detected, not prevented.	All ingestion around corporate-action windows.
W4 Notice attestation	A boundary declaration rewrites both sides of the seam and, once its moves are emitted, cannot be repaired by the price-correction path.	Admission requires issuer or exchange signature, or content-hash agreement across $N_{CA} \geq 2$ independent sources (owner: data governance). Same- t_{eff} weld: a boundary whose t_{eff} equals that of another boundary on the same unit — committed or co-presented — is admitted only if the attested notices declare mutual precedence (<i>prec</i> , Definition 8.2); absent declaration, the colliding notice is refused and the unit enters pending-transition, fail-closed. Between first notice and confirmation the unit is <i>pending-transition</i> for consumption across the boundary: W1 blocks, W2 quarantines, W3 partitions.	Every boundary event.

8.12 The 2-for-1 split, end to end

A wallet holds 1,000 shares of u , and $\text{usBasis} = b_3$.

1. *Snap*. Vendor EX1 publishes 100.00. The ingest door stamps the observation $\{u \mapsto b_3\}$, per the standing convention for the pair $(u, \text{EX1})$. The stored record is immutable and logged.
2. *The corporate action*. One atomic transaction τ carries three things: a move of +1,000

shares from the corporate-action virtual wallet ($1,000 \rightarrow 2,000$), the status write **SetBasis** b_4 , and the declaration $\{\text{KSpot} \mapsto \text{Scale } 2, \dots\}$. Admission checks the welds: the quantity legs satisfy $2,000 = 1,000 \times 2$ and the cash legs satisfy $0 = 1,000 \times 0$ (Theorem 8.11), and b_4 is the effective tip after insertion (Principle 8.4). No reachable state holds the doubled quantity without the boundary that explains it.

3. *Snapshot at t_{price} .* The assignment gives $\beta(u) = b_4$; the fold gives a balance of 2,000 at b_4 ; the stored spot is stamped b_3 . The door carries the spot across the one declared conversion: $A(b_3 \rightarrow b_4)(100) = 50$.
4. *Mark.* **markV** 2,000 50 = €100,000. No phantom.
5. *The bug, attempted.* Feeding the raw 100 into the mark requires a value of type **PriceAt**’ b . The only functions that produce one are **snapPrice** and **transport**, and each either refuses or converts. The basis index carries a nominal role annotation (§8.9), so the type is distinct at distinct bases and a raw number cannot be coerced across the boundary. In the didactic Tier 1 listing of §8.9, pairing the epoch- $(n+1)$ balance with the epoch- n quote does not compile. The €200,000 is therefore a program that does not exist on the compile side and a **Left BasisMismatch** on the runtime side; it never reaches a committed **Cash**.
6. *Attribution.* The true post-split spot is 55 — up 10% like-for-like. Open one **Snapshot** pinned before the split and one pinned after it; **transport** carries the opening pair along the declared conversion, $(1,000, 100) \mapsto (2,000, 50)$, and Theorem 8.11 with $c = 0$ makes the carried value equal. Then $\text{PnL}_{\text{price}} = 2,000 \times (55 - 50) = +10,000$ and $\text{PnL}_{\text{flow}} = (2,000 - 1,000) \times 55 = 0$. A naive decomposition that compares the €55 close against the un-transported €100 open instead splits the €10,000 into $1,000 \times (55 - 100) = -45,000$ of “price” and $(2,000 - 1,000) \times 55 = +55,000$ of “flow” — two large terms that are artefacts of mixing the pre- and post-split frames. Transporting the opening pair across the split removes them: the decomposition, not only its sum, is basis-invariant, and it can be written only through the one declared door.
7. *Late notice.* If τ is not yet logged at t_{price} , C3 keeps the moves and the boundary together, so the snapshot is coherent at b_3 : $1,000 \times 100 = €100,000$ — the right answer under the information held. Meanwhile the exchange disseminates prices near 50: W3 quarantines the series, and W1 blocks settlement-grade use. At t_{notify} , τ commits with economic time t_{eff} . If boundaries with later effective times committed in the interim, four mechanisms reconcile them, each already stated. The tip weld (Principle 8.4) makes τ ’s **SetBasis** re-assert the standing tip, so the coordinate never moves backward. The chain projection places τ ’s boundary at its effective position, mid-chain. Re-stamp events (§8.10) move the window’s observations to their corrected coordinates. Replay of the corrected log is then coherent end to end. At no point does $2,000 \times 100$ or $1,000 \times 50$ reach a committed **Cash**.

The same split read from the boundary — two consumers snapping one series on opposite sides of the open — is the worked example of the market-data contract (§8.5).

The dividend. The same walk on a 1,000-share holding priced at 102, for a €2 dividend: the declaration is **Shift** -2 with quantity factor $f = 1$. Transport gives $102 - 2 = 100$. The 2-per-share cash leg books once, in the same transaction, and the weld checks booked cash $= q \cdot c$: $1,000 \times 2 = €2,000$, so the total, €102,000, is unchanged.

This recovers the **Cum/Ex** pricing of §D exactly: that appendix treats this very dividend, with $f = 1$ and a chain only one boundary long. Its quote-ex flag and this section’s per-(unit, source) stamp carry the same one bit of information — whether a quote is cum or ex — read from the two sides of the door. In the cum-to-ex direction, a source still quoting cum past the ex-date is the lagging convention of §8.4, and the seam converts its prints by $p - c$ on read. In the ex-to-cum direction, a source quoting ex before the ledger’s transition is the one case the appendix’s flag cannot express. Here it is a large innovation — a price jump no committed boundary explains — and W3 quarantines the series (§8.11), fail-closed.

The bond coupon is the same event with $f = 1$. The coupon date books the cash move and advances the series from cum to ex in one atomic transaction (C3). Decoupling the two admits exactly two corruptions. Cash paid while the mark stays cum counts the coupon twice; the mark gone ex with no cash booked loses it. Atomicity makes both unreachable, because the price jump at the coupon date exactly offsets the booked cash (Theorem 8.11).

The index divisor. Index I is computed over constituents A and B. With A at 60 and B at 84, the published level $(60 + 84)/D = 100$ fixes the divisor $D = 1.44$, snapped as a derived observation whose joint stamp covers everything it consumed: $\{I \mapsto b_I, A \mapsto b_A, B \mapsto b_B\}$. B then splits 2-for-1 ($b_B \rightarrow b'_B$), and fresh prices arrive: A 60, B 42. The corrupt level $(60 + 42)/1.44 \approx 70.83$ — a phantom -29.2% index move — can be computed only by consuming the snapped D at an assignment where $\beta_t(B) = b'_B \neq b_B$. That consumption is pointwise inadmissible — the stamp must agree with the assignment at every unit it names, and it disagrees at B — and is refused fail-closed. The check is well defined even though the valuation scope holds only I : β_t is total over registered units, and admissibility ranges over the stamp's full domain $\{I, A, B\}$ (Invariant 8.17(ii)). The framework will not transport a derived datum (Principle 8.14). Three legal paths remain: the index authority's maintenance boundary on I declares **Recompose** with $D' = 1.02$, so $(60 + 42)/1.02 = 100$, entering under W4 like any corporate action; or a fresh divisor observation arrives stamped at the new assignment; or the level is re-derived in scope from transported constituents. Until one of these happens, **snapPrice** I returns a **Left: Unpriced** I , enumerated, never a silent 70.83.

The delisting. A wallet holds 1,000 shares of u ; the last stamped print is €12.00. The confirmed notice declares liquidation with no successor and no consideration: class **CATermination**, declaration **Terminal** on every kind, any residual distribution booked as terminal moves in the same transaction. **Terminal** gives no conversion in either direction (Definition 8.8), so the stale 12.00 cannot cross: **snapPrice** u returns a **Left**, and the book values as (**Cash**, [**Unpriced** u]) (W2). The €12,000 stale carry never reaches a settlement-grade use. That is not a failure to price; it is the only correct mark for a dead series, since any number produced from the stale print would be a guess. The flagged stale carry (W2') remains available for management PnL, and it arrives as **PricedStale**, pattern-matched downstream — never a silent 12.00. Termination is a fact of the confirmed notice, never of the venue. A series that leaves its exchange but continues over the counter has no boundary event at all, because no per-share economics changed: what ends is the exchange source convention. The feed goes silent, the stale-data gate and W2 do their work, and fresh prints re-price the same unit at the same basis under a new (unit, source) convention (O4).

The spin-off. A wallet holds 1,000 shares of A, stamped €100 at basis b . The confirmed notice: one share of A1 and one of A2 per share of A, effective at the ex-date; class **CASpinOff**. When the children later print on their own, A1 will open near €80 and A2 near €20. Neither number is in the notice, because no notice can contain a price formed after it. The confirmed terms therefore carry exactly as far as they logically can, and the two planes carry different halves.

On the entitlement plane, moves issue 1,000 A1 and 1,000 A2, the children registered at their origins with their datum kinds (§8.6). On the quotation plane, the declaration is a **Recompose** of old A onto the basket $\{A1 \mapsto 1, A2 \mapsto 1\}$. The basket mechanics, pointed to from Definition 8.7, are these. A **Recompose** σ whose target names successor units declares one forward conversion. It carries the parent's price-like kinds onto a basket datum, jointly stamped at the successors' origins; the basket weights are the confirmed entitlement ratios. The basket datum is consumable only jointly: pointwise stamp agreement (C13) refuses its projection onto any single component. No backward or per-component conversion exists until fresh per-component

stamped observations arrive, because the decomposition is not in the notice. The transported basket value carries its lineage (`txCause`, the declaring boundary). And the basket step is a bridge, not a standing conversion: it retires on arrival of fresh per-component observations, after which composition proceeds component-wise under Proposition 8.9.

Pair-level valuation is therefore all the stale €100 mark can license; component pricing stays refused. The stale €100 may value the pair — 2,000 shares combined, marked through the basket at $1,000 \times 100 = €100,000$, phantom-free, so the overnight risk of the combined position is readable at the open. It may never value a component: C13 refuses the projection of the joint stamp onto A1 or A2 alone, because that projection is the 80/20 decomposition the notice never contained. Per-component kinds stay **Pending**, and the pending spin-off is the normal case, not the edge case: the basis id advances at effectiveness, and until fresh per-component stamps arrive no number can price a component at the new basis (P34). Once A1 and A2 carry their own stamps — the 80 and the 20 — the basket step has retired, and the book marks $1,000 \times 80 + 1,000 \times 20 = €100,000$ from observations, component-wise on each child's chain.

The refused error, in numbers: carrying the stale 100 onto A1 by an assumed identity (€100,000) and adding the fresh A2 print at 20 (€20,000) books €120,000 against a true €100,000 — A2's value counted twice, once embedded in the stale parent price and once as the fresh print. The seam refuses it twice over. AId is declared, never assumed, so no conversion carries A onto A1 alone. And C13 refuses the mixed consumption of a stale pair value beside a fresh component print.

The elective merger. A wallet holds 1,000 shares of target T, stamped €100; the acquirer is Q. The confirmed terms after election and proration: per T share, €102 cash or 2.04 Q shares, under a 50% aggregate cash cap. Holder A (500 shares) elected cash, holder B (500 shares) elected stock, and the cap does not bind.

Resolution comes first. All of it is in the log, and none of it is an operator. The announcement opens the window, and the unit enters pending-transition, so consumption across the boundary fails closed. Election instructions are logged per holder. Proration is computed. Confirmation publishes the final blend. Only now does an operator exist.

On the entitlement plane the two holders receive different moves: A's 500 T become €51,000 cash, B's 500 T become 1,020 Q. On the quotation plane the series has one fate. The confirmed blend per T share is €51 cash plus 1.02 Q, and the declaration on T's price-like kinds is $\text{Subst}(Q, 1.02) \circ \text{Shift}(-51)$ — the composite the benchmark-cessation fallback of §8.6 already uses, no new case in the menu. Transport of the stale datum: $100 \mapsto (100 - 51)/1.02 = 2450/51$ per Q share, exact rational.

Admission is the elective case of Principle 8.6, stated here, where its data are on the page. *Elective aggregate weld*: the confirmed notice declares an election structure and a proration result (`CAMergerElective`), and the transaction is admitted only if the legs in aggregate satisfy $\sum(\text{quantity legs}) = f \cdot \sum q$ and $\sum(\text{cash legs}) = c \cdot \sum q$ at the confirmed blend (f, c) , up to explicit cash-in-lieu legs. Here the check reads: quantity legs $1,020 = 1,000 \times 1.02$; cash legs $€51,000 = 1,000 \times 51$. No single holder's legs match either factor — that is what an election means — and the blend exists exactly because proration is confirmed.

Value before, $1,000 \times 100 = €100,000$. Value after, $1,020 \times 2450/51 + 51,000 = 49,000 + 51,000 = €100,000$ — continuous to the minor unit. Per holder the values differ by the elected consideration: A holds €51,000 against €50,000 before, and B holds €49,000. The €1,000 A gains is the €1,000 B cedes, determined by the logged elections and summing to zero. If Q's next fresh print is 50, the transported $2450/51 \approx 48.04$ is not wrong: it is old information in new coordinates. The gap — €2,000 on the €100,000 aggregate — is marking PnL, old information against a fresh mark, not a phantom. The operator never absorbs it.

8.13 CDM corporate-action mapping

Real corporate actions arrive as a message pipeline: announcement, election instructions, confirmation. The pipeline is not mechanical. Terms are indicative at announcement. Per-holder elections and proration change what is delivered. Mixed consideration blends cash and stock. All of that is lifecycle *resolution*: events in the log, entitlements as moves, no operator. The message standards that carry the pipeline — ISO 15022 (MT564/565/566), ISO 20022 (seev.031/033/036), or a vendor feed — are sources under the W4 attestation regime (§8.11). They are named here as instances, never as a type layer: each standard is one more attested source under rules that already exist, and no message format adds a type or a rule to this specification. “Confirmed” is a property of attestation, not of format: a notice is admitted on its W4 attestation alone, whichever format carried it.

An operator exists only from confirmation. This is the resolution/consumption boundary, hereafter the *confirmation gate*: “the declared terms of the notice” in Definition 8.7 means the *confirmed* terms — the terms carried by the W4-attested final-terms notice. The attested object is terms-confirmation; movement confirmation — the message class that reports postings after the fact — has no gating role, since it can post days after the ex-date and gating on it would quarantine every split from ex-date to pay-date. The boundary’s t_{eff} is the ex-date. Between announcement and confirmation no operator has been declared, so nothing adjusts: consumption that would cross the announced boundary fails closed or quarantines, while consumption in the unit’s standing basis — fresh prints before t_{eff} — is unaffected, since the basis advances only at effectiveness. The unit is pending-transition for the crossing (W4, §8.11).

The sentence of §8.2 — an event that declares no operator adjusts nothing — is exactly this window’s safety clause, and P32 witnesses it: pending blocks consumption, and an empty declaration yields refusal, never identity. Confirmation is where the declaration of Principle 8.10 is written; both of its uses begin there. For mandatory actions — splits, ordinary dividends — final terms confirm before the ex-date in the ordinary course, so the window is empty and consumption never blocks; the window bites exactly where marking would be guessing: elective and prorated events, late notices, contested terms.

The pipeline’s guarantees restate in four steps. The operator is *declared*: F — the map of §14 that turns a business event into a ledger transaction — derives it from the notice’s confirmed terms, never from its name (Definition 8.7). The terms are *attested*: they are the W4-attested final terms, and no other terms count. The operator is *welded*: admission checks it against the booked moves (Principle 8.6). And it is *recomputable*: any party recomputes it from the logged terms and diffs (P27). Declared, attested, welded, recomputable.

Every confirmed notice declares its class, drawn from a closed taxonomy of eight:

```
data CAClass
  = CASplit          -- split, reverse split, stock dividend: any pure ratio
    event
  | CADividendOrdinary -- ordinary cash distribution, coupon detachment
  | CADividendSpecial  -- special or extraordinary distribution, return of
    capital
  | CASpinOff          -- distribution of a new series to holders of the parent:
    -- spin-off, demerger, rights issue (the right is the
    registered child)
  | CAMergerStock      -- stock-for-stock succession
  | CAMergerElective   -- elective or mixed consideration, tender with proration
  | CATermination      -- merger for cash, liquidation, delisting with
    termination
  | CAREbalance        -- index composition or divisor change
deriving (Eq, Ord, Show, Enum, Bounded)
```

The class is declared data on the attested notice — the issuer’s or exchange’s classification — never inferred from a name; the operator still comes only from the declared terms (Defi-

nition 8.7). Ordinary and special dividends are two classes because the adjustment-schedule rule language reads only the declared parameters: an ordinary €2 and a special €2 dividend are indistinguishable by (f, c) , yet listed conventions treat them oppositely, and only the class lets a schedule say so (Principle 8.15). **CATermination** unifies three names because their fate is identical: **Terminal**. The consideration amount is notice data, not a class.

A rights issue is **CASpinOff**: the right is a tradable series distributed pro rata to holders of the parent. The right registers as the child and is priced only by fresh stamps, never by a theoretical value. Its later exercise is subscription activity of the right itself — a lifecycle of that unit, not a corporate action on the parent. An issuer-level scrip election is the elective class, whose content is any declared election structure with a confirmed proration blend. A pure stock dividend is **CASplit**. A reinvestment plan is a standing instruction executing as market purchases after the ordinary distribution, and declares nothing.

Two events are excluded on purpose: delisting with continuation, a venue fact with no boundary event (§8.12); and non-corporate-action terms amendment, the table’s last row, which declares nothing. Extending the taxonomy is a change to this specification, never a change to data: a notice outside the eight classes is refused at the door, fail-closed. The schedule totality gate C14 quantifies over exactly this closed set, and each row of the table below carries its class.

One wall stands over the whole pipeline. An operator re-expresses a datum — the same fact in new coordinates — and never updates its information content. The repricing an action itself causes is new information: implied volatility collapses on a takeover announcement, the target pins to the offer, a spun child finds its level. New information enters only as fresh stamped observations through the ingest door (Principle 8.16); under the model-free directive (§8.2) the discipline never manufactures it by transforming old observations. Transported data mark the book at old information in new coordinates; the gap to the next fresh print is market PnL and appears as such, never absorbed into the boundary.

CDM **BusinessEvents** already map to transactions through the map F of §14, which keeps a business event’s economic content and drops its CDM-specific wrapping; “CDM” denotes Common Domain Model v6.0.0 throughout, as pinned in Appendix F. The extension is one rule. A corporate-action event’s instructions supply both halves of τ : the moves, as today, and the declaration, keyed on the event’s declared terms and never on its taxonomy name. F lands in (moves, **SetBasis** + declaration) instead of moves alone. It remains total and deterministic, and the one economic fact it now keeps — the basis change the action causes — is exactly the fact the old map used to discard.

CDM representation	Moves (existing)	Declaration (new)
StockSplit / ReverseSplit / StockDividend (QuantityChange, ratio r) — class CASplit	entitlement $\times r$ via the corporate-action virtual wallet (§6)	$\text{KSpot} \mapsto \text{Scale } r$, the quantity factor, per-share reference kinds likewise; weld checks $q' = q \cdot r$
CashDividend / coupon detachment (Transfer) — class CADividendOrdinary or CADividendSpecial , as the notice declares	cash issuer $\rightarrow$ holders	$\text{KSpot} \mapsto \text{Shift}(-d)$ on cum-quoted kinds; weld checks cash = $q \cdot d$; subsumes §D

CDM representation	Moves (existing)	Declaration (new)
SpinOff — class CASpinOff	issuance moves of the child; child registered at its origin	parent kinds Pending <i>ref</i> until allocation publishes; the publication event crystallises the parameter; pair-level bridge per the basket mechanics of §8.12 (Definition 8.7), which exits on fresh per-component stamps (when-issued prints in the ordinary course), not only on the allocation publication
Merger, stock-for-stock (TermsChange + succession) — class CAMergerStock	C8 Breaking amendment + paired issuance; usSupersededBy	Subst <i>u' r</i> — the cross-unit conversion; predecessor data reach the successor's basis through it
Merger for cash — class CATermination	terminal moves	Terminal — no conversion in either direction; fail-closed forbids any further crossing
Index rebalance / divisor change — class CAREbalance	rebalance moves on the strategy unit	boundary on the index unit via the basis-notice handler; $KComposition \mapsto Recompose \sigma$, under W4
Election / proration windows — class CAMergerElective	per-holder entitlement differences are moves	boundary Pending during the window; the unit-level operator crystallises at proration — Status stays read identically by every holder (Principle 8.5)
TermsChange (non-corporate-action amendment)	C6/C8 as today	no declaration; the basis advances only on declared basis change

Key Invariants and Consequences

- **One coordinate, one seam (C13).** The corporate-action basis is a field of **UnitStatus**, folded from logged boundary events. Every consumed datum carries a joint basis stamp. Consumption requires pointwise agreement with the total assignment β_t , or a ledger-authored conversion. C13 is the data-plane counterpart of P3. (§8.1, §8.8.)
- **Value invariance is a theorem, not a property claim.** The invariance weld makes the identity $(qf) \cdot \frac{p-c}{f} = qp - qc$ a condition of admission, so marked value is continuous through every boundary up to exactly the booked cash (Theorem 8.11) — per position across uniform boundaries, in aggregate at the confirmed blend across elective ones. The lifecycle value-invariance property of §1 is thereby proved on both factors for basis boundaries.

- **No regression, no reordering.** `SetBasis` is absolute and idempotent, so P6 is unchanged. The tip weld makes a basis regressed by a late notice unrepresentable, and replay in booking order agrees with the effective-order chain projection on `usBasis` at every prefix of the log (Principle 8.4).
- **Fail-closed by expressibility.** A declaration is a partial map over datum kinds: a kind it does not mention has no conversion and cannot cross the boundary. `Pending` and `Terminal` generate no conversion. Identity is declared, never assumed. (§8.2.)
- **The phantom is unrepresentable in a well-typed program.** Within the stated language fragment, every (quantity, price-like) pair that a well-typed pricing function can form lies in one basis assignment; at the erased boundary the same discipline is a typed refusal, never a silent pick. A mixed-basis value is unrepresentable as a `Cash` and representable only as an enumerated `Valuation` exclusion. (§8.9, §8.11.)
- **Two flows at the market-data boundary.** The basis projection flows out: read-only, versioned, and free of arithmetic. Raw observations flow in through the single ingest door, where the ledger attaches the stamp. Door soundness (P25) admits nothing whose stamped basis the log has not committed, and the stamp is a pure function of pinned recorded data, so real-time, batch, and multi-vendor operation coincide and no corporate-action logic can exist outside the ledger. (§8.5.)
- **Time travel needs no new machinery.** `clone_at(t)` rebuilds `usBasis` by the existing replay fold (P8). The basis axis is effective order; the knowledge axis is bitemporal. Late notices repair through re-stamp events with citable lineage, and the live coordinate never moves backward. (§8.10.)
- **Protected elements untouched.** The atomic move, conservation, and log immutability (P1, P2, P4) hold in unchanged obligation. The whole discipline consists of a logged event class, the `SetBasis` status write, admission conditions at the single door, a named projection, and a typed seam.

The invariant checks the two numbers together, not one at a time. A quantity and a price can each be individually correct and still belong to different bases; the phantom PnL is exactly what multiplying them in that state produces. When both are typed in the same basis assignment, that product cannot be formed at all, so the phantom PnL is unrepresentable.

9 The Futures Lifecycle

A listed future stresses every primitive at once. An equity trade is one transaction: cash one way, shares the other. A futures trade reads the same, “buy 10 at 100,” yet moves no cash at inception. Its economic effect unfolds over days, through variation margin, partial closes, and settlement. The future is therefore the canonical lifecycle. It is recorded, traded, settled daily, expired, and flattened to zero. Each step is one atomic `FutStateDelta` over the move and conservation primitives of §2 and the three-home state model of §4. The `Fut*` types form a futures-engine kernel that *projects onto* the core `Transaction`, exactly as the CDM and settlement boundary types do, under the same naming discipline. The kernel is not a second core. Conservation is shown at each step, not asserted. The centrepiece is variation-margin settlement: one event writes the shared settlement price once, fans out over the current holders, resets each accumulated cost, and moves cash. That settlement price is an externally sourced mark, consumed under single-basis consumption (Invariant 8.17): the mark and the positions it settles stand in one basis.

9.1 Notation, and where the future’s state lives

The primitives are those of §2: wallet, unit, move, conservation. The homes are those of §4. Write $\text{net_qty}(w, u) := h(w, u)$ for the signed holding of wallet w in unit u .

UnitStatus is a projection, not a source of truth: a shared per-unit read cache of the event log, rebuilt exactly by replay (§4.3).

For the future, the fields divide by who reads them and how often they change:

Field	Home	Reason
multiplier, currency, expiry, clearinghouse, exchange, product_id	<i>ProductTerms</i> [u_F]	immutable; the CME and ICE contracts are distinct units.
lifecycle_stage, last_settlement_price, last_settlement_date	<i>UnitStatus</i> [u_F]	one settlement value per contract, read identically by every holder.
accumulated_cost (ac)	<i>PositionState</i> [w, u_F]	two wallets holding the same contract carry different ac.

Accumulated cost **ac** is a *conserved* field: $\sum_w \text{ac}(w, u) = 0$ for every u . The identity is enforced one level up, at the event handler (C2), because no production language provides a refinement type over a sum of exact integers for free.

The atomic delta. One event proposes one **FutStateDelta** for one unit across the three maps: at most one shared **FutStage** write, a map of per-holder position deltas ($\Delta \text{net_qty}$, Δac), and a map of per-holder cash legs. The delta applies in full or not at all (C3). A partially applied delta is not representable. A delta reaches application only through **futValidate**. That gate discharges conservation — $\sum_w \Delta f(w, u) = 0$ for each conserved field — and returns the abstract witness **FutValidDelta**. An unconserved delta cannot be applied.

The futures kernel is a boundary, not a core. **FutStateDelta** and its validated form **FutValidDelta** are the futures engine’s own constructions, named apart from the core **Transaction**. Each validated futures delta *projects onto* one core **Transaction**, recorded through the single core door **applyTx**. The projection carries the conserving moves (the edge-sum of the per-holder legs) together with the non-balance rows (the **ac** resets and the shared stage write). The local **futValidate** discharges conservation *within* the futures kernel before it projects out; the core therefore keeps no separate validate gate and remains sealed.

The single dimension bridge. Contracts, money, and price are distinct types (**Qty**, **Cash**, **Price**). The only multiplication in the lifecycle,

$$\text{futMark}(\text{net_qty}, S, m) = \text{net_qty} \cdot S \cdot m : \text{Cash},$$

converts contracts at a price into money. **Price** carries no addition: two prices are never summed, nor moved between wallets. The load-bearing identity $\text{VM} = \text{net_qty} \cdot S \cdot m + \text{ac}$ typechecks only because both summands are **Cash**. **Price** and **Cash** share one fixed minor-unit scale, chosen so $\text{net_qty} \cdot S \cdot m$ lands in **Cash**’s unit. The worked example carries **Price** in whole index points and **Cash** in whole dollars for readability. A production scale carries **Price** in scaled minor units — a quarter-point for ES, a cent for a bond quoted in 32nds — so sub-point ticks are exact. The arithmetic is integer throughout and never divides, so the choice of scale is free: at any scale there is no rounding to break conservation.

The scalar layer — the three dimensions, **futMark**, and the **PosQty** parse boundary of §9.4 — is the kernel’s foundation. The type discipline encodes one mental model: quantities and cash add, prices do not, so “a price summed into a balance” is a type error rather than a bug to catch.

-- *Three dimensions kept apart. Qty and Cash are additive groups; Price is not*
 -- *(you never sum two prices, nor move one between wallets) -- so Price has no*

```

-- Monoid instance, and that absence makes "a price summed into a balance"
-- unspellable rather than merely unintended.
newtype Qty    = Qty    Integer deriving (Eq, Ord, Show)    -- additive group: the
    (<>)/mempty
newtype Cash   = Cash   Integer deriving (Eq, Ord, Show)    -- instances of sec:
    contracts
newtype Price  = Price  Integer deriving (Eq, Ord, Show)    -- deliberately NO Monoid
newtype Day    = Day    Int    deriving (Eq, Ord, Show)

qneg    :: Qty -> Qty ; qneg    (Qty n) = Qty (negate n)
cashNeg :: Cash -> Cash; cashNeg (Cash n) = Cash (negate n)

-- The single dimension bridge: contracts at a price, scaled by the multiplier,
-- become money. The only multiplication in the lifecycle, and the only place the
-- dimensions cross. VM = futMark ... <> ac typechecks only because both are Cash.
futMark :: Qty -> Price -> Integer -> Cash
futMark (Qty q) (Price s) mult = Cash (q * s * mult)

newtype WalletId = WalletId String deriving (Eq, Ord, Show)
newtype UnitId   = UnitId   String deriving (Eq, Ord, Show)

-- The positive-quantity parse boundary (G5). PosQty is abstract: mkPosQty is its
-- sole constructor, so q <= 0 is unrepresentable downstream of the boundary.
newtype PosQty = PosQty Qty deriving (Eq, Ord, Show)

mkPosQty :: Qty -> Maybe PosQty
mkPosQty q@(Qty n) | n > 0      = Just (PosQty q)
                  | otherwise = Nothing

unPosQty :: PosQty -> Qty
unPosQty (PosQty q) = q

```

Evaluations. futMark (Qty 10) (Price 100) 50 = Cash 50000; futMark (Qty (-10)) (Price 102) 50 <> Cash 51000 = Cash 0 (a Cash sum); Price 100 <> Price 102 is a type error — Price has no <>.

9.2 The instrument and the worked example

Definition 9.1 (The worked instrument). *One cash-settled listed future $u_F = ES-FUT$, with $m = 50$, currency USD, expiry Day 3, clearinghouse CME-CH, exchange CME. Wallets A , B , C are traders; CH is the clearinghouse virtual wallet, the variation-margin settlement hub whose cash leg is the residual that balances the holders' legs.*

CH stands in as a settlement hub, not as a *novating* central counterparty. Faithful novation would interpose CH on the *contract* leg of every trade ($A+q/CH-q$ and $CH+q/B-q$), so that the two cleared sides never face each other. Clearing infrastructure is out of scope. Here CH carries only the variation-margin cash residual. Two rules govern every figure:

- **Trade.** For a signed-quantity change Δ at price p , each leg sets $ac \ += \ -\Delta \cdot p \cdot m$.
- **Settlement.** At price S , the per-holder variation margin is $VM(w) = net_qty(w) \cdot S \cdot m + ac(w)$ (cash, routed through CH); then $ac(w) \leftarrow -net_qty(w) \cdot S \cdot m$.

The whole life, with net_qty , ac , and per-step variation-margin cash for (A, B, C) :

Step	Event	net_qty (A,B,C)	ac (A,B,C)	VM cash (A,B,C)
Listing	u_F registered; stage REGISTERED, no mark; no rows	(0, 0, 0)	(-, -, -)	-
T1	A buys 10 from B @100	(10, -10, 0)	(-50000, +50000, 0)	-
Settle d1	$S_1 = 102$; shared price $\leftarrow 102$	(10, -10, 0)	(-51000, +51000, 0)	(+1000, -1000, 0)
T2	C buys 4 from A @103	(6, -10, 4)	(-30400, +51000, -20600)	-
Settle d2	$S_2 = 101$; shared price $\leftarrow 101$	(6, -10, 4)	(-30300, +50500, -20200)	(-100, +500, -400)
T3	B buys 4 from C @101 (C $\rightarrow$ flat)	(6, -6, 0)	(-30300, +30300, 0)	-
Expiry	stage $\rightarrow$ EXPIRED; final $S = 105$	(6, -6, 0)	(-31500, +31500, 0)	(+1200, -1200, 0)
Close	units returned to CH; positions $\rightarrow 0$	(0, 0, 0)	(0, 0, 0)	-

The subsections below walk this table in event order. Each event is one `FutStateDelta` showing the three conservation sums $\sum \Delta \text{net_qty}$, $\sum \Delta \text{ac}$, $\sum \text{VM}$. The example is the argument; the prose names the mechanism.

9.3 Listing

Registration writes `ProductTerms[uF]` (the six immutable terms) and `UnitStatus[uF]` together, establishing “ $u \in \text{terms} \iff u \in \text{status}$ ” by construction. The status initialises to stage REGISTERED with no settlement mark. No `PositionState` row exists: `futPosition(w, uF) = None` for every w .

REGISTERED denotes a unit recorded in and known to the ledger before any settlement, not admission to an exchange. An OTC option or an internal strategy reaches it identically. Two status states are unreachable: a never-traded unit carrying a price, and an expired unit lacking a final mark. Both are made unspellable by fusing the mark onto the stage:

`FutStage = FutRegistered | FutActive(Maybe Settlement) | FutExpired Settlement.`

`FutRegistered` carries no mark; `FutExpired` always carries one. `last_settlement_price` and `last_settlement_date` are projections of the `Settlement` carried by the stage, not independent fields. Conservation holds vacuously: no holders, so every sum is the empty sum, zero (C9). Re-registering an existing unit id is a hard error, never a silent reset (C10).

The shared status type and its registration follow.

```
-- FutTerms: the six immutable contract terms, one per unit, co-registered with
-- FutStatus. ptMultiplier drives mark value and the settlement fan-out. The Fut*
-- prefix marks this engine as a futures-boundary construction, distinct from the
-- core types: it PROJECTS onto the core Transaction (see "The atomic delta").
```

```
data FutTerms = FutTerms
  { ptMultiplier    :: !Integer
  , ptCurrency      :: !String
  , ptExpiry        :: !Day
  , ptClearinghouse :: !String
  , ptExchange      :: !String
  , ptProductId     :: !String
  } deriving (Eq, Show)
```

```
-- The mark is FUSED onto the stage, so the two unreachable states
-- (FutRegistered, Just p) and (FutExpired, Nothing) are unspellable.
-- FutRegistered carries no mark; FutExpired always carries one; FutActive may or
-- may not.
```

```
data Settlement = Settlement { settlePrice :: !Price, settleDate :: !Day }
  deriving (Eq, Show)
```

```
data FutStage
  = FutRegistered           -- recorded, never traded: no holders, no mark
  | FutActive (Maybe Settlement) -- trading; Nothing = not yet settled, Just s =
    marked
```

```

    | FutExpired Settlement          -- final mark ALWAYS present; absorbing
    deriving (Eq, Show)

newtype FutStatus = FutStatus { fusStage :: FutStage } deriving (Eq, Show)

-- last_settlement_price/_date are PROJECTIONS of the mark the stage carries,
-- total over all three stages -- not independent fields.
settlement :: FutStage -> Maybe Settlement
settlement FutRegistered = Nothing
settlement (FutActive m) = m
settlement (FutExpired s) = Just s

settlementPrice :: FutStatus -> Maybe Price
settlementPrice = fmap settlePrice . settlement . fusStage
settlementDate  :: FutStatus -> Maybe Day
settlementDate  = fmap settleDate  . settlement . fusStage

-- The coarse rank REGISTERED < ACTIVE < EXPIRED: the one fact futApplyDelta needs
-- to reject a regressing stage. It never regresses.
stageRank :: FutStage -> Int
stageRank FutRegistered = 0
stageRank (FutActive _) = 1
stageRank (FutExpired _) = 2

-- Rank alone cannot separate FutExpired 105 from FutExpired 110, so a strict new<
-- cur
-- guard would re-admit a second Expire. isExpired is the precise absorbing test.
isExpired :: FutStage -> Bool
isExpired (FutExpired _) = True
isExpired _              = False

futRegisteredStatus :: FutStatus
futRegisteredStatus = FutStatus FutRegistered

```

Evaluations. `settlementPrice (FutStatus FutRegistered) = Nothing`; `settlementPrice (FutStatus (FutExpired (Settlement (Price 105) (Day 3)))) = Just (Price 105)`.
The states `(FutRegistered, Just p)` and `(FutExpired, Nothing)` cannot be written.

9.4 First trade

A buys 10 contracts from B at 100. The trade emits one `FutStateDelta`:

- **Shared.** REGISTERED $\rightarrow$ ACTIVE with no mark (`FutActive Nothing`); the first trade activates the unit. Any existing mark is preserved — a trade never wipes the shared price.
- **Per-position.** The contract quantity moves from seller to buyer: $\Delta_{\text{net_qty}}(A) = +10$, $\Delta_{\text{net_qty}}(B) = -10$. Each leg sets $\text{ac} += -\Delta \cdot p \cdot m$:

$$\Delta_{\text{ac}}(A) = -(+10) \cdot 100 \cdot 50 = -50000, \quad \Delta_{\text{ac}}(B) = -(-10) \cdot 100 \cdot 50 = +50000.$$

- **Cash.** None: entering a futures position pays no notional.

Conservation: $\sum \Delta_{\text{net_qty}} = 0$, $\sum \Delta_{\text{ac}} = 0$, $\sum \text{VM} = 0$. State after T1: `net_qty = (10, -10, 0)`, `ac = (-50000, +50000, 0)`. A and B cross from None to Some on first touch; C is still None.

Positive quantity. A trade leg carries a strictly positive quantity $q > 0$, parsed at the boundary (`mkPosQty`). $q \leq 0$ is rejected (`NonPositiveQty`) and never recorded. $q = 0$ would promote `REGISTERED`→`ACTIVE` and create Some-flat rows for two never-held wallets, collapsing the never-held/held-flat distinction (§9.8). $q < 0$ would swap buyer and seller, violating the move primitive’s positivity. The sign of the position change is supplied by the leg — buyer $+q$, seller $-q$ — not by q .

The per-event delta, its conserved image, and the conservation witness follow. The mental model: totalling a delta is addition in a monoid, “conserving” means the total is zero, and the witness type makes an unconserved delta impossible to apply.

```
-- FutPos per (wallet, unit). ac is per (w,u): two wallets holding the same
-- contract carry different ac -- the field that makes intraday margin correct.
data FutPos = FutPos { fpNetQty :: !Qty, fpAccumCost :: !Cash }
  deriving (Eq, Show)

futZeroP :: FutPos
  -- flat row created on first touch; held-and-flat
  -- baseline
futZeroP = FutPos mempty mempty

-- Conservation is a monoid identity. Totalling a delta is a monoid homomorphism
-- -- a map that respects (<>) and mempty -- into Conserved; "conserving" means
  -- the
-- image is mempty. The zero-holder case is the empty fold, mempty (C9); we SUM
-- deltas, never apportion, so no divide-by-holder-count bug can arise.
data Conserved = Conserved !Qty !Cash !Cash deriving (Eq, Show) -- (sum net, sum
  ac, sum cash)
instance Semigroup Conserved where
  Conserved a b c <> Conserved a' b' c' = Conserved (a<>a') (b<>b') (c<>c')
instance Monoid Conserved where mempty = Conserved mempty mempty mempty

data FutRowDelta = FutRowDelta { rdNetQty :: !Qty, rdAc :: !Cash } deriving (Eq,
  Show)

-- One event's proposed change: at most one shared FutStage write, per-holder
  -- additive
-- row deltas, per-holder cash legs. Applied in full or not at all (C3).
data FutStateDelta = FutStateDelta
  { fsdUnit :: UnitId, fsdStage :: Maybe FutStage
  , fsdRows :: Map WalletId FutRowDelta, fsdCash :: Map WalletId Cash } deriving (
  Show)

futNetDelta :: FutStateDelta -> Conserved
futNetDelta sd =
  foldMap (\(FutRowDelta nq ac) -> Conserved nq ac mempty) (fsdRows sd)
  <> foldMap (\c -> Conserved mempty mempty c) (fsdCash sd)

-- The C2 witness, ABSTRACT: futValidate is the only constructor, so an
  -- unconserved
-- delta cannot reach futApplyDelta. This is the futures engine's OWN conservation
-- discharge, distinct from the core (which has no validate gate); the validated
-- delta projects onto one conserving core Transaction. Conservation is a value-
  -- level
-- fact (a sum of exact integers), so this is a parse boundary, not a free type
  -- fact.
newtype FutValidDelta = FutValidDelta FutStateDelta

futValidate :: FutStateDelta -> Either FutLedgerError FutValidDelta
```

```

futValidate sd | net == mempty = Right (FutValidDelta sd)
                | otherwise      = Left (FutNotConserved (fsdUnit sd) net)
    where net = futNetDelta sd

-- A trade activates without wiping any existing mark, and sets each leg's
-- ac += -(delta signed_qty) * p * m (buyer +q, seller -q). No cash: entry pays no
-- notional.
activateTrade :: FutStage -> FutStage
activateTrade cur = FutActive (settlement cur)

tradeDelta :: UnitId -> FutStage -> Integer
              -> WalletId -> WalletId -> PosQty -> Price -> FutStateDelta
tradeDelta u cur m buyer seller pq p = FutStateDelta
  { fsdUnit = u, fsdStage = Just (activateTrade cur)
  , fsdRows = Map.fromList
    [ (buyer , FutRowDelta q      (cashNeg (futMark q p m)))
    , (seller, FutRowDelta (qneg q) (futMark q p m)) ]
  , fsdCash = Map.empty }
  where q = unPosQty pq

```

9.5 Daily variation-margin settlement: the mechanism and the clean case (day 1)

Settlement pays each holder the day's mark-to-market in cash and re-bases the position at the new price. Formally: settlement at price S , multiplier m , over the current holders. For each holder the reset target is $\text{target} = -\text{net_qty} \cdot S \cdot m$, the row delta is $\Delta\text{ac} = \text{target} - \text{ac}$, and the cash leg is

$$\text{VM}(w) = -\Delta\text{ac}(w) = \text{net_qty}(w) \cdot S \cdot m + \text{ac}(w)$$

The cash leg is the mirror of the change in accumulated cost. Therefore $\sum_w \text{VM} = -\sum_w \Delta\text{ac}$, and variation-margin zero-sum is the *same* fact as ac conservation, not a separate reconciliation. The reset discards the entry prices: with $\text{ac} \leftarrow -\text{net_qty} \cdot S \cdot m$, the next day's margin needs only S and the position. Each daily settlement economically closes and reopens the position at the new price (the resettable-forward property).

Settlement is one atomic `FutStateDelta` touching both layers:

- **Shared**, one write: `UnitStatus[uF]` acquires the mark (S, d) . The *coarse rank* `REGISTERED < ACTIVE < EXPIRED` is unchanged: the unit stays `ACTIVE`. The *embedded mark* updates: the stage becomes `FutActive (Just (Settlement S d))`. The settlement price enters the log only as the `FSettleVM` event. `futReplay` rebuilds the mark, and the fold stays pure at the boundary: the shared value is a projection of the log, never an off-log write.
- **Per-position**, a fan-out over the current holders: each row's ac is reset to target, and each holder gets the cash leg $\text{VM}(w)$.

The cash legs are emitted over the holders; the clearinghouse leg is the residual that balances them. Since the holder legs already sum to zero, CH's leg is zero throughout this example and no CH cash row materialises. This is the special case of a *closed* holder set $\{A, B, C\}$, where $\sum_w \text{net_qty}(w) = 0$ at every step. In the Ledger's intended use — an internal firm system of record facing an external clearinghouse — the firm is generally *not* flat. When $\sum_w \text{net_qty}(w) \neq 0$ over the firm's holders, the CH leg is the firm's nonzero net variation margin: the boundary figure reconciled against the clearing broker's daily statement.

Day 1: $S_1 = 102$. Holders A and B; C has no row and is untouched — the contract settles vacuously over C; the shared price still updates (C9). With pre-settle $\mathbf{ac} = (-50000, +50000)$ and $\mathbf{net_qty} = (10, -10)$:

$$\mathbf{VM}(A) = 10 \cdot 102 \cdot 50 - 50000 = +1000, \quad \mathbf{VM}(B) = -10 \cdot 102 \cdot 50 + 50000 = -1000.$$

Reset: $\mathbf{ac}(A) \leftarrow -51000$, $\mathbf{ac}(B) \leftarrow +51000$. Conservation: $\sum \Delta \mathbf{net_qty} = 0$, $\sum \Delta \mathbf{ac} = 0$, $\sum \mathbf{VM} = 0$. State: $\mathbf{net_qty} = (10, -10, 0)$, $\mathbf{ac} = (-51000, +51000, 0)$.

The fan-out, shared by settlement and expiry, is one pure function of the current holders. Per holder it does two things: reset $\mathbf{ac}$ to the new-price baseline, and pay the mirror of that reset as cash.

```
-- Shared by FSettleVM and FExpire. For each holder at price S, multiplier m:
--   delta ac = (-(net*S*m)) - ac      VM = -(delta ac) = net*S*m + ac
-- so VM = -(delta ac) exactly: cash conservation IS ac conservation, not a
--   separate
-- reconciliation. A flat holder gives delta ac = 0, VM = 0; no holders -> empty
--   fan-out.
settlementFanout :: Price -> Integer -> [(WalletId, FutPos)]
                  -> (Map WalletId FutRowDelta, Map WalletId Cash)
settlementFanout s m holders =
  ( Map.fromList [ (w, FutRowDelta mempty (deltaAc ps)) | (w, ps) <- holders ]
  , Map.fromList [ (w, cashNeg (deltaAc ps))                | (w, ps) <- holders ] )
  where deltaAc ps = cashNeg (futMark (fpNetQty ps) s m) <> cashNeg (fpAccumCost ps)
```

9.6 A subsequent, intraday trade (day 2)

C buys 4 contracts from A at 103, after the day-1 settle and before the day-2 settle — A trades intraday. One `FutStateDelta`, stage already `FutActive` (mark preserved):

$$\Delta \mathbf{net_qty}(C) = +4, \quad \Delta \mathbf{net_qty}(A) = -4; \quad \Delta \mathbf{ac}(C) = -20600, \quad \Delta \mathbf{ac}(A) = +20600.$$

Conservation: $\sum \Delta \mathbf{net_qty} = 0$, $\sum \Delta \mathbf{ac} = 0$, $\sum \mathbf{VM} = 0$. State after T2: $\mathbf{net_qty} = (6, -10, 4)$, $\mathbf{ac} = (-30400, +51000, -20600)$. C crosses from None to Some on first touch.

9.7 Settlement under intraday trading (day 2): the anchor

The day-2 settle at $S_2 = 101$ runs the same fan-out over holders A, B, C:

$$\begin{aligned} \mathbf{VM}(A) &= 6 \cdot 101 \cdot 50 - 30400 = -100, \\ \mathbf{VM}(B) &= -10 \cdot 101 \cdot 50 + 51000 = +500, \\ \mathbf{VM}(C) &= 4 \cdot 101 \cdot 50 - 20600 = -400. \end{aligned}$$

Conservation: $\sum \Delta \mathbf{net_qty} = 0$, $\sum \mathbf{VM} = -100 + 500 - 400 = 0$. Reset $\mathbf{ac} \leftarrow (-30300, +50500, -20200)$ with $\sum \Delta \mathbf{ac} = 0$. State: $\mathbf{net_qty} = (6, -10, 4)$, $\mathbf{ac} = (-30300, +50500, -20200)$.

Principle 9.2 (The intraday-margin subtlety forces stored per-position $\mathbf{ac}$ (C11)). *A's day-2 variation margin is -100 , not the naive $\mathbf{net_qty} \cdot (S_2 - S_1) \cdot m = 6 \cdot (101 - 102) \cdot 50 = -300$. A sold 4 contracts at 103 intraday, one point above the prior mark 102, gaining $4 \cdot (+1) \cdot 50 = +200$, which offsets the -300 mark loss on the held position. Only the stored per-wallet $\mathbf{ac}$, which absorbed the intraday trade, yields the correct -100 . A price-derived model gives the wrong cash. Accumulated cost is therefore per-position stored state with a single canonical writer (C11), not a consequence read off the shared price.*

The three settlement facts, stated once:

1. *Settlement is a state update split by layer.* The shared part — `last_settlement_price`, `last_settlement_date` — is one write on `UnitStatus[uF]`, one value per contract. The per-position part — the `ac` reset and the cash leg — is a fan-out over the current holders on `PositionState[w, uF]`. One atomic `FutStateDelta` touches both.
2. *One atomic event that fans out, not a derived consequence of the price.* The cash leg is real daily money moving between longs and shorts through CH. Every cash move is a recorded, conservation-bearing event, so a per-holder pass is unavoidable. The `ac` reset rides in the same atomic delta. The fan-out is forced by the cash, not chosen for the bookkeeping.
3. *Price only in shared state, consequence only in per-wallet state.* The consequence of the shared price — the `ac` reset and the variation-margin cash — lives only in `PositionState[w, uF]` and the move stream.

Remark 9.3 (Direction reversal needs no special case). *The accumulated-cost rule handles a long-to-short reversal in one trade with no branch. From `net_qty = +5`, `ac = -500` (entered at 100, $m = 1$), a single sell of 8 at 105 sets $\Delta ac = -(-8) \cdot 105 \cdot 1 = +840$, so `ac = -500 + 840 = +340` and `net_qty = -3`. The economic value at 105 is $340 + (-3) \cdot 105 = 25$, exactly the gain from closing the long 5 at \$5 per contract; the new short 3 marks forward from here with no extra bookkeeping. The same arithmetic that accumulates a position unwinds and reverses it.*

9.8 Closing a position to flat

B buys 4 contracts from C at 101; C goes flat. One `FutStateDelta`:

$$\Delta net_qty(B) = +4, \Delta net_qty(C) = -4; \quad \Delta ac(B) = -20200, \Delta ac(C) = +20200.$$

Conservation: $\sum \Delta net_qty = 0$, $\sum \Delta ac = 0$, $\sum VM = 0$. State: `net_qty = (6, -6, 0)`, `ac = (-30300, +30300, 0)`. C is now flat. Its row is *retained*, Some at (0, 0) — not deleted, not None. The two readings are distinct and both load-bearing. None means never held. Some-flat means held and now flat; it is read after close-out for tax reporting, wash-sale lookback, and trade reconstruction (C1(a)). The carrier is monotone: once a row is created it is never removed (C1(b)). No row deleter exists, because the ledger is abstract.

9.9 Expiry, final settlement, and the Close

Expiry is the settlement fan-out of §9.5 composed with a stage write to **EXPIRED**. Like a settle, it acts only on an **ACTIVE** unit, never a never-traded one (§17). The final mark $S = 105$ fans out over the remaining holders A and B (C is flat; its row is touched to no effect):

$$VM(A) = 6 \cdot 105 \cdot 50 - 30300 = +1200, \quad VM(B) = -6 \cdot 105 \cdot 50 + 30300 = -1200.$$

Conservation: $\sum \Delta net_qty = 0$, $\sum VM = 0$; `ac` $\leftarrow (-31500, +31500, 0)$ with $\sum \Delta ac = 0$. The coarse rank advances to **EXPIRED**: the stage becomes `FutExpired (Settlement 105 (Day 3))`; the final mark is always present.

Cash settlement and the Close. No unit is delivered. The economic result is the cumulative variation margin already moved. The *Close* step extinguishes every holder's position against the clearinghouse, flattening `net_qty` and `ac` by an additive negation of each row, with no further cash. Wallet A held +6 and B held -6; closing them returns the contracts to CH:

$$\Delta net_qty(A) = -6, \Delta net_qty(B) = +6; \quad \Delta ac(A) = +31500, \Delta ac(B) = -31500.$$

The clearinghouse legs are the residual of the holder legs — zero, since the holder legs balance — so no CH row materialises. Conservation: $\sum \Delta \text{net_qty} = 0$, $\sum \Delta \text{ac} = 0$, $\sum \text{VM} = 0$. State after Close: $\text{net_qty} = (0, 0, 0)$, $\text{ac} = (0, 0, 0)$; rows retained at zero (monotone carrier). Close carries no stage write, so it is the one event still admissible on an EXPIRED unit.

Physical settlement. The variant differs only at the Close. Instead of flattening against CH for cash, the contract converts to delivery. The deliverable unit and the cash payment move together as one atomic transaction (delivery versus payment; the settlement interface is §13). The future’s net_qty goes to zero, and the holder’s wallet gains the deliverable. The cash leg is struck at the *final settlement price* S , not the trade price. Expiry has already marked every position to S through variation margin, so delivery exchanges the deliverable for $\text{net_qty} \cdot S \cdot m$ and moves no further mark-to-market. Pricing the delivery at any other figure would double-count the variation margin already settled. Per-unit conservation holds on each unit separately — the future, the deliverable, the cash — and the cross-unit delivery is one all-or-nothing transaction. The worked example is cash-settled; the figures above are unchanged by this variant.

Theorem 9.4 (Closing identity: cash ties to economic PnL). *Cumulative variation margin per wallet equals the wallet’s economic profit and loss. Here $A = +1000 - 100 + 1200 = +2100$, $B = -1000 + 500 - 1200 = -1700$, $C = -400$, $CH = 0$, summing to 0; and $A = 4 \cdot (103 - 100) \cdot 50 + 6 \cdot (105 - 100) \cdot 50 = +2100$, $B = 4 \cdot (100 - 101) \cdot 50 + 6 \cdot (100 - 105) \cdot 50 = -1700$, $C = 4 \cdot (101 - 103) \cdot 50 = -400$.*

The terminal flatten delta has no stage write and no cash leg. It conserves for free: each conserved field already sums to zero over the holders, so the row-by-row negation sums to zero too.

```
-- Terminal flatten: NO stage write (stays EXPIRED, so it does not breach the
-- absorbing rule), NO cash; every row driven to zero by additive negation. It
-- conserves because net and ac each already sum to zero, so their negations do
-- too.
closeDelta :: UnitId -> [(WalletId, FutPos)] -> FutStateDelta
closeDelta u holders = FutStateDelta
  { fsdUnit = u, fsdStage = Nothing
  , fsdRows = Map.fromList
    [ (w, FutRowDelta (qneg (fpNetQty ps)) (cashNeg (fpAccumCost ps)))
    | (w, ps) <- holders ]
  , fsdCash = Map.empty }
```

9.10 After expiry: state versus derivation

After the Close, every row is retained at zero: $\text{PositionState}[w, u_F] = \text{Some flat}$ for A, B, C. $\text{UnitStatus}[u_F]$ holds stage EXPIRED and final mark 105. $\text{ProductTerms}[u_F]$ is unchanged — immutable.

`first_touch_date` is *not* stored: it is derived from the event log on demand. Caching it in PositionState would make a replay disagree with itself under a back-dated correction: the cached value would reflect the pre-correction order while the fold reflects the corrected one. The rule is general. What the fold over the log determines is derived, not stored. Only what the fold cannot reconstruct from prior events is state.

9.11 Invariants threaded through the life

Invariant 9.5 (Per-event conservation, C2). *Every FutStateDelta satisfies $\sum_w \Delta \text{net_qty} = 0$, $\sum_w \Delta \text{ac} = 0$, $\sum_w \text{VM} = 0$, discharged by futValidate before application. Induction over the*

stream lifts the per-event identity to the global invariant (P1). The vacuous case — a unit with no holders — is the empty sum, zero (C9); conservation sums deltas and never divides by a holder count.

Invariant 9.6 (Deterministic replay as a fold, C1(b), P8). *futReplay* is a Kleisli fold (an error-stopping fold whose combining step halts at the first failure) of the event stream over the ledger: $\text{futReplay}(xs \triangleleft ys) = \text{futReplay}xs \gg \text{futReplay}ys$ — replaying a concatenation equals composing the replays. Checkpoint independence is a consequence of this law, not a test; the monotone carrier keeps the key set stable across cuts. The stream is assumed unique — de-duplicated at the ingestion boundary by an event key — because the two event kinds differ in idempotency: a duplicated *FSettleVM* at a fixed mark is inert ($\Delta ac = 0$, $VM = 0$), whereas a duplicated *FTrade* accumulates a second position delta. Duplicate suppression is a boundary obligation, not a ledger function.

Invariant 9.7 (Monotone, absorbing stage). The coarse rank *REGISTERED* < *ACTIVE* < *EXPIRED* never regresses; a proposed stage of strictly lower rank is rejected (*StageRegression*). Settle and expire act only on an *ACTIVE* unit; both reject a *REGISTERED* (never-traded) unit (*NotActive*), which has no holders and no mark slot — so a never-traded unit is never silently promoted, and *EXPIRED* is reachable only from *ACTIVE*: the chain is *REGISTERED* → *ACTIVE* → *EXPIRED* with no skips. *EXPIRED* is absorbing: the rank guard alone is too weak — a second expiry has equal rank, and $2 < 2$ is false — so an explicit absorbing test rejects every stage-writing delta on an expired unit; *FClose*, which carries no stage write, is the one event still admissible. Re-settling at the same price before expiry is a no-op: with $ac = -\text{net_qty} \cdot S \cdot m$ already, $\Delta ac = 0$ and $VM = 0$.

The event dispatcher, the abstract ledger, and the replay fold realise these invariants. One event flows through three total functions: *futHandle* decides what should happen, *futValidate* certifies conservation, and *futApplyDelta* writes. The stage and positivity guards live once in *futHandle*. The monotone carrier is the absence of any deleter; the absorbing rule is the *isExpired* test in *futApplyDelta*.

```
-- The four event classes. FTrade carries a RAW Qty (the unparsed input of the
  free
-- monoid of events); futHandle parses it once into PosQty. FClose is the one
  event
-- with no stage write, so it alone is admissible on an EXPIRED unit.
data FutEvent
  = FTrade    UnitId WalletId WalletId Qty Price    -- buyer, seller, qty, price
  | FSettleVM UnitId Price Day                      -- settlement price, date
  | FExpire   UnitId Price Day                      -- final price, date
  | FClose    UnitId                               -- terminal flatten to zero
  deriving (Eq, Show)

futEventUnit :: FutEvent -> UnitId
futEventUnit (FTrade u _ _ _) = u
futEventUnit (FSettleVM u _ _) = u
futEventUnit (FExpire u _ _)   = u
futEventUnit (FClose u)        = u

-- The dispatcher: a pure function of (event, current ledger), hence deterministic
.
-- The stage/positivity guards live here once: self-trade (G3); EXPIRED absorbing
-- (G2); positivity parsed (G5); settle/expire reject REGISTERED, never silently
-- promoting a never-traded unit (G4); close only after expiry (G1).
futHandle :: FutEvent -> FutLedger -> Either FutLedgerError FutStateDelta
futHandle ev l = case Map.lookup u (flUnits l) of
```

```

Nothing          -> Left (FutUnknownUnit u)
Just (terms, us) ->
  let m = ptMultiplier terms; cur = fusStage us in case ev of
    FTrade _ buyer seller q p
      | buyer == seller -> Left (SelfTrade u buyer)          -- G3
      | isExpired cur   -> Left (UnitExpired u)              -- G2
      | otherwise       -> case mkPosQty q of                 -- G5: parse once
        Nothing -> Left (NonPositiveQty u q)
        Just pq  -> Right (tradeDelta u cur m buyer seller pq p)
    FSettleVM _ s d -> case cur of
      FutExpired _ -> Left (UnitExpired u)                  -- G2
      FutRegistered -> Left (NotActive u)                   -- G4
      FutActive _  -> let (rows, cash) = settlementFanout s m (futHoldersOf u l
    )
                        in Right (FutStateDelta u (Just (FutActive (Just (
Settlement s d)))) rows cash)
      FExpire _ s d -> case cur of
        FutExpired _ -> Left (UnitExpired u)                -- G2
        FutRegistered -> Left (NotActive u)                 -- G4 (symmetric)
    )
      FutActive _  -> let (rows, cash) = settlementFanout s m (futHoldersOf u l
    )
                        in Right (FutStateDelta u (Just (FutExpired (Settlement s
d)))) rows cash)
      FClose _ -> case cur of
        FutExpired _ -> Right (closeDelta u (futHoldersOf u l)) -- G1
        _            -> Left (NotExpired u)
  where u = futEventUnit ev

-- The ledger, ABSTRACT: no field setter or row/status/cash deleter is exported,
-- so
-- futApplyDelta (insert/update only) is the sole mutator and that absence IS the
-- monotone-carrier discipline (C1). Terms and status are FUSED into one map, so
-- the
-- desynchronised state (one map has u, the other does not) is unrepresentable.
data FutLedger = FutLedger
  { flUnits :: Map UnitId (FutTerms, FutStatus)
  , flPos    :: Map (WalletId, UnitId) FutPos
  , flCash   :: Map WalletId Cash }

data FutLedgerError
  = FutReRegistration UnitId | FutUnknownUnit UnitId
  | StageRegression UnitId FutStage FutStage | FutNotConserved UnitId Conserved
  | UnitExpired UnitId | NotActive UnitId | NotExpired UnitId
  | SelfTrade UnitId WalletId | NonPositiveQty UnitId Qty
  deriving (Eq, Show)

futEmptyLedger :: FutLedger
futEmptyLedger = FutLedger Map.empty Map.empty Map.empty

-- Co-registers terms AND status as one tuple. Re-registration is a hard error (
C10).
futRegister :: UnitId -> FutTerms -> FutLedger -> Either FutLedgerError FutLedger
futRegister u terms l
  | u `Map.member` flUnits l = Left (FutReRegistration u)
  | otherwise = Right l { flUnits = Map.insert u (terms, futRegisteredStatus) (
flUnits l) }

```

```

-- Apply a validated delta: total, atomic, monotone. EXPIRED rejects ANY stage
  write
-- (absorbing); a bare new<=cur would wrongly reject the ACTIVE->ACTIVE daily
-- re-settle, so the precise rule is isExpired plus a rank-regression guard.
  FClose's
-- fsdStage = Nothing bypasses both. Rows and cash are inserted/updated, never
  removed.
futApplyDelta :: FutValidDelta -> FutLedger -> Either FutLedgerError FutLedger
futApplyDelta (FutValidDelta sd) l
  | not (u `Map.member` flUnits l) = Left (FutUnknownUnit u)
  | otherwise = case fsdStage sd of
    Nothing -> Right written
    Just new
      | isExpired cur -> Left (UnitExpired u)
      | stageRank new < stageRank cur -> Left (StageRegression u cur new)
      | otherwise -> Right written
        { flUnits = Map.adjust (\(t,_) -> (t, FutStatus new)) u (flUnits l) }
where
  u = fsdUnit sd
  cur = maybe FutRegistered (fusStage . snd) (Map.lookup u (flUnits l))
  written = l
    { flPos = Map.foldrWithKey applyRow (flPos l) (fsdRows sd)
    , flCash = Map.foldrWithKey (\w c -> Map.insertWith (<>) w c)
      (flCash l) (fsdCash sd) }
  applyRow w (FutRowDelta dnq dac) mp =
    let key = (w, u); p0 = Map.findWithDefault futZeroP key mp -- first touch
    -> flat
      in Map.insert key p0 { fpNetQty = fpNetQty p0 <> dnq
        , fpAccumCost = fpAccumCost p0 <> dac } mp

-- One event end to end: read state -> discharge conservation -> apply. Each
  futStep
-- projects onto one core Transaction recorded through the core applyTx; futStep
  is
-- the futures engine's narration of that projection, not a second core door.
futStep :: FutEvent -> FutLedger -> Either FutLedgerError FutLedger
futStep ev l = futHandle ev l >>= futValidate >>= \vd -> futApplyDelta vd l

-- futReplay is a Kleisli fold -- a fold whose step composes in the Either monad.
  Its
-- homomorphism law futReplay (xs <> ys) = futReplay xs >=> futReplay ys makes
-- checkpoint independence a CONSEQUENCE, not a test. The stream is assumed
-- de-duplicated at ingestion: a duplicate FSettleVM at a fixed mark is inert, but
  a
-- duplicate FTrade accumulates a second delta.
futReplay :: [FutEvent] -> FutLedger -> Either FutLedgerError FutLedger
futReplay evs l0 = foldM (flip futStep) l0 evs

-- Total accessors. futPosition is the C1 Option accessor: Nothing = never held,
-- Just futZeroP = held and now flat -- never collapsed.
futProductTerms :: FutLedger -> UnitId -> Maybe FutTerms
futProductTerms l u = fst <$> Map.lookup u (flUnits l)
futUnitStatus :: FutLedger -> UnitId -> Maybe FutStatus
futUnitStatus l u = snd <$> Map.lookup u (flUnits l)
futPosition :: FutLedger -> WalletId -> UnitId -> Maybe FutPos
futPosition l w u = Map.lookup (w, u) (flPos l)
futCashOf :: FutLedger -> WalletId -> Cash
futCashOf l w = Map.findWithDefault mempty w (flCash l)

```



```
futHoldersOf :: UnitId -> FutLedger -> [(WalletId, FutPos)]
futHoldersOf u l = [ (w, ps) | ((w, u'), ps) <- Map.toList (f1Pos l), u' == u ]
```

9.12 Escalations

The two escalations that follow are carried to the register in §21.

FE1 — Fan-out cost at scale. A settlement writes one *PositionState* row and emits one cash leg per open holder, every day. Across $\sim 10^6$ contracts each with up to thousands of holders, daily settlement is $O(\text{open positions})$ writes and cash legs — a large daily fan-out and write amplification. This is intrinsic to per-wallet variation margin. Mitigations (batching, snapshotting) trade against the conservation-per-event and single-writer disciplines and must be shown to preserve C2 and C11. The cost stands. *Owner*: performance and risk engineering.

FE2 — The derived-consequence alternative, declined. Storing only the shared `last_settlement_price` and computing each wallet’s mark lazily saves nothing on the dominant cost: the cash leg is per-wallet and unavoidable (FE1). It cannot produce the correct variation margin for an intraday trader without reconstructing `ac` anyway (§9.7). It would also split the single-writer discipline for `ac` (C11).

Key Invariants and Consequences

Valuation and lifecycle processing are projections of state; the following hold once conservation is granted. Each is catalogued in §17.

- **State-sufficiency.** Portfolio value depends only on current state—wallet balances, unit state (including embedded lifecycle state), and the current price vector—not on transaction history. Valuation is the linear functional $V_t = \sum_u w_t(u) P_t(u)$. (§5.)
- **PnL path-independence (P10).** $\text{PnL} = V_{t_1} - V_{t_0}$, independent of the intermediate transactions. Dual valuation applies two price vectors to one state. (§5, Theorem 5.3.)
- **Lifecycle value invariance.** Across deterministic lifecycle events—coupons, dividends, resets, redemptions—total value under a given price vector is invariant: the instrument’s price jump is offset by the explicit cash move. Under optionality the invariance holds for intrinsic value only. (§1; §7.)
- **Contracts as move generators.** A smart contract is a deterministic program emitting sequences of moves; because every lifecycle effect is a move, conservation P1 is preserved with no separate validation. (§6.)
- **Lifecycle idempotency (P6).** Applying the same lifecycle event to the same unit state produces no additional moves. (§7, Invariant 17.2.)
- **Lifecycle purity (P9).** For fixed inputs, lifecycle functions produce identical outputs and no effect beyond the return, so valuation and time travel are deterministic. (§7.)

Part IV

Accounts, Reporting, Operations, Settlement, and CDM Integration

10 Managed Accounts, Virtual Portfolios, and TRS

A managed account is not a primitive. It is the composition of four primitives that already exist (§2, §4). The first is a wallet partition that holds positions and produces PnL. The second is

a mandate unit u_{MA} , issued by the manager and held by the client, conserved by the issuance law. The third is a deterministic reset contract that observes performance, crystallises one net cash flow, and resets its baseline. The fourth is the three-home state model of §4, which homes the mandate’s terms, the shared observables, and the per-client economic state. A total return swap is the same composition with two changes: the wallet partition lives in a virtual ledger, and the reset contract’s settlement crosses into the real one. Both products reduce to a single conservative cash move. Its magnitude is a function of observed performance and a recorded baseline. The worked example (§10.11) carries one account through subscription, trade, NAV, crystallisation against a high-water mark, and redemption, with $\sum_w w(u) = 0$ at every step.

10.1 The mandate is a unit

A managed account is opened by issuing the mandate unit u_{MA} from the manager w_M to the client w_C . This gives $w_M(u_{\text{MA}}) = -1$, $w_C(u_{\text{MA}}) = +1$, $\sum_w w(u_{\text{MA}}) = 0$. The issuance is a real two-named-wallet transfer with a real issuer and holder, not a self-reflexive sentinel. Conservation holds by the issuance law (P1).

A *Move* is one conserved transfer of a unit between two named wallets: from $- = q$, to $+ = q$. Conservation (P1) holds by construction, not by check: the per-wallet deltas are group negatives, so they sum to the identity. The mandate issuance is the canonical instance. In code, the conservation witness is explicit: a move’s two deltas cancel.

```
-- Move in abridged four-field form, used by this section's listings (illustrative
);
-- the reference Move carries the magnitude as PosQty and adds time and source.
data Move = Move
  { mFrom :: WalletId, mTo :: WalletId, mUnit :: UnitId, mQty :: Qty } -- q > 0

-- The per-wallet deltas a move induces; their sum is the conservation witness.
moveDelta :: Move -> [(WalletId, Qty)]
moveDelta (Move s d _ q) = [(s, qneg q), (d, q)] -- qneg q <> q = mempty :
  conserved

-- Mandate issuance: a real two-named-wallet transfer of  $u_{\text{MA}}$ , manager -> client,
-- q = 1. A real issuer and holder, not a self-reflexive sentinel.
issueMandate :: WalletId -> WalletId -> UnitId -> Move
issueMandate manager client uMA = Move manager client uMA (Qty 1)
```

One mandate binds exactly one manager to exactly one client. The invariant states this as a support condition on the unit’s balances.

Invariant 10.1 (Mandate is a singleton bilateral contract). *While the mandate is live,*

$$\text{support } (w \mapsto w(u_{\text{MA}})) = \{(w_M, -1), (w_C, +1)\}.$$

Conservation alone does not establish Invariant 10.1: P1 admits $(-5, +5)$ or a three-way split. Four existing primitives together enforce the invariant across the mandate’s life, each covering one phase. Before issuance, the `Option` accessor reads `None`, distinct from the `Some(zero)` of a terminated but retained mandate. The `UnitStatus` lifecycle guard admits issuance only from an unissued status, so a terminated u_{MA} cannot be resurrected. Through the active phase, the singleton support restriction (Invariant 10.1) holds. C8 routes a re-mandate to a fresh u'_{MA} stamped `SetSupersededBy`.

The mandate entitles the client to an account whose contents are already valued. Pricing the mandate itself would value that account a second time.

Principle 10.2 (Mandate is non-valued). *u_{MA} is excluded from the domain of valuation: $P_t(u_{\text{MA}})$ is undefined, not zero. A price on u_{MA} would add $+1 \cdot P_t(u_{\text{MA}})$ to the client’s per-wallet NAV and double-count the underlying exposure; the term cancels only in the global sum.*

What is recorded where.

Datum	Home
Mandate text, fee schedule, benchmark identity, position limits, HWM/hurdle methodology, crystallisation frequency	$ProductTerms[u_{MA}]$ (immutable, versioned, append-only)
Lifecycle stage, superseded_by	$UnitStatus[u_{MA}]$ (shared)
HWM value, entry NAV, accrued fees, mandate-breach flags, subscription/redemption cursor, benchmark NAV at this wallet's inception	$PositionState[w_C, u_{MA}]$
Current benchmark level (externally sourced, captured as a logged observation event)	$UnitStatus[u_{bench}]$

The benchmark level is a shared cached observable, governed by the same discipline as every $UnitStatus$ value (§4.3), and consumed in NAV under single-basis consumption (Invariant 8.17): the benchmark level and the account balances it is struck against stand in one basis.

The per-client economic scalars are homed in $PositionState$. Each is nevertheless a fold of the event log. Whether they are stored or re-derived is escalation ME1 (§21).

10.2 Subscription

Capital enters as a cash move from the client's external funding wallet w_{ext} into w_C . w_{ext} is a virtual wallet at the ledger boundary. The move is tagged **SUBSCRIPTION** and indexed by the subscription/redemption cursor in $PositionState[w_C, u_{MA}]$. The tag is load-bearing: it lets the net external flow over any interval be reconstructed as a projection over the stream (§10.5). At subscription the entry NAV and the initial high-water mark are set to the subscribed capital base. Before subscription the high-water mark is **None** (unrepresentable), never 0. A 0 mark would charge a performance fee on the entire contributed capital at the first crystallisation. The benchmark NAV at inception is recorded from $UnitStatus[u_{bench}]$ for performance reporting.

10.3 Trading under mandate

A trade is an ordinary two-legged transaction against a virtual market wallet w_{mkt} , admitted only if it satisfies the mandate guards. A guard sits in front of trade admission: either the trade passes and executes unchanged, or it is rejected and nothing happens.

Definition 10.3 (Mandate guard). *A quantitative mandate constraint is a precondition g on the move-generation function. The contract evaluates g against the current state and the recorded price snapshot; if g fails, no moves are emitted and the state is unchanged.*

Guard determinism is price-relative. “Same wallet state + same trade $\Rightarrow$ same decision” is false for value-based limits such as leverage, concentration-by-value, and currency caps: holding positions fixed and doubling a price flips the same trade from admit to reject. Determinism is recovered only when the guard is a pure function of $(ProductTerms, state, trade, P_t)$ with P_t an explicit, snapshot-pinned input. Two totality obligations follow. First, a missing price ($P_t(u) = \text{None}$) makes the guard fail closed: a typed reject, never a silent admit. Second, a passive breach — a limit crossed by price appreciation with no admitted trade — lies outside the domain of a move-time guard and is detected only by a periodic valuation sweep. The pinned-snapshot requirement is escalation ME2 (§21).

10.4 Valuation and NAV

The account NAV is the wallet valuation with the mandate excluded (Principle 10.2):

$$\text{NAV}_t = \sum_{u \neq u_{\text{MA}}} w_{C,t}(u) P_t(u).$$

By state-sufficiency, NAV_t depends only on current balances, current unit state, and current prices, not on history. Performance against the benchmark is computable from the inception benchmark NAV and the current level in $\text{UnitStatus}[u_{\text{bench}}]$. It feeds reporting. It feeds the fee only when the mandate’s methodology names a benchmark hurdle.

10.5 Fee logic

Performance is net of capital flows. Over a reset interval $[t_{k-1}, t_k]$ the performance is the gross NAV change less the net external flow:

$$\text{Perf}_k = (\text{NAV}_{t_k} - \text{NAV}_{t_{k-1}}) - \text{NetFlow}_{[t_{k-1}, t_k]}, \quad \text{NetFlow} = \sum_{\substack{m \text{ tagged sub/redemption} \\ t_{k-1} < m.t \leq t_k}} \pm m.q.$$

The gross change conflates performance with capital flows. By the PnL decomposition, a mid-period subscription is a flow; crystallising it as performance would charge a fee on the client’s own contributed capital. NetFlow is a projection over the already-tagged cursor moves (§10.2), so the correction adds no new coordinate.

Fee is not PnL settlement. A client managed account pays the manager only a fee fraction; the gain stays in the client book. This differs from the internal-desk case, where a single move of the whole Perf to a beneficiary sweeps a book to Treasury. Two fees are computed at each reset: the management fee accrues on assets in every period, and the performance fee applies only to gains above the high-water mark, which never decreases. The fee handler is total over $\text{sign}(\text{NAV} - \text{HWM})$:

$$\begin{aligned} \text{management fee } f_k^m &= r_m \cdot \Delta t_k \cdot \text{NAV}_k^{\text{base}} && (\text{accrues on AUM regardless of sign}), \\ \text{performance fee } f_k^p &= r_p \cdot \max(0, \text{NAV}_k^{\text{net}} - \text{HWM}_{k-1}) && (\text{floored at zero; no clawback}), \\ \text{HWM}_k &= \max(\text{HWM}_{k-1}, \text{NAV}_k^{\text{net}} - f_k^p) && (\text{monotone ratchet; C11 writer } \text{fee_crystallise}), \end{aligned}$$

where $\text{NAV}_k^{\text{net}} = \text{NAV}_{t_k} - f_k^m$. On a loss period the performance fee is 0 and the high-water mark is unchanged. The management fee still accrues.

Crystallisation is double-entry. A fee is realised as a real cash move $w_C \rightarrow w_M$, so it is conserved and carries its own contra-entry. A non-conserved stored scalar would be single-entry and sit outside P1.

A signed amount x crystallises into *at most one* conserved cash move: magnitude $|x|$, direction by $\text{sign}(x)$, none at $x = 0$. The **Maybe** makes this total: $x = 0$ maps to **Nothing**, never to an illegal $q = 0$ move.

The fee handler emits two such moves $w_C \rightarrow w_M$ atomically (C3) and ratchets the high-water mark through one canonical writer, `WHwm :: FieldWrite 'FeeCrystallise`, reused verbatim from the three-home model (§4). The phantom type index carries the handler’s name in the write’s type, so a settle or trade handler bumping the HWM is a compile error: the mark has exactly one author. `applyWrite (WHwm _)` applies the `qmax` ratchet, so the monotone law above holds by construction, not by a runtime branch.

```
-- Crystallise(magnitude, from, to): |x|, direction by sign(x), no move at x = 0.
crystallise :: Qty -> WalletId -> WalletId -> Maybe Move
crystallise (Qty x) from to
```

```

| x > 0      = Just (Move from to usd (Qty x))           -- from -> to
| x < 0      = Just (Move to   from usd (Qty (negate x))) -- direction flips
| otherwise = Nothing                                   -- no move at zero

data FeeEvent = FeeEvent
  { fcMgmt :: Qty, fcPerf :: Qty, fcNavNet :: Qty } -- f_m, f_p, NAV net of f_m

-- Two cash moves w_C -> w_M (mgmt, perf), atomic (C3), + the HWM ratchet write.
-- f_m, f_p >= 0; on a loss period f_p = 0, so crystallise returns Nothing and the
-- performance move is absent (no zero-quantity move).
crystalliseFees :: WalletId -> WalletId -> FeeEvent
  -> ([Move], FieldWrite 'FeeCrystallise)
crystalliseFees wC wM fc =
  ( [ m | Just m <- [ crystallise (fcMgmt fc) wC wM
                               , crystallise (fcPerf fc) wC wM ] ]
    , WHwm (fcNavNet fc <> qneg (fcPerf fc)) ) -- HWM_k = max(HWM_{k-1}, NAV^net - f_p)

```

At the worked example's Step E, `crystalliseFees wC wM (FeeEvent (Qty 5750) (Qty 28850) (Qty 1144250))` yields the two legs `Move wC wM USD (Qty 5750)` and `Move wC wM USD (Qty 28850)` together with `WHwm (Qty 1115400)`. Applied to a row with $HWM_{k-1} = 1,000,000$, the `qmax` ratchets the mark to 1,115,400. On a loss period $f_k^p = 0$ leaves one leg and the mark unchanged.

The fee is a transfer, not an accrual: whatever leaves the client wallet arrives in the manager wallet in the same transaction.

Theorem 10.4 (Fee zero-sum per crystallisation). $\sum_w \Delta_{fee}(USD) = 0$ at each crystallisation: the client pays exactly what the manager receives, creating and destroying no value.

Proof. Each leg is $w_C - = q$; $w_M + = q$; summing over wallets gives $-q + q = 0$ per leg, hence 0 over the transaction. $\square$

Quantity, baseline, residual. Every settlement leg here is a `crystallise` call (above). The reset baseline is the post-settlement capital base $B_k = NAV_{t_k} - (f_k^m + f_k^p) + \text{NetFlow}$. Reading the pre-settlement NAV instead would claw the distributed performance back next period. The rounding residual $\text{Perf}_k - \text{round}(\text{Perf}_k)$ is retained in w_C as a conserved un-crystallised remainder, never dropped. It is the reconciling item between economic performance and moved cash. Whether B_k , HWM_k , and the accrued fee are stored or re-derived is escalation ME1 (§21). Crystallising unrealised mark-to-market can drive the cash wallet negative. Conservation alone cannot distinguish a funded obligation from an insolvent overdraft; this liveness gap is recorded as escalation ME5 (§21).

10.6 Segregation

Conservation checks that a move balances; it never checks whose wallets the move names. Keeping clients apart is therefore a question of authorisation, not of arithmetic.

Theorem 10.5 (Segregation). *Client partitions are mutually isolated if and only if conservation is combined with locality and capability scoping. A cross-client move is an illegal state ruled out by C4 under locality, not by conservation.*

Conservation alone is not segregation. A move $m = (w_A, w_B, \text{USD}, q)$ between two different clients' partitions is perfectly conservative ($\sum_w \Delta = 0$), so P1 does not forbid it. Two independent facts are at work: conservation (CONS, $\sum_w \Delta = 0$) and locality (LOC, $\text{supp}(\Delta) \subseteq \{w_s, w_d\}$). Locality gives “no effect beyond the named wallets”. Isolation of distinct clients additionally requires an authorisation predicate: C4 capability scoping, which forbids a handler

from naming or reading across a (w, u_{MA}) boundary it has no capability for. Segregation is therefore a theorem under $\text{CONS} \wedge \text{LOC} \wedge \text{C4}$. It supplies the logical-segregation component of CASS 6 / MiFID II Art. 16(8). Legal segregation, meaning distinct custodial accounts and trust or nominee structure, is outside scope: the ledger evidences it, it does not establish it. That C4 is asserted in prose rather than typed on the accessor signature is escalation ME4 (§21).

10.7 CSA margin and collateral

The CSA margin contract is a wallet-level smart contract attached to a per-counterparty collateral wallet. It reads, in the real ledger, the aggregate mark-to-market of the trades the CSA governs. It computes the required collateral under the threshold, minimum-transfer-amount, eligible-collateral, and haircut rules. It emits margin-call moves to or from the collateral wallet. Margin eligibility and haircut are contractual data in *ProductTerms*; the exposure is the observed valuation.

For a synthetic managed account the netting set is the single real unit u_{TRS} , never the simulated constituents in the virtual ledger. Reading those constituents would breach the isolation invariant (§10.8) and over-margin unmarginated positions. The realm tag and C4 realm-scoped reads enforce this. The margin exposure $N_k \cdot \text{TR}_k$ is itself an observation of the virtual book, lifted across the realm boundary as a number. That this observation is unattested and unversioned is escalation ME2 (§21).

10.8 TRS as synthetic managed account

A total return swap connects a virtual ledger $\mathcal{L}_v$ to real counterparty wallets in the real ledger $\mathcal{L}_r$ through periodic cash settlement. Ledger isolation is invariant P7: no move has one endpoint in $\mathcal{L}_v$ and the other in $\mathcal{L}_r$. A minimal virtual ledger is not a second engine. It is a realm $\in \{\text{real}, \text{virtual}\}$ tag enforcing P7.

At reset t_k the TRS contract in $\mathcal{L}_r$ observes the virtual book's performance and emits one real move, $w_{\text{payer}} \rightarrow w_{\text{receiver}}$ in USD, tagged `TRS_NET_SETTLEMENT`: magnitude $|\text{Payment}_k|$, direction by sign, where $\text{Payment}_k = N_k \cdot \text{TR}_k - N_k \cdot r_k \cdot \Delta t_k$. The total return $\text{TR}_k = (V_{t_k}^v - V_{t_{k-1}}^v) / V_{t_{k-1}}^v$ is partial: it requires $V_{t_{k-1}}^v > 0$, and failure of that precondition is a typed failure, not a division by zero.

Viewed from the ledger, a funded managed account and a TRS do the same thing at each reset: observe a performance, compare it with a recorded baseline, and pay one net cash amount.

Theorem 10.6 (Equivalence to the funded managed account). *The TRS settlement and the funded managed-account fee/PnL settlement are the same ledger operation: one net conservative cash move whose magnitude is a deterministic function of an observed performance and a recorded baseline. The funded book plays the role of $\mathcal{L}_v$; the periodic settlement is the TRS payment.*

Proof sketch. Both reduce to `Crystallise(magnitude, from, to)` with magnitude = observed performance over the interval and direction by sign; both reset a baseline to the post-settlement value; neither moves a position across a boundary. The settlement primitive is one; the products differ, and that difference is a CDM (reporting) distinction, not a ledger one: a CDM `TotalReturnSwap` is two-legged ($N \text{ TR}$ and $N r \Delta t$), so the report derives from the retained CDM `BusinessEvent`, never from the netted move (§10.12). $\square$

Price consistency. $\mathcal{L}_v$ valuation and $\mathcal{L}_r$ settlement use one price vector P_t . Because P7 forbids a cross-ledger move, a price divergence produces unexplained PnL with no internal reconciliation path. This is the one place the no-internal-break guarantee can be silently voided. Both realms read the price from the same logged event, so a fresh replay reconstructs one price

for both. What remains is attestation of that logged price: escalation ME2, the attestation-envelope requirement (§21).

10.9 Redemption and wind-down

Wind-down strikes a final NAV, settles the final fee, returns capital, and retires the mandate. Positions are liquidated against the market wallet. The cash is paid to the client's external wallet. The mandate unit is returned to the manager. All closed-out rows are retained at zero under the monotone carrier (C1). This preserves the audit trail, the final high-water mark, and the quantities needed for tax reconstruction.

```
Move(from: w_C,      to: w_mkt, unit: <asset>, quantity: <held>)      -- liquidate
Move(from: w_mkt,    to: w_C,   unit: USD,    quantity: <proceeds>)
Move(from: w_C,      to: w_ext, unit: USD,    quantity: <final NAV>) -- capital-
                        out
Move(from: w_C,      to: w_M,   unit: u_MA,   quantity: 1)          -- retire
                        mandate
```

10.10 Account-level balance-sheet substantiation

Each account's balance at any date is a deterministic projection of the move stream filtered to its wallets. Client statements, desk PnL, and regulatory position reports differ only in the filter. Within the ledger there is no separate account-level quantity record to reconcile against. The move stream substantiates quantities and provenance. Valuations depend on the external price vector; disclosures and legal status require external evidence. Net settlement is operational netting. Presenting a balance sheet net requires, in addition, a legally enforceable right of set-off and the intent to settle net (IAS 32.42 / ASC 210-20), a determination outside the ledger. The accounts' assets = liabilities + equity reconciliation holds at the quantity level by P1: the trial balance balances by construction. The valued and classified presentation does not follow from P1 alone.

10.11 Worked example

One managed account w_C under mandate u_{MA} issued by manager w_M ; virtual market wallet w_{mkt} ; client external wallet w_{ext} . USD reference. Terms: management fee 2%/yr, performance fee 20% over a high-water mark (no hurdle), quarterly ($\Delta t = 0.25$). Conventions: the management fee accrues on end-of-period NAV; the performance fee is charged on NAV net of the management fee, above the mark; the mandate is non-valued. $\sum_w w(u) = 0$ holds for every unit after every move.

Step	Moves (src $\rightarrow$ dst : qty unit)	NAV
A. Mandate issuance	$w_M \rightarrow w_C : 1 u_{MA}$	—
B. Subscription	$w_{ext} \rightarrow w_C : 1,000,000 \text{ USD}$	1,000,000
C. Trade (buy 5,000 AAPL @100)	$w_C \rightarrow w_{mkt} : 500,000 \text{ USD}; w_{mkt} \rightarrow w_C : 5,000 \text{ AAPL}$	1,000,000
D. Q1 NAV (AAPL $\rightarrow$ 130)	(price move; no ledger move)	1,150,000
E. Fee crystallisation	$w_C \rightarrow w_M : 5,750 \text{ USD (mgmt)}; w_C \rightarrow w_M : 28,850 \text{ USD (perf)}$	1,115,400

F. Wind-down	$w_C \rightarrow w_{\text{mkt}} : 5,000 \text{ AAPL}; w_{\text{mkt}} \rightarrow w_C : 650,000 \text{ USD}; w_C \rightarrow w_{\text{ext}} : 1,115,400 \text{ USD}; w_C \rightarrow w_M : 1 u_{\text{MA}}$	0
--------------	--	---

Fee arithmetic (Step E). No capital flow occurred inside the period, so $\text{Perf} = (1,150,000 - 1,000,000) - 0 = 150,000$. Management fee $= 1,150,000 \times 0.02 \times 0.25 = 5,750$. NAV net of management $= 1,144,250$. Performance fee $= 0.20 \times (1,144,250 - 1,000,000) = 28,850$. NAV after both fees $= 1,115,400$. New high-water mark $= \max(1,000,000, 1,115,400) = 1,115,400$; reset baseline $B_1 = 1,115,400$. Fee zero-sum (Theorem 10.4): $\sum_w \Delta_{\text{fee}}(\text{USD}) = -34,600 (w_C) + 34,600 (w_M) = 0$.

Closing identity (all wallets). Final non-zero balances: $w_M = +34,600$ (fee revenue), $w_{\text{mkt}} = -150,000$ (the gross gain paid out by the market), $w_{\text{ext}} = +115,400$ (client net). Their sum is $34,600 - 150,000 + 115,400 = 0$. The client received principal 1,000,000 plus net gain 115,400. The gross gain 150,000 equals net 115,400 plus fees 34,600. Per-unit conservation holds across the chain. u_{MA} is issued $(+1, -1)$ then returned $(0, 0)$. USD nets to 0 at w_C and globally after every move. AAPL is acquired and liquidated with w_{mkt} as the mirror. Every position row is retained at zero under the monotone carrier.

10.12 Conformance and flags

No conformance is asserted; each item is demonstrated against a cited source or flagged for the responsible owner. The CDM strata named below (**LegalAgreement**, **Trade**, **TotalReturnSwap**, **TradeState**, **BusinessEvent**) are identified at the type level only. Pinning them to a named CDM release is owed to the CDM owner and is flagged, not asserted.

- **CDM mandate representation.** u_{MA} maps to the CDM **LegalAgreement** stratum, the ISDA Create-digitised investment-management agreement: parties, fee schedule, benchmark identity, methodology, limits. It does not map to the **Trade**/product stratum. A discretionary mandate is a legal agreement, not a tradable contract.
- **CDM TRS representation.** A CDM **TotalReturnSwap** is two-legged. The report is the retained **BusinessEvent** (performance leg and financing leg $N r \Delta t$), never the netted settlement move. The ledger “same mechanism” identity (Theorem 10.6) is a settlement-primitive identity; it is flagged as not a CDM/reporting identity.
- **SFTR/EMIR reporting surface.** u_{MA} issuance is not reportable: a discretionary mandate is the MiFID II Annex I § A(4) portfolio-management service, not a Section C instrument, not an SFT, not a derivative. The reportable artefact is never u_{MA} ; it is the underlying book trade or u_{TRS} , each carrying its own UTI/UPI/LEI. Reportability is a per-relationship, regime-cited attribute on the wrapped unit’s *ProductTerms*; it is not inferable from the coarse unit-type key. Governance of that attribute is escalation ME3 (§21). So is the dual-sided EMIR/CFTC/SFTR reporting, which needs validated ISO-17442 counterparty LEIs and classification. Neither is a quantity fact; neither has a home in the move stream.
- **IFRS classification.** Economic NAV and economic PnL are derived here. Accounting classification of fair-value changes (FVTPL/FVOCI/amortised cost) and net presentation (IAS 32.42) are outside scope and require external evidence. Accrued fees are modelled as conserved moves rather than stored scalars, so they carry their own contra-entry and are auditable as double-entry.

- **CDM TradeState alignment.** That the CDM TradeState-per-Trade view and *PositionState* $[w, u]$ stay replay-consistent (both projections of one minted event) is asserted, not yet verified. It is the outstanding F6 deliverable: a Rosetta mapping rerun against the three-map schema.

Escalations ME1–ME5 and the attestation-envelope requirement are cited at their points of origin above and collected in the flagged register (§21). The workflow is sound as quantity algebra: every step is atomic $q > 0$ moves with $\sum_w w(u) = 0$ verified, and the worked example closes to the penny. The escalations mark the points at which the performance and reporting layer outruns the framework’s treatment of recorded observations and external identity.

11 Balance Sheet Substantiation and Dual Valuation

The balance sheet is a projection of the move stream, not a separate record. Each move carries CDM-native metadata linking its balance to the originating product, business event, and counterparty. Every line item therefore traces to its business event along a standardised evidence chain. Substantiation is the demonstration to an external auditor that a balance is complete, accurate, and properly valued. Under this construction it reduces to replaying the log. The balance sheet and the ledger are the same stream read two ways, so the two cannot disagree.

11.1 Balance Reconstruction

The move stream is an immutable, chronological log, so any balance can be recovered by replaying it. The recipe: start from the opening balance, add every move in, subtract every move out. The balance of wallet w in unit u at time t is its opening balance plus the signed sum of every move on u up to t :

$$w_t(u) = w_0(u) + \sum_{\text{moves } m: m.\text{unit}=u, m.\text{time} \leq t} \begin{cases} +m.\text{quantity} & m.\text{destination} = w \\ -m.\text{quantity} & m.\text{source} = w. \end{cases}$$

The result is deterministic and reproducible from the raw stream alone. An auditor verifies any balance at any date t by replaying the stream to t and applying this formula. No independent balance record exists to reconcile against.

The reconstruction is a projection of the move stream: each move maps to a two-cell delta, minus at its source and plus at its destination, and the balance map is the sum of these deltas. Pointwise addition, with the all-zero balance as unit, makes the per-(wallet, unit) balance map a *monoid*. The projection balances is foldMap over that monoid.

```
-- Scalars and keys in local pedagogical form: Qty is the additive abelian
-- group of exact minor units (Section 2); the Move is abridged to four
-- fields with a raw Qty magnitude. The consolidated form -- the reference
-- Move, magnitude PosQty -- is reference/Ledger.hs (Part B).
newtype Qty      = Qty Integer      deriving (Eq, Show)
newtype WalletId = WalletId String  deriving (Eq, Ord, Show)
newtype UnitId   = UnitId   String  deriving (Eq, Ord, Show)

instance Semigroup Qty where Qty a <> Qty b = Qty (a + b)
instance Monoid    Qty where mempty         = Qty 0
qneg :: Qty -> Qty
qneg (Qty n) = Qty (negate n)

data Move = Move { from, to :: WalletId, unit :: UnitId, qty :: Qty }
```

```

-- The projection target: one signed total per (wallet, unit). The monoid
-- is POINTWISE addition -- combine cell by cell, all-zero map as unit. This
-- is NOT the library default for Map, which is left-biased union and would
-- silently drop the second contribution to a cell. Choosing the right
-- monoid is the whole correctness content of this type.
newtype Balances = Balances (Map (WalletId, UnitId) Qty)

instance Semigroup Balances where
    Balances a <> Balances b = Balances (Map.unionWith (<>) a b)
instance Monoid Balances where
    mempty = Balances Map.empty

-- One move debits its source and credits its destination: the two-cell
-- signed contribution the displayed formula writes with + / -.
contribution :: Move -> Balances
contribution m = Balances (Map.fromListWith (<>)
    [ ((to m, unit m), qty m)
    , ((from m, unit m), qneg (qty m)) ])

-- foldMap: map every move to its contribution, then combine them all with
-- one associative operation; the empty stream maps to the all-zero balance.
-- Folding the chronological prefix up to t gives w_t; the opening balance
-- is just the moves before the window, so there is no separate opening term
-- and no separate balance record to reconcile against.
balances :: [Move] -> Balances
balances = foldMap contribution

netBal :: WalletId -> UnitId -> [Move] -> Qty
netBal w u ms = let Balances b = balances ms
                in Map.findWithDefault mempty (w, u) b

```

Remark 11.1 (Balance-as-fold). *Plain reading: balances are not stored and maintained; they are recomputed by folding the move stream. Formally, `balances` is a monoid homomorphism from the free monoid of move lists (lists under concatenation, empty list as unit) into the monoid of balance maps—it carries concatenation to combination,*

$$\text{balances } (xs ++ ys) = \text{balances } xs \text{ } \langle \rangle \text{ } \text{balances } ys.$$

This one law is the substantiation guarantee read three ways. With `ys` empty and the operands reassociated, the grouping of independent batches cannot change the result, so replay is order-stable and deterministic. With `xs` the moves up to a snapshot and `ys` the moves since, the snapshot balance combined with later moves equals the balance of the whole stream—the $O(k)$ snapshot query of Section 11.4 is correct by this law, not by separate argument. And summing any unit’s column over all wallets cancels each $+q$ against its $-q$ to `mempty`: that is the conservation law P1 (Section 2), here a consequence of `contribution` emitting equal and opposite legs, not an assertion to enforce.

11.2 Substantiation Scope

Position quantities and transaction provenance are substantiated by construction. The position set, the flows between wallets, and unit state at any historical date all derive from the move stream by the reconstruction above. This eliminates the manual comparison of independent records that substantiation otherwise requires.

Valuation and disclosure are not so substantiated. Fair value hierarchy classification under IFRS 13 (Level 1/2/3, with sensitivity analysis for Level 3 inputs) depends on inputs outside the move stream. So do the disclosures of IAS 1 paragraphs 117–124 (accounting policies,

significant estimates and judgments). Those inputs are external price feeds, valuation-model documentation, management judgment, and audit assessment. Legal enforceability depends on external documentation. The design claim is therefore exact: the ledger substantiates quantities and transaction provenance by construction; valuations, disclosures, and legal status require supplementary evidence.

Unit state read during substantiation comes from UnitStatus, a projection of the same log (§4.3).

Appendix C enumerates, in one place, the internal reconciliation failure modes the construction renders structurally unreachable, the boundary modes that remain detectable, and the by-design treatment of securities lending.

11.3 Dual Valuation

No single price serves settlement and risk management at once. Settlement requires the price at which an external party will transact. Risk management requires a single coherent price across related positions. The resolution is to carry two prices per unit while keeping one position set: both views read the same positions, and only the price vector differs.

MENTAL MODEL. Picture each position as water sealed in a fixed run of pipes, with two rate sheets posted on the outside of the seal, each converting the one water level into money by its own rule. Settlement values the water off one sheet and risk off the other; because both read the same water, the quantity beneath them can never diverge. The two money figures do diverge, and either can move while not a drop of water shifts — unlike the sealed water, neither figure is conserved, for the sheets hang outside the seal.

Definition 11.2 (Dual Valuation). *Each unit $u \in \mathcal{U}$ carries two price functions:*

- $P_t^{\text{MtMk}}(u)$, *mark-to-market, from observable market data—used for settlement, margin, external reporting, and custodian reconciliation;*
- $P_t^{\text{MtMd}}(u)$, *mark-to-model, from internal valuation models—used for risk management, portfolio optimisation, and performance attribution.*

Their differential is $\Delta_t(u) = P_t^{\text{MtMd}}(u) - P_t^{\text{MtMk}}(u)$.

Multi-exchange futures. Nikkei 225 futures settle on three exchanges at different cut-off times:

$$\begin{aligned} P_t^{\text{MtMd}}(\text{NKY}) &= 35,500 && \text{(theoretical fair value)} \\ P_t^{\text{MtMk}}(\text{NKY-SIMEX}) &= 35,480 && \text{(Singapore settlement)} \\ P_t^{\text{MtMk}}(\text{NKY-CME}) &= 35,490 && \text{(Chicago settlement)} \\ P_t^{\text{MtMk}}(\text{NKY-OSE}) &= 35,495 && \text{(Osaka settlement).} \end{aligned}$$

The three cannot share a mark-to-market price. Singapore closes hours before Chicago, and Osaka closes between them. Each settlement price fixes the market at a different moment. Margin must therefore use the exchange-specific price. They share one mark-to-model price because all converge to the same final settlement value, the Nikkei 225 index. A desk holding offsetting positions across the three reads net exposure from the single MtMd price. Each leg settles at its own MtMk price.

Fair value adjustment. The model price admits an adjustment for the gap between theoretical and executable prices: liquidity cost, bid-ask spread, model uncertainty. The adjusted price is

$$P_t^{\text{MtMd, adj}}(u) = P_t^{\text{MtMd}}(u) + \text{FVA}_t(u).$$

Prudent valuation under CRR Article 105 requires the adjusted price for regulatory capital. Risk management uses the unadjusted price. Dual valuation carries both.

Single position set. Desk-level consistency and firm-level reconciliation coexist because both valuations read the same wallet balances. Each desk prices related positions coherently through P^{MtMd} . The firm reads valuation discrepancy directly from $\Delta_t(u)$. Internal transfers may book on either basis by policy. Position divergence between the two views is impossible: there is one position set and two price vectors.

11.4 Snapshots and Audit Trail

A snapshot is a substantiated checkpoint: a consistent cut of the move stream, carrying both pricing bases and the provenance metadata of each move. A balance query starting from the most recent snapshot replays only the moves since it. The query is therefore $O(k)$ in the moves since the snapshot rather than $O(n)$ in the total. A snapshot is consistent only when taken at a quiescence point (no transactions in flight) or under multi-version concurrency control.

Quantities and prices separate cleanly: the quantity layer is closed and constrained by conservation, and prices enter only afterwards as a multiplier. Conservation enforces double-entry accounting on quantities, independent of price:

$$\sum_{w \in \mathcal{W}} w_t(u) = 0 \quad \forall u, \forall t,$$

the closed-system law of Section 2 (P1). Valuation applies the price vector to the conserved quantities:

$$V_t^{\text{MtMk}} = \sum_w w_t(u) \cdot P_t^{\text{MtMk}}(u), \quad V_t^{\text{MtMd}} = \sum_w w_t(u) \cdot P_t^{\text{MtMd}}(u).$$

Quantities are conserved and shared. Only the two valuations differ.

12 Implementation and Operations

The architecture is specified at the design level: data structures, algorithms, and interfaces an implementation must provide. No implementation language is assumed.

12.1 Three-Layer Architecture

The system has three layers. One layer stores facts; the other two compute from them. A CDM business event enters at the persistence layer, is decomposed into balance deltas at the computation layer, and is reflected as state at the projection layer.

1. **Move stream** (persistence). The immutable, append-only log of atomic moves: the canonical internal record. Each move carries full provenance—timestamp, source contract, metadata, external references.
2. **Aggregation engine** (computation). A pure GroupBy from a set of moves to net change per (wallet, unit) key. Having no side effects, it parallelises across independent keys.
3. **Projection** (state). Wallet balances and unit status, derived from the move stream through the aggregation engine.

The projection is a cache, not a record. Corruption of the projection (hardware fault, lost node) is repaired by replaying the move stream from the last verified snapshot. Nothing is lost: the move stream is append-only and immutable (P4).

Remark 12.1 (The projection is a rebuilt value, not a second source of truth). *UnitStatus is a projection of the event log (§4.3); so are wallet balances: replay rebuilds the exact balance and status that held at any chosen instant (P8).*

12.2 Balance-Update Algorithm

A transaction $\tau = \{m_1, \dots, m_n\}$ is applied in four steps. The steps are mechanical: check the transaction, net its moves, write the result, record a checkpoint.

1. **Validate.** Every move references an existing wallet and unit; τ satisfies conservation (C2, P1).
2. **Aggregate.** Net change per (wallet, unit) key by GroupBy over the moves.
3. **Apply.** Update balances atomically; on any failure, roll back the whole transaction (P2).
4. **Snapshot.** Persist the updated projection as a checkpoint for later queries.

12.3 Constant-Cost Profit and Loss

Profit and loss over $[t_0, t_1]$ loads the snapshots and prices at the endpoints, computes V_0 and V_1 , and returns $V_1 - V_0$. The cost is $O(n)$ in the number of instruments held, not the number of moves: the result depends only on the endpoint states (state-sufficiency, P10). A portfolio of 10,000 instruments and 1,000,000 moves reads 10,000 balances, not 1,000,000 moves. Prices at t_0 and t_1 are taken as given; they are supplied by the market-data and valuation satellites.

12.4 Fault Tolerance

Late events. Each event carries two timestamps: economic (“what happened”) and booking (“when it was learned”). Because insertion is append-only, an event learned after its economic time is appended like any other; nothing already recorded is mutated. Financial statements and regulatory returns project on the economic timestamp at the reporting date. The booking timestamp supports the as-known view.

Duplicate events. Idempotency is structural at two levels. A transaction carries a unique identifier; a re-applied identifier is rejected (P5). A lifecycle event is gated on unit status: if the status shows the event complete, the lifecycle function returns no moves and no state change (P6). CDM event-qualification functions perform this check, excluding both duplicate and out-of-sequence processing.

Contradictory external state. Virtual wallets mirror external counterparties. A disagreement between an external confirmation (ISO 20022 feedback) and the internal record is detected by comparing virtual-wallet balances to the external statement. Resolution is not automated. The discrepancy is surfaced, and a human or external process determines the correct state. That resolution is recorded as a transaction in the move stream.

Stale market data. The conservation layer (moves and balances) is independent of price feeds and operates correctly when they fail. Only the valuation projection degrades. Value-dependent settlements—managed-account settlements (Section 10), margin, total-return-swap resets—produce economically incorrect moves if driven by stale prices. An implementation therefore gates such settlements on data-quality checks. When price quality is insufficient, settlement is deferred, not booked at a known-stale value for later correction. Staleness handling and degradation policy are specified by the market-data and deferred-settlement satellites.

Move-stream integrity. The canonical record is replicated and protected against corruption by cryptographic hash chaining (P4). Periodic verification detects silent corruption: balances are recomputed from the raw stream and compared to the cached projection. The canonical-record design removes internal inconsistency between records but concentrates availability risk. Deployment addresses this risk through replication, backup, and disaster recovery.

Partial failures. Atomicity is all-or-nothing (P2). A transaction that cannot complete rolls back entirely, leaving the move stream consistent. Preserving atomicity across process restarts requires write-ahead logging or equivalent crash-recovery semantics.

12.5 Corrections as Events

The move stream is immutable. An error is corrected by a compensating transaction that reverses the original and books the correct version. This is event sourcing: state derives from an append-only log, and a correction is an event, not a mutation. The incorrect moves remain in the stream, preserving error history for audit.

The compensating transaction references the original through a `corrects` field in its metadata, forming an explicit correction chain. This linkage is a first-class metadata requirement: it distinguishes economic events from error corrections on replay.

A trade amendment (a change of terms, such as a notional increase) is distinct from a cancellation. An amendment is a CDM `TermsChangeEvent`. The contract's unit status advances to the new terms, and any economic difference is captured as a settlement move. The original terms remain in the stream. A formal correction algebra (anti-moves, amendment chains, their composition, and the interaction of corrections with snapshot consistency) is an open problem (Section 21).

12.6 From Moves to Settlement Instructions

A lifecycle event that generates moves—option exercise, coupon payment, margin call—also settles externally. The ledger move is the internal record of the economic event. The settlement instruction is the external message that transfers cash or securities between custodian, CCP, or payment-system accounts.

The mapping is a projection. Each move carries source wallet, destination wallet, unit, and quantity. The settlement layer translates wallet identifiers to external accounts (BIC, IBAN, CSD account), unit identifiers to security identifiers (ISIN, CUSIP), and quantity to settlement amount. Because the event model is CDM and CDM maps to ISO 20022, the translation is a struct-to-struct transformation, not a business computation. The information content of every settlement instruction is fully determined by the move and requires no additional capture. The settlement-layer interface and instruction timing are specified in Section 13 and the deferred-settlement satellite.

12.7 ISO 20022 Interface

Message generation is projection: the CDM object is the semantic content, the ISO 20022 message its serialisation. Every field of a `sese.023` Securities Settlement Instruction is already present in the CDM representation of the move.³

ISO 20022 field (<code>sese.023</code>)	CDM source
<code>SecurityIdentification</code> (ISIN)	<code>Product.security.identifier</code>
<code>SettlementQuantity</code>	<code>Move.quantity</code>
<code>SettlementAmount</code>	<code>Move.quantity</code> × price
<code>TradeDate</code>	<code>BusinessEvent.effectiveDate</code>
<code>SettlementDate</code>	<code>Transfer.settlementDate</code>
<code>DeliveryReceiveIndicator</code>	Move direction

Cash messages (`pacs.008`, `pacs.009`) follow the same principle. The CDM `Transfer` carries payer, payee, amount, currency, and value date. The ISO message adds only transport fields (BIC routing, message identifiers).

³`sese.023` is the ISO 20022 business message identifier for the Securities Settlement Instruction; under the ISO 20022 message-definition identifier scheme the full identifier `sese.023.001.NN` fixes variant and version *NN*.

The traceability chain is deterministic at every step: smart contract $\rightarrow$ move (with contract reference) $\rightarrow$ ISO 20022 instruction (with `EndToEndId` and `TxId`) $\rightarrow$ external confirmation (matching identifiers) $\rightarrow$ status update in the event log. A returned confirmation (`sese.025` for securities, `camt.054` for cash) is ingested back into the move stream, closing the settlement cycle within one framework.

13 The Settlement Layer Interface

The Ledger records economic events. The settlement layer executes real-world transfers. The boundary between them is a *projection*: a pure, total function that reads one committed transaction and produces an optional settlement instruction. Everything the settlement layer needs is already present in the transaction. Everything the settlement layer adds lies outside the Ledger’s scope: custodian accounts, CSD connectivity, message formatting.

13.1 The Settlement Projection

The whole interface is one function. It looks at a committed transaction and either reads off the instruction that settles it, or reads off nothing. It consults nothing else and remembers nothing.

Definition 13.1 (Settlement Projection). *The settlement projection*

`settleProjection` : `SettlementTx` $\rightarrow$ `Maybe SettlementInstruction`

is pure, deterministic, stateless, and idempotent. It is total: defined on every settlement transaction, returning `Just` an instruction for the settlable classes (`SETTLEMENT`, `COLLATERAL`) and `Nothing` for the rest.

Its input, `SettlementTx`, is a *boundary type*: the settlement-layer view onto which a committed core `Transaction` (Section 2) *projects*, never the core `Transaction` itself. The core `Transaction` is the one atomic event recorded in the log: an ordered list of moves over ledger units, with the state delta they carry. Its boundary view holds the transaction id, the settlement class, and the moves; each move’s unit is already resolved to its settlement identity (`Asset`: an ISIN or a currency). The projection is the unit-to-asset resolution the contract performs at move generation. `settleProjection` then carries `SettlementTx` to the wire instruction.

The projection reads only from the transaction:

- **Transaction id** — the settlement reference (`EndToEndId` in ISO 20022).
- **Each move’s source wallet, destination wallet, settlement asset, and quantity** — the economic content of the transfer.
- **The CDM event payload** (Section 14) — settlement date, counterparty identifiers (LEI, BIC), and execution venue (MIC).

It reads nothing else: not current wallet balances, not other transactions, not any external system. Purity follows. The same settlement transaction always yields the same instruction. The function is safe to re-run after a failure. The settlement layer need not understand conservation, unit state, or lifecycle logic.

The instruction is already latent in the transaction; the types below make it impossible to extract a malformed one. Two facts carry the argument. First, a settlable move’s `smAsset` is its unit’s settlement identity: the security ISIN or the currency the unit denotes. It is resolved from the unit’s registered terms (Section 4) and carried into the `SettlementTx` by the contract, so the projection needs no off-transaction lookup. Second, the settlement *type* and the *legs* are one value. Each constructor of `SettlementLegs` carries exactly the legs its type names. An instruction with no legs therefore cannot be written, and the type can never disagree with the legs.

```

-- The settlement classification of a unit: a transferable security, or a
-- currency. Every settleable move is exactly one or the other. (CashCcy, not
-- Cash, so the settlement currency case never collides with a scalar.)
data Asset = Security ISIN | CashCcy Currency

-- A settlement-layer move: it ranges over the settlement identity (Asset),
-- not the core ledger UnitId. It is what a core Move projects to once the
-- contract has resolved the unit to its settlement asset.
data SettleMove = SettleMove { smFrom, smTo :: WalletId, smAsset :: Asset, smQty
    :: Qty }

-- Settability is a field set by the contract, never inferred. Total over all
-- five classes; only SETTLEMENT and COLLATERAL settle. LIFECYCLE/CORRECTION
-- "sometimes" settle only by DECOMPOSING into separate transactions at the source
.
data TxClass = SETTLEMENT | COLLATERAL | LIFECYCLE | ACCOUNTING | CORRECTION
settles :: TxClass -> Bool
settles SETTLEMENT = True
settles COLLATERAL = True
settles LIFECYCLE = False
settles ACCOUNTING = False
settles CORRECTION = False

data CdmPayload = CdmPayload { cdmTradeDate, cdmSettleDate :: Day
    , cdmCounterparty :: LEI, cdmVenue :: MIC }

-- The boundary type. NOT the core Transaction (the log event of Sec. 2):
-- it is the settlement-layer view a committed core Transaction PROJECTS onto,
-- its moves already resolved to settlement Assets.
data SettlementTx = SettlementTx { stId :: TxId, stClass :: TxClass
    , stMoves :: [SettleMove], stCdm :: CdmPayload }

data SecuritiesLeg = SecuritiesLeg { slIsin :: ISIN, slQty :: Qty
    , slDeliver, slReceive :: WalletId }
data CashLeg = CashLeg { clCcy :: Currency, clAmount :: Qty
    , clPayer, clReceiver :: WalletId }

-- type and legs are ONE value: "an instruction with no legs" is unrepresentable.
data SettlementLegs = DvP SecuritiesLeg CashLeg | FoP SecuritiesLeg | CashOnly
    CashLeg

data SettlementType = DVP | FOP | CASH
settlementType :: SettlementLegs -> SettlementType -- a projection, not a stored
    field
settlementType (DvP _ _) = DVP
settlementType (FoP _) = FOP
settlementType (CashOnly _) = CASH

data SettlementInstruction = SettlementInstruction -- plain data, no
    behaviour
    { siRef :: TxId, siTradeDate, siSettleDate :: Day
    , siLegs :: SettlementLegs, siCounterparty :: LEI, siVenue :: MIC }

-- A security move -> securities leg (source delivers, destination receives);
-- a cash move -> cash leg (source pays, destination receives). Total over Asset.
classify :: SettleMove -> Either SecuritiesLeg CashLeg
classify (SettleMove from to (Security i) q) = Left (SecuritiesLeg i q from to)
classify (SettleMove from to (CashCcy c) q) = Right (CashLeg c q from to)

```



```

-- Nothing when there is no standard leg shape: no moves (a contract error), or a
-- non-decomposed multi-leg structure. Both give no instruction, never a partial
one.
legsOf :: [SettleMove] -> Maybe SettlementLegs
legsOf ms = case partitionEithers (map classify ms) of
  ([s],[c]) -> Just (DvP s c)
  ([s],[])  -> Just (FoP s)
  ([],[c])  -> Just (CashOnly c)
  _         -> Nothing

settleProjection :: SettlementTx -> Maybe SettlementInstruction -- pure, total
settleProjection tx
  | settles (stClass tx) = mk <$> legsOf (stMoves tx)
  | otherwise           = Nothing
where mk legs = SettlementInstruction (stId tx) (cdmTradeDate (stCdm tx))
      (cdmSettleDate (stCdm tx)) legs (cdmCounterparty (stCdm tx)) (cdmVenue (
      stCdm tx))

-- Behaviour, made concrete (settlementType . siLegs <$> settleProjection tx):
-- security + cash move (a DvP equity trade) -> Just DVP
-- a securities move alone                    -> Just FOP
-- a cash move alone (a margin call)          -> Just CASH
-- an ACCOUNTING transaction                  -> Nothing -- not settlable
-- a SETTLEMENT with no moves                 -> Nothing -- contract error,
no instruction

```

Remark 13.2. *The projection is total and deterministic by construction. `settles` classifies every `TxClass`; `legsOf` is exhaustive, its wildcard absorbing both the empty and the non-decomposed multi-leg cases as `Nothing`; and `settleProjection` reads only `tx`—no clock, no balances, no external system—so the same settlement transaction always yields the same instruction and re-running after a failure is safe.*

The instruction itself is a filled-in form with no logic of its own. Its type is read off its legs, so the form cannot claim one shape and carry another.

Definition 13.3 (SettlementInstruction). *The `SettlementInstruction` is plain data with no behaviour (the `data` declaration above): instruction id, trade and settlement dates, the settlement legs, counterparty LEI, and execution venue MIC. The settlement type $\in \{DVP, FOP, CASH\}$ is a projection of which legs are present (`settlementType`), never an independent field that could disagree with them. Both legs present give DVP; securities alone give FOP (Free of Payment); cash alone gives CASH; no legs is unrepresentable, so a settlable transaction with no moves yields `Nothing`—a contract error, not an instruction.*

The standard cases carry at most one securities leg and one cash leg: DvP equity trades, FOP transfers, cash-only settlements. Multi-leg structures decompose at the source, not in the projection. The smart contract emits one `SETTLEMENT` transaction per independently settleable leg. A repo emits two: one for the opening leg, one for the closing leg, each a standard DvP structure.

13.2 Boundary Separation

The division of labour: the Ledger knows the trade; the settlement layer knows the plumbing.

Principle 13.4 (Boundary Separation). *The Ledger provides what settles: parties, securities, quantities, dates. The settlement layer provides how: standing settlement instructions (SSIs),*

custodian account identifiers, CSD connectivity, ISO 20022 message generation, and exception management. Neither side reads the other’s internals.

The **SettlementInstruction** is the sole object shared across the boundary. Wire-message generation is two steps: projection, then enrichment. The projection produces the instruction from the transaction. The settlement layer enriches it with the operational detail the Ledger does not hold: SSIs, CSD participant ids, settlement priority. It then generates the ISO 20022 message (e.g. `sese.023`). This is the precise sense of “message generation is projection, not translation” (Section 12): the projection step is the Ledger’s; enrichment is the settlement layer’s.

13.3 Transaction Types and Settability

Settability is a first-class field on the transaction. The generating smart contract assigns it at the point of move generation; it is never inferred at runtime. The contract knows the business intent.

Type	Settles?	Description
SETTLEMENT	Yes	Maps to a real-world settlement instruction (e.g. equity trade → <code>sese.023</code>).
COLLATERAL	Yes	Settles through the collateral channel (e.g. margin call → cash transfer).
LIFECYCLE	Sometimes	Decomposes into settlement and accounting sub-transactions.
ACCOUNTING	No	Internal adjustment, no external transfer (e.g. PnL crystallisation, revaluation).
CORRECTION	Sometimes	Compensating transaction; settles a reversal only if the original already settled.

The projection returns **Just** for **SETTLEMENT** and **COLLATERAL**, **Nothing** for the rest. When a **LIFECYCLE** event requires both settlement and accounting, the smart contract emits both transactions: a **SETTLEMENT** for the legs that transfer externally, an **ACCOUNTING** for the internal adjustment. Physical exercise of an option, for instance, emits a **SETTLEMENT** (deliver shares, receive cash) and a separate **ACCOUNTING** (close the option position, crystallise PnL). The decomposition is the contract’s responsibility, not the projection’s.

13.4 Lifecycle-Originated Settlement

Some settlements originate from internal contract logic rather than an external instruction: an option exercise, a coupon falling due, a barrier breach. The Ledger is the originator, so there is no external instruction to match. The projection treats them exactly as it treats execution-originated settlements. The contract tags the transaction **SETTLEMENT**; the projection extracts the instruction; the settlement layer enriches and delivers it. The sole difference is provenance. Execution-originated instructions match an external trade confirmation; lifecycle-originated instructions are generated unilaterally. The settlement layer receives the instruction with no awareness of its origin.

One lifecycle transaction may yield several instructions when it carries several legs. Physical exercise of an equity option with lot-size constraints (Section 6) carries a securities delivery (whole lots) and a cash payment (strike value plus any fractional residual). The projection classifies each move by unit type and emits the matching instruction for each.

13.5 DvP: The Two-Level Guarantee

Delivery versus Payment (DvP) is the principle that securities delivery and cash payment occur together or not at all. The framework provides it at two independent, mutually reinforcing levels.

Ledger-level. A DvP trade is one transaction holding both the securities move and the cash move. The executor commits all moves atomically: both take effect or neither does (Section 2). This is a structural consequence of the transaction primitive, not a runtime check. The smart-contract specification (Section 6) supplies the other half of the guarantee by requiring both legs within one transaction.

Settlement-level. The CSD or CCP provides DvP at the real-world level: securities delivery and cash payment settle simultaneously on its books. This depends on external infrastructure and lies outside the Ledger’s control.

The temporal gap. Between trade date and settlement date the Ledger shows the economic position under trade-date accounting (Section 1); the CSD confirms the real-world transfer at settlement. The gap is not a DvP failure. It is the normal state under trade-date accounting, where exposure exists from execution.

Settlement failure. On failure (the counterparty cannot deliver), the Ledger records a settlement-failure event and does *not* reverse the economic position. The exposure existed from trade date regardless of settlement outcome. It persists until the failure resolves through buy-in, cancellation, or partial delivery. The discrepancy between Ledger and custodian positions is detected by virtual-wallet comparison (Section 12).

External regimes. Legal title, beneficial ownership, fail treatment, and the regulatory-capital treatment of unsettled transactions are governed by external frameworks, not by the Ledger. The Ledger supplies the position and timing data those treatments require; it does not determine the applicable regime.

Remark 13.5. *The mechanics of the trade-date-to-settlement-date interval—deferred settlement, fail lifecycle, buy-in, partial delivery, and re-instruction—are specified in the standalone deferred-settlement specification. This section fixes only the interface: what the projection produces and how settlement status returns. The satellite is out of scope here.*

13.6 Gross-to-Net Reconciliation

The Ledger records gross transactions, one per trade. The settlement layer may net: for trades in the same security with the same counterparty on the same settlement date, it produces a single net instruction. Netting is a settlement-layer optimisation; the Ledger’s gross records are authoritative and are never modified by netting.

Gross and net reconcile by an algebraic identity, not a heuristic. For each (security, counterparty, settlement date) group, the algebraic sum of gross instruction quantities equals the net instruction quantity; deliveries count positive, receipts negative. The sum is a `foldMap` from the gross legs into the additive group of quantities. A violation localises the fault: either the projection produced an incorrect instruction (a Ledger fault) or the netting computation is wrong (a settlement-layer fault).

```
-- Signed securities quantity from one party's view: delivery positive, receipt
   negative.
signedFor :: WalletId -> SecuritiesLeg -> Qty
```

```

signedFor me l | slDeliver l == me = slQty l
               | slReceive l == me = negQty (slQty l)
               | otherwise         = mempty

-- The net leg's signed quantity equals the sum of the gross legs'. An identity,
-- checked, not a reconciliation heuristic.
reconciles :: WalletId -> [SecuritiesLeg] -> SecuritiesLeg -> Bool
reconciles me gross net = foldMap (signedFor me) gross == signedFor me net

```

13.7 The Confirmation Return Path

Settlement status records one thing: how far the real-world transfer has progressed. Settlement confirmations (`sese.025` for securities, `camt.054` for cash) flow back into the Ledger as lifecycle events recording the confirmation timestamp and external reference. No moves are generated; positions were already correct from trade date. The settlement status advances:

Status	Meaning
EXECUTED	Committed in the Ledger; the settlement layer has not yet processed it.
INSTRUCTED	The settlement layer has submitted the instruction; the CSD has not confirmed.
SETTLED	The CSD confirms successful settlement; custody now matches Ledger positions.
FAILED	The CSD reports a failure. Ledger positions remain correct (economic exposure); the external transfer has not occurred. A failed trade may re-instruct (return to <code>INSTRUCTED</code> after buy-in or extension), partially settle (a lifecycle event), or cancel (a <code>CORRECTION</code> transaction reversing the original moves).

The transitions are `EXECUTED` → `INSTRUCTED` → `SETTLED`, with `INSTRUCTED` → `FAILED` as the alternate terminal. Each transition is itself a logged lifecycle event in the move stream. Settlement status is therefore a projection of those events, not a separate mutable store. Replaying the events to any point rebuilds the status that held then; the stored value is only a read cache, overwritten in place. This completes the traceability chain of Section 12: smart contract → move → settlement instruction → external confirmation → status.

14 ISDA CDM Integration

The framework needs a canonical vocabulary for products and lifecycle events: one normalised description of what is traded, one normalised description of how obligations evolve, and one deterministic rule for when a transition is permitted. The ISDA Common Domain Model, governed by the Fintech Open Source Foundation (FINOS), supplies all three. The ledger adopts it as its domain model. The correspondence is derived from the framework's own primitives: wallet, unit, move, transaction, conservation. Each CDM record is then stored intact alongside the transaction it generates.

14.1 The CDM

The CDM is a machine-readable, machine-executable model of financial products and their lifecycle events. It is expressed in the Rune domain-specific language and distributed as open-source executable code. It comprises five components:

1. A *product model*: tradable contracts and instruments in normalised form (quantity, price, parties, payouts, legal agreements).
2. An *event model*: lifecycle events as state transitions built from a fixed set of primitive operators (execution, quantity change, terms change, reset, transfer).
3. A *process and workflow model*: pre- and post-trade steps (proposals, acceptances, rejections, allocations, novations).
4. A *reference-data model*: parties, legal entities, and the static data needed to interpret trades and events.
5. A *mapping layer* (synonyms): a machine-readable correspondence between CDM objects and external message formats and identifiers (FpML, FIX, ISO 20022).

A single instrument makes the abstractions concrete: a European call on AAPL, strike \$200, expiry 2026-06-20, 100 contracts, cash-settled in USD, bought by Firm A from Firm B on 2026-03-15. The CDM describes the contract as an `OptionPayout` (underlier, option type, strike, expiry, settlement terms). It wraps the payout in a `Trade` that binds counterparties and execution context. It records the exercise as a `BusinessEvent` whose `before TradeState` (ACTIVE) becomes its `after TradeState` (EXERCISED). The chain is *product* (what the contract is) → *tradable product* (who traded it) → *business event* (what happened) → *ledger moves* (how value transfers). The full product walkthrough is in Appendix F.

14.2 Derivation: CDM Maps to the Framework Primitives

Each framework primitive has a CDM counterpart, so the integration follows from the definitions rather than from adoption.

Products as units. The wallet universe $\mathcal{U}$ instantiates directly from the CDM product model. Each CDM trade or contract corresponds to a unit type or a bounded set of unit types (underlyings, legs, payouts). The Unit Store manages these types (Section 3). CDM’s normalised quantity, price, and party supply the primitives of wallet balances and move units.

Event model as state graph. The CDM event model already treats each lifecycle event as a functional state transition. `TradeState` and `State` are the nodes of the lifecycle state graph. The primitive operators composed in a `BusinessEvent` are its edges. The lineage between `before` and `after` states, and between `WorkflowSteps`, is the evidence chain that reconstructs any path through the graph. CDM is thus a concrete state machine for the abstract lifecycle.

Embedded logic as transition contracts. CDM embeds executable Rune logic: primitive functions, validation rules, event-qualification functions. These are the deterministic predicates a transition contract requires. A CDM function applied to a `TradeState` is a transition-contract instance. Its validation and state-transition checks are the proof of execution that must pass before moves are emitted. The ledger treats CDM functions as the authoritative contracts from which wallet moves are generated.

Mapping layer as oracle interface. External messages are oracle outputs. Synonym mappings are deterministic transforms from those messages into CDM trades and events. CDM functions then drive transition contracts and emit moves. Because the mapping is explicit and shared, no bilateral adapter is required per counterparty.

CDM describes *what* trades and events are; the closed ledger describes *how* value moves between wallets in response.

14.3 The Synonym Layer and Ingestion

Each external message maps to a CDM object through a declared synonym. The message may be an FpML confirmation, a FIX execution report, or an ISO 20022 settlement instruction. The mapping is deterministic, versioned, and machine-readable. It is reused across systems in place of bespoke bilateral adapters. Ingestion is a fixed pipeline:

raw message → CDM synonym mapping → CDM object → lifecycle function → moves and
state changes

The only bespoke code is at the transport layer; the semantic mapping is shared.

The move stream stores CDM objects that may originate from earlier CDM versions, so version coexistence is required. Each stored event carries its CDM version identifier. The lifecycle engine either processes an older-version event natively or applies a version migration before processing. A CDM enumeration addition (a new lifecycle intent or product type) requires a corresponding update to the lifecycle state machines and the test generators.

14.4 The Forgetful Map F

The forgetful map reads a rich CDM business event and keeps only the bare value movements the ledger acts on. The legal and business detail is held aside in storage, not discarded.

Let **CDM** be the collection of CDM states and business events and **Ledg** the collection of ledger states and transactions. The map $F : \mathbf{CDM} \rightarrow \mathbf{Ledg}$ sends each **BusinessEvent** to a boundary record. The record carries the event's economic content: the moves. It stores the originating CDM event as its payload. The moves it carries are exactly the edges a core ledger **Transaction** records, so the boundary record projects onto the core transaction. It is not itself the core **Transaction** and keeps a distinct name. F is *forgetful*: it drops the legal and ISDA-business detail the ledger does not track and keeps only the moves. Nothing is lost, because the dropped detail survives verbatim in the payload.

F has the signature below: it consumes a CDM **BusinessEvent** and produces a boundary **CdmTransaction** that projects onto the core ledger **Transaction**. The listing has three parts: the CDM types F consumes, the boundary type it produces, and the definition of F itself.

```
-- The CDM side. A BusinessEvent carries the primitive operators F reads, the
-- lifecycle states it transitions between, and the business detail F forgets.
data Intent          = Execution | Exercise | Termination | QtyChange | ResetIntent
data LifecycleState = ActiveS    | Exercised | Terminated | SettledS
newtype TradeState  = TradeState LifecycleState

data PrimitiveInstruction
  = PiTransfer  WalletId WalletId UnitId Qty  -- value leg (cash or asset)
  | PiQtyChange WalletId WalletId UnitId Qty  -- position leg (contracts)
  | PiTerms     -- amendment: no move
  | PiReset     -- observation/reset: no move

data BusinessEvent = BusinessEvent
  { beInstructions :: [PrimitiveInstruction] -- read by F (economic content)
  , beBefore       :: TradeState             -- lifecycle before / after: read
  , beAfter        :: TradeState             -- sequencing only
  , beIntent       :: Intent                 -- FORGOTTEN: business intent
  , beLineage      :: [String]               -- FORGOTTEN: workflow lineage
  , beStructure    :: String                 -- FORGOTTEN: product structure,
  , legalTerms     -- legal terms
  }
```

```

-- F emits the ledger Move in its six-field shape (mFrom, mTo, mUnit, mQty, mTime,
-- mSource); the magnitude is shown as a raw Qty (illustrative) -- the reference
-- builds both legs through the checked move/mkPosQty door and records the dropped
-- non-positive leg as a refinement. A
-- Move names BOTH parties, so a single Move conserves BY CONSTRUCTION (a credit
-- with no offsetting debit is unrepresentable); the whole event's conservation is
-- then the signed edge-sum of its move list -- mempty for the empty list, so a
-- move-less event (amendment, reset) is conserved for free. There is no separate
-- validation step, and none is representable.
--
-- CdmTransaction is the CDM BOUNDARY type, kept DISTINCT from the core ledger
-- Transaction (Sec. 4): its ctMoves ARE the edges a core Transaction records, so
-- it
-- PROJECTS onto the core Transaction; ctPayload keeps the originating event
-- verbatim.
data CdmTransaction = CdmTransaction
  { ctMoves    :: [Move]          -- economic content the ledger acts on (-> core
    txMoves)
  , ctPayload :: BusinessEvent    -- originating CDM event, kept verbatim (
    recoverable)
  }

-- F = forget. Total over every BusinessEvent; deterministic (pure). Move
-- extraction is 'concatMap instructionMoves', a MONOID HOMOMORPHISM from the free
-- monoid of instruction lists to the free monoid of move lists --- meaning it
-- distributes over concatenation, moves (xs <> ys) = moves xs <> moves ys. The
-- restricted composition and sequencing properties below are consequences of this
-- one law, not separate guarantees.
instructionMoves :: PrimitiveInstruction -> [Move]
instructionMoves (PiTransfer a b u q) = [Move a b u q (Timestamp 0) (SourceId "
  cdm")]
instructionMoves (PiQtyChange a b u q) = [Move a b u q (Timestamp 0) (SourceId "
  cdm")]
instructionMoves PiTerms                = []          -- move-less event (FAQ Q1)
instructionMoves PiReset                 = []

forget :: BusinessEvent -> CdmTransaction
forget be = CdmTransaction (concatMap instructionMoves (beInstructions be)) be

```

A move-less event (an amendment, a reset) maps to a `CdmTransaction` with `ctMoves = []` and the full payload. `forget` is total there; it does not fail. It is *forgetful* in code: `ctPayload` retains the whole event, so the detail dropped from `ctMoves` (intent, lineage, structure) is recoverable verbatim — `ctPayload (forget e) == e`.

Worked example: exercising the AAPL call. Firm A exercises the 100-contract call on 2026-06-20 at settlement price $S_T = 215$. The arriving `BusinessEvent` carries an `Exercise Instruction` with `before TradeState` (ACTIVE, 100 contracts) and `after TradeState` (EXERCISED). *F* reads the primitive instruction: a cash `Transfer` of $100 \times \max(215 - 200, 0) = \text{USD } 1,500$ from seller to buyer. It emits `Move(seller[Firm B], buyer[Firm A], USD, 1500)`. Firm B's USD balance falls by 1,500 and Firm A's rises by 1,500. The net change is zero, so conservation holds. The option unit's state becomes EXERCISED and accepts no further events. The full CDM `BusinessEvent` is stored in the generated `CdmTransaction`'s payload for audit and reporting: exercise terms, counterparty identifiers, product structure, workflow lineage.

Remark 14.1. *UnitStatus* is a projection of the event log (§4.3). The exercise above changes

the option unit's status to *EXERCISED* solely by logging its *BusinessEvent*; the stored *EXERCISED* flag is the replay of that log.

What F preserves. F forgets detail; it preserves the structure the ledger relies on: how events compose, that value is conserved, in what order moves appear, and when repetition is inert.

- **Composition (restricted).** For referentially independent events e_1, e_2 , meaning neither references the other's state, $F(e_2 \circ e_1) = F(e_2) \circ F(e_1)$. This is the monoid-homomorphism law of `concatMap instructionMoves` above: move extraction distributes over concatenation. F therefore assembles the composite's moves from the parts without reference to neighbours. Events that carry cross-references (a novation referencing the original trade) or dependent cascades (a margin call triggered by a reset on the same unit) must first be resolved and processed in causal order. Composition then holds for the resolved sequence, provided the resolution is deterministic and alters no event's economic content.
- **Conservation.** Every business event maps to a transaction satisfying the conservation law; F neither creates nor destroys value.
- **Sequencing.** The temporal order of CDM events is the order of the resulting move stream.
- **Idempotency.** If a CDM event is idempotent, so is its transaction. Exercising the same option twice produces no additional moves: the absorbing lifecycle guard (**Exercised** accepts no further exercise, as **Expired** is absorbing for futures, Section 9) makes the second event inert before F runs.

What F forgets. Business intent (why the trade was placed), workflow lineage (the approval and negotiation history), product structure (payout schedules and legal terms), and regulatory classification (IFRS 9 measurement category; see Section 18). Each is preserved in the CDM payload, not in ledger state.

The division is deliberate. The ledger keeps what its guarantees use; the CDM payload keeps everything else.

Principle 14.2. *The ledger carries economic substance—positions, cashflows, conservation; CDM carries business meaning—intent, structure, classification. Embedding full CDM graphs in ledger state would enlarge it without strengthening any guarantee, since balance computation and conservation do not depend on business intent. The forgetfulness of F is therefore forced by the separation of the two abstractions, and the dropped detail is recoverable from the payload at all times.*

The concrete per-type translation rules that implement F are tabulated in Appendix A: which CDM types map to ledger primitives, and which are preserved as metadata.

14.5 CDM Enumerations as Generator Universe

The CDM enumerations form a closed, finite universe of valid constructs: product types (interest rate, equity, credit, FX, commodity), event types (execution, termination, novation, exercise, reset, transfer, quantity change), party roles, date-adjustment rules, settlement types. The CDM validation rules state which event types are valid for which product types; enumerations and rules together define the generator universe for the property-based tests of Section 17. The test framework enumerates the valid (product, event) combinations and verifies that F produces a conservation-satisfying transaction for each.

Part V

Obligations, Generalised Positions, and Securities Lending

15 Orchestration and Obligation Liveness

The framework fixes *what* must hold: conservation, atomicity, idempotency, purity. It leaves open *how* lifecycle events are scheduled, retried, and sequenced. Safety alone does not move the system forward: a coupon due on a date causes no error if it never fires. This section states the requirements an execution engine must meet to close that gap.

MENTAL MODEL. Picture an alarm set for a future date whose winding is written into a ledger rather than held in the clock’s spring: cut the power, swap the clock, or move it to another building, and at the appointed moment it still rings, because whoever rebuilds it reads the same date and re-arms it. The ring is a duty falling due, and arming the alarm the instant the duty is created is what keeps a dated duty from being silently forgotten. Unlike a household alarm, whose ring is satisfied by the ring itself, this one demands the duty actually be met: when it rings the duty is either already discharged, or a prescribed fallback runs, or—if no fallback can be computed—the duty is recorded as defaulted.

15.1 Execution-engine requirements

Four requirements follow from the framework’s own primitives. They bind any execution engine. The engine itself is an implementation choice that satisfies them, not a result derived here.

1. **At-least-once delivery.** Idempotency (P5, transaction level; P6, lifecycle level) makes duplicate execution harmless. It does not make an event fire. The engine must dispatch every scheduled event and re-dispatch on worker failure. At-least-once delivery composed with idempotent activities yields exactly-once semantics.
2. **Durable timers surviving restarts.** A coupon scheduled for a fixed future date fires on that date irrespective of process restarts, deployments, or infrastructure failures in the interval. Timer state is persisted by the engine, not held in process memory.
3. **Deterministic replay.** Time travel (Property 6) requires ledger state at any past instant to be reconstructable by replaying the move stream. The engine carries the complementary property for orchestration: workflow state at any point is reconstructable by replaying an append-only workflow history. Both derive correctness from one principle, deterministic replay of an immutable log.
4. **Single-threaded execution per instance.** Each workflow instance processes signals, timer fires, and activity completions one at a time, in deterministic order. This is the single-writer model. It eliminates concurrency races without a command queue or consensus protocol.

The reference architecture adopts Temporal.io, whose durable-execution model satisfies all four. “The engine” hereafter denotes the adopted instance. Its guarantees enter the liveness proof as named lemmas (§15.6.7).

15.2 Processor requirements

The ledger admits transactions; it does not manufacture them. The machinery that computes a settlement or a corporate-action adjustment and submits it is a *processor*: external to the ledger, untrusted by it, and verifiable against it. The engine of §15.1 delivers events; the four conditions here bind whatever receives them.

Two kinds of processor exist, and they are one architecture. A *lifecycle processor* catches a due event from the scheduler (§15.4) — an expiry, a coupon, an exercise. It reads the unit’s declared terms and pulls the stamped observations the payoff names. It computes the settlement and submits the transaction. A *corporate-action processor* catches a confirmed notice. It reads the declared operator, the declared dimensions, and the adjustment schedule. It submits the transaction the boundary requires, and that transaction has three parts. The first is the new terms version, appended under C6 and carrying the adjusted strikes and references. The second is the entitlement moves. The third is the status writes: **SetBasis**, together with the matching unit-status updates on any related units the boundary touches. The two kinds differ in what they read, not in how they run: catch, read declared data, compute, submit. An option expiry and a stock split are processed by the same shape of machine.

The pure function **handle** (§7) is one conforming implementation — the reference one. Running in-process, it satisfies the four conditions by construction. The conditions exist so that every other implementation — a separate service, a vendor system, a rewrite — is held to exactly what **handle** already satisfies.

1. **Closed inputs.** A processor reads the committed log and the stamped observations the log records; nothing else. No ambient clock, no direct feed read, no configuration outside the log. The timer fire is delivery, not data: it says *when*; everything the event means — terms, schedule, observations — is log data. An expiry processor woken an hour late reads the same terms and the same stamped close it would have read on time, and submits the same transaction.
2. **Prefix-determinism.** Given a log prefix, the submission is determined: same prefix, same transaction. Two runs, or two independent implementations, over one prefix produce equal transactions. **canonicalTx** fixes a deterministic move order, so equality is decided move for move, not judged.
3. **Idempotence keys.** Every submission carries a key derived deterministically from its cause. For a lifecycle settlement the key is the obligation id (Definition 15.2). For a corporate-action follow-on the key is the causing boundary id — the value recorded as **txCause** — paired with the target unit. A retried submission is bit-identical (item 2), so it carries the same transaction id and the door commits it once (P5). Crash-and-retry cannot double-book.
4. **Proposed, never authoritative.** A submission is a proposal. It becomes ledger fact only through **applyTx** (Principle 7.2), under the same admission checks as every other transaction. A processor holds no write privilege, and no submission channel bypasses the door.

What the door guarantees about these producers, and what it leaves to be checked, divide along one line.

Principle 15.1 (Admission and recomputation). *Admission guarantees that no transaction can break the ledger; recomputability from declared data guarantees that a wrong transaction can always be detected. Structural correctness is enforced at the door. Economic correctness is not enforced at the door: it is made checkable by recomputing the transaction from the log, and a failed check marks a producer defect, repaired downstream.*

The first layer is *structural correctness*. Conservation rides on the Transaction type: an unconserved move set is unrepresentable, so no processor can submit one. Every submission passes **applyTx**, so no processor can break an invariant, leave a partial state, or delete a row. Structural correctness holds against a defective processor and a malicious one alike, because it never depended on the processor at all.

The second layer is *economic correctness*, and the door does not check it — by design, not by omission. Whether a transaction is economically right is not a predicate on the transaction alone; it is agreement between the transaction and the declared data that determine it, and

agreement is checked by computing, not by inspecting. The declared terms of an option settlement state that €100 is owed; a defective processor submits a settlement paying €98. The transaction is balanced, conserving, and basis-consistent. Admission has no ground to refuse it, and it books €2 of wrong economics.

The error is caught in exactly one way: recompute the transaction from the log prefix and compare. The terms, the adjustment schedule, the dimensions, the datum-kind registry, and the operator are all declared, logged data, so a second computation of the same settlement is always possible from the log alone. The recorded and the recomputed transactions are compared in canonical form: `canonicalTx` fixes the form, and equality on `Transaction` decides the comparison. P27 (Producer agreement) is the executable witness. Its oracle, `recomputeOK`, recomputes over the prefix and asserts a zero diff.

A nonzero diff is a producer defect, never a ledger defect. It is repaired through the compensating-transaction path (§12.5): the producer is corrected and the transaction resubmitted.

The corporate-action half runs the same check on the same numbers as the lifecycle half. A 2-for-1 split's declared `Scale 2` and the unit's adjustment schedule take a €100 strike to €50; a defective corporate-action processor submits a terms append carrying €49 — the 49-versus-50 error of the June-1 timeline (§8.2). The append is structurally admissible, and the error is caught the same way: `recomputeOK` re-runs the schedule against the logged notice, obtains €50, and reports a nonzero diff, repaired through the same compensating path. One architecture, one verification mechanism, on both halves.

Untrusted therefore has exact content. A processor is trusted to produce transactions, not trusted to be right about them. The door re-checks the structure of every submission, and every output the processor produces can be recomputed from declared data by an independent implementation. The single-door consequence of §7 (Principle 7.2) is exactly the first layer of this division; the second layer is what recomputation adds on top of it.

Replay re-reads recorded transactions; it never re-executes a processor, and it fetches nothing from outside the log. History is therefore closed against processor change: a processor can be upgraded, replaced, or discarded, and every past state still reconstructs exactly. The settlements these processors originate are the lifecycle-originated settlements of §13.4: the Ledger is the originator, so the settlement projection carries them outward as instructions, and there is no inbound instruction the ledger must wait for or reconcile against.

15.3 The executor as an activity

The executor is the single component permitted to mutate the ledger. It validates conservation, checks idempotency by transaction ID (P5), and commits atomically. It runs as a retrievable engine *activity*: a unit of work the engine re-invokes on transient failure.

The mapping is exact on two points. **Retry.** The engine retries on transient failures: network partition, process crash, transient database error. The executor's transaction-ID idempotency check makes a retried commit a no-op. At-least-once dispatch is thereby upgraded to exactly-once effect. Conservation violations, referential-integrity errors, and idempotency rejections are non-retryable. They signal logic errors, never transient ones, and a retry never succeeds. **Idempotency contract.** The activity's idempotency key is the transaction ID. If that ID is already committed, the executor returns the prior result without re-applying moves. This satisfies the engine's requirement that an activity called repeatedly with the same input produce one observable effect.

15.4 The due-event scheduler

Every contractual obligation with a deterministic future date is mapped, at unit registration, to a durable timer. This covers bond coupons, futures daily settlement, option expiry, IRS resets,

CSA margin calls, and managed-account resets (Table 10, deterministic-date rows). The timer is persisted by the engine. It survives server restarts, worker deployments, version upgrades, and data-centre failovers. This resolves the liveness gap for dated obligations. The timer fires regardless of infrastructure failures, and idempotency guarantees it fires at most once.

A due event does not proceed on stale or unavailable market data. Each lifecycle workflow runs a data-quality check before invoking the lifecycle function. On failure it defers the event and retries after a fixed interval. The deferral itself is durable across worker crashes. Market-data staleness thresholds and cross-source validation are specified in the market-data satellite specification. This section consumes that gate; it does not define it.

15.5 Concurrency: single-writer by construction

One lifecycle workflow is assigned per unit, or per wallet for per-wallet state such as futures accumulated cost. Each instance processes events single-threaded in deterministic order, so concurrent lifecycle events on the same unit serialise. Two margin calls on one CSA cannot interleave. Two coupons on one bond cannot race. A daily settlement and an intraday trade on one futures contract process in strict causal order. The executor therefore never receives concurrent mutations for a unit from the orchestration layer. Its idempotency check (P5) is a second line of defence against a duplicate reaching it through an activity retry.

Operations spanning units, such as corporate actions, close-out netting, and CSA-level margin, are coordinated by a parent workflow. The parent’s deterministic execution fixes causal order across its children. No shared mutable state exists between them. Serialisation is per unit, not global. Distinct units execute in parallel, so throughput scales with the number of active units and the worker pool, not with any single unit’s lifecycle complexity.

15.6 Obligation liveness

The scheduler closes the gap only for obligations whose deadline is known at registration. A second class, *event-triggered* obligations, arises during the lifecycle with no timer at registration. An SBL recall creates a return-by deadline absent at loan inception. A collateral-eligibility change creates a substitution demand with a cut-off. A margin call creates a delivery obligation at T+1 from the call. A counterparty default triggers close-out netting on master-agreement deadlines. Obligations handled only inside workflow code are invisible to the framework. The framework then cannot distinguish “tracked” from “forgotten by the workflow author.” The obligation as a first-class object closes this gap.

15.6.1 The obligation type

An obligation packages three things: a deadline, a test on ledger state that says whether the duty has been met, and a prescribed consequence if the deadline passes with the test unmet. The definition makes each part a field.

Definition 15.2 (Obligation). *An obligation is a tuple $o = (id, type, source, t_d, D, \kappa)$:*

- *id*: a unique identifier, derived deterministically from the source event and obligation type.
- *type*: a member of the obligation-type enumeration (Table 10).
- *source*: the unit, agreement, or regulatory event that created it.
- *t_d*: the deadline by which the obligation must be discharged.
- *D*: the discharge predicate on ledger state. The obligation is discharged when a move set *M* journaled before *t_d* gives $D(\text{state_after}(M)) = \text{true}$.
- *κ*: the compensation action, a deterministic function computing compensating moves when the deadline passes undischarged.

The obligation record and its discharge predicate are expressed directly in the type system. The discharge predicate D and the compensation κ are *total* functions of ledger state: same state, same verdict. A discharge decision is therefore deterministic and replayable. κ returns **Just** moves when it can compute valid compensating moves and **Nothing** when it cannot; the latter forces **DEFAULTED**. The record below transcribes the definition field for field.

```
-- D and kappa are total functions of ledger state (deterministic verdicts).
data Obligation = Obligation
  { obId      :: ObId                -- derived from source event +
    type
  , obType    :: ObType              -- taxonomy (obligation-types
    table)
  , obSource  :: Source              -- unit / agreement / regulatory
    event
  , obDeadline :: Time                -- t_d
  , obDischarge :: LedgerState -> Bool -- D : the discharge predicate
  , obCompensate :: LedgerState -> Maybe [Move] -- kappa : Just on success,
    Nothing fails
  }
```

The lifecycle state is a one-way street: once an obligation settles, no code path can reopen it. The state type is split so that the type, not a convention, forbids leaving a terminal state. **PENDING** and **ATTEMPTED** are the only *live* states, and the handler consumes only live states. **DISCHARGED**, **COMPENSATED**, and **DEFAULTED** are terminal and have no exit edge. “An obligation leaves a terminal state” is unrepresentable, not merely unreachable.

```
data Live      = Pending | Attempted      -- non-terminal
data Terminal = Discharged | Compensated | Defaulted -- terminal: no exit edge
data ObState  = Active Live | Settled Terminal -- stored per obligation

data Trigger = DischargeSignal | DeadlineFired -- the two things that move
o

-- Total over (Live x Trigger). ‘step’ takes only Live, so a Terminal can never
-- be transitioned. On DeadlineFired every arm returns Right (a terminal state):
-- that absence of a Left IS lemma L5 (handler totality), the handler half of P21.
step :: Obligation -> LedgerState -> Live -> Trigger -> Either Live Terminal
step o s _ DischargeSignal
  | obDischarge o s = Right Discharged
  | otherwise       = Left  Attempted
step o s _ DeadlineFired
  | obDischarge o s = Right Discharged
  | otherwise       = case obCompensate o s of
    Just _ms    -> Right Compensated -- kappa succeeded
    Nothing     -> Right Defaulted   -- kappa failed
```

The whole mechanism is a record, a sum type split into live and terminal, and one total function. Anything further would remove neither a bug nor a line of code. The state-table transitions of §15.6.1 are exactly the cases of **step**. The CSA call below (§15.6.8) makes the behaviour concrete: **step csaVM** on a post-delivery state with **DischargeSignal** returns **Right Discharged**; on a no-delivery state with **DeadlineFired** it returns **Right Compensated**.

From	To	Condition
PENDING	ATTEMPTED	discharge attempted
PENDING	DISCHARGED	$D = \text{true}$ (early)
PENDING	COMPENSATED	$t > t_d$, κ fires
ATTEMPTED	DISCHARGED	$D = \text{true}$
ATTEMPTED	COMPENSATED	$t > t_d$, κ fires
PENDING	DEFAULTED	$t > t_d$, κ fails
ATTEMPTED	DEFAULTED	$t > t_d$, κ fails

15.6.2 Taxonomy

Obligation type	Scope	Trigger	Compensation κ
<i>Deterministic-date (timer at registration)</i>			
Bond coupon	Unit	Timer at coupon date	Default event
Option expiry	Unit	Timer at termination date	N/A (auto-expire)
IRS reset	Unit	Timer at reset date	Default event
Futures daily VM	Unit	Cron at exchange EOD	Position liquidation
SBL term loan return	Unit	Timer at maturity	Buy-in (GMSLA 9.3)
<i>Event-triggered (timer at event time)</i>			
SBL recall return	Unit	Recall signal	Buy-in (GMSLA 9.3)
SBL manuf. dividend	Unit	Corp. action record date	Cost attribution
SBL collateral subst.	Unit	Demand / eligibility change	Shortfall escalation
CSA VM delivery	Agreement	Margin call computation	Close-out netting (ISDA)
CSA IM delivery	Agreement	Regulatory schedule	Close-out netting
Collateral substitution	Agreement	Schedule amendment	Close-out netting
SBL collateral top-up	Unit	MTM threshold breach	Recall / default
Close-out netting	Agreement	Default event	Waterfall / loss alloc.
<i>Regulatory</i>			
SFTR report	Unit	SBL lifecycle event	Regulatory penalty
SLATE report	Unit	SBL lifecycle event	Regulatory penalty
EMIR report	Unit	Trade/lifecycle event	Regulatory penalty
Settlement instruction	Unit	Settlement-type transaction	Failed settlement

Table 10: Obligation types by trigger and scope. Deterministic-date obligations are scheduled at unit registration; event-triggered obligations are created when the triggering event is processed; regulatory obligations are created alongside the business event.

15.6.3 The obligation store

Obligations live where everything else lives: in the one event log. The store is what a reader recovers from that log, not a second place where truth is kept.

Definition 15.3 (Obligation store). *The obligation store $\mathcal{O}$ is the set of obligations recovered from the event log by filtering to obligation-typed entries. Each obligation is added atomically within the transaction that creates its triggering event; each state transition is a logged state-only transaction.*

The obligation store is a projection of the event log. Each change to an obligation’s state is a logged transaction (Definition 15.3). The stored value is a read cache, overwritten in place. Replaying the log to any instant rebuilds the value that held then, so overwriting loses nothing. A logged transaction is the only thing that changes the value. The value exists from the obligation’s creation onward.

15.6.4 Registration

Obligations register through two channels. At **unit registration**, the lifecycle function extracts dated obligations from the CDM **EconomicTerms** (coupon, reset, termination schedules), one per date. These correspond one-to-one with the scheduler’s timers (§15.4). They make explicit what the timer alone left implicit: each has a defined discharge predicate and compensation action. At **event processing**, the lifecycle function returns, alongside the event’s moves and state deltas, any obligation the event creates. The executor commits the obligation registration atomically with the triggering event.

Registration cannot rest on a workflow author remembering to create the obligation. The principle below places it in the lifecycle function’s output contract instead.

Principle 15.4 (Obligation completeness). *A lifecycle function processing an obligation-creating event includes the obligation in its output. A function that omits a required obligation is incorrect, irrespective of whether its moves are valid.*

The CDM event-type enumeration bounds the test space, so this principle is enforceable by property-based testing. For each obligation-creating event type (Table 10), the generator produces events and the postcondition verifies a non-empty, well-formed obligation list. A typed return that mandates an obligation list for obligation-creating event types makes omission a compile-time error.

15.6.5 The obligation workflow

Each obligation is managed by one workflow, either spawned per obligation or embedded in the unit or agreement lifecycle workflow that owns its source. The pattern is uniform: a selector waits on a discharge signal channel and a deadline timer. On a discharge signal, the handler checks D against ledger state. If D is satisfied, the obligation becomes **DISCHARGED** and the workflow completes. Otherwise the obligation becomes **ATTEMPTED** and the workflow waits. When the timer fires at t_d , the handler re-checks D . If D is satisfied, the obligation becomes **DISCHARGED**. Otherwise the handler commits κ through the executor, reaching **COMPENSATED** on success and **DEFAULTED** (raising a default event) on failure. The workflow ID is derived deterministically from $o.id$, so the engine’s duplicate-start rejection prevents double-spawning.

15.6.6 Liveness invariants

Three invariants govern the subsystem. In plain terms: every obligation reaches a terminal state by its deadline (P21); discharging or compensating it moves value only through the executor, so conservation is never bypassed (P22); and discharging it twice changes nothing (P23).

Invariant 15.5 (P21—Obligation Liveness). *For every $o \in \mathcal{O}$ with deadline t_d :*

$$\forall t > t_d : \quad o.state \in \{\text{DISCHARGED}, \text{COMPENSATED}, \text{DEFAULTED}\}.$$

*No obligation persists in **PENDING** or **ATTEMPTED** beyond its deadline.*

Invariant 15.6 (P22—Obligation Conservation). *Every discharging move set and every compensation move set for o satisfies the conservation law $Q(u) = 0$ (P1). The obligation mechanism does not bypass the executor.*

Invariant 15.7 (P23—Obligation Idempotency). *Discharging the same obligation twice, by $o.id$, has no incremental effect: D is checked before moves are emitted, and if already satisfied no moves are produced. This composes with transaction-ID idempotency (P5) and lifecycle idempotency (P6).*

15.6.7 Proof of liveness

The proof follows the obligation’s own path. The obligation is registered; a workflow with a timer exists for it; the timer survives failures and fires; the total handler then lands in a terminal state. Each step is a lemma.

Theorem 15.8 (Obligation liveness). *Under the executor, engine, and obligation store, P21 (Invariant 15.5) holds: no obligation persists in PENDING or ATTEMPTED beyond its deadline.*

Proof. Five lemmas compose.

L1 (Registration completeness). Every obligation-creating event registers an obligation in $\mathcal{O}$. By Principle 15.4, registration is part of the lifecycle function’s output contract; the executor commits it atomically with the triggering event; property-based testing verifies a non-empty obligation list per obligation-creating event type.

L2 (Workflow creation). Every $o \in \mathcal{O}$ is managed by a workflow with a timer at t_d . The executor’s post-commit hook spawns it; the deterministic workflow ID makes the engine reject double-spawning.

L3 (Timer durability). The timer at t_d is persisted by the engine and survives server restarts, worker deployments, version upgrades, and data-centre failovers—a guarantee of the engine (§15.1).

L4 (Timer fires). The timer fires at t_d , by the engine’s durable-execution guarantee. *Prerequisite:* the engine cluster is available at t_d . Cluster availability is an operational prerequisite, not a framework property; multi-region replication mitigates but does not eliminate the dependency.

L5 (Handler totality). On firing, the handler transitions o out of PENDING. The handler is total: $D = \text{true}$ yields DISCHARGED; otherwise κ fires, yielding COMPENSATED on success and DEFAULTED (with a default event) on failure. Every branch terminates in a terminal state.

Composition. For o with deadline t_d : $o \in \mathcal{O}$ (L1); a workflow with a timer at t_d exists (L2); the timer is durable (L3) and fires (L4); the handler reaches a terminal state (L5). Hence for all $t > t_d$, $o.state \in \{\text{DISCHARGED}, \text{COMPENSATED}, \text{DEFAULTED}\}$. $\square$

P22 follows from P1 and executor exclusivity: every discharging and compensating move passes through the executor, which validates $Q(u) = 0$ before commit. P23 composes with P5 and P6. The obligation ID is part of the transaction source metadata. The discharge predicate prevents re-emission. The transaction ID prevents re-commitment. The composition is summarised below.

Property	Guarantee source
Every obligation fires	engine durable timer (L4)
No obligation fires twice	idempotency (P5, P6, P23)
Discharge preserves conservation	executor (P1, P22)
Handler reaches terminal state	workflow totality (L5)

Every obligation is thereby tracked, timed, and resolved. This includes obligations created dynamically during the lifecycle, not only those with a deterministic registration-time date.

15.6.8 Worked example: CSA variation margin call

Alice and Bob hold a CSA over five IRS trades, threshold \$500,000, minimum transfer amount (MTA) \$250,000. The daily margin workflow computes an aggregate mark-to-market of \$2.3M in Alice's favour; required margin is $\max(0, |\$2.3M| - \$500K) = \$1.8M \geq \$250K$, so a call is generated and registers an obligation `csaVM`, committed atomically with the call event: type `CSA_VARIATION_MARGIN` on source CSA-AB; deadline T+1 next business day close; discharge predicate $D = \text{Bob's coll_post}(\text{USD}, \text{CSA-AB})$ increased by at least \$1.8M; compensation $\kappa = \text{ISDA close-out netting of CSA-AB}$. **Discharge:** Bob delivers \$1.8M before the deadline — $\Delta w_{\text{Bob}}(\text{USD})[\text{own}] = -1,800,000$, $\Delta w_{\text{Alice}}(\text{USD})[\text{coll_recv}] = +1,800,000$, $Q(\text{USD}) = 0$ — the signal satisfies D and the obligation lands `DISCHARGED`. **Compensation:** Bob does not deliver; the timer fires, D fails, and κ terminates all five trades, computes close-out values per the ISDA Master Agreement, and emits one net settlement move through the executor: `COMPENSATED`.

15.6.9 Worked example: SBL collateral substitution

A lender demands substitution: bond XYZ is no longer eligible under the updated collateral schedule (IBP references denote clauses of the ISLA Best Practice Handbook [1]). The eligibility check registers an obligation on loan L123: type `SBL_COLLATERAL_SUBSTITUTION`; deadline one hour before the DVP cut-off (IBP-170); discharge = new eligible collateral posted *and* old released; compensation = collateral-shortfall escalation. **Discharge:** the borrower delivers German Bund in one four-move transaction, each move satisfying the Single-Coordinate Move Principle (Principle 16.1) — release `coll_post/coll_recv` of XYZ by the old amount, post `coll_post/coll_recv` of BUND by the new — so per-unit conservation holds on both units ($\Delta Q(\text{XYZ}) = 0$, $\Delta Q(\text{BUND}) = 0$): `DISCHARGED`. **Compensation:** the cut-off passes without delivery; the escalation marks L123 collateral-deficient in the SBL unit state and raises an exposure-breach notification, which beyond the GMSLA grace period triggers recall or default per GMSLA §10: `COMPENSATED`.

15.6.10 Relationship to SBL invariants

P21 supplies the temporal guarantee underpinning the SBL invariants (P11–P20). It guarantees collateral top-up obligations resolve by deadline, by delivery or by compensation, so P13 (Collateral Sufficiency) cannot be violated by an untracked call. It guarantees each reportable event's reporting obligation fires before its T+1 deadline, or escalates, sustaining P16 (SFTR Completeness). It guarantees recall obligations resolve, so shares cannot remain on loan past a recall deadline without return or buy-in, sustaining P18 (Lender Ownership Invariance).

15.6.11 Property-based testing for obligations

Three properties extend the testing framework. **Registration completeness:** for random obligation-creating events, the output includes a non-empty obligation list with valid deadline, discharge predicate, and compensation action. **Discharge conservation:** every discharge move set satisfies $Q(u) = 0$. **Handler totality:** over random obligation states $\times \{\text{discharge-before-deadline, no-discharge, compensation-success, compensation-failure}\}$, the handler reaches a terminal state in every case. The CDM event-type enumeration bounds the obligation-creating event space, making coverage structurally complete.

15.6.12 Assumptions and limitations

Theorem 15.8 depends on four assumptions, each at the framework boundary. **Engine availability at t_d :** an unavailable cluster delays the timer until recovery; liveness holds eventually, not instantaneously. **Executor availability:** an unavailable executor leaves the obligation

PENDING under retry until recovery. This is a bounded delay, not a permanent failure. **External cooperation:** discharge conditions requiring counterparty action cannot be compelled; compensation is the contractual fallback. **Lifecycle-function correctness:** L1 assumes lifecycle functions register all required obligations. This is enforceable by property-based testing and, in a typed implementation, by a mandatory obligation list in the return type. These assumptions match the framework’s existing boundary: correctness within the ledger, given infrastructure availability and correct lifecycle functions. External actors and infrastructure availability lie outside it.

16 Generalised Positions and Securities Borrowing and Lending

The scalar model of Sections 2–17 stores one number per (wallet, unit) pair. That one number is complete for every instrument whose ownership and possession never diverge: cash, equities, derivatives, swaps. It fails the instant they diverge: securities lending, repo, pledged collateral. This section generalises the scalar balance to a six-coordinate vector that resolves the failure. It then derives Securities Borrowing and Lending (SBL) from the framework’s primitives. Repo, collateral management beyond SBL, and novation are open problems (Section 21).

16.1 Where the Scalar Model Breaks

Alice owns 1,000 VOD. A single number records it, $w_{\text{Alice}}(\text{VOD}) = 1,000$, and that number is complete. She lends 400 to Bob. A scalar balance now admits only three encodings, and each is wrong:

1. *Transfer:* $w_{\text{Alice}} = 600$, $w_{\text{Bob}} = 400$. Conservation holds, but Alice’s PnL drops 40%—wrong: under IFRS 9 §3.2.6 the lender does not derecognise lent securities.
2. *Do not transfer:* $w_{\text{Alice}} = 1,000$, $w_{\text{Bob}} = 0$. Bob possesses 400 shares with no record of them.
3. *Credit both:* total = 1,400 against 1,000 in the external world. Conservation is violated.

A scalar balance cannot encode two operational states at once. After the loan Alice holds 600 available and 400 on loan: two facts, and one number cannot carry both. “On-loan” is a coordinate because “lend” is a physical action that changes operational status without changing ownership.

16.2 The Generalised Coordinate Vector

A *coordinate* is a stored quantity that passes the *physical-action test*: a distinct physical action modifies it and only it. A quantity fixed by other coordinates through an algebraic identity is a *projection*. A projection is computed on read, never stored, and never written by a move.

The intuition: one move changes one number; anything larger is a list of moves.

Principle 16.1 (Single-Coordinate Move). *An atomic move modifies exactly one coordinate of one unit across the two entities it connects: one move, one coordinate, one unit, source and destination. A move is the smallest state change. Any operation requiring multiple coordinate changes is a transaction—a list of moves executed atomically.*

The scalar core is the one-coordinate case of this principle: there each entity has a single coordinate, and a move modifies one wallet of it. The vector model carries the same guarantee across six coordinates.

MENTAL MODEL. One number per holding becomes six. Each records a distinct operational fact about the same shares: how many the entity owns, how many it has lent out, how many it has borrowed, and how much collateral it has posted, received, and re-used. Only ownership carries economic value; the other five record where the shares sit, who holds title to them, and

who owes them back. Unlike moving a crate to another aisle of a warehouse, which leaves it just as reachable, marking shares on-loan or posted-as-collateral removes them from what the entity may lend or sell again while moving no value out of ownership: the lent share stays the lender's asset, yet is no longer available. Available inventory is ownership less what is lent, plus what is borrowed, with the collateral coordinates holding shares aside. The familiar scalar balance is the degenerate case in which only the ownership number is non-zero.

Definition 16.2 (Position Vector). *For entity e and unit u , the position vector $\vec{w}_e(u) \in \mathbb{R}^6$ is*

$$\vec{w}_e(u) = (w_e^{\text{own}}(u), w_e^{\text{onloan}}(u), w_e^{\text{borr}}(u), w_e^{\text{coll-post}}(u), w_e^{\text{coll-recv}}(u), w_e^{\text{coll-rehyp}}(u)).$$

Each coordinate passes the physical-action test:

- *own: economic ownership. Modified by buy/sell. May be negative (short). Drives PnL.*
- *onloan: securities lent out. Modified by lend/recall-return. Lender retains ownership.*
- *borr: borrowed shares received (the return obligation count). Modified by borrow-receive/return. Non-negative. Unchanged when the borrower sells.*
- *coll_post: collateral delivered to a lender or triparty agent. Modified by post/release.*
- *coll_recv: collateral held from the borrower. Modified by receive/release.*
- *coll_rehyp: non-cash collateral re-used by the taker. Modified by rehypothecate/recall. Tracked for SFTR Article 15.*

The first projection answers one operational question: how much of the unit the entity can trade right now.

Definition 16.3 (Available Inventory). *For every entity e and lendable unit u , available inventory is the read-time projection*

$$\text{avail}(e, u) \stackrel{\text{def}}{=} \vec{w}_e(u)[\text{own}] - \vec{w}_e(u)[\text{onloan}] + \vec{w}_e(u)[\text{borr}]$$

—what is tradable equals what is owned, less what is lent, plus what is borrowed. Because *avail* is never stored, the identity holds by definition: no stored value can drift from the coordinates that determine it.

The model's computational content is three pieces: the six-field record, *avail* as a total projection, and *applyMove* writing one coordinate.

```
-- Qty: exact minor units, the additive abelian group reused from Sec.2.
-- (One scale; own may be negative, the operational coords are guarded
-- at value level, not by type -- see below.)
newtype Qty = Qty Integer deriving (Eq, Ord, Show)
instance Semigroup Qty where Qty a <> Qty b = Qty (a + b)
instance Monoid Qty where mempty = Qty 0
qneg :: Qty -> Qty
qneg (Qty n) = Qty (negate n)

-- The position vector: one field per coordinate (Def. position-vector).
data Position = Position
  { own      :: !Qty -- economic ownership; may be negative (short)
  , onloan   :: !Qty -- lent out; lender retains ownership
  , borr     :: !Qty -- borrowed; the return-obligation count
  , collPost :: !Qty -- collateral delivered
  , collRecv :: !Qty -- collateral held
  , collRehyp :: !Qty -- collateral re-used
  } deriving (Eq, Show)

zeroPos :: Position -- the never-held position
zeroPos = Position mempty mempty mempty mempty mempty mempty
```

```

-- avail: a TOTAL projection, computed on read, never stored (Def. avail,
-- P20). It cannot drift because no value holds it -- it IS this function.
avail :: Position -> Qty
avail p = own p <> qneg (onloan p) <> borr p

-- A move names exactly ONE coordinate. The Coord tag is what makes the
-- Single-Coordinate Move Principle hold by construction: there is no
-- constructor that writes two fields.
data Coord = Own | OnLoan | Borr | CollPost | CollRecv | CollRehyp
  deriving (Eq, Show)

applyMove :: Coord -> Qty -> Position -> Position
applyMove Own      q p = p { own      = own p      <> q }
applyMove OnLoan   q p = p { onloan   = onloan p    <> q }
applyMove Borr     q p = p { borr     = borr p      <> q }
applyMove CollPost q p = p { collPost = collPost p  <> q }
applyMove CollRecv q p = p { collRecv = collRecv p  <> q }
applyMove CollRehyp q p = p { collRehyp = collRehyp p <> q }

```

avail is a group homomorphism. The effect of any move on availability is fixed by the coordinate it writes: $\text{avail}(\text{applyMove } c \ q \ p) = \text{avail}(p) \diamond \delta_c(q)$ with $\delta_{\text{Own}} = \delta_{\text{Borr}} = q$, $\delta_{\text{OnLoan}} = \text{qneg } q$, and $\delta_c = \text{mempty}$ for the three collateral coordinates. A *stored* availability could therefore only drift; the projection cannot. After Alice lends 400 (`applyMove OnLoan (Qty 400)` on (1000, 0, 0, 0, 0, 0)) the record reads (1000, 400, 0, 0, 0, 0) and `avail` returns `Qty 600`. Ownership is untouched; availability is down by the loan.

Remark 16.4 (Discharged by construction). `applyMove` is total (Coord matched exhaustively; Qty is a total group, so no input is rejected and none diverges) and deterministic; the Coord tag makes a two-coordinate move unrepresentable, discharging Principle 16.1; `avail` is a pure function of three stored fields, so P20 (Available-Inventory Identity) holds with no stored value to violate. Non-negativity of onloan, borr, and the collateral coordinates is not typed away (they share Qty with own, which is legitimately negative); the listing’s comment records this. It is a value-level pre-condition: the lending gate $\text{avail}(e, u) \geq Q$ of Section 16.5, enforced where the transaction is built, not here.

Remark 16.5 (Special cases).

- Pure holder/lender ($\text{borr} = 0$): $\text{avail} = \text{own} - \text{onloan}$.
- Non-SBL entity (all SBL coordinates zero): $\text{avail} = \text{own}$.
- Borrower who sold Q of N borrowed ($\text{own} = -Q$, $\text{borr} = N$): $\text{avail} = N - Q$.

Why own is a coordinate, not a projection. Four reasons. (1) `PnL` is a direct lookup: no summation, no regime-dependent read logic. (2) For non-lendable instruments only `own` is non-zero, so the vector is a scalar. (3) `own` is not derivable from the five operational coordinates: two states with identical operational vectors but distinct `own` exist whenever a borrower has sold some but not all borrowed shares. (4) Buy/sell is a distinct physical action touching no other coordinate.

Conservation needs no new idea: a move still takes a quantity from one slot and adds it to another; only the number of slots has grown.

Proposition 16.6 (Conservation in the Generalised Model). *Each move subtracts q from one (entity, coordinate) slot and adds q to another, so the algebraic sum over all slots is preserved—the $\text{src} -= q$; $\text{dst} += q$ pattern of the core framework (Section 2). For the physical-quantity check, substitute the `avail` projection. For each unit u the lender’s `+onloan` and the per-relationship virtual wallet’s `-onloan` cancel, leaving five terms per real entity:*

$$\sum_{e \in \mathcal{E}} (\vec{w}_e(u)[\text{own}] + \vec{w}_e(u)[\text{borr}] + \vec{w}_e(u)[\text{coll_post}] + \vec{w}_e(u)[\text{coll_recv}] + \vec{w}_e(u)[\text{coll_rehyp}]) = 0,$$

$\mathcal{E}$ ranging over real entities and virtual wallets. The cancellation of onloan encodes the physical fact that lent shares are counted once—as the borrower’s borr.

Portfolio value uses own directly: $V_e(t) = \sum_u \vec{w}_e(u)[\text{own}] \cdot P_t(u)$. This is the scalar formula of Section 5 with own in place of the scalar balance; no projection or conditional logic enters.

In-flight collateral. Trade-date accounting updates the position vector at execution. The unsettled delivery lives in the virtual wallet as the custodian contra-entry. SBL prepay collateral (IBP-177⁴) is instructed before the loan settles. The in-flight collateral is then a virtual-wallet entry; on settlement it resolves to a move writing coll_recv (lender) or coll_post (borrower). Custodian reconciliation checks $\text{avail}(L, u) + \vec{w}_L(u)[\text{onloan}] + \text{inflight}(L, u) \stackrel{?}{=} \text{depot}(u)$. Substituting avail simplifies this to $\vec{w}_L(u)[\text{own}] + \vec{w}_L(u)[\text{borr}] + \text{inflight}(L, u) \stackrel{?}{=} \text{depot}(u)$.

16.3 Graceful Degeneration to the Scalar Model

Remark 16.7 (Non-lendable instruments). *For non-lendable instruments—OTC derivatives, listed options and futures, FX forwards, credit derivatives, cash—only own is non-zero: $\vec{w}_e(u) = (q, 0, 0, 0, 0, 0)$. The avail projection yields q , valuation reduces to one lookup per unit, and the scalar model of Sections 2–17 is recovered exactly. The generalisation is invisible when not needed.*

16.4 The Lending Resolution: Named Wallets

After Alice lends 400 VOD to Bob:

$$\vec{w}_{\text{Alice}}(\text{VOD}) = (1000, 400, 0, 0, 0, 0), \quad \vec{w}_{\text{Bob}}(\text{VOD}) = (0, 0, 400, 0, 0, 0).$$

Ownership is unchanged. $\text{avail}_A = 1000 - 400 + 0 = 600$; $\text{avail}_B = 0 - 0 + 400 = 400$. A per-relationship virtual wallet absorbs the delivery contra-entry: $w_{A \leftrightarrow B}^{\text{virtual}}(\text{VOD}) = -400$. Proposition 16.6 gives conservation:

$$\underbrace{1000}_{\text{own}_A} + \underbrace{0}_{\text{borr}_A} + \underbrace{0}_{\text{own}_B} + \underbrace{400}_{\text{borr}_B} + \underbrace{(-400)}_{\text{virtual}} + \underbrace{(-1000)}_{\text{custodian}} = 0.$$

Alice’s PnL reads $\vec{w}_{\text{Alice}}(\text{VOD})[\text{own}] = 1,000$, correct.

Reclassification vs. transfer. A reclassification is a two-move transaction within one entity: one coordinate down, another up. Rehypothecation is the example: coll_recv ↓, coll_rehyp ↑. A transfer is a move between entities. The primitive is identical; the distinction is entity identity. Lending writes onloan; the drop in avail is read, not written.

16.5 Analytical Projections and Encumbrance

Three read-time questions recur: what can be traded, what is physically held, and what is owned but locked. Each has a projection; none has storage.

Definition 16.8 (Projections). *A coordinate-projection is a function of the six stored coordinates. A store-projection folds, in addition, over records in the Unit Store. Neither is stored or written by a move; a materialised aggregate of either is a rebuildable index, never an authority. The coordinate-projections:*

$$\begin{array}{ll} \text{avail}(e, u) = \vec{w}_e(u)[\text{own}] - \vec{w}_e(u)[\text{onloan}] + \vec{w}_e(u)[\text{borr}] & \text{tradeable inventory} \\ \text{possess}(e, u) = \text{avail}(e, u) + \vec{w}_e(u)[\text{coll_recv}] & \text{physical custody} \\ \text{encumb}(e, u) = \vec{w}_e(u)[\text{onloan}] + \vec{w}_e(u)[\text{coll_post}] & \text{owned, not tradeable} \end{array}$$

⁴IBP references denote clauses of the ISLA Best Practice Handbook: International Securities Lending Association, “ISLA Best Practice Handbook,” various editions.

One store-projection exists: $\text{reserved}(e, u, t)$, the sum of *remaining_quantity* over e 's active put-on-hold locates in u at t (Section 16.14). It is not a function of the six coordinates, which is why the second class is needed.

An entity may sell q only if $\text{avail}(e, u) \geq q$. A scalar balance of 1,000 passes a sell of 500; with 700 on loan, only 300 are available. An in-ledger lender may lend or confirm locates only against

$$\text{AvailableToLend}(e, u, t) = \max(0, \text{avail}(e, u) - \text{reserved}(e, u, t) - \text{regulatory_hold}(e, u, t)),$$

regulatory_hold a recorded, as-of-pinned restriction input.

16.6 The SBL Smart Contract

The claim: SBL needs no new primitive, only new data and new invariants.

Principle 16.9 (SBL Representability). *SBL is a smart contract over existing primitives. It adds: (1) an SBL loan unit type in the Unit Store; (2) a `legal_regime` field; (3) invariants P11–P20 (Section 16.18). It changes neither the move primitive, the conservation law, nor the execution engine. Every SBL lifecycle event decomposes into moves satisfying Principle 16.1.*

MENTAL MODEL. A smart contract is a machine bolted into the sealed pipework, its gears fixed so that identical inputs always drive identical outputs, and its only outputs balanced transfers—each draining one tank by exactly what it fills into another. A wrong setting makes it compute the wrong transfer, the wrong quantity into the wrong tank, but never an unbalanced one: a faulty machine can misroute the water yet cannot create or destroy a drop. Unlike a general program, its fault corrupts the amounts it moves, never the conservation of what it emits — the division §15.2 states precisely.

Why SBL is native, not a companion system. The decisive argument is double-lending prevention. A companion system reintroduces a TOCTOU race: between querying inventory and executing the loan, another process lends the same shares. Cross-boundary conservation becomes unprovable. Collateral becomes invisible, breaking P13 (Section 16.18). Exposure requires a two-system join. The companion “reserved” bucket discards load-bearing information.

The loan unit’s lifecycle state is held in `UnitStatus` and read identically by every holder:

```
SBLUnitState:
  loan_id, lender, borrower, agent
  isin, quantity, original_qty, term_type, maturity_date
  fee_rate, rebate_rate
  collateral_type, margin_pct, haircut_pct, collateral_ccy
  triparty_agent, legal_regime, rehyp_consent
  lifecycle_stage, settlement_status, recall_date, recall_qty
  sftr_uti, slate_loan_id, execution_ts, trade_date
  last_mark_date, accrued_fee, fee_accrual_log
```

Remark 16.10 (Loan state is a projection of the log). *UnitStatus is a projection of the event log (§4.3). The loan unit’s `lifecycle_stage`, `settlement_status`, and `accrual` fields are this cache; the SBL state machine (Section 16.7) is the function the log replays.*

The SBL contract is a pure function $f_{\text{SBL}} : (u_\ell, \text{state}_t(u_\ell), e, \text{data}) \rightarrow (\tau, \text{state}_{t+1}(u_\ell))$.

Loan unit valuation (bond analogy). The loan unit is priced. The lender holds +1, the borrower −1; the holdings conserve to zero. Its price is the accrued unpaid fee, $P_t(u_{\text{loan}}) = \text{loan_value}(t) \times \text{fee_rate} \times \text{DCF}(\text{last_settle}, t)$. Fee payment is a cash move that resets the accrual. The price drops to zero; the cash move offsets; net PnL from payment is zero, as for a bond coupon. No standalone fee-accrual term enters the PnL formula. Under IFRS 9 §3.2.6 the lender does not derecognise lent securities, because the risks and rewards of ownership are not transferred. Lent securities therefore remain in own.

16.7 SBL State Machine

The state machine is total: every (state, event) pair yields a valid transition or an explicit rejection. Terminal states are RETURNED, CANCELLED, DEFAULTED; $S_{\text{live}} = \{\text{ACTIVE}, \text{RECALLED}, \text{PARTIALLY_RETURNED}\}$. The table answers two questions for every event: which state follows, and which moves fire.

From	Event	To	Moves Generated
—	new_loan	PENDING	None
PENDING	loan_settled	ACTIVE	Securities + collateral
PENDING	cancel	CANCELLED	None
ACTIVE	recall	RECALLED	None (notification)
ACTIVE/RECALLED/PART. RET.	partial_return	PART. RET.	Partial securities + collateral
ACTIVE/RECALLED/PART. RET.	full_return	RETURNED	Remaining securities + collateral
S_{live}	rate_change	S_{live}	State-only
S_{live}	mark_to_market	S_{live}	Margin call moves (if threshold)
S_{live}	collateral_sub	S_{live}	Paired collateral swap
S_{live}	corp_action	S_{live}	Manufactured payment
S_{live}	reallocation	S_{live}	State-only (lender change)
ACTIVE	maturity	RETURNED	Full securities + collateral
S_{live}	buy_in	RETURNED	Buy-in + close-out moves
any non-terminal	default	DEFAULTED	Close-out moves per GMSLA

16.8 Atomic Moves for SBL Lifecycle Events

Each lifecycle event is a move-carrying Transaction with `txIntroduce = Nothing` and `txAppend = Nothing`: it operates on units already registered, introducing no unit and appending no terms. Its `txMoves` are the ordered moves below. Each move touches one coordinate per entity. Conservation holds by construction from the signed edge-sum, so there is no separate validation step. Loan value (IBP-163): $LV = \lceil Q \times P \times M\% \times \chi \rceil_{0.01}$.

Loan initiation (four moves):

```

tau_new.txMoves:                                -- txIntroduce = Nothing
  Move 1: Lender  onloan    += Q                  -- securities, u_s
  Move 2: Borrower borrh    += Q
  Move 3: Borrower coll_post += LV                -- collateral, ccy
  Move 4: Lender  coll_recv += LV

```

Move 1 draws from $w_{L \leftrightarrow B}^{\text{virtual}}$; Move 2 adds from it. Moves 3–4 pair; cash collateral writes own instead of `coll_post`, with the virtual wallet absorbing the non-cash contra-entry. No move writes the lender's own: P18 holds. After: $\text{avail}_L = 1000 - 400 + 0 = 600$ (down Q), $\text{avail}_B = 0 - 0 + 400 = 400$ (up Q). When the loan converts a locate, the conversion transaction co-admits these four moves with the locate's drawdown and terminalisation; a partial conversion leaves a residual live locate (Section 16.14).

Recall and return. Collateral release on partial return: $LV_{\text{new}} = \lceil (Q_{\text{out}} - Q_{\text{ret}}) \times P \times M\% \times \chi \rceil_{0.01}$, $C_{\text{release}} = C_{\text{held}} - LV_{\text{new}}$.

```

tau_return.txMoves:                             -- txIntroduce = Nothing
  Move 1: Borrower borrh    -= Q_ret
  Move 2: Lender  onloan    -= Q_ret
  Move 3: Lender  coll_recv -= C_rel
  Move 4: Borrower coll_post -= C_rel

```

Lender's avail rises by Q_{ret} (onloan $\downarrow$); borrower's falls by Q_{ret} (borr $\downarrow$).

Short sale. Bob sells 400 borrowed VOD to Carol. This is a standard buy/sell writing only own:

```
tau_sell.txMoves:                                -- txIntroduce = Nothing
  Move 1: Bob   own -= 400
  Move 2: Carol own += 400
```

Bob's borr is untouched; it tracks the obligation, not possession. $\text{avail}_B = -400 - 0 + 400 = 0$: Bob has consumed his available inventory.

Mark-to-market margin call (one move per entity for the collateral unit):

```
tau_margin.txMoves:                             -- txIntroduce = Nothing
  Move 1: Borrower coll_post += delta_LV
  Move 2: Lender   coll_recv += delta_LV
```

Cash collateral writes the borrower's own; paired moves sum to zero.

Other events. Collateral substitution (IBP-170) takes four moves: release old, post new. A manufactured dividend is one cash move per entity writing own, of amount $\text{dividend} \times Q_{\text{onloan}}$. Buy-in (GMSLA 9.3) comprises market purchase, close-out, collateral release, and cost attribution, each move satisfying Principle 16.1.

16.9 Title Transfer vs Security Interest

The legal regime determines what the collateral taker may do with received collateral:

Regime	Collateral Treatment	Rehypothecation
TITLE_TRANSFER (GMSLA 2000/2010)	Receiver owns collateral	Permitted by default
SECURITY_INTEREST (GMSLA 2018)	Pledged; poster retains ownership	Forbidden absent GMSLA Schedule consent
US_15C3_3	Regulated possession/control	Capped at 140% of customer debit

The wallet structure and the avail projection are identical under every regime. The regime affects only the PnL projection. The `legal_regime` is set at inception and immutable.

Remark 16.11 (Pledge and the own coordinate). *Under security interest the poster retains economic ownership of pledged collateral, so*

$$V_e^{\text{SI}}(t) = \sum_u (\vec{w}_e(u)[\text{own}] + \vec{w}_e(u)[\text{coll_post}]) \cdot P_t(u),$$

whereas under title transfer $V_e^{\text{TT}}(t) = \sum_u \vec{w}_e(u)[\text{own}] \cdot P_t(u)$. The own coordinate therefore does not alone capture economic ownership under pledge: ownership splits between own (lent securities) and coll_post (pledged collateral). The framework keeps own a single, regime-independent meaning (ownership from buy/sell/lend) and makes the pledge adjustment explicit in the valuation function—as the dirty-price function carries bond accrual rather than mutating the balance.

16.10 Collateral and Margin

Each collateralisation method reduces to a fixed pattern of moves; the table gives the mapping.

Method	Mechanism	Ledger Representation
Cash rebate	Cash collateral; lender pays rebate	Single cash move per loan
Non-cash bilateral	Securities B→L directly	Securities to $w_L^{\text{coll-recv}}$; haircut applied
Non-cash triparty	Securities to triparty agent	RQV instruction; daily agreement (IBP-189)
Cash pool (standard)	Single pool per cpty-cy	Aggregate marking; single margin move (IBP-322)
Cash pool (EU)	Per-loan margin tracking	Individual marks; net movement (IBP-323)
Uncollateralised	No collateral	No collateral moves (IBP-326)

Exposure (IBP-163): $LV = \lceil Q \times P_{\text{close}} \times M\% \times \chi \rceil_{0.01}$. Margin calls are conservation-preserving transfers between the borrower's `coll_post` (or `own` for cash) and the lender's `coll_recv`. Rehypothecation is a reclassification: two moves carrying `coll_recv` ↓, `coll_rehyp` ↑ within the taker's position. Substitution demands and top-up obligations with deadlines are formalised as obligations under the obligation workflow (Section 15.6). A worked SBL substitution is at Section 15.6.9.

16.11 Short Selling and Inventory

Short selling creates a negative own coordinate; a negative own is the defining feature of a short position. Alice borrows 1,000 NVDA from Bob, then sells 500:

$$\text{borrow: } \vec{w}_{\text{Bob}} = (1000, 1000, 0, 0, 0, 0), \quad \vec{w}_{\text{Alice}} = (0, 0, 1000, 0, 0, 0),$$

$$\text{after sale: } \vec{w}_{\text{Alice}} = (-500, 0, 1000, 0, 0, 0), \quad \text{avail}_{\text{Alice}} = -500 - 0 + 1000 = 500.$$

The sale writes only `own`. `borr` stays at 1,000 because the return obligation is unchanged. The negativity lives in the economic dimension alone; no operational coordinate goes negative.

Locate and available-to-lend. EU Short Selling Regulation (SSR) Article 12(1) requires a reasonable expectation of borrowing before short selling. The gate realises the `AvailableToLend` projection of Section 16.5:

```
def available_to_lend(entity, security, t):
    # entity: an in-ledger lender; t: the admitting transaction's log timestamp
    pos = position_vector(entity, security, t)
    avail = pos.own - pos.onloan + pos.borr
    reserved = sum(loc.remaining for loc in active_locates(entity, security, t)
                    if loc.kind == PUT_ON_HOLD)
    regulatory_hold = regulatory_restrictions(entity, security, t)
    return max(0, avail - reserved - regulatory_hold)
```

`active_locates` is the activity predicate of Section 16.14, decidable from the committed log alone; the due-event scheduler's terminal sweep at expiry is hygiene, not authority. `regulatory_restrictions` is a recorded, as-of-pinned input. Fail-to-deliver (FTD) close-out: Reg SHO Rule 204 (T+3); CSDR daily cash penalties.

16.12 Cash Collateral Reinvestment

US cash-collateral lending falls under Rule 15c3-3, which subjects the cash to possession-or-control and reserve-formula requirements. The lender reinvests received cash collateral. Two economic positions then exist and must be tracked independently:

1. **Collateral obligation:** the duty to return cash collateral on termination—a fixed amount (mark-adjusted) in the loan unit state (`collateral_amount`), reflected in the lender’s `coll_recv`.
2. **Reinvestment asset:** the money-market fund (MMF) units, paper, or repo bought with the cash—a separate position in the lender’s own for the reinvestment instrument, carrying its own market risk.

Reinvestment is a standard buy/sell transaction. Cash leaves `coll_recv`; the reinvestment unit enters own; the MMF takes the cash for its units. Conservation holds per leg. The cash collateral is thereby transformed: `coll_recv(USD)` falls to zero while `own(MMF)` holds the reinvestment. The return obligation persists, fixed in the loan unit state. The separation is the point: the reinvestment asset may move in value while the obligation does not.

Reinvestment risk. If the MMF breaks the buck (NAV below \$1.00), the reinvestment position falls below the fixed obligation. The gap is uncollateralised credit risk borne by the lender. This risk materialised in September 2008 (Reserve Primary Fund). SEC Rule 2a-7 reforms have not eliminated it for institutional prime MMFs.

Unwind on return. Termination sells the reinvestment units back for cash, restoring `coll_recv`. The loan-return transaction then releases it. CDM: the reinvestment buy/sell is a standard **Transfer** (no gap); the obligation maps to `SecurityLendingPayout.collateral`. The full numeric walkthrough is in Appendix I (US SEC 15c3-3 worked example).

16.13 On-Lending Chains

Borrowed shares may be re-lent, forming on-lending chains of independent bilateral loans. The six-coordinate model handles them unmodified. Alice owns 1,000 XYZ and lends 1,000 to Bob (ℓ_1); Bob re-lends 500 to Charlie (ℓ_2). Each loan is its own unit with its own fee, collateral, and reporting. Columns abbreviate the coordinates: `onl` = onloan, `c_p` = `coll_post`, `c_r` = `coll_recv`, `c_rh` = `coll_rehyp`; `avail` = `own` – `onl` + `borr` is the read projection.

Entity	own	onl	borr	c_p	c_r	c_rh	avail
Alice	1,000	1,000	0	0	C_1	0	0
Bob	0	500	1,000	C_1	C_2	0	500
Charlie	0	0	500	C_2	0	0	500

Verification: $\text{avail}_{\text{Alice}} = 0$ (all lent), $\text{avail}_{\text{Bob}} = 0 - 500 + 1000 = 500$, $\text{avail}_{\text{Charlie}} = 500$; system `own` = 1,000 (physical shares unchanged).

P18 scope. P18 (Lender Ownership Invariance) fixes `own` across lending for *pure* lenders (`borr` = 0). It applies to Alice, not to Bob (`borr` = 1,000, `onloan` = 500); Bob’s exposure is through `borr`. For an intermediary re-lender (`own` = 0, `borr` > 0) the binding identity is `onloan` ≤ `borr`: an entity cannot re-lend more than it has borrowed plus what it owns. The identity follows from the lending pre-condition $\text{avail}(e, u) \geq Q_{\text{lend}}$, i.e. `own` – `onloan` + `borr` ≥ Q_{lend} .

Cascade recall. When Alice recalls 1,000 from Bob, Bob can return `avail` = 500 at once and must recall the remaining 500 from Charlie. Charlie returns 500; Bob’s vector becomes (0, 0, 1000, C_1 , 0, 0) with `avail` = 1000. Bob returns 1,000 to Alice. The cascade carries settlement-timing risk: a downstream failure fails the upstream return, triggering CSDR penalties or Reg SHO close-out.

The cascade is orchestrated as a saga: a long-running workflow split into steps, each with a compensating action that undoes it on failure (Section 15.6). The steps are: compute `avail`; return immediately what is available; recall the shortfall from downstream loans where the entity is lender, itself possibly cascading; await downstream returns under a settlement-deadline

timeout; then execute the upstream return. On timeout the saga compensates by market buy-in with cost attribution per GMSLA 9.3 (IBP-328).

Individual recalls and returns map to CDM `QuantityChange/Termination`. The coordinated cascade has no CDM equivalent (gap); it is orchestration, not a single business event.

16.14 Locates

A *locate* is a pre-trade confirmation by a lender (or agent lender) to a prospective short seller that the requested securities are identified and deliverable. A locate is *not* a ledger move: no securities or collateral move, no coordinate changes, conservation is not engaged. Conservation engages only when the locate is converted into a borrow (the loan-initiation transaction of Section 16.8).

MENTAL MODEL. A locate is a cloakroom attendant glancing at the rack and saying “yes, there is a coat here you could borrow”—without lifting it off the rack, so the same coat may be promised to the next caller too, and the holder is free never to come for it. It moves no garment and conserves nothing. Only the put-on-hold kind actually takes the coat off the rack and sets it aside.

Regulatory basis. EU SSR Article 12(1)(c): a person may short sell only where a third party has confirmed the share is located and measures taken give a reasonable expectation of settlement (ESMA 70-448-10 proposes reinforcing the locate provider’s commitment). Implementing Regulation (EU) No 827/2012 Article 6 fixes four confirmation types: standard, same-day, easy-to-borrow, and put-on-hold. US Reg SHO Rule 203(b)(1): before a short sale the broker-dealer must borrow, arrange to borrow, or have reasonable grounds to believe the security can be borrowed for delivery when due; easy-to-borrow reliance rests on Rule 203(b)(1)(ii).

Lifecycle. The stages are: request (u , Q , kind) $\rightarrow$ admission $\rightarrow$ a confirmed locate with `remaining_quantity` = Q and an expiry fixed at confirmation $\rightarrow$ drawdown by short-sale admissions (P14) $\rightarrow$ conversion to a borrow or expiry. Admission has two cases: for an in-ledger lender it is the capacity condition of the locate-capacity weld below; for an external lender it is attestation.

Modelling. A locate is registered as a time-limited unit (TTL) in the Unit Store, carrying lender, security, kind, expiry, and `remaining_quantity`. It has no position-vector entries. A locate is *active* at t iff it is confirmed, its remaining quantity is positive, and t precedes its expiry, with t the admitting transaction’s log timestamp. This predicate is the sole authority for activity: decidable from the committed log, exact under replay, immune to scheduler lag. The due-event scheduler’s terminal sweep at expiry is hygiene, not authority; no status read decides activity. The unit satisfies the EU SSR / ESMA 70-448-10 record-keeping obligation natively; a purely external trading-system locate does not.

Remark 16.12 (Why a unit and not a coordinate). *A coordinate holds an anonymous, conserved quantity and is written only when something legally moves. A locate fails this on every axis. It is plural: several stand against one security at once, each with its own lender, quantity, kind, and expiry—a relation, and a per-lender relation has no home in a scalar slot on the security’s shared status. It is identified: each confirmation carries its own record-keeping obligation and one-shot lifecycle. It is non-conserved: confirmation moves nothing, so there is no quantity for a coordinate to conserve. The remaining candidate, a reservation coordinate written free $\rightarrow$ reserved at confirmation, would record an encumbrance the law does not create: under SSR Article 12(1)(c) and Reg SHO Rule 203(b)(1) a locate confirms deliverability and blocks nothing—the lender may lawfully sell located stock the next instant. Freedom from over-location is therefore a condition at the moment of confirmation, not an invariant of the states that follow. A property of that shape is enforced at the admission door—the locate-capacity weld below—while identity, expiry, and audit belong to a unit. Only the put-on-hold confirmation binds the provider to hold the quantity, and there the bound is law, not policy: it enters*

the reserved deduction and, where binding, an obligation record (Section 15.6)—still no move. Should ESMA 70-448-10’s reinforcement be enacted, the locate’s commitment strengthens as a gate parameter and an obligation; the primitive’s type does not change.

Over-location prevention. A lender must not confirm more locates than inventory supports. The gate is one inequality, and the deduction lives in one place: `available_to_lend` (Section 16.11) already nets outstanding locates, and the locate under confirmation is not yet active, so nothing is subtracted twice:

```
def confirm_locate(lender, security, quantity, kind, t):
    # lender: in-ledger; t: the transaction’s log timestamp
    atl = available_to_lend(lender, security, t)
    return CONFIRMED if atl >= quantity else DECLINED
```

The chain reads: owned → minus on-loan → plus borrowed → minus reserved (active put-on-hold locates) → minus regulatory holds = available to lend. For a put-on-hold confirmation the bound is the provider’s legal undertaking; for the other three kinds it is firm policy, and the kind may lawfully over-locate.

The gate is total only over in-ledger lenders. An inbound locate from an external or agent lender is an attested boundary observation: admitted as a unit on its attestation, ungated, since capacity is the issuer’s obligation. Under SSR Article 12(1)(c) the confirming party is a third party, so the attested case is P14’s main case, not an edge.

Principle 16.13 (Locate-capacity weld). *Locate confirmation is a move-less transaction. `applyTx` admits a locate confirmation by an in-ledger lender ℓ for security u and quantity Q only if $\text{AvailableToLend}(\ell, u, t) \geq Q$, with t the transaction’s log timestamp. The condition is decidable from the committed log and the transaction alone, evaluated at the transaction’s position in the total order.*

Race-freedom is inherited from the total order, not argued case by case. Two confirmations are totally ordered, and the later one’s fold sees the earlier’s locate. Confirmation against conversion cannot tear: conversion’s loan moves, drawdown, and terminalisation commit as one transaction (Section 16.8). Confirmation against expiry is not a race: expiry is a timestamp predicate, not an event, so there is nothing to tear against. A per-(lender, security) workflow owner would not suffice: capacity is also written from outside any locate family—a sale writes own—and only the one admission door owns those writers too.

Invariant 16.14 (P26—Locate-Capacity Admission). *At every commit position p carrying a locate confirmation by an in-ledger lender ℓ in unit u ,*

$$\sum_{loc \in \text{activeHold}(\ell, u, p)} \text{remaining}(loc) \leq \max(0, \text{avail}(\ell, u) - \text{regulatory_hold}(\ell, u)),$$

both sides evaluated at p , `activeHold` the set of active put-on-hold locates. The property is step-indexed to confirmation positions and falsifiable by replaying the log alone.

Over-location freedom is a condition at the moment of confirmation, not an invariant of the states that follow: a lawful later operation—a sale of located stock—lowers the right-hand side without touching a locate. No artifact, in specification, test, or reconciliation, may restate P26 as a standing reachable-state invariant or a solvency identity over locates.

Executable oracle. P26 is log-shaped, like P4, P8, and P24: it quantifies over the commit positions of a run, not over single transactions. The oracle replays the command stream, re-evaluating the fold at every position that admits an in-ledger confirmation. The oracle splits as in P1/P2: a wrongful *decline* falls on the vacuous side, and completeness is discharged by the split metamorphic below. The check reads the committed run alone—no cached aggregate to vouch for itself, since `reserved` is a fold, never stored. The oracle and its kernel are verbatim

from the reference (Part G); *avail* is the six-coordinate projection of Section 16.18, and *t* the command’s log position (the *k*-th command commits at Timestamp *k*).

```
-- Activity is a TIMESTAMP PREDICATE and the sole authority: no status read
-- competes with it; the scheduler’s terminal sweep is hygiene, not authority.
locActive :: Timestamp -> Locate -> Bool
locActive t loc = locState loc == LocLive
                && locRemaining loc > mempty && t < locExpiry loc

-- reserved: a read-time fold over the ACTIVE PUT-ON-HOLD locates -- never
-- stored, so no counter drifts and the deduction lives in ONE place.
locReserved :: LocLedger -> LenderId -> UnitId -> Timestamp -> Qty
locReserved ll e u t = foldMap locRemaining
  [ loc | loc <- Map.elims (llLocs ll)
    , locOrigin loc == LocInLedger e, locSecurity loc == u
    , locKind loc == LocPutOnHold, locActive t loc ]

-- P26 Locate-capacity admission: at every ADMITTED in-ledger confirmation
-- position, reserved <= max(0, avail - regulatory_hold), both sides at that
-- position (the admitted put-on-hold locate already in the fold). Attested
-- external locates are OUTSIDE the quantification -- ungated by design.
p26 :: [LocCmd] -> Bool
p26 = go (1 :: Integer) locEmpty
  where
    go _ _ [] = True
    go k ll (c : cs) =
      let t = Timestamp k; (d, ll') = locStep t c ll
      ok = case (c, d) of
        (LConfirm e u _ _ _, LocConfirmed _) ->
          locReserved ll' e u t
            <= qmax mempty (avail (locPosOf ll' e u)
                                <> qneg (locHoldOf ll' e u))
        _ -> True
      in ok && go (k + 1) ll' cs
```

The confirmation gate is *locStep*’s *LConfirm* arm: it admits iff $\text{locAt1 } \ell \text{ } u \text{ } t \geq Q$, where *locAt1* subtracts *reserved* once and the locate under confirmation is not yet in the fold. The generator *genLocCmds* draws command streams over a small lender/security pool: confirmations of all four kinds with near expiries; attested external locates; draws and conversions against the deterministic mint sequence; and inventory writes, including sales that lower *own* from outside any locate family. This is exactly the interleaving that defeats a per-(lender, security) workflow owner. It is paired with *shrinkLocCmds* (prefixes first, then pointwise quantity shrinking).

P14, structural. A short sale is admissible only by referencing a live locate unit and decrementing its *remaining_quantity* in the same admission. Drawdown is monotone, terminal states (converted, expired) are absorbing, and a double-spend of a locate is a non-state under the unit’s serialised lifecycle. The capacity bound stays at the confirmation door (P26); the drawdown is structure, not a check.

Records. Locate has no CDM equivalent (gap, Section 16.17, which fixes the lineage carried by the conversion’s *ContractFormation*). The locate unit is retained in the Unit Store for the retention obligation.

Test obligations. Four checks, none stated as a standing invariant, each a runnable oracle in the reference (Part G):

- the split metamorphic (*locSplitOK*): one put-on-hold confirmation of *Q* is admitted iff two successive put-on-hold confirmations of *Q/2* are—the one check that kills a double-counted deduction;
- the conversion postcondition (*locConvertNeutralOK*): converting a put-on-hold locate

leaves AvailableToLend unchanged, since the loan write and the drawdown are one atomic command;

- the drawdown-monotonicity check (`locDrawMonotoneOK`), P14’s structural half;
- the P26 replay check (`p26`).

Byte-equality of the `CONFIRMED/DECLINED` stream under replay is discharged by the type: `locRun` is a total pure function of the command list, so a run-it-twice oracle would be vacuous. `regulatory_restrictions` and the kind parameter are recorded, as-of-pinned inputs.

16.15 Cross-Border Regulatory Treatment

A single loan can span jurisdictions, triggering SFTR and FINRA SLATE simultaneously. A loan from BNP Paribas (EU, SFTR) to Goldman Sachs (US, SLATE) carries both `sftr_reportable/sftr_uti` and `slate_reportable/slate_loan_id` in the unit state. Both workflows fire per lifecycle event, independently. The schemas are distinct. The identifiers are distinct: LEI for SFTR; LEI + FINRA Loan ID for SLATE. The deadlines are distinct: T+1 for SFTR; same-day before 8pm ET for SLATE.

The `legal_regime` is fixed at inception by the governing agreement:

Governing Agreement	Use	legal_regime
GMSLA 2010 (title transfer)	EU/UK bilateral	TITLE_TRANSFER
GMSLA 2018 (security interest)	EU/UK pledge	SECURITY_INTEREST
MSLA	US bilateral	US_15C3_3
PBA	US prime broker	US_15C3_3

Venue, not domicile. The locate regime is fixed by the trading venue: EU SSR Article 12(1) for EU venues, US Reg SHO Rule 203(b)(1) for US venues. A cross-border short may face both. Settlement discipline is fixed by the settlement venue: a loan settled through an EU CSD faces CSDR cash penalties; one settled through DTC faces Reg SHO Rule 204 (T+3) and FINRA Rule 4320. Each is independent of the counterparty’s domicile.

16.16 External Reconciliation

Four participants each see a slice of the same state; the table names the ledger object each slice reads.

Participant	View	Ledger Mapping
Beneficial owner	On-loan; collateral; fee income	w_L^{onloan} , $w_L^{\text{coll-recv}}$, fee accrual
Agent lender	Pool across BOs	Aggregate own
Borrower	Borrowed; collateral posted; fee expense	w_B^{borr} , $\text{avail}(B, \text{ccy})$
Custodian	Depot position	Nostro/depot wallet

Contract compare (IBP-153/155) runs daily at COB. Custodian reconciliation, substituting the avail projection, is $\vec{w}_L(u)[\text{own}] + \vec{w}_L(u)[\text{borr}] + \text{inflight}(L, u) \stackrel{?}{=} \text{depot}(u)$: owned plus borrowed plus in flight equals what the custodian holds. CSDR penalty attribution is tracked per instruction. Settlement-instruction generation branches by collateral type: non-cash bilateral yields two FOP instructions; cash rebate one FOP plus one CASH; cash pool one FOP.

16.17 CDM Alignment and Regulatory Reporting

Each SBL event maps to a CDM type and an SFTR action; three events have no CDM home.

SBL Event	CDM Type	SFTR Action
New loan	ContractFormation	NEWT
Rate change	TermsChange	MODI
Partial return	QuantityChange	MODI
Full return	Termination	ETRM
Recall	<i>No CDM equivalent (gap)</i>	—
Margin call	Transfer (collateral)	COLU
MTM	Observation + Valuation	VALU
Substitution	Transfer $\times 2$	COLU
Locate	<i>No CDM equivalent (gap)</i>	—
Rehypothecation	<i>No CDM equivalent (gap)</i>	COLU

CDM gaps: Recall, Locate, Rehypothecation. The locate lifecycle is off-model; the conversion’s **ContractFormation** carries the locate id as lineage (retention: Section 16.14). SFTR: both counterparties report, UTI per ESMA waterfall. FINRA SLATE: same-day reporting, 24 required fields. Tokenised securities posted as collateral occupy coll_post/coll_recv with the token unit, not the underlying (Section F.4). The haircut covers both the underlying’s market risk and the tokenisation platform’s operational risk. CDM has no native type for tokenised-collateral eligibility: a fourth gap.

16.18 SBL-Specific Invariants (P11–P20)

SBL adds ten invariants that narrow the valid states without relaxing any core invariant: every transaction satisfying P11–P20 also satisfies P1–P10. Worked correctness witnesses are in Appendix H (European GMSLA 2010) and Appendix I (US SEC 15c3-3); eight-scenario avail verification in Appendix G.

Invariant 16.15 (P11–P20). 11. **P11—Paired Legs.** *Every SBL settlement transaction carries both a securities and a collateral leg (exceptions: uncollateralised, cash pool, tri-party RQV).*

12. **P12—On-Loan Consistency.**

$$w_e^{\text{onloan}}(u) = \sum_{\substack{u_\ell: \text{lender}(u_\ell)=e \\ \text{isin}(u_\ell)=u, \text{stage}(u_\ell) \in S_{\text{live}}}} \text{quantity}(u_\ell).$$

13. **P13—Collateral Sufficiency.** *Held collateral covers loan value; three formulations—per-loan cash, cash pool, non-cash bilateral.*

14. **P14—Locate Drawdown.** *Every short-sale admission draws down a named, live locate: the admission references the locate unit and decrements its **remaining_quantity**; draw-down is monotone and terminal states are absorbing. Structural under the locate unit’s serialised lifecycle; the capacity bound at confirmation remains an admission condition (Principle 16.13).*

15. **P15—Partial Return Monotonicity.** *Loan quantity is monotonically non-increasing.*

16. **P16—SFTR Completeness.** *Each reportable event has exactly one SFTR report.*

17. **P17—Settlement State Monotonicity.** *Settlement states advance monotonically.*
18. **P18—Lender Ownership Invariance.** *For pure lenders under title transfer, excluding buy-in, $\vec{w}_e(u)[\text{own}]$ is constant across lending operations. By construction under Principle 16.1: initiation and return write the lender's onloan, never own; MTM, substitution, and dividend write collateral or cash coordinates only. Buy-in is excluded because its market purchase writes own.*
19. **P19—Rehypothecation Regime Compliance.** *Rehypothecation is validated against the legal regime.*
20. **P20—Available Inventory Identity.** *For all e and lendable u , $\text{avail}(e, u) = \vec{w}_e(u)[\text{own}] - \vec{w}_e(u)[\text{onloan}] + \vec{w}_e(u)[\text{borr}]$. Under the six-coordinate model this is a definition, not a cross-check: avail is computed on read from three stored coordinates and cannot be violated because it is never stored.*

Key Invariants and Consequences

Obligations and the six-coordinate position vector generalise the scalar core without relaxing any of P1–P10: every transaction satisfying P11–P23 also satisfies P1–P10. Each guarantee carries its canonical number in §17.

- **Conservation generalises (P1).** The scalar conservation law extends unchanged to the six-coordinate vector: each move writes exactly one coordinate of one unit across the two entities it connects (Single-Coordinate Move Principle), so the signed sum over all coordinate slots is preserved. (§16, Proposition 16.6.)
- **Available-inventory identity (P20).** $\text{avail}(e, u) = \vec{w}_e(u)[\text{own}] - \vec{w}_e(u)[\text{onloan}] + \vec{w}_e(u)[\text{borr}]$, computed on read from three stored coordinates and never stored, hence not violable. (§16.18, Invariant 16.15.)
- **SBL narrows without relaxing (P11–P19).** Paired legs (P11), on-loan consistency (P12), collateral sufficiency (P13), locate drawdown (P14), partial-return monotonicity (P15), and lender ownership invariance under title transfer (P18, by construction—lending writes *onloan*, never the lender’s *own*) narrow the valid states. (§16.18.)
- **Locate-capacity admission (P26).** At every locate-confirmation commit position, the lender’s active put-on-hold reservations are bounded by available inventory net of regulatory holds. Step-indexed and log-checkable; a later lawful sale may falsify the inequality, so it is never restated as a standing state invariant. (§16.14, Invariant 16.14.)
- **Obligation liveness (P21).** For every obligation o with deadline t_d : $\forall t > t_d$, $o.\text{state} \in \{\text{DISCHARGED}, \text{COMPENSATED}, \text{DEFAULTED}\}$; none persists in *PENDING* or *ATTEMPTED* past its deadline. (§15.6.6, Invariant 15.5.)
- **Obligation conservation (P22).** Every discharging and every compensating move set satisfies $Q(u) = 0$; the mechanism does not bypass the executor. Follows from P1. (§15.6.6, Invariant 15.6.)
- **Obligation idempotency (P23).** Discharging the same obligation twice by $o.\text{id}$ has no incremental effect; it composes with P5 and P6. (§15.6.6, Invariant 15.7.)

Part VI

Formal Properties, Limitations, and Context

17 Invariants, Conservation Laws, and Property-Based Testing

This section states the framework’s correctness claims once, as precise testable invariants, under the canonical numbering P1–P31, with three further conformance-tier properties P32–P34 stated at their homes (Section 17.7). The conditions C1–C14 and the basis-layer invariants of Section 8 are indexed here against the guarantees they discharge. The invariants serve two roles at once. They are the design specification: the system is correct exactly when all of them hold. They are the test oracle: property-based testing checks every generated input against them. Executable pre/postcondition forms are given in Appendix B; this section fixes the prose statements and the cross-links.

17.1 Conservation as Central Invariant and Universal Oracle

Conservation states that nothing is created or destroyed inside the ledger: every quantity that leaves one wallet arrives in another, so across all wallets the holdings of each unit always cancel to zero. This is the one property every generated test checks.

The conservation law $Q(u) = \sum_w w_t(u) = 0$, for every unit u and time t , is the central invariant. It is an algebraic consequence of move semantics ($\text{src} \dashv=\ q$; $\text{dst} \dashv=\ q$) and holds for every transaction by construction (Section 2). The conserved-charge analogy is exact: the charge is the total quantity of each unit.

Conservation governs *quantities per unit*, not values (the FX witness is given in Section 5). Value changes from price movements, including FX, are carried by the valuation projection $V_t = \sum_u w_t(u) \cdot P_t(u)$ (Section 5). Valuation is a linear functional on the state. Dual valuation applies two price vectors to one state. PnL is the functional’s difference at two times. All three derive from conservation and the additive structure of moves.

Conservation is therefore a universal oracle. For any transaction sequence, checking $Q(u) = 0$ is constant-time after the sum, and it detects any violation of the system’s fundamental guarantee. A conservation failure is an implementation bug, never a financial event.

17.2 The Core Invariants P1–P10

The ten core invariants are the acceptance criteria for any implementation. Each is a universally quantified sentence about the ledger—over all transactions, all times, or all inputs of the named kind—so a single counterexample falsifies an implementation regardless of its other outputs. Appendix B gives each as an executable oracle; the prose statement is fixed here.

- P1 Conservation.** $Q(u) = \sum_w w_t(u) = 0$ for all units u , all times t . (Stated as System Closure in Section 2, Theorem 2.7.)
- P2 Atomic commitment.** A transaction applies completely or not at all; no partial state is observable.
- P3 Referential integrity.** Every move references existing wallets and an existing unit.
- P4 Log monotonicity.** The event stream is append-only; no entry is retroactively rewritten. Hash chaining—each entry carries the hash of its predecessor—provides tamper-evidence: it detects tampering but does not prevent it. Prevention requires deployment controls (WORM storage, consensus, or hardware security modules). The design requires tamper-evidence as the minimum; stronger immutability depends on the environment.
- P5 Transaction idempotency.** Applying the same transaction twice (by `tx_id`) has no incremental effect.
- P6 Lifecycle idempotency.** Applying the same lifecycle event to the same unit state produces no additional moves. The basis coordinate obeys the same algebra: `SetBasis` is an absolute replacement write, so re-applying it is the identity (§8).
- P7 Virtual/real ledger isolation.** No move has one endpoint in a virtual ledger and the other in the real ledger.
- P8 Snapshot consistency.** `clone_at(t)` reconstructs the complete view—balances *and* unit state, the basis coordinate `usBasis` included—identical to the ledger state at time t , under retro-effective boundary insertions as under any other event (§8.10).
- P9 Lifecycle purity.** For fixed inputs, lifecycle functions produce identical outputs and no effect beyond the return.
- P10 PnL path-independence.** $\text{PnL} = V_{t_1} - V_{t_0}$, independent of the intermediate transactions (Section 5, Theorem 5.3).

Two further oracles exercised by the same generators are global rather than numbered among the ten. *Totality*: every valid input returns a success or a well-typed error, never an unhandled exception. *Valid transitions only*: every product/state/event triple absent from the product's transition table is rejected. Both are stated in executable form in Appendix B.

17.3 SBL Invariants P11–P20

Securities borrowing and lending adds ten invariants. They narrow the valid-state set without relaxing any core invariant: every transaction satisfying P11–P20 also satisfies P1–P10. The full statements are in Section 16.18, together with the position vector and Single-Coordinate Move Principle they rest on. The index follows; entries marked *by construction* are not tested but structural—the write discipline makes a violation unbuildable.

P11 Paired legs

Every SBL settlement transaction carries both a securities and a collateral leg (exceptions: uncollateralised, cash pool, triparty RQV).

P12 On-loan consistency

$w_e^{\text{onloan}}(u)$ equals the summed quantity of that entity's live loans of u .

P13 Collateral sufficiency

Posted collateral covers exposure, by one of three formulations: per-loan cash, cash pool, non-cash bilateral.

P14 Locate drawdown

Every short-sale admission draws down a named, live locate: it references the locate unit and decrements its `remaining_quantity` in the same admission; drawdown is monotone and terminal states are absorbing. *By construction*: structural under the locate unit's serialised lifecycle. The capacity bound at confirmation is P26.

P15 Partial-return monotonicity

Loan quantity is monotonically non-increasing.

P16 SFTR completeness

Each reportable SBL event produces exactly one SFTR report.

P17 Settlement-state monotonicity

Settlement states advance monotonically.

P18 Lender ownership invariance

A pure lender's own coordinate is constant across lending operations under title transfer (buy-in excluded). *By construction*: under the Single-Coordinate Move Principle, loan initiation and return write onloan, never the lender's own.

P19 Rehypotheication regime compliance

Rehypotheication is validated against the legal regime.

P20 Available-inventory identity

$\text{avail}(e, u) = \vec{w}_e(u)[\text{own}] - \vec{w}_e(u)[\text{onloan}] + \vec{w}_e(u)[\text{borr}]$. *By construction*: a definition computed on read from three stored coordinates, never stored, hence not violable (Invariant 16.15).

17.4 Obligation Liveness Invariants P21–P23

Recalls, collateral substitutions, margin deliveries, and close-out deadlines are event-triggered obligations. Obligations are first-class objects governed by three invariants, stated in full in Section 15.6.6. The three apply the ledger’s core algebra to deadlines: an obligation must resolve by its deadline (P21), resolve without creating or destroying quantity (P22), and gain nothing from a repeated discharge (P23).

P21 Obligation liveness

For every obligation o with deadline t_d : $\forall t > t_d, o.state \in \{\text{DISCHARGED}, \text{COMPENSATED}, \text{DEFAULTED}\}$. No obligation persists in `PENDING` or `ATTEMPTED` past its deadline (Invariant 15.5).

P22 Obligation conservation

Every discharging and every compensating move set satisfies $Q(u) = 0$; the mechanism does not bypass the executor. Follows from P1 (Invariant 15.6).

P23 Obligation idempotency

Discharging the same obligation twice (by $o.id$) has no incremental effect; the discharge predicate is checked before any move is emitted. Composes with P5 and P6 (Invariant 15.7).

17.5 Basis Invariants P24 and P25

The basis layer contributes two testable invariants: the property form of the tip weld (Principle 8.4), and door soundness at the market-data boundary (§8.5). In plain terms: the two ways of computing the basis coordinate never disagree (P24), and nothing crosses the market-data door without a registered unit and a committed basis point (P25).

P24 Basis tip agreement

At every accepted prefix of the log, the booking-order status fold and the effective-order chain projection agree on `usBasis` for every registered unit (executable form in Appendix B).

P25 Ingest-door soundness

For every observation offered to the ingest door: on acceptance, every stamped unit is registered and every stamped basis id is a committed point of that unit’s chain; on refusal, nothing is admitted and the payload is retained — the refusal is the quarantine (executable form, with the boundary property catalogue P-DET through P-LAG, in §8.5).

17.6 Locate Invariant P26

The locate layer contributes one testable property. Unlike P1–P20, it is neither structural nor discharged by a condition: over-location freedom is a *precondition* on admission, not an *invariant* of every reachable state. A lawful later sale of located stock lowers the capacity bound without touching a locate, so the property is enforced at the confirmation door (the locate-capacity weld, Principle 16.13) and witnessed at each confirmation position, not over reachable states. No artifact may restate it as a standing reachable-state invariant. Its full statement is in Section 16.14; the runnable oracle and its generator are `p26 (reference/Ledger.hs, Part G)`.

P26 Locate-capacity admission

At every commit position that admits a locate confirmation by an in-ledger lender ℓ in unit u , the summed remaining quantity of ℓ ’s active put-on-hold locates of u is at most

$\max(0, \text{avail}(\ell, u) - \text{regulatory_hold}(\ell, u))$, both sides at that position. Falsifiable by replaying the log alone (Invariant 16.14). The companion short-sale drawdown of P14 is *structural*: a short sale references a live locate and decrements its remaining quantity monotonically in the same admission; a converted or expired locate is absorbing. A double-spend is therefore a non-state.

17.7 Producer and Vocabulary Invariants P27–P31

The processor contract (§15.2), the declared-operator discipline, the adjustment schedule, and the datum-kind registry (§8) contribute five testable invariants. Each runs in the property suite; the oracles are in `reference/Ledger.hs`, Part L (P27, P28, P30) and with their gates (P29, P31).

P27 Producer agreement

For every processor-originated transaction on a committed log, an independent recomputation from the log prefix alone — declared terms, adjustment schedule, dimensions, datum-kind registry, stamped observations — yields the identical transaction in canonical form (`canonicalTx`; equality on `Transaction`, zero diff). Oracle `recomputeOK`, higher-order over the processor; the terms leg is the named oracle `propTermsRecompute`; the corporate-action instance joins follow-ons on `txBoundary` $\cup$ `txCause`.

P28 Cause committed

Every committed transaction carrying `txCause` names a boundary id committed strictly before it; a dangling or self-referential cause is refused (`CauseNotCommitted`), never admitted. Oracle `pCauseCommitted`.

P29 Schedule totality

The witness of C14: registration and every terms append are admitted exactly when the written version’s adjustment schedule yields one well-formed entry per (corporate-action class, field), the classes drawn from the closed `CAClass` (§8.13); the refusal channel is `ScheduleIncomplete` and only that. Oracle `propScheduleTotalGate`.

P30 Dimension invariance

Under a boundary declaring $(f \neq 0, c)$, the per-dimension action (Theorem 8.13) preserves the value identity for every declared field and every registered kind’s components; the composite-kind oracle `dkDivSplitOK` checks the dividend-list identity and fails the misdeclared variant. Oracle `propDimInvariance`.

P31 Kind totality

For every observation offered to the ingest door: on acceptance its claimed kind is in the pinned registry; on refusal of an unregistered kind nothing is admitted and the payload is retained (`UnregisteredKind`) — the P25 shape at the vocabulary plane. Oracle `pKindTotal`.

Three further properties are conformance-tier: their executable forms live where `transport` lives, since the reference carries no operator arithmetic. P32 (confirmation gate: a pending boundary blocks consumption, and an empty declaration yields refusal, never identity — §8.13); P33 (elective aggregate invariance, against the elective case of Principle 8.6); P34 (basket joint consumption: the spin-off pair is derivable while any single-component projection is refused until fresh per-component stamps — §8.12). The transport-leg obligations of the composition law attach to Proposition 8.9 and ride P30’s suite run.

17.8 The Conditions C1–C14

The state model imposes the conditions C1–C14. Each is defined once at the site that forces it: C1–C12 by the three-home construction (Section 4), C13 and C14 by the data plane’s basis edge (Section 8). A condition is not a test but a constraint on what can be written down: each renders an illegal state unrepresentable. The index below cross-links them; the discharge map (Section 17.9) gives the mechanism by which each acts.

Cond.	Content (defined in Section 4, except C7/C10 in Section 3 and C13/C14 in Section 8)
C1	Position accessor returns <code>Option</code> ; monotone carrier keeps the key set stable.
C2	Handler-level conservation per event class, vacuous base case included.
C3	Atomic Transaction across all three maps; partial application unrepresentable.
C4	Capability-scoped reads; cross-overlay reads forbidden; exports via <i>UnitStatus</i> .
C5	<i>UnitStatus</i> registration-total; product-declared defaults at registration.
C6	<i>ProductTerms</i> versioned, append-only; no in-place mutation.
C7	<i>ProductTerms</i> registration-total; first write at registration.
C8	Two-track amendment by a total fungibility predicate (Preserving / Breaking).
C9	Zero-holder handlers discharge $\sum = 0$ vacuously.
C10	Re-registration is a hard error, never a silent reset.
C11	Each <i>PositionState</i> field has a canonical writer set; an outside writer is a type error at authorship.
C12	All per-(wallet, mandate/strategy) economic state lives in <i>PositionState</i> ; the <i>W</i> -sector is empty by schema.
C13	Basis edge: every consumed datum names its joint basis; consumption requires pointwise agreement with the total assignment β_t or a ledger-authored arrow—the data-plane sibling of P3.
C14	Adjustment-schedule totality: registration and every terms append admitted only if the schedule yields exactly one well-formed entry per (corporate-action class, field); witnessed by P29 (C14, §8.3).

C13 rests on three basis-layer correctness claims, stated once in Section 8 and indexed here:

- *single-basis consumption* (Invariant 8.17): over each stamp’s whole domain, every balance and every datum a valuation consumes either agrees pointwise with one total assignment β_t or crosses by a ledger-authored arrow;
- the *tip weld* (Principle 8.4): `applyTx` admits `SetBasis b` only as the post-insertion effective-order tip, so the booking-order status fold and the effective-order chain projection agree on `usBasis` at every log prefix;
- the *invariance weld* (Principle 8.6): a basis-changing corporate action is admitted only if its moves and declaration jointly satisfy the invariance identity of Theorem 8.11.

17.9 Which Condition Discharges Which Invariant

The core of this section is the map from conditions to the guarantees they discharge, and the mechanism in each case. The guarantees carry the P1–P31 names this section fixes; Section 4 presents the same map under its own local labels, and the names fixed here are canonical over them. P26 is not in the table below: it is an admission-gated precondition, not a guarantee discharged by a structural condition. P27 is likewise not below: it is a recomputation check over committed logs, not a structural discharge.

“Unrepresentable” is used precisely. An *abstract type* whose only constructor is a checked smart constructor makes an illegal value impossible to build, so it never reaches the operation that would act on it. A *non-empty list type* makes a versionless state untypable. A *GADT write-tag* is a per-field marker checked at compile time, then erased once writes share a stored row. A *value-level check* is a runtime guard returning an error, not a type fact. Conservation, by contrast, holds by construction: each **Move** edge names both parties, so the signed per-unit sum of any move list is the group identity **mempty**. That is a structural fact of the **Transaction** type, not a runtime check.

Guarantee	Cond.	Mechanism
P1 Conservation	C2, C3	A Transaction carries its flow as Move edges; each edge contributes $\text{qneg } q \triangleleft q = \text{mempty}$, so the signed per-unit sum of any move list is the group identity by construction—the empty list (registration) included. No unconserved Transaction exists to reject, hence no validate gate. C3 makes one Transaction span all three maps, so partial application is unrepresentable.
P8 Snapshot / replay	C1	Replay is the foldM of the event stream, a catamorphism: replaying the events always rebuilds the exact value. It is therefore a homomorphism $\text{replay}(xs \triangleleft ys) = \text{replay } xs \gg \text{replay } ys$; the Kleisli $\gg$ is glossed at mechanism note (4). The monotone carrier (C1) keeps the key set stable across any checkpoint cut, so checkpoint-independence follows from the law, not a test.
P6 Lifecycle idempotency	C5	The lifecycle stage lives in $\text{UnitStatus}[u]$, u -keyed and written by replacement, so re-applying a stage write is the identity (EXPIRED over EXPIRED is EXPIRED)—structural at one key. Non-conserved <i>PositionState</i> fields share the algebra (hwm by max, entry_nav write-once); conserved fields draw replay-safety from P1.
Terms immutability (underpins P4)	C6, C7	<i>ProductTerms</i> is an append-only $\text{NonEmptyList}[\text{TermsVersion}]$, so a versionless unit is untypable; registration totality fixes the first version at registration.

Guarantee	Cond.	Mechanism
Identity immutability (underpins P4)	C10, C8	Re-registration is a hard error; a breaking amendment allocates a new unit u_{new} and never rewrites u_{old} .
P7 Isolation / read scoping	C4	Isolation holds by the per-move locality predicate (both endpoints in one ledger), not by conservation; cross-overlay reads are forbidden at the capability layer wrapping the accessors, not in the data reference.
Handler-field canon (underpins P9)	C11	The canonical writer set per field is a FieldWrite GADT: a write by a field-writer outside the set is a type error at authorship, erased once writes share a delta row.
Single-basis consumption (Invariant 8.17)	C13	Every consumed datum carries a joint basis stamp; the snapshot seam admits it only on pointwise agreement with β_t over the stamp's whole domain, so mis-based value is a typed refusal, never a Cash (§8.9).
Basis tip agreement (P24; underpins P6, P8)	C13	The tip weld: applyTx admits SetBasis b only as the post-insertion effective-order tip, so a retro-effective boundary reasserts the standing tip and the coordinate never regresses (Principle 8.4).
Value invariance at boundaries	C13	The invariance weld: a basis-changing corporate action is admitted only if its moves and declaration satisfy the invariance identity, a value-level gate at the single door (Theorem 8.11).

The lifecycle-idempotency and snapshot guarantees rest on the status discipline. **UnitStatus** is a shared per-unit read cache of the event log, rebuilt exactly by replay: it holds the lifecycle stage, the basis coordinate **usBasis**, the last settle mark, and the successor pointer (§4.3).

Each mechanism above is a type or a value-level gate carrying a theorem. The Haskell rendering makes the correspondence explicit.

Types as theorems. The five anchors below render the discharge mechanisms as Haskell, one type or function per guarantee. They are verbatim excerpts of the reference (**reference/Ledger.hs**, the three-home model of Section 4; see Appendix B). Deriving clauses and the record fields marked ‘...’ are elided. Quantities are exact **Integer** minor units forming an additive abelian group (**Qty**, $\langle \rangle =$ addition, **mempty** = zero). Floating point is excluded because conservation and replay are arithmetic facts it would forfeit.

(1) **P1 conservation — Move edges / unitDelta (C2).** Conservation is a property of the **Transaction** type, not a checked side-condition. Each **Move** names both parties, so its per-unit contribution is $\text{qneg } q \diamond q = \text{mempty}$. Summing those contributions over any move list is a *monoid homomorphism* from the free monoid of moves to **Qty**. Per unit, the column sum over wallets is taken edge by edge. The composite is identically **mempty** by the monoid laws. The empty list (a move-less registration or amendment) is **mempty**, so a zero-holder event (C9) conserves with no special case. Summing edges, never dividing by a holder count, excludes

the divide-by-zero-holders bug class. There is no unconserved `Transaction` to reject, hence no validate gate.

```
data Move = Move
  { mFrom :: !WalletId, mTo :: !WalletId, mUnit :: !UnitId
  , mQty   :: !PosQty, ... } -- positive magnitude (sole door mkPosQty); from ==
    q, to += q

-- Conservation witness: the signed per-unit sum over the edges. By construction:
unitDelta :: UnitId -> Transaction -> Qty
unitDelta u tx =
  foldMap (\m -> if mUnit m == u then let n = unPosQty (mQty m) in qneg n <> n
    else mempty) (txMoves tx)
-- law: unitDelta u tau == mempty for every unit u and transaction tau

netDelta :: Transaction -> Qty
netDelta tx = foldMap (\m -> let n = unPosQty (mQty m) in qneg n <> n) (txMoves tx)
-- law: netDelta tau == mempty for every tau (the empty/registration case too)
```

A one-edge transfer shows there is nothing to reject: a `Move` names both endpoints, so a half-edge is unrepresentable. The fragment is illustrative usage, not reference text:

```
-- Just m = move a b u (Qty 1000) t s -- the checked door builds the PosQty edge
-- tx = Transaction u [m] mempty [] Nothing Nothing Nothing
netDelta tx == mempty -- by construction; a one-sided delta cannot be
  built
```

(2) **P4 terms immutability — NonEmpty (C6/C7).** `ProductTerms` wraps a *non-empty* list, so a “registered but versionless” unit is untypable, and `currentTerms` is total without a `Maybe`. No setter exists. The only growth operations are `register` (singleton) and `appendVersion` (append, never rewrite).

```
newtype ProductTerms = ProductTerms (NonEmpty TermsVersion) -- abstract

currentTerms :: ProductTerms -> TermsVersion -- total: no Maybe
currentTerms (ProductTerms vs) = NE.last vs

appendVersion :: TermsVersion -> ProductTerms -> ProductTerms -- append-only (C6)
appendVersion tv (ProductTerms vs) = ProductTerms (vs <> (tv :| []))
```

(3) **P8 monotone carrier — the Option accessor (C1).** `position` returns `Maybe`. `Nothing` is *never held*; `Just zeroP` is *held and currently flat*; the spec reads these as distinct states. The carrier is monotone by absence: `Ledger` is abstract, `applyTx` only inserts or updates a row, and no row-deleter is exported. The key set therefore never shrinks, and a checkpoint cut cannot lose a row.

```
position :: Ledger -> WalletId -> UnitId -> Maybe PositionState
position l w u = Map.lookup (w, u) (ledgerPS l)
-- Nothing = wallet has NEVER held this unit
-- Just zeroP = held once, currently flat (the two cannot be collapsed)
```

(4) **P8 replay homomorphism — Kleisli $\Rightarrow$.** `Replay` is an error-stopping fold. Determinism and checkpoint-independence are the homomorphism law `replay(xs <> ys) = replay xs  $\gg$  replay ys`, not a test: replaying two batches in sequence equals replaying them together. With the stable key set from (3), the placement of a cut cannot change the result.

```

-- (>=) runs the first Either-returning step, feeds its result to the next,
-- and stops at the first error.
replay :: [Transaction] -> Ledger -> Either LedgerError Ledger
replay ds l0 = foldM (\l d -> applyTx d l) l0 ds

```

(5) Handler-field canon — FieldWrite GADT (C11, underpins P9). The GADT constructors *are* the field-to-writer table: a write by a handler outside a field’s writer set is a type error at authorship. A handler is typed by the `Handler` it speaks for, so the phantom type index constrains at a live call site, not merely in an alias. Once authored, the index is erased to `SomeWrite`: the check has already happened.

```

data Handler = Settle | Trade | Transfer | FeeCrystallise | Subscribe

-- The balance field is NOT in this table: its sole canonical writer is the Move
-- edge (handler 'Transfer), applied in applyTx -- so a balance change without a
-- conserving move is unrepresentable. The rest are non-balance per-position rows.
data FieldWrite (h :: Handler) where
  WAc      :: Qty -> FieldWrite 'Settle
  WAcTrade :: Qty -> FieldWrite 'Trade
  WHwm     :: Qty -> FieldWrite 'FeeCrystallise
  WEntryNav :: Qty -> FieldWrite 'Subscribe

settleHandler :: [(WalletId, Qty)] -> Map WalletId [FieldWrite 'Settle]
settleHandler legs = Map.fromList [ (w, [WAc q]) | (w, q) <- legs ]
-- A settle body bumping hwm would emit WHwm :: FieldWrite 'FeeCrystallise and
-- fail to typecheck against the declared 'Settle output -- C11 IS a type error.

```

17.10 Invariants as Specification and Oracle

The invariants define correctness independently of any implementation. The system is correct if and only if all of them hold, so any property-based test checking an invariant thereby tests the specification. An example-based test verifies one input/output pair; an invariant-based test verifies a property over a whole input class.

A bond coupon payment illustrates the dual role. P1 requires $Q(\text{USD}) = 0$ before and after the coupon move. P6 requires that paying the same coupon twice produces no additional moves. Applied to randomly generated bonds with random coupon schedules, these two checks exercise scenarios no hand-written suite would enumerate. The equation $Q(u) = 0$ governs both the ledger and its test suite.

17.11 Property-Based Testing over the CDM Generator Universe

Each property has a *generator* (the random-input space) and a *postcondition* (the invariant). The framework draws inputs, applies the operation, and checks the postcondition. A failure yields a minimal counterexample. The executable pre/postcondition for each of P1–P10 and for P24, together with the totality and valid-transition oracles, is given once in Appendix B. P25’s, with its generators, is stated with the market-data contract (§8.5).

The generators draw from the CDM enumerations: product types, event types, party roles, date rules (Section 14). These enumerations are closed and finite, so the state space of any product’s lifecycle is bounded and coverage is a checklist rather than a probability. Where a random fuzzer might miss a rare combination after millions of runs, the enum-driven generator covers every combination in its first pass. The strategy is compositional: draw a CDM product, draw a sequence of valid lifecycle events, map through F to ledger transactions, and check every invariant.

Completeness by enumeration. The CDM `EventIntentEnum` is the closed set of lifecycle intents. For an equity option the applicable intents map one-to-one to unit-state transitions:

EventIntentEnum	Unit-state transition	Status
OPEN	$\emptyset \rightarrow \text{ACTIVE}$	Covered
EXERCISE	$\text{ACTIVE} \rightarrow \text{EXERCISED}$	Covered
EXPIRE	$\text{ACTIVE} \rightarrow \text{EXPIRED}$	Covered
ASSIGN	$\text{ACTIVE} \rightarrow \text{ASSIGNED}$	Covered
NOVATION	$\text{ACTIVE} \rightarrow \text{NOVATED}$	Covered
PARTIAL_TERMINATION	—	Not applicable (option is all-or-nothing)

Because the enum is closed, the generator drawing intents uniformly from it achieves full intent coverage by construction. Incompleteness is surfaced, not hidden: a new enum value with no transition (say `RESTRUCTURING`) is produced by the generator, finds no matching rule, and is flagged by the valid-transitions oracle. A missing rejection is as much a bug as a missing acceptance. The `OptionTypeEnum` `{CALL, PUT, PAYER, RECEIVER, STRADDLE}` bounds payout directionality the same way. Every variant must have payout logic; a missing case is a compile-time or test-time error, not a runtime surprise.

17.12 Specification-First Development

The properties are written before the implementation. Typed state schemas make illegal field combinations unrepresentable. The core properties are executable specifications. The simplest lifecycle case anchors the simulation harness. The full product universe is reached using CDM enumerations as the generator source. Correctness is thereby defined independently of language, storage backend, or deployment, and the same properties hold from single-process prototype to distributed system. The approach is falsifiable: a property satisfiable by no reasonable implementation reveals a design contradiction before implementation begins.

18 Regulatory Obligations and the Direction of Travel

Post-trade regulatory reporting is the dominant compliance cost for derivatives market participants. The same economic event must be reported to multiple trade repositories, each in a jurisdiction-specific format: EMIR Refit (EU 2024/1856, 203 reportable fields), MiFIR (RTS 22, 65 fields), SFTR (Regulation EU 2015/2365, 155 fields), and CFTC Parts 43 and 45 (rewritten 2022–2024). Dual-sided reporting under EMIR and SFTR makes the reconciliation burden structural. Both counterparties report every lifecycle event independently, and the two reports must match. Mismatches trigger regulatory queries. Firms therefore maintain parallel pipelines per regime, each with bespoke field mappings, validation, and exception management.

CDM and Digital Regulatory Reporting. The ISDA Common Domain Model, governed by FINOS, supplies a single machine-readable vocabulary for trade description. ISDA’s Digital Regulatory Reporting (DRR) encodes each regime’s reporting rules as executable code over the CDM. The interpretation of a regulation thus becomes a shared, testable artefact rather than a firm-specific implementation. DRR is in production, not sandbox. Four institutions file DRR-generated reports with live trade repositories, further institutions are in proof of concept, and reported acknowledgement rates under EMIR Refit exceed 98%.⁵ The convergence point

⁵ISDA, “Digital Regulatory Reporting: 2025 Year in Review,” January 2026. Figures are pinned to that publication and not restated as current.

is a unified post-trade stack: a trade described once in CDM, reports generated mechanically by DRR, settlement executed by CDM-native contracts.

Framework alignment. The architecture derives this alignment from its own primitives, not from a standard’s mandate. Transactions are expressed in CDM (Section 14). Instruments are deterministic move-generating contracts (Section 6). The canonical internal record is the immutable event log (Section 2). The economic content of a DRR report is the lifecycle event, its quantities, and its timing. That content is already present in the move stream and is generated from it by projection. No separate trade-data pipeline is needed.

Two boundaries limit the claim. First, a complete report (e.g. EMIR Refit’s 203 fields) also requires reference-data enrichment: UTI generation, LEI validation, counterparty classification, collateral detail. This enrichment does not reside in the move stream. It is supplied by the reference-data authority at the ledger boundary (§1.6), not by this framework. Second, the framework supplies technical substrate, not organisational compliance.

Conformance checkpoints. The event log satisfies the audit-trail requirements of BCBS 239 Principle 6 (accuracy and integrity) and Principle 11 (distribution): every report projects from one immutable record. The EU Digital Operational Resilience Act (DORA, Regulation EU 2022/2554) sets testing and traceability requirements in Article 8 (ICT risk management) and Chapter IV (digital operational resilience testing). Their technical substrate is the property-based testing strategy (Section 17), the deterministic lifecycle functions, and the immutable audit trail. Full DORA conformance requires operational and organisational measures outside the scope of a design specification.

Mandate issuance reporting. Representing a managed-account mandate as a unit creates an SFTR/EMIR reporting surface for mandate issuance. The framework supplies the mechanism (flag F5). The `reportable` predicate on `ProductTerms` gates two things: whether a unit’s issuance requires a UTI/LEI pair, and which units project into the reporting layer. The managed-account treatment (Section 10) carries the construction. F5 remains flagged pending external regulatory sign-off (Section 21).

19 Scope and Limitations

The framework’s guarantees hold for activity recorded within the ledger boundary of §1.6, which states the inside and outside lists once. This section establishes no new result. It bounds the claims of the preceding sections: what the framework does not do, and what standing its theorems have.

19.1 What the Framework Does Not Do

The following limitations are inherent in the design.

1. **Does not guarantee executable prices or replace pricing models.** P_t is an external input. The framework is agnostic to pricing methodology. It requires only that prices be supplied. Under stress, P_t may be stale, unavailable, or detached from achievable liquidation values.
2. **Does not replace legal documentation or determine legal ownership.** Legal ownership depends on custodial arrangements, nominee structures, registrar entries, and governing law, all outside the boundary.
3. **Does not guarantee settlement finality.** The ledger provides ledger-level atomicity: all moves in a transaction commit or none do. Real-world finality is a property of the external settlement system and its governing law (e.g. the EU Settlement Finality Directive).
4. **Does not determine accounting policy.** IFRS 9 classification, hedge-accounting designation under IAS 39/IFRS 9, expected-credit-loss provisioning under IFRS 9, and fair-value-hierarchy classification under IFRS 13 require judgment beyond the ledger’s data. The framework supplies the position and valuation data that support these decisions. It does not make them.
5. **Does not eliminate market, credit, or operational risk from external systems.** The framework eliminates classes of internal reconciliation failure. It does not eliminate price risk, counterparty-default risk, liquidity risk, or operational failure in custodians, payment systems, or exchanges.
6. **Does not replace model governance or validation.** Pricing and risk models and their independent validation (CRR Article 368, SR 11-7) are external. The framework records model outputs; it does not validate the models.
7. **Does not eliminate external reconciliation at the boundary.** Virtual wallet balances reconcile against custodian statements, counterparty confirmations, and CSD records. The framework makes this reconciliation narrowly scoped and detectable. It cannot eliminate it: external systems are independent actors.
8. **Does not replace regulatory submission infrastructure.** The framework supplies the economic content of regulatory reports. Formatting, submission to trade repositories, acknowledgement handling, and query response require infrastructure beyond the ledger.
9. **Does not guarantee reproducibility without dependency versioning.** Replay reproduces results identically only under identical versions of market-data snapshots, reference data, and pricing-model parameters. An implementation must version-pin these dependencies. The framework specifies the requirement, not the versioning infrastructure.

19.2 Theorems and Modelling Choices

Several results stated as theorems are consequences of the framework's modelling choices, not independent mathematical discoveries. The conservation law ($Q(u) = 0$) is an algebraic identity forced by the move semantics ($\text{src} \text{--} = q; \text{dst} \text{+} = q$). Path-independent valuation follows from defining portfolio value as a function of current state. State-sufficiency follows from making unit state carry all economically relevant information. Their formal value is that any implementation that violates one exhibits a bug, and that any extension that breaks an invariant must justify the violation. They serve as executable specifications for property-based testing.

19.3 Architecture and Operational Reality

The framework specifies a target architecture, not an operational system. The architecture reduces the structural complexity of post-trade processing. It does not remove the need for operational controls. Four-eyes trade approval, exception management, manual intervention for settlement failures, regulatory query response, and business-continuity planning remain necessary. The architecture makes these controls more effective by supplying better data and removing certain failure modes. It does not replace them.

20 Frequently Asked Questions

Each answer states its own content and points to the section that establishes the result.

Q1: Are move-less lifecycle events valid, and must the executor accept them?

Yes. A lifecycle event may produce a state change with an empty move list. The transaction is recorded in the move stream as a state-change entry carrying the full CDM event payload. The audit trail and time travel are preserved. Conservation holds trivially: the empty sum is zero for every unit in every wallet.

Such events are not optional records. A barrier knock-out on a product where the owner holds no position transitions the unit from **ACTIVE** to **TERMINATED** and emits no moves. Without the recorded state change, `clone_at(t)` for t after the knock-out cannot distinguish “terminated by barrier breach” from “active with zero holders”. The two states are economically distinct: the first carries zero contingent liability, the second a potentially large one. Under IFRS 9 the fair-value liability drops to zero on termination, not on payment. The state change records *why* the liability disappeared.

Further instances: out-of-the-money expiry whose position-closing move has zero economic effect, suspension and delisting events, and intraday futures trades that update unit state without cash moves (Section 9). The futures lifecycle already establishes state-only transactions. The principle applies to all product types, and the executor must not reject them.

Remark 20.1. *The state a lifecycle event changes is held in `UnitStatus`, a shared per-unit read cache of the event log (§4.3). “Move-less” therefore names a transaction that overwrites this cache without altering any wallet balance.*

Q2: Does the Ledger guarantee Delivery-versus-Payment atomicity?

Internally, yes, by construction (Section 13.5). Conservation alone does not deliver DvP: a single-leg transfer balances quantities across wallets and is conservation-preserving. The guarantee is the conjunction of two properties. **Transaction atomicity:** the executor commits all moves of a transaction together or not at all (Definition 2.5). **Contract correctness:** the trade contract emits both legs, cash and securities, in the same transaction (Section 6). External settlement atomicity is provided independently by the CSD (Section 13.5).

Q3: How does the framework settle physical delivery under lot-size constraints?

By the lot-split construction of Section 6. The construction delivers as many whole lots as the exercised quantity contains and cash-settles what remains. One atomic transaction delivers the whole-lot quantity $N_{\text{deliver}} = L \lfloor N/L \rfloor$ against strike cash and cash-settles the residual at intrinsic value. The construction is property-testable for conservation, lot compliance ($N_{\text{deliver}} \bmod L = 0$), and economic equivalence (total value transferred equals the payoff for any lot split).

Boundary cases follow from the formula. Exercise below one lot ($N < L$) delivers zero and cash-settles entirely. Exercise an exact multiple of L leaves zero residual and no cash adjustment. Odd-lot markets set $L = 1$, so the residual is always zero. DvP (Q2) applies at both Ledger and CSD level.

Q4: Why is securities lending inside the Ledger rather than in a companion system?

Because a companion system reintroduces the TOCTOU race that enables double-lending. With it come four further failures: unprovable cross-boundary conservation, invisible collateral

(breaking P13), a two-system exposure join, and a “reserved” bucket that discards load-bearing information. The full argument is in Section 16.6.

Q5: Why is `own` a stored coordinate while `avail` is a projection?

One coordinate records a physical fact; the other is arithmetic over recorded facts. `own` passes the physical-action test and is not derivable from the five operational coordinates. `avail` fails the test and is fixed by the identity $\text{avail} = \text{own} - \text{onloan} + \text{borr}$, so storing it would be a cache, not a fact (Section 16.2).

Q6: Is a scalar balance sufficient for cash, including cash collateral in securities lending?

Yes. A cash position is one number. For any cash unit, the six-coordinate position vector of Section 16.2 degenerates to $(q, 0, 0, 0, 0, 0)$: the scalar q (§16.3). Cash is not a lendable security: ownership and possession never diverge, so the lending-specific coordinates (`onloan`, `borr`, `coll_post`, `coll_rehyp`) are never populated for cash. Only `own` and `coll_rcv` are engaged, and `coll_rcv` is itself scalar per cash unit. Receipt, reinvestment, and return of cash collateral are ordinary atomic moves over these two coordinates; the return obligation is held in the SBL loan unit state (Sections 16.9 and 16.12; Appendix I). The rehypothecation cap (140 % of customer debit balance, US Rule 15c3-3, P19) constrains *securities* rehypothecation, not cash reinvestment.

The scalar cash balance, the scalar `coll_rcv` coordinate, and the SBL loan unit state jointly represent all cash-collateral operations under both GMSLA 2010 title transfer and US Rule 15c3-3 / MSLA pledge. No extension to the cash model is required.

Q7: What does the Ledger require of market-data providers, and who tracks corporate actions?

The Ledger does. Everything at this boundary is one contract, the market-data contract (§8.5). Its parts answer the five forms the question takes.

Minimum requirements. A provider owes a value, an observation time, a signed source id, and a resolvable unit reference — nothing more (obligation O1). In particular it owes no corporate-action awareness (O3): a pre-adjusting source and a lagging source are both representable.

How incoming data are reconciled with internal state. The datum does not say whether it reflects a corporate action. The ledger attaches a basis stamp at the ingest door. The stamp is computed from the read-only basis projection and the source basis convention on file for the (unit, source) pair (O2/O4). A datum whose basis cannot be determined is quarantined, never guessed (O5).

Scope beyond dividends. One table covers spot and quotes; implied-vol observations; index levels, compositions, weights, and divisors; fixings and benchmarks; and derived curves. For implied-vol observations, the basis-sensitivity of their coordinates is declared data; no vol model enters.

Direction of flow. Exactly two arrows: the projection out, stamped observations in. The market-data layer never writes ledger state.

Operating modes. Real-time, end-of-day batch, and multi-vendor feeds face the same door. The stamp is a pure function of pinned recorded data, so mode cannot enter the result. No corporate-action logic exists outside the Ledger: the projection is arithmetic-free, and adjustment arrows are declared once, in the log.

The one-step pedagogical case, a quote lagging one ex-dividend boundary, is Appendix D.

21 Conclusion and Open Problems

21.1 Summary

The architecture rests on five concepts: wallet, unit, move, transaction, and contract. It rests on one invariant: every financial event is a set of quantity transfers between wallets, and the total quantity of each unit is conserved. Every instrument is a deterministic contract that emits moves. One immutable event log records them. Pure lifecycle functions extend the log; this gives idempotent processing and replay to any past state. Product complexity is isolated in contracts. The ledger engine is invariant under what is traded. The scalar position suffices while ownership and possession coincide. Where they diverge, the position vector of §16 extends the balance to six stored coordinates and degenerates back to the scalar when they coincide. Auditability, reproducibility, time travel, and encoded correctness follow; reproducibility holds under version-pinned market data, reference data, and pricing models (§19).

The standard is self-consistency: internally inconsistent states are unreachable by construction, not detectable after the fact. Where a failure depends on an external actor and cannot be eliminated, it is made detectable and narrowly scoped. Appendix C records this assessment per failure mode. External reconciliation against custodians, counterparties, and regulatory authorities remains necessary at the ledger boundary.

21.2 Resolved Design Problems

Three problems intrinsic to an event-sourced post-trade ledger are resolved; they are stated here only to close them.

Concurrency. Resolved by the single-writer workflow model (§15). Each lifecycle workflow instance serialises events for its unit. Cross-unit coordination is handled by parent workflows. The executor never receives concurrent mutations for one unit.

Liveness. Resolved in two stages (§15). Deterministic-date obligations are covered by durable timers. Event-triggered obligations (SBL recalls, collateral substitution, CSA margin, close-out deadlines) are covered by the obligation as a first-class object with invariants P21–P23. The guarantee covers all obligation types. Durable timers (liveness) composed with the idempotency chain (safety) yield exactly-once lifecycle processing.

State attachment. The unit-versus-wallet question of where unit state lives is resolved by the three-home state model (§4): unit state is a materialised projection of the immutable event log. It is a read cache the log always rebuilds, not an authoritative mutable store.

21.3 Open Problems

The following are design-level questions, not implementation details. All must be resolved before production, with two exceptions: repos and access control can be addressed incrementally.

Netting and close-out. Atomic moves are gross. Netting is a higher-level operation: it inspects pending gross moves, computes net amounts per (counterparty, unit) pair, and emits reduced net moves. Conservation holds by construction: the net moves form a transaction whose signed edge-sum is zero. The algebra of netting (commutativity, associativity, interaction with CCP default management) is unformalised, particularly for multilateral netting through CCPs. Close-out netting under the ISDA Master Agreement is a mass lifecycle event over many contracts. The per-contract model requires a cross-contract close-out mechanism that terminates multiple contracts, computes close-out values, and emits one net settlement move.

Correction algebra. Compensating transactions are described informally. A formal algebra of corrections (anti-moves, amendment chains, composition rules) would make corrections first-class objects.

Multi-entity federation. The design assumes one shared ledger. Multi-entity deployments require a federation protocol preserving conservation across ledger boundaries. Inter-company eliminations for consolidated reporting are unaddressed.

XVA. CVA, DVA, and FVA are non-transfer value changes that do not decompose into wallet-to-wallet moves and may require a dedicated adjustment layer (valuation mechanics: §5).

Bitemporal semantics. Economic and booking timestamps are distinguished informally. A full bitemporal model would handle market-data restatements without corrupting the historical record (market data: §8).

Tax-lot tracking. Wallets store net quantities. US wash-sale and specific-identification reporting require per-lot tracking (date, quantity, cost basis) as move-stream metadata, with a lot-selection method (FIFO, specific identification). Its interaction with the futures accumulated-cost method (§9), which deliberately avoids per-trade tracking, is unresolved.

Impairment and ECL. IFRS 9 expected credit losses and IAS 36 impairment reduce carrying value without a counterparty transfer. These may require a virtual impairment-reserve wallet or an extension of the move primitive.

Data privacy and GDPR. The append-only log conflicts with GDPR Article 17 (erasure) when moves carry personal data. Candidate resolutions: pseudonymous identifiers linked to a separate mutable store; crypto-shredding; or a schema in which personal data never enters the log. Crypto-shredding trades against time travel: once keys are destroyed, `clone_at(t)` returns anonymised counterparty references. That is acceptable for financial replay but incomplete for audit requiring counterparty identification. The choice fixes the move schema and must be made at design time.

Access control and actor attribution. The move primitive carries no authenticated actor identity. Production requires per-move attribution to an authorised actor, maker-checker workflows, cryptographic non-repudiation, and wallet-level read/write permissions scoped to role and organisational boundary.

Repos. Repurchase agreements (GMRA) share collateral and settlement primitives with SBL (§16) but differ in title-transfer semantics. Extension requires the GMRA-specific lifecycle events (open leg, close leg, repricing, substitution, classic repo versus buy-sell back).

Default management. Counterparty or clearing-member default requires close-out, portfolio auction, and loss allocation. Conservation must accommodate loss mutualisation (transfer of losses to surviving participants), which needs governance and waterfall rules the framework does not provide.

Pre-trade validation. The framework is silent on pre-trade checks (credit limits, position limits, margin sufficiency, regulatory constraints). A pre-condition gate on the move primitive is a natural extension point. Its interaction with the conservation law is unspecified.

Migration from legacy architectures. Adoption requires migrating from separate trading systems, risk engines, and general ledgers: parallel running with reconciliation during transition, progressive routing of new trades, and back-loading of historical positions. The political economy of the transition is as consequential as the technical path.

Tokenised securities. Appendix F establishes the tokenised-equity model, the double-counting resolution, and the custodian-is-flat principle. It identifies four CDM v6.0.0 gaps. Open: cross-chain interoperability; partial backing and proof-of-reserves; regulatory classification under MiFID II and SEC frameworks; tokenised-collateral eligibility; and the interaction of on-chain governance with custodian-mediated shareholder voting.

21.4 Flagged Open-Items Register

The items below are the forward agenda; they are kept distinct from the proven quantity algebra. They are migration, governance, operational, and conformance risks. None invalidates the design on correctness grounds. Each escalation is also cited at its home section: futures in §9, managed-account in §10.

21.4.1 State-Model Adoption Risk Register (F1–F8)

These accompany the three-home state model (§4). F2, F5, F6 require external stakeholders before any code ships. F1, F3, F4, F7, F8 are internal engineering risks manageable within the delivery programme. Each row pairs a risk with its mitigation.

	Risk	Mitigation
F1	Migration scope: every downstream consumer that reads unit state as one combined field must be retrofitted.	Ship an accessor that presents the split state as the one combined field, preserving read compatibility; split additively before cutover; parallel-run $\geq$ one quarter with reconciliation.
F2	Ownership of the C8 fungibility predicate is ungoverned.	Per-product RACI in the Unit Store registry; Legal/Product/Risk sign-off for predicate changes; version the predicate.
F3	Monotone carrier key-space growth at $10^7 \times 10^6$ scale is unbenchmarked.	Benchmark Option-accessor cost and cache behaviour before merge; tombstone-compaction policy for rows Some(zero) beyond the retention horizon.
F4	C1–C14 is a multi-sprint test programme, not a release.	Land the schema with the type-enforced conditions first; stage C2/C3/C8/C11 over later releases; never merge below C1+C2+C3.
F5	Mandate-as-unit creates an SFTR/EMIR reporting surface for mandate issuance.	Pre-flight with Regulatory; determine whether u_{MA} issuance needs a UTI/LEI pair; possible reportable opt-out flag.
F6	CDM alignment (TradeState per Trade vs PositionState $[w, u]$) is asserted, not verified.	Rerun the Rosetta NS1–7 mapping against the three-map schema before any CDM adapter work; publish a delta document.
F7	Onboarding cost of three maps, Option/Monotone carriers, and thirteen conditions is real.	Ship a decision tree, runbook, and cheat sheet; mandatory training before handler work.
F8	Forward migration is irreversible once the pre-migration combined wallet-state representation is collapsed into the three maps.	Keep a read-only mirror of that pre-migration representation $\geq$ four quarters post-cutover; document the inverse mapping explicitly.

21.4.2 Testing Commitments (Mutation Score)

Concrete targets, to be met before production. The generator universe (CDM enum $\times$ product-type cross-product) is finite and enumerable, so coverage is structurally bounded.

- Event handlers (decimal arithmetic): 85–90 %. Conservation kills most sign and coefficient mutants.
- Lifecycle guards: 70–80 %. Boundary-comparison mutants ($>$ versus $\geq$) require targeted tests.

- Overall core: $\geq 80\%$ (the Feathers change-safety threshold).
- State-machine model at $|W| = 3$, $|U| = 2$, depth ≤ 6 : 10^5 – 10^6 reachable states (TLC-tractable); conservation-violating handlers are caught at depth 1.

21.4.3 Managed-Account Escalations (ME1–ME5)

Open items from §10, distinct from the segregation theorem ($\text{CONS} \wedge \text{LOC} \wedge \text{C4}$) proved there.

Open item
ME1 Store-versus-derive for account-level scalars (NAV, high-water mark): which are stored and which recomputed on read is unsettled.
ME2 The observation surface is unattested: the external observations driving mandate evaluation carry no cryptographic attestation; an attestation envelope is required.
ME3 Counterparty legal identity: the mapping from client to a governed LEI is outside the ledger and must be bound.
ME4 Localisation condition C4 is asserted in the segregation argument, not yet discharged by a type.
ME5 No in-ledger solvency liveness: continuous solvency monitoring of the managed account is not a framework guarantee.

21.4.4 Futures Escalations (FE1–FE2)

Open items from §9, distinct from the futures lifecycle and its closing identity (cumulative VM = economic PnL) proved there.

Open item
FE1 Variation-margin settlement fan-out cost at scale is unbenchmarked.
FE2 The derived-consequence alternative to storing accumulated cost per position (C11) was declined; the trade-off is recorded, not closed.

Concurrency, liveness, and state attachment are resolved (§15, §4). The items in this register and in Open Problems are not. The quantity algebra is sound within the boundary of §1.6. The open items bound what the framework does not yet establish.

Part VII

Appendices

A CDM Type Mapping Tables

The mapping F of Section 14 is a set of translation rules, one per CDM type. Types carrying economic substance map to ledger primitives: products, events, transfers. Types carrying business context are preserved as event-log metadata: workflow steps and lineage.

Each row of the table states one rule: the CDM type, the ledger concept it becomes, and the shape of the correspondence.

CDM Type	Ledger Concept	Notes
Product	Unit type ($u \in \mathcal{U}$)	One product $\rightarrow$ one or more units
TradeState	Unit state ($\text{state}_t(u)$)	Lifecycle position and flags
BusinessEvent	Transaction (τ)	One or more moves
PrimitiveEvent	Move (m)	Atomic transfer
Transfer	Move (m)	Cash or securities movement
QuantityChange	Move (m)	Position adjustment
Party	Wallet (w)	One party $\rightarrow$ one or more wallets
Account	Wallet (w)	Direct correspondence
WorkflowStep	Transaction metadata	Event-log payload
Lineage	Transaction metadata	Event-log payload

The unit state $\text{state}_t(u)$ to which **TradeState** maps is a materialised projection of the immutable event log. Its value at t is rebuilt by replaying the events up to t ; the state is never an authoritative store written off-log.

B Property Test Catalogue

This appendix restates the ten core invariants of Section 17 as executable oracles. Each oracle is a precondition that constrains the generated input and a postcondition the operation under test must satisfy. The CDM enumeration universe (Section 14) supplies the generators. The framework draws random inputs, applies the operation, and checks the postcondition. A failure yields a minimal counterexample. The invariant prose is stated once in Section 17; this catalogue gives only the test forms.

Each row of the table reads the same way: given an input satisfying the precondition, the outcome must satisfy the postcondition.

Invariant	Precondition	Postcondition (oracle)
P1 Conservation	τ applied to state s , accepted	$\forall u : Q_{s'}(u) = \sum_w w(u) = 0$
P2 Atomic commitment	$\tau = \langle m_1, \dots, m_n \rangle$; move m_k fails validation, $1 \leq k \leq n$	$\text{execute}(s, \tau) = s$; no partial state observable
P3 Referential integrity	m a move in an accepted τ	$m.\text{src} \in \mathcal{W} \wedge m.\text{dst} \in \mathcal{W} \wedge m.\text{unit} \in \mathcal{U}$
P4 Log monotonicity	$\log L = \langle e_1, \dots, e_n \rangle$ after any operation sequence	$\forall i > 1 : e_i.\text{prevHash} = H(e_{i-1})$; no e_j , $j < n$, rewritten (append-only, hash-chained)
P5 Transaction idempotency	state s , transaction τ	$\text{execute}(\text{execute}(s, \tau), \tau) = \text{execute}(s, \tau)$ (dedup by <code>tx_id</code>)
P6 Lifecycle idempotency	$f(u, st, d) = (\text{moves}, st')$; st' already reflects the event	$f(u, st', d) = (\emptyset, st')$; second call emits no moves

Invariant	Precondition	Postcondition (oracle)
P7 Ledger isolation	m a move in virtual ledger $\mathcal{L}_v$	$m.\text{src} \in \mathcal{L}_v \wedge m.\text{dst} \in \mathcal{L}_v$; no move crosses the virtual/real boundary
P8 Snapshot consistency	time t in the log; s_t the state at t	$\forall w, u$: $\text{clone_at}(t).\text{balance}(w, u) = s_t.\text{balance}(w, u)$ $\wedge$ $\text{clone_at}(t).\text{unit_state}(u) = s_t.\text{unit_state}(u)$
P9 Lifecycle purity	lifecycle function f , fixed inputs (u, st, d)	repeated evaluation yields identical $(moves, st')$; no effect beyond the return
P10 PnL path-independence	price vectors P_{t_0}, P_{t_1} ; any transaction sequence on $[t_0, t_1]$	$\text{PnL}_{\text{price}} + \text{PnL}_{\text{flow}} = V_{t_1} - V_{t_0}$, independent of the intermediate path

Remark B.1 (P8 and the projection discipline). *The P8 oracle reconstructs both balances and unit state from the log. `unit_state` is read from `UnitStatus`, a materialised projection of the immutable event log—a read cache the log always rebuilds, never a separate source of truth (Section 7). The oracle is the executable form of that discipline: a catamorphism—replaying the events up to t rebuilds the exact value that held then—so $\text{clone_at}(t).\text{unit_state}(u) = s_t.\text{unit_state}(u)$ holds by construction, and a failure exposes any drift between the cache and the log. The observation `sameLedger` makes through `unitStatus` covers `usBasis`, so P8’s generator universe must include boundary transactions with retro-effective t_{eff} .*

Totality oracles. Two further oracles are global. The same generator exercises them, but they are not numbered among the core ten:

- *Totality.* For every generated input i , `execute(s, i)` returns success or a well-typed error; it never raises an unhandled exception.
- *Valid transitions only.* For every (product type, state, event) triple absent from the product’s transition table, the lifecycle function returns an error. This is the executable witness that illegal lifecycle transitions are unrepresentable (Section 7).

PnL inputs. V_t and the price vectors $P_t(u)$ that drive P10 are produced by the valuation projection (Section 5). The state-aware price function is specified in Appendix D.

SBL, obligation, basis, producer, and vocabulary extensions. Section 16.18 adds ten invariants P11–P20 specific to Securities Borrowing and Lending (collateralisation, lender ownership invariance, partial-return and settlement monotonicity). Section 15.6.6 adds P21–P23 for obligation liveness. Section 8 adds P24, basis tip agreement, and P25, ingest-door soundness. Section 16.14 adds P26, locate-capacity admission. Section 17.7 adds five more: P27, producer agreement (oracle `recomputeOK`, with the terms-leg oracle `propTermsRecompute`); P28, cause committed (`pCauseCommitted`); P29, schedule totality, the witness of C14 (`propScheduleTotalGate`, quantifying over the closed `CAClass`); P30, dimension invariance (`propDimInvariance`, with the composite-kind oracle `dkDivSplitOK`); and P31, kind totality at the ingest door (`pKindTotal`, the P25 shape).

These extensions never relax a core invariant. Every transaction satisfying P11–P23 also satisfies P1–P10. P24 quantifies over log prefixes rather than single transactions, P26 over

locate-confirmation commit positions, and P27 over committed logs with a recomputing implementation; each narrows the accepted logs without weakening any oracle.

The executable forms of P11–P23 follow the same precondition/postcondition shape. Those of P24–P31 are given separately: P24 below, P25 with the market-data contract (§8.5), P26 as the log-replay check of Section 16.14, and P27–P31 as the named suite oracles of Section 17.7 (reference/Ledger.hs, Part L and the gates they witness). Three conformance-tier properties — P32, confirmation gate; P33, elective aggregate invariance; P34, basket joint consumption — are specified in Section 17.7; their executable forms live where **transport** lives, outside the reference.

P24 asserts that two computations of a unit’s basis tip agree: the fold that applies boundary events in booking order, and the projection that ranks them in effective order. The oracle checks the agreement at every accepted prefix of the log.

Invariant	Precondition	Postcondition (oracle)
P24 Basis tip agreement	a log L ; any prefix $L_k = \langle \tau_1, \dots, \tau_k \rangle$ whose replay from the empty ledger is accepted	$\forall u$ registered in L_k : $\text{usBasis}(u) = \text{effTip}(u, L_k)$ — the maximum, in the lexicographic order $(t_{\text{eff}}, \text{prec}, \text{bid})$, of the boundary events L_k carries on u , or $\text{Origin}(u)$ if it carries none. The booking-order status fold and the effective-order chain projection agree at every prefix

Remark B.2 (P24 and the time-travel claim). *The basis-time-travel claim of §8.10 is discharged by P8 together with P24: each accepted-prefix check is a `clone_at(k)`, and `sameLedger` observes `usBasis` through `unitStatus`. No-regression is a corollary, not a separate test: the maximum of a growing committed set is monotone. The typed oracle is `p24` (reference/Ledger.hs, Part L); its recomputation `effTip` reads the transactions’ own boundary events, never the `ledgerBounds` cache.*

Typed oracles. Each oracle above has a typed counterpart. Its precondition and postcondition are both total, so the generator and the checker share one type. The operation under test returns a typed **Outcome**. **Accepted** is the *only* constructor that carries a ledger, so a partial (“half-committed”) ledger is not representable. That absence is the *Totality* oracle and the “no partial state” half of P2, made structural rather than tested.

The listing fixes the shared vocabulary: the outcome type, the property shape, and the observational equality the oracles use.

```
-- execute = the canonical (Sec. 4) three-home transaction-apply, lifted to
-- return Outcome; supplied by the implementation, not a symbol defined here.
data Outcome = Accepted Ledger | Rejected TxError    -- no third "crashed" case

-- A property: a precondition constraining the generated input and a
-- postcondition over the outcome of running the operation on it. Both total.
data Property i o = Property
  { pre  :: Ledger -> i -> Bool
  , post :: Ledger -> i -> o -> Bool }
```

```

data TxInput = TxInput { tiTx :: OracleTx, tiScope :: Scope }
data Scope   = Scope   { scWallets :: [WalletId], scUnits :: [UnitId] }

registered :: Ledger -> UnitId -> Bool
registered l u = maybe False (const True) (productTerms l u)

-- Sum a wallet's holding of u across a wallet set. Summation is a monoid
-- homomorphism into the Qty group -- the empty set sums to mempty (=0) with no
-- special case, so a zero-holder unit conserves vacuously (no divide-by-holders).
unitTotal :: Ledger -> [WalletId] -> UnitId -> Qty
unitTotal l ws u = foldMap (\w -> maybe mempty psBalance (position l w u))
  (Set.toList (Set.fromList ws))

-- Equality of ledgers BY OBSERVATION over the scope -- the only equality the
-- oracles need; the abstract Ledger exports no structural Eq.
sameLedger :: Scope -> Ledger -> Ledger -> Bool
sameLedger sc a b =
  and [ position a w u == position b w u | w <- scWallets sc, u <- scUnits sc ]
  && and [ unitStatus a u == unitStatus b u | u <- scUnits sc ]

```

Five oracles test the same operation, `execute`, and share the one Property `TxInput Outcome` shape. Each postcondition that asserts a positive fact is guarded on `Accepted`, so it claims nothing about a rejected transaction.

```

-- P1 Conservation. On any accepted transaction every unit's holdings sum to
-- zero across the scope (closed system); a rejected one asserts nothing.
p1 :: Property TxInput Outcome
p1 = Property (\_ _ -> True) $ \_ (TxInput _ sc) o -> case o of
  Rejected _ -> True
  Accepted l -> all (\u -> unitTotal l (scWallets sc) u == mempty) (scUnits sc)

-- P2 Atomic commitment. Any move failing validation rejects the whole txn;
-- no partial ledger exists (Outcome cannot carry one).
p2 :: Property TxInput Outcome
p2 = Property (\l (TxInput tx _) -> any (invalidMove l) (otMoves tx))
  (\_ _ o -> isRejected o)

-- P3 Referential integrity. Every move in an accepted txn names a registered
-- unit and in-scope wallets.
p3 :: Property TxInput Outcome
p3 = Property (\_ _ -> True) $ \_ (TxInput tx sc) o -> case o of
  Rejected _ -> True
  Accepted l -> all (\m -> registered l (mUnit m)
    && inScope sc (mFrom m) && inScope sc (mTo m)) (otMoves tx
  )

-- P7 Ledger isolation. With Scope = the virtual ledger's wallets, no move
-- crosses its boundary -- a property of the moves, independent of the outcome.
p7 :: Property TxInput Outcome
p7 = Property (\_ (TxInput tx _) -> not (null (otMoves tx)))
  $ \_ (TxInput tx sc) _ ->
    all (\m -> inScope sc (mFrom m) && inScope sc (mTo m)) (otMoves tx)

-- P5 Transaction idempotency (dedup by tx id). Re-running an accepted txn is a
-- no-op; this oracle re-runs the operation, so it takes it directly.
idempotentTx :: (Ledger -> OracleTx -> Outcome)
              -> Scope -> Ledger -> OracleTx -> Bool
idempotentTx exec sc l tx = case exec l tx of

```



```

Rejected _ -> True
Accepted l' -> case exec l' tx of Accepted l'' -> sameLedger sc l' l''
Rejected _ -> False

```

The remaining five test four other operations: the log append, the snapshot rebuild, the pure lifecycle handler, the PnL decomposition. Forcing them into the `Property` shape would buy nothing. Each is therefore a bespoke total predicate (the restraint rule: stop at the cheaper guarantee).

```

-- P4 Log monotonicity. The log is append-only and hash-chained BY CONSTRUCTION
-- (built only by 'append', which stamps the predecessor's hash; no rewrite or
-- delete is exported). The oracle re-checks the link; a break needs API-tamper.
data Hashed = Hashed { hPrev :: Hash, hPayload :: Transaction, hSelf :: Hash }
chainOK :: [Hashed] -> Bool
chainOK es = and (zipWith (\a b -> hPrev b == hSelf a) es (drop 1 es))

-- P8 Snapshot consistency. cloneAt t rebuilds the state at t by replaying the
-- events up to t (a catamorphism -- replaying the log always rebuilds the exact
-- value that held then), so snapshot must equal s_t by observation (both
-- balances and unit state -- sameLedger covers both).
p8 :: Scope -> Ledger -> Ledger -> Bool      -- cloneAt t vs s_t
p8 = sameLedger

-- A lifecycle handler is a PURE TOTAL function. Purity (no IO in the type) IS
-- P9: re-evaluation cannot differ and there is no effect to leak.
type LifecycleFn st ev = UnitId -> st -> ev -> ([Move], st)

-- P6 Lifecycle idempotency. Feeding the handler its own output state emits no
-- further moves and fixes the state.
idempotentL :: (Eq st) => LifecycleFn st ev -> UnitId -> st -> ev -> Bool
idempotentL f u st ev = let (_, st') = f u st ev in f u st' ev == ([], st')

-- Valid-transitions-only. For every (product, state, event) absent from the
-- transition table, the guarded handler returns an error: not in table => Left.
-- An illegal transition is thus not representable as success.
validTransitionsOnly
  :: (p -> st -> ev -> Bool)                -- in the table?
  -> (p -> st -> ev -> Either TxError ([Move], st)) -- the guarded handler
  -> p -> st -> ev -> Bool
validTransitionsOnly inTable h p st ev
  | inTable p st ev = True
  | otherwise       = case h p st ev of Left _ -> True; Right _ -> False

-- P10 PnL path-independence. V_t and the price vectors P_t(u) come from the
-- valuation projection; the decomposition closes regardless of the
-- intermediate transaction path.
p10 :: Cash -> Cash -> Cash -> Cash -> Bool      -- pnlPrice pnlFlow V_t0 V_t1
p10 pnlPrice pnlFlow v0 v1 = pnlPrice <> pnlFlow == v1 <> cashNeg v0

```

P24 is log-prefix-shaped, like P4 and P8. The check replays every prefix from the empty ledger and compares the cached tip with a recomputation from the transactions' own boundary events. Its oracle and the spec function it recomputes are verbatim from the reference (Part L):

```

-- / The effective-order tip a log prefix mandates for u, recomputed from the
-- transactions' own BoundaryEvents -- INDEPENDENT of the ledger's ledgerBounds
-- cache, so a drifted cache cannot vouch for itself. The spec function of the
-- basis-chain definition (sec:state-basis), written once, here: the maximum,
-- in the lexicographic order (t_eff, prec, bid), of the boundary events the
-- prefix carries on u, or Origin u if it carries none.

```

```

effTip :: UnitId -> [Transaction] -> BasisId
effTip u txs =
  case Set.lookupMax (Set.fromList
    [ (bevTEff be, bevPrec be, bevId be)
    | tx <- txs, txUnit tx == u, Just be <- [txBoundary tx] ]) of
    Nothing      -> Origin u
    Just (_, _, bid) -> Boundary bid

-- P24 Basis tip agreement: at EVERY accepted prefix of the log, the booking-order
-- status fold (replay) and the effective-order chain projection (effTip) agree on
-- usBasis for every registered unit. Quantifying over prefixes makes each check a
-- clone_at(k), so the retro-effective time-travel claim (sec:basis-time-travel)
-- rides on this oracle plus the existing P8 (sameLedger already observes usBasis
-- through unitStatus, whose structural Eq covers all four fields). No-regression
-- is a corollary, not a test: the max of a growing committed set never recedes.
-- Wrongful ADMISSION of a non-tip SetBasis fails this oracle at that prefix;
-- wrongful REJECTION is the vacuous side, as in P1/P2's split -- consistent with
-- catalogue precedent. Log-prefix-shaped like P4/P8, not Property-shaped: the
-- claim quantifies over prefixes of a log, not over single transactions.
-- (Generators: BoundaryId is abstract by design, so boundary transactions for
-- this oracle are compiled IN-MODULE, minting deterministic content addresses --
-- no production caller gains a minting door.)
p24 :: [Transaction] -> Bool
p24 txs = all prefixOK [ take k txs | k <- [0 .. length txs] ]
  where
    prefixOK pfx = case replay pfx emptyLedger of
      Left _  -> True  -- a rejected prefix asserts nothing (rejection is P2's
      Right l -> all (agrees l pfx) (Set.toList (Set.fromList (map txUnit pfx)))
    agrees l pfx u = fmap usBasis (unitStatus l u) == Just (effTip u pfx)

```

The full self-contained listing is collected in `reference/Ledger.hs`. It supplies the helpers `inScope`, `isRejected`, `invalidMove`, the `Move/OracleTx` types, the `TxError` and `Hash` types, the `Cash` group inverse `cashNeg`, and the total coverage witness. It reuses the `Ledger` library and its total accessors.

C Reconciliation Failure Mode Taxonomy

For activity recorded in the ledger, each class of internal reconciliation break carries one of three assessments: *structurally unreachable*, *easier to detect*, or *addressed by design*. Three modes are structurally unreachable: the architecture makes the divergent state unrepresentable. Six modes are easier to detect but not eliminable. Each of the six depends on an external actor that the ledger only mirrors. Securities-lending breaks are addressed by design: the split-system break no longer exists. Operational failures outside the architecture remain possible: incomplete booking, duplicate booking through external channels, and stale-data ingestion. These require operational controls.

The table follows one pattern. Where the two sides of a historical break are a single object in the ledger, the divergent state does not exist, and the break is unreachable. Where one side lives in an external system, the ledger holds an independent record to compare against, and the break becomes visible rather than impossible.

Failure Mode	Assessment	Rationale
Trading / GL divergence	Structurally unreachable	Trade and accounting entry are one object
PnL / balance-sheet mismatch	Structurally unreachable	Both project the same position set and price vector
Inter-entity breaks (shared ledger)	Structurally unreachable	An inter-entity move is one atomic event
Custodian / depot breaks	Easier to detect	The virtual wallet is an independent comparison point against the external system
Nostro / vostro breaks	Easier to detect	The virtual wallet mirrors the external account; ISO 20022 feedback (deferred-settlement spec) narrows the window
Corporate action mismatches	Easier to detect	Lifecycle events are transactions; idempotence prevents double-processing
Collateral disputes	Easier to detect	Internal valuation is recorded; the counterparty may disagree. Posting and return are moves; eligibility, haircuts, and substitution under UMR are deferred to further specification
FX rate discrepancies	Easier to detect	The dual-valuation framework (valuation spec) surfaces the differential
Securities-lending breaks	Addressed by design	SBL integration (§16) removes the split-system break

D Coordination Between Pricing, Contract State, and Market Data

A unit’s valuation and the cash recorded in wallets are consistent at every instant: the atomicity of the lifecycle event (C3) commits a coupon’s or dividend’s cash flow and its state transition as one indivisible transaction, so their divergence is structurally unreachable. The worked coupon and dividend, with the two corruptions atomicity excludes, are in §8.12; this appendix fixes the one-step vocabulary that the State-Basis Discipline (§8) generalises.

The pricing of a unit u is state-aware:

$$P_t(u) = P(u, \text{state}_t(u), \text{market data at } t).$$

The state tells the pricer how to read the quote; the quote alone is never a price. The pricing-relevant slice of state is the pair Cum / Ex d : the distribution is either still embedded (Cum) or detached and paid (Ex d , carrying the paid amount, so the adjustment is total and “ex but amount unknown” is unrepresentable). The external quote may lag the ledger’s own ex

transition by one step — the first ex quote may appear only at the next market open — and the flag `mQuoteEx` records whether the quote has itself gone ex. The price is a total, pure function of that state *and* the quote: cum reads the quote as is; ex with a caught-up quote is the identity; ex with a lagging quote removes the paid amount exactly once, $q - d$, so total value (cash plus mark) is continuous through the event. A pricer typed on the quote alone could not see the state and would double-count by construction. The reference implements the pricer as `statePrice` (`reference/Ledger.hs`, Part F), checked for all (u, q, d) by the witness `fibreOK` (Part L) — an unnumbered finite algebraic identity of one function, not a ledger invariant.

This appendix is the one-step, $f = 1$ fibre of the market-data contract (§8.5): its two basis points are Cum and Ex d ; its conversion is the generator `Shift` $(-d)$; and `mQuoteEx` is the per-(unit, source) basis stamp in degenerate form — the source basis convention (O4) collapsed to `True` as `TracksChain` and `False` as `LagsBy 1`, under TA-BASIS, with the lag-only cases as its domain. A quote gone ex while the unit is still cum lies outside the one-bit model, since the flag carries no refusal channel; the general discipline handles that case fail-closed, by W3 partition quarantine (§8.11). The construction and provenance of the external quote feeding $P_t(u)$ lie outside the ledger boundary (§1.6); the contract the quote crosses to enter is §8.5.

E Glossary

Terms carry these meanings throughout. A defined term is used thereafter without re-explanation. Property identifiers P1–P34 and condition identifiers C1–C14 are stated where they are established, and referenced by identifier elsewhere. Each entry below gives the term, its meaning, and, where one exists, a pointer to the section that establishes it.

Accumulated cost

Per-wallet signed sum $\sum_i (-\text{qty}_i \times \text{price}_i \times \text{mult})$ over the trades of a futures position: negative net-long, positive net-short, zero when flat. A conserved field.

Adjustment schedule

The per-unit answer, carried in **ProductTerms** from registration, to “what does each corporate-action class do to each terms field”: a class default, a declared override in a closed rule language, or a designated party owing **Resolved terms**. Total by condition C14 (§8.3).

Atomic move

The indivisible primitive: transfer of a positive quantity of one unit from a source wallet to a destination. It is the edge presentation of a conserved flow (§2).

Balance-sheet substantiation

Demonstration that each balance-sheet figure is a projection of the move stream, reconciled against external authorities at the boundary (§11).

Basis id (`BasisId`)

A unit’s basis identifier: its origin or a boundary id. A unit’s boundary events form its basis chain in effective order, the lexicographic total order on $(t_{\text{eff}}, \text{prec}, \text{bid})$. Stamps and state store basis ids, never ordinals (Definition 8.2).

Basis projection (`BasisView`)

The read-only, versioned view of the log the ingestion layer stamps against: per unit, the committed boundary chain in effective order and its tip. It has the same rank as **UnitStatus**: derivable, discardable, never authoritative. It is arithmetic-free: no operator kinds, parameters, or arrows, so no consumer can re-implement corporate-action logic (§8.5).

Basis stamp

The joint basis named by a consumed datum: a finite map $\text{UnitId} \rightarrow \text{BasisId}$ assigned per (unit, source) at the ingest door of the **Market-data contract** (§8.5). It is never defaulted from ledger state. Over the stamp’s whole domain, consumption requires either pointwise agreement with the prevailing total assignment β_t or a ledger-authored arrow (C13; Invariant 8.17).

CAClass

The closed eight-member taxonomy of corporate-action classes a confirmed notice declares (§8.13). Declared data, never inferred from a name; the schedule totality gate C14 quantifies over it.

CDM

ISDA Common Domain Model: a standardised, machine-executable model of financial products and lifecycle events.

Clone_at(t)

Reconstruction of ledger state as of historical time t by replaying the move stream and the logged state changes up to t .

Closed system

A system in which quantity is neither created nor destroyed, only moved between wallets (§2).

Conservation law

The total quantity of each unit across all wallets is zero at all times: $Q(u) = 0$ (P1).

Conserved field

A **PositionState** field whose per-event-class deltas sum to zero across wallets: $\sum_w \Delta f(w, u) = 0$. The property is established per handler, with a vacuous zero-holder base case (C2, C9). Net quantity, accumulated cost, and variation margin are conserved fields.

Confirmation gate

The resolution/consumption boundary: an operator exists only from the W4-attested final-terms notice; between announcement and confirmation nothing adjusts and consumption fails closed or quarantines (§8.13; witnessed by P32).

Coordinate

A component of the position vector; a position type in the wallet model. Each coordinate passes the physical-action test: a distinct physical action modifies it and only it.

Datum-kind registry

The logged vocabulary of consumable market-datum kinds: each kind registers once, with a component dimension declaration and a mandatory invariance witness, before any datum of that kind can enter; an unregistered kind is refused at the door (§8.6; TA-KIND, P31).

Dimension declaration

The declaration every terms field and every registered datum component makes at definition — quantity-of- u , price-in-basis, cash, or dimensionless — fixing its map under any boundary’s (f, c) by the per-dimension action (Definition 8.12; Theorem 8.13).

Dual valuation

Two price functions over the same position set: mark-to-market and mark-to-model.

Encumbrance

Owned but not freely deployable: $\text{encumb} = \text{onloan} + \text{coll_post}$.

Event sourcing

All state is a projection of an append-only log of events (moves).

Forgetful mapping

The CDM-to-ledger map F , which preserves economic substance while forgetting business intent and workflow lineage.

Idempotency

Applying the same operation twice has no incremental effect beyond the first application.

Invariance weld

`applyTx` admits a basis-changing corporate action only if the moves and the declaration jointly satisfy the invariance identity of Theorem 8.11. Such an action is one atomic transaction carrying its entitlement moves, its `SetBasis`, and its declaration (Principle 8.6).

Lifecycle event

A state transition on a unit (coupon, margin call, exercise, reset) expressed as a transaction.

Managed account

An internal book treated as a virtual portfolio with periodic cash settlement of accumulated PnL.

Market-data contract

The boundary contract for external pricing data (§8.5). It has exactly two arrows: the **Basis projection** flows out; raw observations flow in through the single ingest door. At the door the ledger attaches the **Basis stamp** from the projection and the **Source basis convention**. Providers owe value, observation time, signed source, and unit reference only, and need not track corporate actions. A datum whose basis cannot be determined is quarantined, never guessed. Real-time, batch, and multi-vendor feeds face the same door (door soundness P25).

Monotone carrier

A state carrier whose accessor commutes with replay: the value rebuilt from the first k events, then advanced by the rest, equals the value rebuilt from all events. Replay is therefore a single fold with no order dependence (C1; discharges P8).

Move stream

The immutable, append-only log of all atomic moves: the canonical internal record.

Obligation

A recorded commitment to perform a move by a stated time. Discharged when a matching committed transaction appears; liveness is read by a total discharge predicate (§15).

One declaration, two consumers

A boundary's declared operator is consumed twice: at the effective time by the corporate-action processor, and forever after at the read seam as the standing conversion on pre-boundary arrivals (Principle 8.10).

Option accessor

The position read `position( $w, u$ )` returns an optional value: *None* until the first event touches (w, u) , a value thereafter. No row is fabricated on read (C1).

Own

First coordinate of the position vector, $\vec{w}_e(u)[\text{own}]$: economic ownership, maintained by buy/sell, determines PnL.

Ownership consistency

$\text{avail} = \text{own} - \text{onloan} + \text{borr}$ for lendable securities. A definitional identity under the six-coordinate model (P20).

PnL

Profit and loss, $V_{t_1} - V_{t_0}$: depends only on initial and final portfolio states (P10).

Position vector

$\vec{w}_e(u) \in \mathbb{R}^6$: (own, onloan, borr, coll_post, coll_recv, coll_rehyp). Available inventory $\text{avail} = \text{own} - \text{onloan} + \text{borr}$ is a projection computed on read. For non-lendable instruments the vector degenerates to a scalar (§16).

PositionState

Per-(wallet, unit) economic state: the third state home, holding conserved-additive fields (balance, accumulated cost) and non-conserved fields (high-water mark, monotone by max; entry NAV, write-once). All per-(wallet, mandate) economic state lives here (C12); no flat per-wallet scalars. Read through the **Option accessor** (§4).

PosQty

The abstract positive magnitude: a quantity known > 0 by type, whose sole constructor is the parse boundary **mkPosQty** and whose reader is **unPosQty** (§9). It is the magnitude the reference **Move** carries and the futures trade boundary consumes; no **PositionState** field carries it.

Processor

The external, untrusted, verifiable machinery — lifecycle or corporate-action — that computes a settlement or an adjustment from logged data and submits it through the single door; bound by the processor contract and checked by recomputation (§15.2; Principle 15.1, P27).

ProductTerms

The first state home: a unit's contractual terms, immutable, versioned, and append-only. It is a non-empty list of term versions, written first at registration (C6, C7) and never mutated in place. Amendment is two-track (C8).

Projection

Any view computed from the move stream rather than stored independently: balances, PnL, balance sheets, reports (§2).

Property-based testing

Checking invariants against randomly generated inputs.

Purity

Lifecycle functions produce identical outputs for identical inputs, with no side effects (P9).

 $Q(u)$

Total quantity of unit u across all wallets, $Q(u) = \sum_{w \in \mathcal{W}} w_t(u)$. Zero by the conservation law: the framework is a closed double-entry system.

Real wallet

A wallet representing an actual portfolio position; changes affect PnL.

Reclassification

A move between wallets of the same entity with different position types (e.g. rehypothecation, `coll_recv` $\rightarrow$ `coll_rehyp`).

Registration

Introduction of a unit, recorded as a move-less **Transaction** that writes the first **ProductTerms** version and the **UnitStatus** default together (C5, C7). Its move list is empty, so it is vacuously conserved. Re-registration of an existing identifier is a hard error (C10). It is the move-less instance of the one event type, not an operation standing outside it (§4).

Resolved terms

Adjusted terms rendered by the party the **Adjustment schedule** designates, entering the log as a W4-attested terms append referencing the boundary — the second admissible source of adjusted terms, beside the schedule’s evaluator (§8.3).

Self-consistency

Internally inconsistent states are structurally unreachable within the ledger’s scope, not merely detectable.

Settlement layer

The interface projecting committed ledger activity into external settlement infrastructure (§13).

Single-Coordinate Move Principle

An atomic move modifies exactly one coordinate of one unit in the position vectors of the entities it connects.

Smart contract

A deterministic program mapping inputs and state to sequences of atomic moves.

Source basis convention

The per-(unit, source) declaration on file of how a source’s dissemination relates to the unit’s basis chain: tracks it, lags by a declared offset, pre-adjusts, declares a fixed basis, or makes no claim (O4, §8.5). The ingest door consults it to compute the **Basis stamp**. It is versioned and logged, and owned by market-data operations under TA-BASIS.

State-sufficiency

Portfolio value depends only on current balances, current unit state, and current prices, not on transaction history.

Three homes

The three places unit state lives: **ProductTerms**, **UnitStatus**, and **PositionState** (§4).

Time travel

Reconstruction of the ledger at any historical date and re-execution of lifecycle logic; the read side of **Clone_at**.

Tip weld

`applyTx` admits **SetBasis** b on unit u only if b is the post-insertion effective-order tip of u ’s basis chain, so a basis regressed by a late notice is unrepresentable (Principle 8.4; property form P24).

Transaction

The one atomic event recorded in the log: an ordered list of moves together with the state delta they carry. The delta comprises per-wallet bookkeeping (accumulated cost,

high-water mark, entry NAV), shared **UnitStatus** writes, and either a **ProductTerms** introduction (registration) or an append (preserving amendment). Conservation holds by construction from the signed edge-sum of the moves. The empty move list is conserved for free, so a move-less event needs no check (P1). A transaction is applied whole or rejected whole through the single operation **applyTx** (P2, C3). There is no separate validation step and no unconserved transaction to reject.

Transfer

A move between wallets of different entities.

Two planes

Entitlements and elections settle per holder as moves (the entitlement plane); the quoted series has one holder-independent fate per boundary, and operators live there (the quotation plane) (Principle 8.5).

Transition function

The pure function carrying a unit's lifecycle state and an event to the next state and its moves (§7).

TRS

Total return swap: a contract converting virtual ledger performance into real cash flows.

Unit

Any tradable or bookable object: currency, security, derivative, contract.

UnitStatus

The second state home: one value per unit, shared, read identically by every holder. The value changes over time, but **UnitStatus** is not a separate source of truth: the immutable event log is. Only a logged event changes it, and the stored value is overwritten in place solely as a read cache; replaying the events up to any point rebuilds the exact value that held then. A value exists from registration onward, product-declared defaults applied there (C5; §4). Its content comprises the lifecycle stage, the basis coordinate **usBasis** (a **Basis id**, tip-welded; C13, §8), the last settle mark **usLastSettle**, and the successor pointer. It is the sole home of unit-level shared state.

Virtual ledger

A separate instance of the same framework in which every wallet is virtual, used for strategy simulation.

Virtual wallet

A wallet representing an external counterparty; exists to close the system for conservation.

Wallet

A named account holding quantities of units: a logical partition of position space, not a bank or custody account (§2).

The conditions. Each of C1–C14 renders a class of illegal state unrepresentable. They are established in §4 (C7 and C10 in §3, C13 and C14 in §8) and cross-referenced from the invariant hub (§17).

C1 Option accessor and monotone carrier, both.

C2 Handler-level conservation, per event class, with a vacuous base case.

C3 Atomic **Transaction** across the three state homes; partial application rejected.

- C4** Capability-scoped reads; cross- (w, u) overlay reads forbidden; strategy exports flow only through **UnitStatus**.
- C5** **UnitStatus** registration-total; product-declared defaults applied at Unit Store registration.
- C6** **ProductTerms** versioned, append-only; no in-place mutation.
- C7** **ProductTerms** registration-total; first write at registration.
- C8** Amendment two-track: a product-declared fungibility predicate decides *Preserving* (append a term version) versus *Breaking* (allocate a fresh unit, with **SupersededBy**).
- C9** Handlers on zero-holder units discharge $\sum = 0$ vacuously; the per-class proof obligation includes the empty case.
- C10** Re-registration of a unit identifier is a hard error, never a silent reset.
- C11** Each **PositionState** field is tagged with the canonical set of writers permitted to write it.
- C12** All per-(wallet, mandate) economic state lives in **PositionState**; no flat per-wallet scalars. Enforced by schema.
- C13** Basis edge: every consumed datum names its joint basis; consumption requires pointwise agreement, over the stamp's whole domain, with the prevailing total assignment β_t or a ledger-authored arrow (§8).
- C14** Adjustment-schedule totality: registration and every terms append admitted only if the schedule yields exactly one well-formed entry per (corporate-action class, field); an append introducing a field extends the schedule in the same transaction (§8.3).

F The CDM Product Model—A Developer's Guide

The ISDA Common Domain Model defines a layered object hierarchy for financial products. Each layer maps to one ledger construct. The map yields a single result: the ledger unit is the CDM **Trade**, not the product. This appendix establishes that result through the six layers. It then exercises three boundaries the basic layers do not reach: payout composition, tokenised custody, and date resolution. Each boundary carries one worked instrument. Throughout, “CDM” denotes Common Domain Model v6.0.0 (FINOS-governed); field names follow the Rosetta/Rune schema.

F.1 The Six-Layer Hierarchy

The layers narrow from shared to unique. Five are reusable infrastructure. Only the Payout is product-specific, and only the Trade is a unit.

Layer 1 — Observable. The market quantity a contract references (an equity index, a rate). It is identical across product type, direction, and venue: an option and a future on the same index share one Observable. It is a market-data input to pricing and lifecycle observation. It is not a unit; it is never held or transferred.

Layer 2 — Payout. A choice type, and the only truly product-specific layer. An option is one `OptionPayout`. A future is a `PerformancePayout` paired with a `FixedPricePayout`, mirroring an equity swap. CDM does not distinguish listed from OTC here: the economic logic of a listed and an OTC option is the same payout with different strike, expiry, and settlement currency. A put flips `optionType`. A short reverses direction (`buyerSeller` for options, `payerReceiver` for swap-like legs). Nothing else changes. The Payout is the move-generating logic of the smart contract: it defines what transfers occur under what conditions. It carries *no* counterparty identity and *no* collateral reference.

Layer 3 — EconomicTerms. Wraps the Payout with `effectiveDate`, `terminationDate`, and calculation-agent metadata. It drives the lifecycle state machine: the effective date triggers `ACTIVE`, the termination date triggers expiry, exercise, or settlement observation. Its `collateral` field is product-intrinsic (repo haircuts, securities-lending collateral). The field is not the variation-margin CSA; that attaches at Trade.

Layer 4 — NonTransferableProduct. Wraps `EconomicTerms` with identifiers and ISDA taxonomy. It is the contract *template*: the code, not the instance, shared by every trade with identical terms. CDM offers two product types. `TransferableProduct` extends `Asset`; it is for instruments whose economics follow from ownership (equities, plain bonds). `NonTransferableProduct` wraps `EconomicTerms`; it is for contracts whose payout must be explicitly modelled. Listed options and futures are `NonTransferableProduct`: exchange-traded and fungible, but contracts with explicit payout logic, not assets transferred by ownership. Their exchange fungibility is captured by a shared template: all trades in one contract specification reference the same `NonTransferableProduct`.

Layer 5 — TradableProduct. Binds a template to a `TradeLot` (agreed price and quantity) and counterparty roles. It is the instance before collateral attachment: counterparties and economics, no collateral terms. A product quoted at many prices to many parties is many `TradableProducts`. Confirmation makes one a Trade.

Layer 6 — Trade. Extends `TradableProduct` with `tradeIdentifier` (UTI), `tradeDate`, resolved parties (LEIs), `executionDetails`, and `collateral`. A listed and an OTC trade on the same economic terms share everything up to here. They diverge completely at Trade: counterparty (named bilateral vs. CCP), collateral source (bilateral CSA vs. CCP rulebook), and venue.

The table restates the hierarchy as a map from CDM layer to ledger construct. Two rows carry the identity result: the Payout is the only product-specific layer, and the Trade is the unit.

CDM layer	Ledger construct	Varies across instruments?
Observable	Market-data input	No
Payout	Move-generating logic	Yes—the only product-specific layer
EconomicTerms	Contract definition	No (structural wrapper)
NonTransferableProduct	Contract template	Shared if terms identical
TradableProduct	Priced instance	Trade-specific
Trade	Unit	Always

F.2 Unit Identity: Trade Is the Unit

Two trades on one product can differ in counterparty and collateral. A unit that equated them would lose that difference. The principle fixes the unit at the level where nothing is lost.

Principle F.1 (Trade = Unit). *The ledger unit is the full CDM Trade object, including its Collateral field. Product identity is not unit identity.*

The unit key is the Trade identifier (UTI for OTC; contract specification for listed). The forgetful map $U : \mathbf{Trade} \rightarrow \mathbf{NonTransferableProduct}$ that strips counterparty and collateral is many-to-one: many distinct trades collapse to one product. The ledger operates on the source **Trade**, never on the quotient. The Unit Store (Section 3) enforces injectivity at registration: distinct Trades map to distinct units.

The dual-CSA test. Two OTC trades reference the same template: a European call on the same index at the same strike. One carries a USD CSA (SOFR discounting, New York law); the other a EUR CSA (ESTR discounting, English law). Because **Collateral** differs, the Trade objects differ, and the units differ. Two otherwise-identical options collateralised in different currencies are **different units**: different discount curves, margin flows, and settlement currencies.

Listed and cleared. For listed instruments the CCP interposes itself by novation as the sole counterparty, so all positions in one contract specification are fungible regardless of original counterparty. Listed-vs-OTC is therefore a Trade-level distinction, not a product-level one: counterparty becomes the CCP, collateral becomes the CCP rulebook, venue becomes the exchange. For futures, daily settlement is itself the PnL realisation rather than collateral against it. The cash mechanics are in the futures lifecycle (Section 9).

CCP margin is a CDM gap. CDM v6.0.0 has no type for CCP margin rules distinct from bilateral collateral provisions; the **Collateral** type on Trade was built for bilateral CSAs. For cleared trades the margin framework is an external reference to the CCP rulebook. The two-contract design (payoff contract plus margin contract) handles both regimes. CDM's representation of the cleared case is the less mature.

F.3 Payout Composition and the TransferableProduct Boundary

A structured equity note tests payout composition and the product-type boundary at once. BankCo issues a 1Y zero-coupon note on an equity index, EUR 1,000,000 notional, to FundCo. Redemption is

$$R = N \times \min\left(1, \frac{S_T}{S_0}\right),$$

par if the index is flat or up, full downside if it falls. Economically the note is a zero-coupon bond *plus* a short put struck at par. It has an ISIN, lists on an exchange, and settles through a CSD: a transferable security in law and market infrastructure.

Result 1: economic decomposition is not CDM decomposition. The correct representation is a single **PerformancePayout** with a return capped at 0%, not a **FixedPricePayout** (par bond) plus an **OptionPayout** (short put). The decomposition is economically correct yet wrong as a CDM model, for three reasons. The embedded put has no independent exercise right: the redemption formula applies automatically, so there is no **OptionPayout**. The note has one contractual cash flow at maturity, not two. And CDM qualification recognises a capped **PerformancePayout** on an equity index as a known structure, but not a premium-less **OptionPayout**. Risk systems decompose into bond + put for Greeks; CDM models the contractual cash flow.

Result 2: TransferableProduct is narrower than “transferable.” The note is a `NonTransferableProduct` despite being a transferable security. The criterion is not legal transferability but modelling complexity. `TransferableProduct` is for instruments whose cash flows follow mechanically from ownership (a plain bond’s coupon and par). Any instrument whose cash flows depend on an observable requires an explicit Payout tree and is therefore `NonTransferableProduct`. The ISIN lives in the `NonTransferableProduct` identifier field; carrying an ISIN does not require `TransferableProduct`. The IFRS 9 SPPI test parallels the criterion: the note fails SPPI because of the equity-linked redemption and is measured at fair value. The signal is the same non-trivial payout that forces `NonTransferableProduct` and trading-book scope (Section 1).

Result 3: one unit, three settlement paradigms. The note is one security, one ISIN, one Trade, one ledger unit; the embedded put is not separately tradable, exercisable, or settleable. Issuance and redemption are two-move transactions that conserve per unit. In the index-down-20% scenario redemption is EUR 800,000:

Move(BankCo → FundCo, EUR, 800,000), Move(FundCo → BankCo, note, 1).

The “missing” EUR 200,000 is the put payoff, already recognised through mark-to-market. Before maturity FundCo holds one note valued at EUR 800,000; after maturity, EUR 800,000 cash. The value jump at redemption is zero: lifecycle value invariance (Section 1) holds. The note also exposes the settlement spectrum: bilateral CSA (well modelled), CCP margin (partially modelled, Section F.2), and CSD settlement (essentially unmodelled). CDM v6.0.0 has no `CSDSettlement` type, so settlement through Euroclear or Clearstream is carried in generic `executionDetails`. The gradient reflects CDM’s OTC-derivative origin.

F.4 Tokenised Securities: Double-Counting and the Flat Custodian

Where the structured note tested the product-type boundary, a tokenised equity tests custody. In the mirror/backed model a regulated custodian holds real shares in segregation and issues on-chain tokens 1:1. Each token is a beneficial claim on one share, held against the custodian. The underlying exposure, dividends, and corporate actions are unchanged. Four things are new: the identifier scheme (contract address, chain ID, token standard), the custodial layer, the settlement (on-chain atomic vs. T+1), and fractional divisibility.

Two units, not one. The share (NVDA) and the token (NVDA_TOKEN) are distinct units in the Unit Store: different settlement mechanisms, different identifiers, and different risk profiles. The token carries custodian credit risk the share does not. Treating them as one unit would erase the distinction between a holder of real shares and a holder of tokenised claims.

The double-counting problem and its resolution. Distinct units invite a naïve aggregation of NVDA exposure that double-counts: 1,000,000 real shares in custody plus 1,000,000 tokens held by investors sums to 2,000,000, though only 1,000,000 shares of exposure exist. The position table nets the double-count away: the custodian’s token liability cancels its share holding.

Entity	own(NVDA)	own(NVDA_TOKEN)	Net NVDA exposure
Custodian	+1,000,000	−1,000,000	0
Holders (all)	0	+1,000,000	+1,000,000
System	+1,000,000	0	+1,000,000

The zero in the custodian row is not an accident of the example; it holds in any backed structure.

Principle F.2 (The custodian is flat). *A backing custodian's net economic exposure is zero by conservation, not by choice. It is long the underlying and short the tokens it issued in equal measure: $\text{own}(\text{NVDA}) = +1,000,000$ and $\text{own}(\text{NVDA_TOKEN}) = -1,000,000$, so a unit price move changes asset and liability identically.*

Economic exposure is the read-time projection $\text{own}(\text{NVDA}) + \text{own}(\text{NVDA_TOKEN})$ for any entity: zero for the custodian, +100 for a holder of 100 tokens. The token balance conserves to zero across issuer and holders; the share balance conserves against the virtual issuance wallet. If the custodian issued tokens without holding shares, the shortfall would surface as a conservation violation. The resolution is a consequence of system closure, not a modelling convention. The structure is the securities-lending custody problem (Section 16): possession is not ownership. SBL distinguishes them by the position vector's **own** and **onloan** coordinates; tokenisation distinguishes them by liability netting. The principle is identical.

Variants. Two cases avoid the problem because no backing inventory exists. A *synthetic* token is a delta-one derivative on the underlier: a **NonTransferableProduct** with a **PerformancePayout**, no custodian. A *native-issuance* token is the share on chain: a **TransferableProduct** with blockchain-native identifiers. The mirror token is likewise a **TransferableProduct**: the first the walkthrough presents, and the only one with a non-trivial custody layer.

Four CDM v6.0.0 gaps for tokenised securities. Each entry names the missing type, the workaround, and the severity.

1. **No blockchain identifier type.** **ProductIdentifier/ProductIdSourceEnum** omit contract address, chain ID, and token standard. Workaround: **Other** with a scheme convention. Significant: every token needs an ad-hoc convention.
2. **No custodial backing model.** No type relates a token to the underlying held by a custodian; backing ratio, custodian identity, and segregation are not representable. Workaround: trade-level **contractDetails**. Significant: the custodial relationship is the defining feature.
3. **DigitalAsset underspecified.** The branch exists but neither extends **TransferableProduct** nor carries **EconomicTerms**; built for crypto-native assets, not tokenised securities. Moderate.
4. **No on-chain settlement type.** **SettlementTypeEnum** has no on-chain atomic value and no structure for chain, confirmation depth, or gas. Workaround: **Other**. Moderate.

The gaps are incremental extensions, consistent with CDM's trajectory from bilateral OTC to cleared to tokenised; none requires an architectural change. On-chain delivery-versus-payment, when both legs settle on chain, is the strongest DvP: atomic within one transaction, it eliminates settlement risk. Mixed on/off-chain settlement degrades to conventional DvP risk, a settlement-layer concern (Section 13), not a ledger-layer one.

F.5 Date Handling: Types, Resolution, and Gotchas

A contract date is never a bare date: it carries adjustment rules, calendar references, and resolution logic that fix the operational date. CDM composes these from small precise types in the **base.datetime** namespace. The types feed every dated computation in this document: the IRS move schedule and day-count fractions (Section 6), coupon adjustment (Section 15.4), and settlement-date adjustment (Section 13). A wrong calendar, convention, or day count is a valuation error, not a rounding error.

F.5.1 The Type Taxonomy

The taxonomy runs from resolved output to composable inputs: **date** is the only resolved value; every other row specifies how to reach one.

Type / enum	Role
date	Resolved ISO 8601 calendar date; the output, not the input.
Period	Multiplier + unit (D/W/M/Y); a zero-length period must be in days.
Offset	Period + dayType (Business/Calendar/Exchange...); dayType present only when in days.
BusinessDayConventionEnum	FOLLOWING, MODFOLLOWING, PRECEDING, NEAREST, NONE...
BusinessCenters	Holiday calendars (USNY, GBLO, JPTO...); a date is good only if good in <i>all</i> listed.
BusinessDayAdjustments	Convention + business centres: the reusable adjustment recipe.
AdjustableDate	unadjustedDate (contractual) + adjustments → adjustedDate (operational).
RelativeDateOffset	“ <i>n</i> days/months from a reference date, adjusted by <i>C</i> against <i>B</i> .”
AdjustableOrRelativeDate	Required-choice union of the two above; used throughout the product model.
CalculationPeriodFrequency	Frequency + rollConvention (EOM, IMM, _1.._30); drives schedule generation.
DayCountFractionEnum	ACT/360, ACT/365F, 30/360, ACT/ACT (ISDA/ICMA)...; each a dispatched YearFraction .

MODFOLLOWING dominates OTC derivatives: it advances to the next business day unless that crosses a month boundary, then retreats. This keeps the payment date aligned with its calculation period. Two day-count formulas recur. ACT/360 is actual days/360; 30/360 (Bond Basis) is

$$\frac{360(Y_2 - Y_1) + 30(M_2 - M_1) + (D_2 - D_1)}{360}, \quad D_1 = \min(d_1, 30), D_2 = \min(d_2, 30) \text{ if } D_1 > 29.$$

F.5.2 The Resolution Chain

An **AdjustableOrRelativeDate** resolves deterministically in three steps; **ResolveAdjustableDate** encapsulates them.

1. **Resolve the reference.** If relative, offset from the anchor (**dateRelativeTo**): Add **BusinessDays** when **dayType** = **Business**, calendar arithmetic otherwise. Yields the unadjusted derived date.
2. **Apply business-day adjustment.** Test the unadjusted date against the centres; if not a good business day, shift by the convention.
3. **The adjusted date is final:** the operational date for the cash flow, exercise, or settlement.

A 5Y USD SOFR swap rolling on the 3rd under MODFOLLOWING/USNY illustrates the chain: 2026-10-03 (Sat) → Mon Oct 5; 2027-10-03 (Sun) → FOLLOWING within month → Mon Oct 4; weekday roll dates pass unadjusted. The first fixed period (2026-04-03 to 2026-10-05, 30/360) gives $\delta = (30 \cdot 6 + 2)/360 = 182/360 \approx 0.5056$. The first floating period (2026-04-03 to 2026-07-03, ACT/360) gives $91/360 \approx 0.2528$. These adjusted dates and fractions are exactly the reset dates t_k and δ_k the IRS contract consumes (Section 6).

F.5.3 Gotchas

Each is a bug class the type system prevents only when the types are used correctly.

1. **Omitted business centres.** A convention with no centres adjusts against no calendar: every day is a business day and the convention is inert. Validate that any non-NONE convention has at least one centre.
2. **Calendar vs. business days.** “T+2” from a Friday is Sunday under Calendar, Tuesday under Business. `dayType` is load-bearing, not decoration.
3. **EOM vs. fixed-day roll.** EOM rolls to each month’s last day (Feb 28/29, Mar 31); `_28` always rolls to the 28th. Confusing them shifts payments by up to three days.
4. **Joint-holiday intersection.** Multiple centres take the *intersection* of business days: good in New York but a London holiday is not good for USNY+GBLO. More dates are non-business under joint calendars than either alone.
5. **Stub periods.** An effective date off the roll cycle produces a short or long first period; `balanceOfFirstPeriod` places the stub. Ignoring it gives a wrong year fraction for that period.
6. **Day-count mismatch.** For a 182-day semi-annual period, ACT/360 gives 0.5056 and 30/360 gives 0.5000. On EUR/USD 50,000,000 at 4%, the difference is $0.005556 \times 0.04 \times 50,000,000 \approx \$11,111$: a valuation error, not a rounding error.

G Available Inventory Identity: Eight-Scenario Verification

The available-inventory identity (P20, 16.15)

$$\text{avail} = \text{own} - \text{onloan} + \text{borr}$$

is verified across eight scenarios. Each scenario exercises a distinct combination of the six stored coordinates: `own`, `onl`, `borr`, `c_p` (`coll_post`), `c_r` (`coll_recv`), `c_rh` (`coll_rehyp`). Available inventory is never stored. It is computed on read from three of those coordinates: `own`, `onl`, `borr`. The identity cannot be violated by construction.

Each row below states the stored coordinates for one entity in one scenario. The final column recomputes available inventory from those coordinates and checks the result.

Entity (unit)	own	onl	borr	c_p	c_r	c_rh	avail = own - onl + borr
<i>A. Alice buys 1,000 AAPL outright.</i>							
Alice (AAPL)	1000	0	0	0	0	0	$1000 - 0 + 0 = 1000$ ✓
Virtual	-1000	—	—	—	—	—	(issuance counterpart)
<i>B. Alice owns 1,000 VOD, lends 400 to Bob.</i>							
Alice (VOD)	1000	400	0	0	0	0	$1000 - 400 + 0 = 600$ ✓
Bob (VOD)	0	0	400	0	0	0	$0 - 0 + 400 = 400$ ✓
<i>C. Bob lends 1,000 NVDA to Alice, Alice sells 500.</i>							
Bob (NVDA)	1000	1000	0	0	0	0	$1000 - 1000 + 0 = 0$ ✓
Alice (NVDA)	-500	0	1000	0	0	0	$-500 - 0 + 1000 = 500$ ✓
<i>D. Swap desk holds 5Y EUR IRS (non-lendable).</i>							
Desk (IRS)	+1	0	0	0	0	0	N/A (non-lendable; own = 1)
<i>E. Bob lends 1,000 NVDA, Alice sells all 1,000.</i>							
Bob (NVDA)	1000	1000	0	0	0	0	$1000 - 1000 + 0 = 0$ ✓
Alice (NVDA)	-1000	0	1000	0	0	0	$-1000 - 0 + 1000 = 0$ ✓
<i>F. Alice holds 1,000 AAPL, lends 400 to Bob, Bob sells 200.</i>							
Alice (AAPL)	1000	400	0	0	0	0	$1000 - 400 + 0 = 600$ ✓
Bob (AAPL)	-200	0	400	0	0	0	$-200 - 0 + 400 = 200$ ✓
<i>G. On-lending chain: Alice lends 600 MSFT to Bob, Bob re-lends 300 to Charlie.</i>							

Entity (unit)	own	onl	borr	c_p	c_r	c_rh	avail = own - onl + borr
Alice (MSFT)	1000	600	0	0	0	0	$1000 - 600 + 0 = 400$ ✓
Bob (MSFT)	0	300	600	0	0	0	$0 - 300 + 600 = 300$ ✓
Charlie (MSFT)	0	0	300	0	0	0	$0 - 0 + 300 = 300$ ✓
<i>H. Alice lends 400 AAPL to Bob under security interest; Bob posts 200 MSFT as pledge.</i>							
Bob (MSFT)	0	0	0	200	0	0	$0 - 0 + 0 = 0$ ✓
Alice (MSFT)	0	0	0	0	200	0	$0 - 0 + 0 = 0$ ✓

Scenario G is the case that separates the correct projection from its naive alternative.

Remark G.1 (On-lending defeats the naive identity). *For intermediary re-lender Bob (G), $\text{own} = 0 \neq \text{avail} + \text{onl} = 300 + 300$: the identity $\text{own} = \text{avail} + \text{onloan}$ fails. The correct projection $\text{avail} = \text{own} - \text{onl} + \text{borr}$ holds because it credits borrowed shares.*

Scenario H shows where the collateral regime becomes visible: not in available inventory, but in profit and loss.

Remark G.2 (Pledge preserves economic ownership). *Under pledge (H), Bob retains economic ownership of posted collateral; the regime difference surfaces in PnL projection, where Bob's MSFT PnL is $c_p(200) \times P$, not $\text{own}(0) \times P$.*

H European SBL Worked Example: GMSLA 2010 Title Transfer with Rehypothecation

This appendix works a complete European securities loan under GMSLA 2010 title transfer. The lifecycle covers non-cash collateral, rehypothecation, daily mark-to-market, a manufactured dividend, a partial recall, and full return. Every move is a single-coordinate write of the six-coordinate position vector (Section 16, Principle 16.1). Conservation is verified at each step. The example is the title-transfer correctness witness; the pledge counterpart is Appendix I.

H.1 Participants and Setup

Five entities take part: a beneficial owner lending through an agent, a prime broker borrowing, the broker's short-selling client, the open market, and a third party that receives re-used collateral.

Entity	Role	Description
CALPERS_EU	Lender (beneficial owner)	Holds 500,000 TotalEnergies (ISIN FR0000120271). Agent lender BNY Mellon.
GOLDMAN	Borrower	Prime broker; borrows TTE to cover a client short.
HEDGE_FUND	Goldman's client	Short seller; sells the borrowed TTE to market.
MARKET	Market buyer	Virtual wallet for the open market.
COUNTERPARTY_C	Third party	Receives rehypothecated Bund collateral in a separate repo.

Loan terms. 500,000 TotalEnergies (TTE), open loan, fee 25 bps p.a. (ACT/360), non-cash collateral (German Bund, ISIN DE0001102580), margin 102%, GMSLA 2010 title transfer. Under title transfer Goldman takes ownership of the Bund collateral and may rehypothecate it. SFTR Article 15 documentation is required.

Units. TTE (equity), BUND (German Bund), EUR (cash). Position vector (own, onloan, borr, coll_post, coll_recv, coll_rehyp), abbreviated own, onl, borr, c_p, c_r, c_rh.

Remark H.1. *Loan, collateral, and exposure values follow the valuation rules of the valuation satellite specification (IBP-163, IBP-129, IBP-191); the figures are stated here as inputs, not re-derived. Intermediate quantities are computed in exact integer minor units; round-half-to-even (IEEE 754) applies only at the settlement-instruction projection boundary.*

H.2 Step 1: Loan Initiation (Day 0)

TTE €58.50; Bund €98.50 per €100 nominal. The collateral requirement is the loan value grossed up by the 102% margin, converted into Bund units and rounded up.

$$LV = 500,000 \times 58.50 = \text{€}29,250,000, \quad RCV = LV \times 102\% = \text{€}29,835,000.$$

$$Q_{\text{Bund}} = \left\lceil \frac{29,835,000}{98.50} \right\rceil = 302,894 \text{ (units of €100 nominal),}$$

market value $302,894 \times 98.50 = \text{€}29,835,059$. Rounding up leaves slight over-collateralisation.

The initiation transaction has two legs. The securities leg records the loan; the collateral leg transfers Bund ownership from Goldman to CalPERS.

```
tau_1 = Transaction(type = SETTLEMENT, day = 0):
    -- Securities leg (TTE)
    Move 1: CALPERS_EU  onloan  += 500,000  [onloan]
    Move 2: GOLDMAN     borr     += 500,000  [borr]
    -- Collateral leg (BUND) -- title transfer
    Move 3: GOLDMAN     own      -= 302,894  [own]
    Move 4: CALPERS_EU  coll_recv += 302,894  [coll_recv]
```

Move 3 writes own, not coll_post. Under title transfer the poster transfers *ownership*: Goldman's Bund ownership decreases, and CalPERS holds the Bunds as received collateral. This is the defining distinction from the pledge regime (Appendix I). Virtual wallets absorb the contra-entries for both legs.

Positions after τ_1 :

Entity	Unit	own	onl	borr	c_p	c_r	c_rh	avail
CALPERS_EU	TTE	500,000	500,000	0	0	0	0	0
CALPERS_EU	BUND	0	0	0	0	302,894	0	0
GOLDMAN	TTE	0	0	500,000	0	0	0	500,000
GOLDMAN	BUND	-302,894	0	0	0	0	0	-302,894
Virtual _{C↔G}	TTE	-500,000 (securities delivery contra)						
Virtual _{mkt}	BUND	+302,894 (Bund delivery contra)						

Conservation (TTE). $500,000_C + 500,000_{G,\text{borr}} - 500,000_{\text{Virt}} - 500,000_{\text{Cust}} = 0$. ✓ **P18.** CalPERS own(TTE) = 500,000, unchanged. ✓ **SFTR.** NEWT.

H.3 Step 2: Delivery to Hedge Fund and Short Sale (Day 0)

Locate (EU SSR Article 12(1)). Before the short sale HEDGE_FUND satisfies the locate obligation. The borrow from CalPERS meets Article 12(1)(a) (the person “has borrowed the share”). Locate is a pre-trade validation the executor checks before τ_3 . It generates no moves and changes no coordinate.

Goldman delivers the borrowed shares to the hedge fund, which sells them to the market for cash. Every move in both transactions writes own.

```

tau_2 = Transaction(type = SETTLEMENT, day = 0):  -- Goldman -> Hedge Fund
    Move 1: GOLDMAN      own -= 500,000      [own]
    Move 2: HEDGE_FUND   own += 500,000      [own]

tau_3 = Transaction(type = SETTLEMENT, day = 0):  -- Hedge Fund short sale to
    market
    Move 1: HEDGE_FUND   own -= 500,000      [own]
    Move 2: MARKET       own += 500,000      [own]
    Move 3: MARKET       own -= 29,250,000    [own, EUR]
    Move 4: HEDGE_FUND   own += 29,250,000    [own, EUR]

```

Goldman's borr is untouched: it tracks the return obligation, not possession.

Positions after τ_3 :

Entity	Unit	own	onl	borr	c_p	c_r	c_rh	avail
CALPERS_EU	TTE	500,000	500,000	0	0	0	0	0
GOLDMAN	TTE	-500,000	0	500,000	0	0	0	0
HEDGE_FUND	TTE	0	0	0	0	0	0	0
MARKET	TTE	500,000	0	0	0	0	0	500,000
HEDGE_FUND	EUR	29,250,000	0	0	0	0	0	29,250,000

Goldman is flat in available shares ($-500,000 + 500,000_{\text{borr}} = 0$) but short in ownership. The economic short passed through from the hedge fund.

Conservation (TTE). $500,000_C - 500,000_G + 500,000_M - 500,000_{\text{Virt}} = 0$. ✓

H.4 Step 3: Rehypothecation of Bund Collateral (Day 1)

Goldman owns the Bunds under title transfer and re-uses 302,894 of them as collateral in a separate repo with COUNTERPARTY_C. P19 (rehypothecation regime compliance) requires SFTR Article 15 re-use documentation. The documentation is an operational compliance flag, not a move.

The re-use is recorded in two transactions: a reclassification inside CalPERS's vector, and the onward delivery of the Bunds against cash.

```

tau_4a = Transaction(type = RECLASSIFICATION, day = 1):  -- within CalPERS vector
    Move 1: CALPERS_EU   coll_recv -= 302,894    [coll_recv, BUND]
    Move 2: CALPERS_EU   coll_rehyp += 302,894    [coll_rehyp, BUND]

tau_4b = Transaction(type = SETTLEMENT, day = 1):  -- onward delivery to
    COUNTERPARTY_C
    Move 1: GOLDMAN      own -= 302,894          [own, BUND]
    Move 2: COUNTERPARTY_C own += 302,894          [own, BUND]
    Move 3: COUNTERPARTY_C own -= 29,835,059      [own, EUR]
    Move 4: GOLDMAN      own += 29,835,059      [own, EUR]

```

Goldman's Bund own moves $-302,894 \rightarrow -605,788$. Goldman is doubly short: it has delivered Bunds to CalPERS and to COUNTERPARTY_C and owes both. The reclassification records the re-use inside CalPERS's vector without changing CalPERS's total Bund holding.

Positions after τ_{4a} and τ_{4b} :

Entity	Unit	own	onl	borr	c_p	c_r	c_rh	avail
CALPERS_EU	BUND	0	0	0	0	0	302,894	0
GOLDMAN	BUND	-605,788	0	0	0	0	0	-605,788
COUNTERPARTY_C	BUND	302,894	0	0	0	0	0	302,894
GOLDMAN	EUR	29,835,059	0	0	0	0	0	29,835,059
COUNTERPARTY_C	EUR	-29,835,059	0	0	0	0	0	-29,835,059

Conservation (BUND). $302,894_C - 605,788_G + 302,894_{CC} + 302,894_{\text{Virt}} - 302,894_{\text{Cust}} = 0$.
✓ **P19.** Regime TITLE_TRANSFER permits rehypothecation; Art. 15 flag set. ✓

H.5 Step 4: Mark-to-Market (Day 3)

TTE €60.25; Bund €98.20.

$$LV_3 = 500,000 \times 60.25 \times 102\% = €30,727,500, \quad \text{collateral} = 302,894 \times 98.20 = €29,744,189.$$

$$\text{Exposure} = 30,727,500 - 29,744,189 = €983,311, \quad Q_{\text{add}} = \left\lceil \frac{983,311}{98.20} \right\rceil = 10,014 \text{ units.}$$

The lender is under-collateralised. A margin call delivers additional Bunds.

```
tau_5 = Transaction(type = MARGIN_CALL, day = 3):
    Move 1: GOLDMAN      own      -= 10,014    [own, BUND]
    Move 2: CALPERS_EU   coll_recv += 10,014    [coll_recv, BUND]
```

The addition enters coll_recv (not coll_rehyp): it is not yet re-used.

Positions after τ_5 :

Entity	Unit	own	onl	borr	c_p	c_r	c_rh
CALPERS_EU	BUND	0	0	0	0	10,014	302,894
GOLDMAN	BUND	-615,802	0	0	0	0	0
COUNTERPARTY_C	BUND	302,894	0	0	0	0	0

P13. CalPERS holds $312,908 \text{ Bunds} \times 98.20 = €30,727,566 \geq €30,727,500$ required. ✓ **Fee accrual (Days 0–3).** Day 0 base $29,250,000 \times 0.0025/360 = €203.13/\text{day}$; from Day 3, $30,125,000 \times 0.0025/360 = €209.20/\text{day}$. Running total €818.58. **SFTR.** VALU.

H.6 Step 5: Manufactured Dividend (Day 7)

TotalEnergies pays €1.25 per share. CalPERS retains economic ownership, so Goldman pays a manufactured dividend of $500,000 \times 1.25 = €625,000$. The manufactured payment is a contractual GMSLA payment, not a distribution. Its net amount may differ from the gross dividend under the applicable withholding-tax treaty.

```
tau_6 = Transaction(type = CORP_ACTION, day = 7):
    Move 1: GOLDMAN      own      -= 625,000    [own, EUR]
    Move 2: CALPERS_EU   own      += 625,000    [own, EUR]
```

EUR positions after τ_6 :

Entity (EUR)	own	onl	borr	c_p	c_r	c_rh
CALPERS_EU	625,000	0	0	0	0	0
GOLDMAN	29,210,059	0	0	0	0	0

Conservation (EUR). $625,000_C + 29,210,059_G + 29,250,000_{HF} - 29,835,059_{CC} + \text{virtual offsets} = 0$. ✓

H.7 Step 6: Partial Recall and Return (Days 10–12)

CalPERS sells 200,000 TTE, so BNY Mellon recalls 200,000 shares. The recall notice (IBP-328) reaches Goldman on Day 10. Settlement follows at $T + 2$ (Euronext Paris) on Day 12.

The recall itself moves nothing; it changes only the loan state.

```
tau_7 = Transaction(type = RECALL, day = 10):
    -- Move-less: empty move list (vacuously conserved); carries only a state delta.
    -- SBL state ACTIVE -> RECALLED; recall_qty = 200,000; recall_date = Day 10
```

Day 12. TTE €59.80; Bund €98.30. Remaining loan 300,000:

$$LV_{12} = 300,000 \times 59.80 \times 102\% = €18,298,800.$$

Collateral held $312,908 \times 98.30 = €30,758,856$; excess €12,460,056:

$$Q_{\text{release}} = \left\lfloor \frac{12,460,056}{98.30} \right\rfloor = 126,755 \text{ Bund units.}$$

Remaining $312,908 - 126,755 = 186,153$ Bunds, value $186,153 \times 98.30 = €18,298,835 \geq LV_{12}$ (P13 ✓). Release draws from coll_recv first (10,014), then coll_rehyp (116,741).

The partial return reverses 200,000 of the loan and releases the excess collateral.

```
tau_8 = Transaction(type = PARTIAL_RETURN, day = 12):
    -- Securities return (TTE)
    Move 1: GOLDMAN      borr      -= 200,000    [borr]
    Move 2: CALPERS_EU   onloan     -= 200,000    [onloan]
    -- Collateral release (BUND): coll_recv then coll_rehyp
    Move 3: CALPERS_EU   coll_recv  -= 10,014     [coll_recv]
    Move 4: GOLDMAN      own        += 10,014     [own]
    Move 5: CALPERS_EU   coll_rehyp -= 116,741    [coll_rehyp]
    Move 6: GOLDMAN      own        += 116,741    [own]

tau_8b = Transaction(type = SETTLEMENT, day = 12): -- Goldman unwinds repo leg
    Move 1: COUNTERPARTY_C own -= 116,741      [own, BUND]
    Move 2: GOLDMAN      own += 116,741        [own, BUND]
    Move 3: GOLDMAN      own -= 11,473,439     [own, EUR]
    Move 4: COUNTERPARTY_C own += 11,473,439    [own, EUR]
```

Releasing 116,741 from coll_rehyp forces Goldman to recall those Bunds from COUNTERPARTY_C first (τ_{8b}).

Positions after τ_8 and τ_{8b} :

Entity	Unit	own	onl	borr	c_p	c_r	c_rh	avail
CALPERS_EU	TTE	500,000	300,000	0	0	0	0	200,000
GOLDMAN	TTE	-500,000	0	300,000	0	0	0	-200,000
CALPERS_EU	BUND	0	0	0	0	0	186,153	0
GOLDMAN	BUND	-372,306	0	0	0	0	0	-372,306
COUNTERPARTY_C	BUND	186,153	0	0	0	0	0	186,153

P18. CalPERS own(TTE) = 500,000, unchanged. ✓ **P15.** Loan quantity 500,000 → 300,000, monotonically non-increasing. ✓ **SFTR.** MODI (not ETRM: partial, not full return).

H.8 Step 7: Full Return and Settlement (Day 20)

Remaining 300,000 TTE returned. TTE €61.00; Bund €98.40.

The full return clears the remaining loan and collateral coordinates and unwinds the rest of the repo.

```
tau_9 = Transaction(type = FULL_RETURN, day = 20):
    Move 1: GOLDMAN      borr      -= 300,000    [borr]
    Move 2: CALPERS_EU   onloan     -= 300,000    [onloan]
    Move 3: CALPERS_EU   coll_rehyp -= 186,153    [coll_rehyp]
    Move 4: GOLDMAN      own        += 186,153    [own]

tau_9b = Transaction(type = SETTLEMENT, day = 20): -- unwind remaining repo
    Move 1: COUNTERPARTY_C own -= 186,153      [own, BUND]
    Move 2: GOLDMAN      own += 186,153        [own, BUND]
    Move 3: GOLDMAN      own -= 18,308,845     [own, EUR]
    Move 4: COUNTERPARTY_C own += 18,308,845    [own, EUR]
```

Fee. Accrued daily on the mark-to-market loan value over 20 days:

Period	Qty	Loan value (no margin)	Daily fee
Days 0–2	500,000	€29,250,000	€203.13
Days 3–6	500,000	€30,125,000	€209.20
Days 7–11	500,000	various	≈€207.64 avg
Days 12–19	300,000	≈€18,000,000	≈€125.00 avg

Total accrued fee €3,548.40 (exact amount depends on daily closing prices). The fee settles as a single cash payment.

```
tau_10 = Transaction(type = FEE_SETTLEMENT, day = 20):
  Move 1: GOLDMAN      own -= 3,548.40   [own, EUR]
  Move 2: CALPERS_EU   own += 3,548.40   [own, EUR]
```

Loan disposal. $\vec{w}_{\text{CALPERS}}(u_{\text{loan}})[\text{own}]: +1 \rightarrow 0$; $\vec{w}_{\text{GOLDMAN}}(u_{\text{loan}})[\text{own}]: -1 \rightarrow 0$. SBL state RETURNED (terminal); SFTR ETRM.

Final positions:

Entity	Unit	own	onl	borr	c_p	c_r	c_rh	avail
CALPERS_EU	TTE	500,000	0	0	0	0	0	500,000
CALPERS_EU	BUND	0	0	0	0	0	0	0
CALPERS_EU	EUR	628,548	0	0	0	0	0	628,548
GOLDMAN	TTE	−500,000	0	0	0	0	0	−500,000
GOLDMAN	BUND	0	0	0	0	0	0	0
GOLDMAN	EUR	−575,773	0	0	0	0	0	−575,773
COUNTERPARTY_C	EUR	−52,775	0	0	0	0	0	−52,775
HEDGE_FUND	EUR	29,250,000	0	0	0	0	0	29,250,000
MARKET	TTE	500,000	0	0	0	0	0	500,000

Goldman retains $\text{own}(\text{TTE}) = -500,000$ from the delivery to the hedge fund, a relationship separate from the loan. The CalPERS loan is fully closed: all 500,000 TTE returned, 0 on loan.

Final conservation. TTE: $500,000_C - 500,000_G + 500,000_M - 500,000_{\text{Virt}} = 0 \checkmark$. BUND: all zero $\checkmark$. EUR: sums to zero across all entities including virtual wallets $\checkmark$.

PnL.

- **CalPERS:** €3,548.40 fee + €625,000 manufactured dividend; TTE position unchanged, unrealised $500,000 \times (61.00 - 58.50) = €1,250,000$.
- **Goldman:** €3,548.40 fee + €625,000 manufactured dividend paid; earned the repo spread on rehypothecated Bunds.
- **Hedge Fund:** short PnL $(58.50 - 61.00) \times 500,000 = -€1,250,000$; must buy back at €61.00 to close.

I US SBL Worked Example: SEC Rule 15c3-3 with Cash Collateral

This appendix is a regulatory correctness witness for the US securities-lending regime. It covers cash collateral, rebate mechanics, and the rehypothecation cap of SEC Rule 15c3-3 [2], and maps FINRA SLATE reporting at each step. It is the counterpart to the European title-transfer example (App. H). Title transfer is not pledge. The two regimes exercise the same Ledger primitives under different law.

I.1 Participants and Setup

Three parties take part: a mutual fund lends, a broker-dealer borrows, and an agent lender manages the loan for the fund.

Entity	Role	Description
FIDELITY	Lender (beneficial owner)	US mutual fund. Holds 1,000,000 shares of NVIDIA (NVDA, ISIN US67066G1040). Agent lender: State Street.
MORGAN_STANLEY	Borrower	US broker-dealer. Borrows NVDA to cover a client short. Subject to Rule 15c3-3.
STATE_STREET	Agent lender	Acts for FIDELITY; negotiates and manages the loan.

Loan terms. 1,000,000 NVDA, open loan, cash collateral (USD), margin 102%. Cash rebate: Fed Funds Effective Rate minus 25 bps. Take Fed Funds = 5.25%, so rebate rate = 5.00%. The lender reinvests the cash collateral and earns the spread between reinvestment return and rebate paid. As in App. H, intermediate quantities are computed in exact integer minor units. The fractional cash figures shown below (rebates, spreads) are display roundings; round-half-to-even (IEEE 754) is applied only at the settlement-instruction projection boundary.

Legal regime US_15C3_3. The regime fixes three things: where the cash goes, how much may be rehypothecated, and how posted collateral enters PnL.

- Cash collateral transfers from Morgan Stanley to Fidelity/State Street, who reinvest it.
- Rehypothecation of customer securities is capped at 140% of the customer’s debit balance (Rule 15c3-3(b)(3)).
- The poster’s PnL includes coll_post: $V_e^{15c3} = \sum_u (\vec{w}_e(u)[\text{own}] + \vec{w}_e(u)[\text{coll_post}]) \cdot P_t(u)$.

Locate precondition (Reg SHO Rule 203(b)(1)) [3]. Before the client’s short sale executes, Morgan Stanley satisfies the locate obligation, either by holding the borrow or by a locate confirmation against inventory. The locate is pre-trade validation. No move occurs, no coordinate changes, and conservation is not engaged until the borrow executes.

I.2 Step 1: Loan Initiation (Day 0)

NVDA price \$875.00. Loan value and required collateral:

$$LV = 1,000,000 \times 875 = \$875,000,000, \quad RCV = 875,000,000 \times 102\% = \$892,500,000.$$

Cash is always owned. The collateral leg writes own on the poster and coll_recv on the receiver; coll_recv is the earmark that must be returned at close.

Each τ below is a single Transaction carrying moves, with **txIntroduce** = NOTHING and **txAppend** = NOTHING: neither a registration nor an amendment. Its move list is the conserved flow, so conservation holds by construction from the signed edge-sum. The trailing comment names the economic role; it is not a type field on the core Transaction.

Initiation is one transaction with two conserved legs: the shares go on loan, and the cash collateral moves against them.

```

tau_1 = Transaction(day 0):  -- securities-loan settlement
    -- Securities leg (unit: NVDA)
    Move 1: FIDELITY          onloan    += 1,000,000    [onloan]
    Move 2: MORGAN_STANLEY    borrr      += 1,000,000    [borrr]
    -- Cash collateral leg (unit: USD)
    Move 3: MORGAN_STANLEY    own        -= 892,500,000  [own]
    Move 4: FIDELITY          coll_recv += 892,500,000  [coll_recv]
```

Units. NVDA (equity), USD (cash). Position vector

(own, onloan, borr, coll_post, coll_recv, coll_rehyp),

abbreviated own, onl, borr, c_p, c_r, c_rh.

Fidelity keeps ownership of the shares and gains the cash earmark; Morgan Stanley records the borrow and the cash outflow.

Entity	Unit	own	onl	borr	c_p	c_r	c_rh	avail
FIDELITY	NVDA	1,000,000	1,000,000	0	0	0	0	0
FIDELITY	USD	0	0	0	0	892,500,000	0	0
MORGAN_STANLEY	NVDA	0	0	1,000,000	0	0	0	1,000,000
MORGAN_STANLEY	USD	-892,500,000	0	0	0	0	0	-892,500,000

P18 check. Fidelity $\vec{w}(\text{NVDA})[\text{own}] = 1,000,000$, unchanged. ✓ **SLATE.** New Loan Event: FINRA Loan ID, Client Loan ID, execution timestamp, ISIN/CUSIP, quantity, collateral type (Cash), collateral amount, rebate rate, term type (Open), borrower/lender LEI.

I.3 Step 2: Rebate Mechanics and Cash Reinvestment

Fidelity reinvests the \$892,500,000 in approved money-market instruments. It earns the money-market yield (take = 5.25%) and pays Morgan Stanley the rebate (5.00%). The reinvestment transforms the cash earmark into MMF units (see §16.12):

```
tau_reinvest = Transaction(day 0):  -- cash reinvestment
    -- Deploys the cash into MMF units; the USD coll_recv tables below
    -- carry the unchanged return obligation, with the MMF overlay separate.
Move 1: FIDELITY coll_recv -= 892,500,000 [coll_recv, USD]
Move 2: FIDELITY own      += 892,500      [own, MMF_UNIT]
Move 3: MMF_FUND own      -= 892,500      [own, MMF_UNIT]
Move 4: MMF_FUND own      += 892,500,000  [own, USD]
```

The reinvestment deploys the cash, but the \$892.5M return obligation persists in the loan unit state (**collateral_amount**): the collateral must be returned in full at close. The position-vector tables that follow therefore carry coll_recv (USD) at the collateral *obligation* amount throughout — the cash-equivalent Fidelity owes back. The 892,500 MMF units are held and tracked in a separate unit. The margin-call (Step 4) and release (Steps 6–7) moves accordingly act on the obligation amount shown. Fidelity now bears reinvestment risk. Conservation holds in each unit independently.

Daily economics. The lender’s income is the spread between what the cash earns and the rebate it pays.

$$\text{Reinvestment income} = 892,500,000 \times 5.25\% / 360 = \$130,104.17$$

$$\text{Rebate payable} = 892,500,000 \times 5.00\% / 360 = \$123,958.33$$

$$\text{Lending spread} = 130,104.17 - 123,958.33 = \$6,145.83$$

Billing here is monthly. Rebate at Day 30: $892,500,000 \times 5.00\% \times 30 / 360 = \$3,718,750$.

```
tau_rebate = Transaction(day 30):  -- rebate (fee settlement)
Move 1: FIDELITY      own -= 3,718,750 [own, USD]
Move 2: MORGAN_STANLEY own += 3,718,750 [own, USD]
```

Reinvestment over the month = \$3,903,906; rebate paid = \$3,718,750; net = \$185,156, the lending spread.

I.4 Step 3: Rehypothecation Constraints under Rule 15c3-3

Rule 15c3-3(b)(3) permits Morgan Stanley to use customer securities only up to 140% of the customer's aggregate debit balance. The contrast with the title-transfer regime of App. H is legal: who owns the collateral determines what the receiver may do with it.

	European (GMSLA TT)	2010	US (Rule 15c3-3)
Rehypothecation	Receiver rehypothecates collateral freely	collat-	Borrower faces a regulatory cap
Legal basis	Title transfer: receiver <i>owns</i> collateral		Pledge/control: customer retains ownership interest
Cap	None (SFTR Art. 15 documentation only)		140% of customer debit balance

Pre-condition enforcement (P19). The SBL contract validates a rehypothecation request before any move commits. The `legal_regime` field selects the path:

```

validate_rehypothecation(entity, unit, quantity, regime):
    if regime == US_15C3_3:
        max_rehyp = customer_debit_balance(entity) * 1.40
        current_rehyp = sum(entity.coll_rehyp[u] for u in units)
        if current_rehyp + quantity * price(unit) > max_rehyp:
            REJECT("15c3-3 rehypothecation cap exceeded")
    elif regime == TITLE_TRANSFER:
        if not sftr_art15_documented(entity): REJECT("SFTR Art 15 missing")
        PERMIT()

```

A request exceeding the cap is rejected, never committed. P19 is a pre-condition, so the illegal state is unreachable rather than reached and rolled back.

I.5 Step 4: Daily Mark-to-Market (Day 5)

NVDA rises to \$910.00.

$$RCV_5 = 1,000,000 \times 910 \times 102\% = \$928,200,000,$$

$$\Delta C = 928,200,000 - 892,500,000 = \$35,700,000.$$

The price rise raises the required collateral; Morgan Stanley posts the shortfall.

```

tau_2 = Transaction(day 5): -- margin call
    Move 1: MORGAN_STANLEY own      -= 35,700,000    [own, USD]
    Move 2: FIDELITY      coll_recv += 35,700,000    [coll_recv, USD]

```

Entity	Unit	own	onl	borr	c_p	c_r	c_rh
FIDELITY	NVDA	1,000,000	1,000,000	0	0	0	0
FIDELITY	USD	0	0	0	0	928,200,000	0
MORGAN_STANLEY	NVDA	0	0	1,000,000	0	0	0
MORGAN_STANLEY	USD	-928,200,000	0	0	0	0	0

P13 check. Collateral held = \$928,200,000 = required. ✓ **SLATE.** Modification: collateral amount and value updated.

I.6 Step 5: FINRA SLATE Reporting

Rule 10c-1a [4] (adopted January 2024) and the FINRA Rule 6500 Series [5] (effective 2025) require every covered securities loan to be reported to SLATE. Each lifecycle event has one SLATE counterpart.

Lifecycle Event	SLATE Event	Key Data Elements
Loan initiation (Day 0)	New Loan	FINRA Loan ID, Client Loan ID, timestamp, ISIN US67066G1040, quantity 1,000,000, collateral type Cash, amount \$892.5M, rebate 5.00%, term Open, borrower/lender LEI
Margin call (Day 5)	Modification	Collateral amount $\rightarrow$ \$928.2M; timestamp
Partial return (Day 15)	Modification	Quantity $\rightarrow$ 500,000; collateral reduced
Full return (Day 25)	Modification	Quantity $\rightarrow$ 0; loan closed

Deadlines. Loans effected on a business day from 12:00am ET through 7:45pm ET report same day before 8:00pm ET. Loans after 7:45pm ET report next business day before 8:00pm ET.

Dual identifiers. Each loan carries a FINRA Loan ID (assigned by SLATE) and a Client Loan ID (the firm's loan_id in the SBL unit state).

Unsettled Loan Flag (Field #44). Set when the loan has not settled on the reporting day. The SBL settlement status (§16.7) maps directly: PENDING $\rightarrow$ Unsettled, ACTIVE $\rightarrow$ Settled.

Remark I.1. The SBL settlement status is a projection of the event log, rebuilt by replay, never an authoritative store; the lifecycle authority is §7.

I.7 Step 6: Partial Recall and Return (Day 15)

Fidelity recalls 500,000 shares; NVDA \$920.00.

$$\text{RCV}_{15} = 500,000 \times 920 \times 102\% = \$469,200,000,$$

$$C_{\text{release}} = 928,200,000 - 469,200,000 = \$459,000,000.$$

Half the loan unwinds: the shares return, and the collateral in excess of the remaining requirement is released.

```
tau_3 = Transaction(day 15): -- partial return
-- Securities return (unit: NVDA)
Move 1: MORGAN_STANLEY  borr      -= 500,000      [borr]
Move 2: FIDELITY         onloan    -= 500,000      [onloan]
-- Cash collateral release (unit: USD)
Move 3: FIDELITY         coll_recv -= 459,000,000 [coll_recv]
Move 4: MORGAN_STANLEY  own        += 459,000,000 [own]
```

Entity	Unit	own	onl	borr	c_p	c_r	c_rh	avail
FIDELITY	NVDA	1,000,000	500,000	0	0	0	0	500,000
FIDELITY	USD	0	0	0	0	469,200,000	0	0
MORGAN_STANLEY	NVDA	0	0	500,000	0	0	0	500,000
MORGAN_STANLEY	USD	-469,200,000	0	0	0	0	0	-469,200,000

P15 check. Loan quantity $1,000,000 \rightarrow 500,000$, monotonically non-increasing. ✓ **P18**

check. Fidelity $\vec{w}(\text{NVDA})[\text{own}] = 1,000,000$, unchanged. ✓

Pro-rated rebate on the returned portion: $459,000,000 \times 5.00\% \times 15/360 = \$956,250$.

```

tau_3b = Transaction(day 15):  -- pro-rated rebate (fee settlement)
    Move 1: FIDELITY           own -= 956,250    [own, USD]
    Move 2: MORGAN_STANLEY     own += 956,250    [own, USD]

```

I.8 Step 7: Full Return and Close-Out (Day 25)

Remaining 500,000 NVDA returned; NVDA \$905.00. The close-out mirrors the partial return: the remaining shares come back, and all remaining collateral is released.

```

tau_4 = Transaction(day 25):  -- full return
    -- Securities return (unit: NVDA)
    Move 1: MORGAN_STANLEY     borr   -= 500,000    [borr]
    Move 2: FIDELITY           onloan -= 500,000    [onloan]
    -- Cash collateral release (unit: USD)
    Move 3: FIDELITY           coll_recv -= 469,200,000 [coll_recv]
    Move 4: MORGAN_STANLEY     own      += 469,200,000 [own]

```

Final rebate: $469,200,000 \times 5.00\% \times 10/360 = \$651,666.67$.

```

tau_4b = Transaction(day 25):  -- final rebate (fee settlement)
    Move 1: FIDELITY           own -= 651,666.67    [own, USD]
    Move 2: MORGAN_STANLEY     own += 651,666.67    [own, USD]

```

The loan unit is disposed: $\vec{w}_F(u_{\text{loan}})[\text{own}] = \vec{w}_M(u_{\text{loan}})[\text{own}] = 0$; SBL state RETURNED.

Entity	Unit	own	onl	borr	c_p	c_r	c_rh	avail
FIDELITY	NVDA	1,000,000	0	0	0	0	0	1,000,000
FIDELITY	USD	-1,607,917	0	0	0	0	0	-1,607,917
MORGAN_STANLEY	NVDA	0	0	0	0	0	0	0
MORGAN_STANLEY	USD	+1,607,917	0	0	0	0	0	+1,607,917

Fidelity's negative USD is the total rebate paid ($956,250 + 651,666.67 = \$1,607,917$). Reinvestment income (§I.3) offsets it but accrues in the separate MMF unit, not in the USD position shown. Fidelity's net loan income is reinvestment return minus rebate paid.

Conservation.

- NVDA: $1,000,000_F + 0_M + \text{virtual}(-1,000,000) = 0$. ✓
- USD: $(-1,607,917)_F + 1,607,917_M = 0$ (plus virtual offsets for the original collateral movements). ✓

SLATE. Final Modification, quantity = 0, closing the loan.

I.9 Cross-Jurisdictional Comparison

The two worked examples differ only in three places: the `legal_regime` field of the SBL unit state, the smart-contract pre-conditions (P19), and the PnL projection. The six-coordinate position vector, the Single-Coordinate Move Principle, and the conservation law are identical across regimes.

Aspect	European (GMSLA 2010 TT)	US (Rule 15c3-3)
Collateral ownership	Title transfers to receiver, who <i>owns</i> collateral.	Pledge: poster retains economic ownership; receiver holds under control.

Aspect	European (GMSLA 2010 TT)	US (Rule 15c3-3)
Collateral type	Non-cash (German Bunds); haircut/margin on collateral market value.	Cash (USD); 102% of loan value posted.
Rehypothecation	Unrestricted under title transfer; SFTR Art. 15 documentation required.	Capped at 140% of customer debit balance (Rule 15c3-3(b)(3)); contract blocks excess.
Fee structure	Lending fee (25 bps on loan value); borrower pays lender.	Rebate (Fed Funds - 25 bps); lender pays borrower on cash, earns reinvestment spread.
Collateral coordinate	own for poster, coll_recv for receiver.	own for poster, coll_recv for receiver.
PnL formula	$V_e = \sum_u \vec{w}_e(u)[\text{own}] \cdot P_t(u)$	$V_e = \sum_u (\vec{w}_e(u)[\text{own}] + \vec{w}_e(u)[\text{coll_post}]) \cdot P_t(u)$
Regulatory reporting	SFTR (NEWT, MODI, VALU, ETRM); dual-sided; UTI per ESMA waterfall.	FINRA SLATE (New Loan, Modification); single-sided (lender reports).
Manufactured dividends	French withholding-tax treatment.	US tax-treaty provisions (IRC §871(m)).
Settlement discipline	CSDR: daily cash penalties for fails; auto-partial (IBP-125).	Reg SHO Rule 204: close-out by T+3; FINRA Rule 4320.
Governing agreement	GMSLA 2010 (title transfer); ISLA Best Practice Handbook.	Master Securities Loan Agreement; SEC/FINRA rules.
Position vector	Identical six coordinates.	Identical six coordinates.
Conservation law	$\sum_e \vec{w}_e(u)[\text{coord}] = 0.$	Identical.
Smart contract	Same SBL contract; legal_regime selects validation.	Same SBL contract; legal_regime selects validation.

The Ledger engine, conservation law, and move primitive are jurisdiction-neutral. Title transfer versus pledge, unrestricted versus capped rehypothecation, and lending fee versus rebate are encoded as SBL unit-state parameters and enforced by P19 pre-conditions. Only the PnL projection and the pre-condition logic differ, as derived in §16.9.

References

- [1] International Securities Lending Association, “ISLA Best Practice Handbook,” various editions.
- [2] U.S. Securities and Exchange Commission, “Rule 15c3-3: Customer Protection—Reserves and Custody of Securities,” 17 CFR §240.15c3-3.
- [3] U.S. Securities and Exchange Commission, “Regulation SHO, Rule 203(b)(1): Locate Requirement,” 17 CFR §242.203(b)(1).
- [4] U.S. Securities and Exchange Commission, “Rule 10c-1a: Reporting of Securities Loans,” 17 CFR §240.10c-1a, adopted January 2024.

- [5] Financial Industry Regulatory Authority, “FINRA Rule 6500 Series: Securities Lending and Transparency Engine (SLATE),” 2024.